

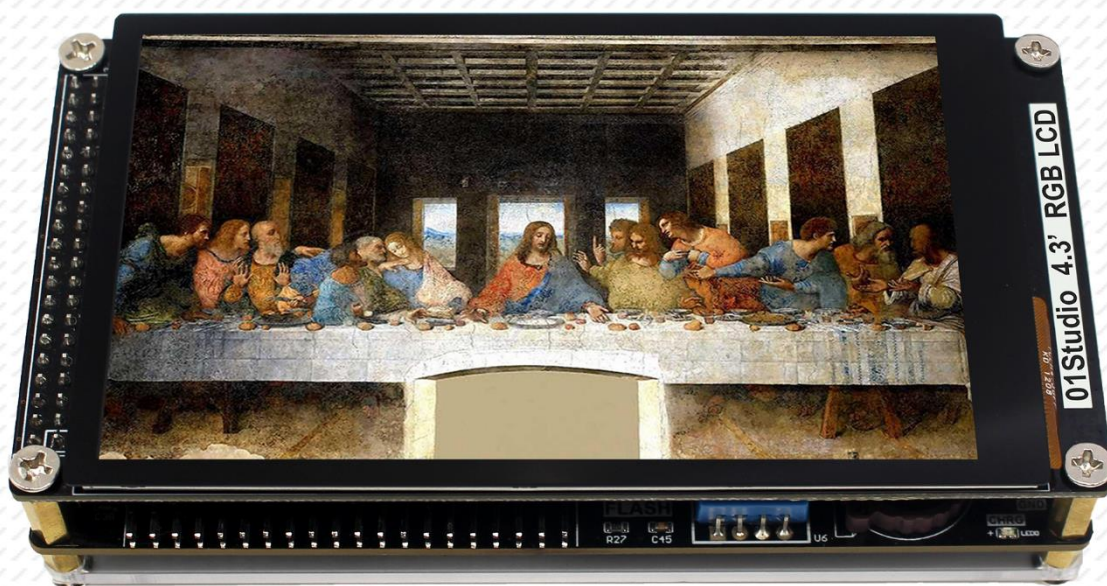
MicroPython

从0到1

用python做嵌入式编程

(基于达芬奇TKM32F499平台)

01Studio团队 编著



 01Studio

- 让编程变得简单有趣 -

MicroPython

产品家族



序

物换星移，人类已经发展到 21 世纪的 2021 年。从信息匮乏到现在的互联时代，在弹指的一瞬间。即便是在 21 世纪初的头十年到最近的十年，电子信息都已经发生了翻天覆地的变化。人们的编程习惯从机器码到汇编到 C 语言一路发展。01 科技紧跟科技时代的步伐，窥探着未来的动态，灵巧地揪住 Python 这年轻但富具活力的编程语言。配合当今主流的物联网模块，向着世界通用，万物互联的美好远景目标迈进。

万物互联，人机交互，正是当今物联网的热满。好钜润科技的 TKM32F499 芯片正是主打人机交互，深耕显示技术的细分领域方向的芯片。该芯片采用 Cortex-M4 内核，大容量 SDRAM，配备 TK80 MCU 接口及 RGB888 LTDC 双重接口，独立电阻触摸 IP，搭配适当的中断及 DMA，十分适合人机交互显示。芯片整体设计独特有个性，液晶显示接口及其它外设尽量考虑实用性，排序整齐划一，不多不少，简洁实用。从设计理念上，就是走君子和与不同的路线，一个不与友商的同质化过度竞争，相互尊重，另外又填补了细分领域的空隙。但总体设计上，又保持了一定的兼容性，在基础架构上不重复造轮子，减轻用户适应及学习成本。体现了我中有你，你中有我，共同向星辰大海发展的设计理念。

01 科技在开发 TK499 MicroPython 过程中，非常努力攻关各种问题，也提出了很多宝贵的意见，为目前正在研发的 TK499 二代提供了有力的支持。从开发交流中也可以看出 01 科技团队，攻坚能力强，遇到问题耐心查找，主观能动性很强。这些工作作风，企业文化也体现在这本书上。本书图文并茂地讲解了 TK499 在 MicroPython 上的应用，各个功能层次分明，底层已经妥当优化好，与应用上层隔离，令初入 MicroPython 的小白也能轻松接受，不用再依据不同的硬件厂商去重新理解底层。从而让用户更专注于算法及应用功能的实现，努力的方向纯朴专一。01 科技已经把底层架构好，这令硬件达到“和”的统一，但是软件应用上层又是灵活开源，支持二次开发，再次推向了君子和而不同中，在和之上实现“不同”的高度。整书给人一种基础扎实，精练，少则是多的哲学感。

好钜润科技创始人 周泰毅

前言

为什么学习 MicroPython?

单片机嵌入式编程经历了汇编、C 语言的发展历程，可以说是一次编程革命，其背后的原因是单片机的速度越来越快，集成度越来越高。而这一趋势并没停止，摩尔定律仍然适用。在未来，单片机上很可能直接跑机器语言。

在 2014 年，MicroPython 在英国诞生了，对于电子爱好者来说无疑拉开了新时代的序幕，用 python 这个每年用户量不断增长的编程语言来开发嵌入式，加上无数开源的函数模块，让嵌入式开发变得从未如此的简单。

MicroPython 致力于兼容 Python。因此，我们在学习完 MicroPython 后除了可以开发有趣的电子产品外，还可以继续深入使用 Python 语言去开发后台、人工智能等领域。

为什么要写《MicroPython 从 0 到 1》教程?

对于初学者，常常需要浪费大量时间来搭建开发环境和安装应用软件，以及需要从不同渠道寻找开发资料。MicroPython 作为新兴的东西，市面上的资料更是参差不齐，被搞得头昏目眩。

“让编程变得简单有趣”作为我们 01Studio 团队的使命，我们一直在寻找最优的学习方法。力求以最简单易懂的方式来讲述 MicroPython 的学习方法，从开发环境快速建立、基础实验、传感器实验到项目进阶实验。《MicroPython 从 0 到 1》诞生了。相比于传统的出版图书，我们会以更平易近人的口吻讲解实验，配上精美的插图，每一行代码都是自己的亲身经历，目的就是让大家更好的入门 MicroPython，将精力放在编程和开发中去。

为什么要打造 MicroPython 学习套件?

MicroPython 在中国是一个新兴的东西，前途无限，但是网上的学习模块套件参差不齐，大多是复制官方开源的开发板来设计，用过的就知道，外国的电路设计跟国内的风格很不同，甚至常常让初学者钻牛角尖。为此，01Studio 团队特意打造的中国风的 MicroPython 开发套件，《MicroPython 从 0 到 1》上的例程也是基于此板开发的，每个例程都能直接跑起。通过一系列系统学习，你甚至可

以用它来完成你的比赛或项目。

为什么要打造 01Studio 社区论坛？

早在数年前我们打造了 WeBee 品牌，专注物联网开发，到现在已经服务了数万名学生、教师和工程师等相关人员。早期依靠着群来维护，但随着开发者的数量增加，我们发现仅依靠群或者技术支持人员已经不能满足需求。

社区论坛是很好的学习交流地方，在这里你可以学习到前人的经验，可以通过搜索内容来解决问题。为什么全世界很火的开源硬件都是由外国人做起来，我们认为很重要的一个点是源于开源和分享精神。大部分开发者都会想着从论坛或网站解决自身问题而不愿意奉献，而我们希望 01Studio 社区是一个大家乐于发表文章和分享心得的地方。我们始终始终坚持开源原则，包括书籍内容、所有例程代码和部分硬件模块的开源。

期待你加入 01Studio 社区大家庭，成为我们的一份子，一起成长。

【社区链接：bbs.01studio.cc】

【微信公众号：01Studio 社区】

【官方商城：01studio.taobao.com】

01Studio
2019.4 于深圳

版本说明

版本	日期	作者	更新内容
v1.0	2021-6-1	Jackey	1. 第 1 版正式发布
v1.1	2021-11-15	Jackey	1. 加入序 2. 修改官方网站链接

版权声明

《MicroPython 从 0 到 1》由 01Studio 团队打造，已于深圳市版权局注册备案，任何单位或个人引用相关文字、图片或相关内容请注明出处【01Studio】，否则我们将保留追究相关法律责任的权利。

目录

第 1 章 MicroPython 简介	8
1.1 MicroPython 是什么	8
1.2 MicroPython 支持的微控制器平台	9
1.3 MicroPython 相关学习资料	11
1.3.1 01Studio 技术社区	11
1.3.2 MicroPython 文档	12
1.3.3 Github 开源代码	13
1.3.4 MicroPython 官方网站	14
1.4 达芬奇开发套件介绍	15
1.4.1 达芬奇 TKM32F499 开发板	16
1.4.2 传感器模块	27
1.4.3 IOT/通讯模块	34
1.4.4 拓展配件	37
第 2 章 Python 基础知识	42
2.1 原始数据类型和运算符	42
2.2 变量和集合	47
2.3 流程控制和迭代器	53
2.4 函数	57
2.5 类	60
2.6 模块	62
2.7 高级用法	63
第 3 章 开发环境快速建立	65
3.1 基于 Windows	66
3.1.1 安装开发软件 Thonny	66
3.1.2 开发套件使用	68
3.2 基于 Mac OS	90
3.2.1 安装开发软件 Thonny	90
3.2.2 开发套件使用	90
3.3 基于 Linux（树莓派）	91
3.3.1 安装开发软件 Thonny	91
3.3.2 开发套件使用	91
第 4 章 基础实验	92
4.1 点亮第一个 LED 灯	93
4.2 流水灯	99
4.3 按键	107
4.4 外部中断	112
4.5 定时器	117
4.6 蜂鸣器	121
4.7 LCD 显示屏	127
4.7.1 4.3 寸 RGB 屏	128
4.7.2 7 寸 RGB 屏	137
4.7.3 图片缓存文件制作	146

4.8 电容触摸屏	155
4.8.1 4.3 寸 RGB 屏触摸.....	156
4.8.2 7 寸 RGB 屏触摸.....	161
4.9 触摸屏按钮	166
4.10 ADC.....	172
4.11 RTC 实时时钟	178
4.12 OLED 显示屏.....	184
4.13 thread（线程）	191
第 5 章 传感器实验	194
5.1 温湿度传感器 DHT11.....	195
5.2 MPU6050 六轴传感器	201
5.3 人体感应传感器	208
5.4 光敏传感器	214
5.5 土壤湿度传感器	220
5.6 水位传感器	226
5.7 大气压强传感器	233
5.8 超声波传感器	239
5.9 继电器	245
第 6 章 通讯实验	250
6.1 UART（串口通信）	251
6.2 网络通讯	258
6.2.1 连接无线路由器	259
6.2.2 Socket 通信	265
6.2.3 MQTT 通信.....	277
6.3 pyIOT（无线通信模块）	294
6.3.1 pyIOT-BLE TLS-01 蓝牙模块.....	295
6.3.2 pyIOT-GPS ATGM336H 卫星定位模块.....	305

第1章 MicroPython 简介

1.1 MicroPython 是什么

第一次接触 MicroPython 的时候,我就想这是个什么玩意,从字面意思来看,就是 Micro 加 Python。难道是阉割版的 Python? 阉割后可以在微控制器上面跑? 当然你也可以这么理解,我们来看看官方的说明:

“MicroPython 是 Python 3 编程语言的精简高效实现,包括 Python 标准库的一小部分,并且经过优化,可以在 Microcontrollers (微控制器)和有限的环境中运行。

MicroPython 包含许多高级功能,如交互式提示,任意精度整数,闭包,列表理解,生成器,异常处理等。然而它非常紧凑,可以在 256k 的代码空间和 16k 的 RAM 内运行。

MicroPython 旨在尽可能与普通 Python 兼容,以便您轻松地将代码从电脑传输到微控制器或者嵌入式系统。”

看完官方说明后,大家应该有所了解,MicroPython 是指在微控制器上使用 Python 语言进行编程,学习过单片机和嵌入式开发的小伙伴应该都知道早期的单片机使用汇编语言来编程,随着微处理器的发展,后来逐步被 C 所取代,现在的微处理器集成度越来越高了,那么我们现在可以使用 Python 语言来开发了。

Python 的强大之处是封装了大量的库,开发者直接调用库函数则可以高效地完成大量复杂的开发工作。MicroPython 保留了这一特性,常用功能都封装到库中了,以及一些常用的传感器和组件都编写了专门的驱动,通过调用相关函数,就可以直接控制 LED、按键、伺服电机、PWM、AD/DA、UART、SPI、IIC 以及 DS18B20 温度传感器等等。以往需要花费数天编写才能实现的硬件功能代码,现在基于 MicroPython 开发只要十几分钟甚至几行代码就可以解决。真可谓:“人生苦短,我用 Python 和 MicroPython”。

1.2 MicroPython 支持的微控制器平台

MicroPython 到目前为止已经可以在多种嵌入式硬件平台上运行：STM32、ESP8266、ESP32、CC3200、K210 等等。由于项目的开源特性，很多开发者在尝试将其移植到更多平台上。

MicroPython 最早支持的硬件平台是 STM32，开发板名称叫 pyboard。使用的芯片型号是：STM32F405RGT6，该芯片具备 1MB flash 和 196k SRAM，168MHZ 主频。

而达芬奇 TKM32 平台则使用 TKM32F499CGT8 芯片，本书主要是围绕达芬奇 TKM32 平台来进行编写，更详细的硬件参数会在后面章节内容讲述。

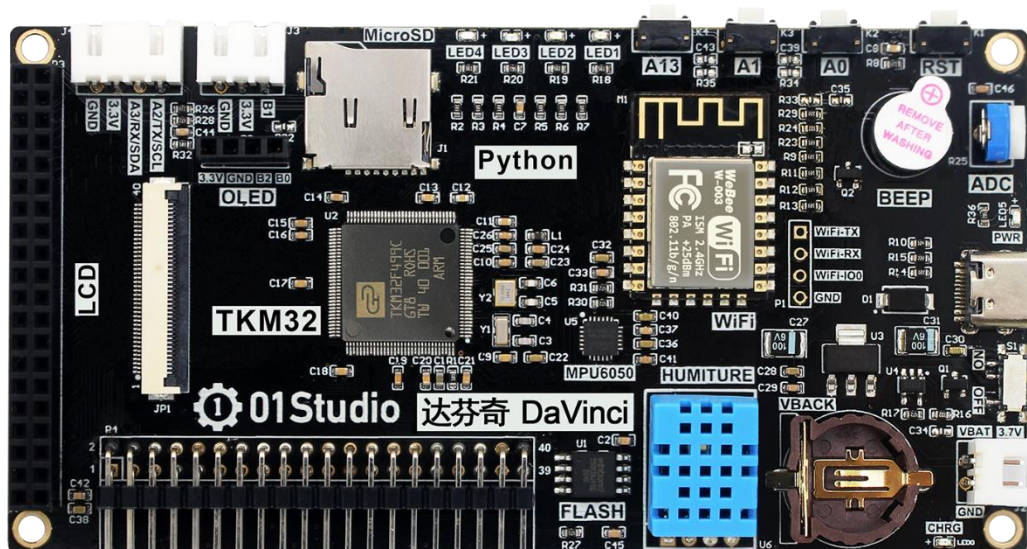


图 1-1 01Studio 达芬奇 TKM32 MicroPython 开发平台

除此之外，上海乐鑫的 WIFI 芯片 ESP8266/ESP32 也非常成熟。用户使用 MicroPython 可以快速开发物联网相关应用，实现 WIFI 无线连接。

不少优秀的开源项目也是基于 MicroPython 衍生出来的，如机器视觉界 Arduino 之称的 OpenMV、人工智能芯片 K210 等。随着社区的日益成熟，MicroPython 必定将嵌入式编程推向新的高度。

STM32 平台	ESP8266 平台	ESP32 平台	NRF52840 平台	OpenMV4 平台	K210 平台
pyBoard v1.1-CN	pyWiFi-ESP8266	pyWiFi-ESP32	pyBLE-NRF52840	pyAI-OpenMV4	pyAI-K210
					
					

图 1-2 01Studio 其它 MicroPython 系列开发平台

1.3 MicroPython 相关学习资料

1.3.1 01Studio 技术社区

【社区网址：bbs.01studio.cc】

01Studio 社区是 MicroPython 开发者交流的技术论坛，我们以极简风格设计，开发者在学习过程中遇到问题可以到论坛搜索或者发帖提问，以提高学习效率。01Studio 团队也会在社区定期发布学习资源。



图 1-3 01Studio 社区

1.3.2 MicroPython 文档

限于篇幅，本教程部分实验只介绍关键的函数和模块应用，如果在学习过程中希望深入了解 MicroPython 函数和模块，请查阅：

MicroPython 文档（中文） 【网址：docs.01studio.cc】



图 1-4 MciroPython 文档（中文）

1.3.3 Github 开源代码

当前 01Studio 所有硬件平台对应项目源代码均开源，用户可以通过 01Studio 社区查看。链接：<https://github.com/01studio-lab>

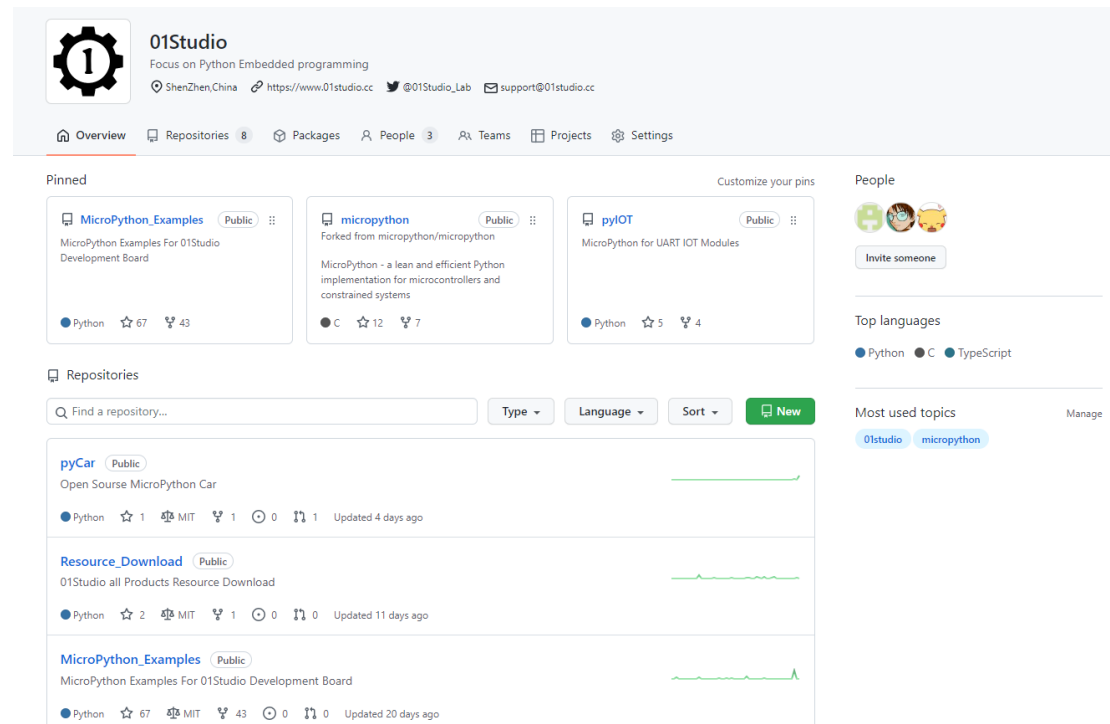


图 1-5 源代码在社区的位置

1.3.4 MicroPython 官方网站

【网址：www.micropython.org】

英文版官网有官方文档(DOCS)和英文论坛, 适合比较英语比较好的小伙伴。

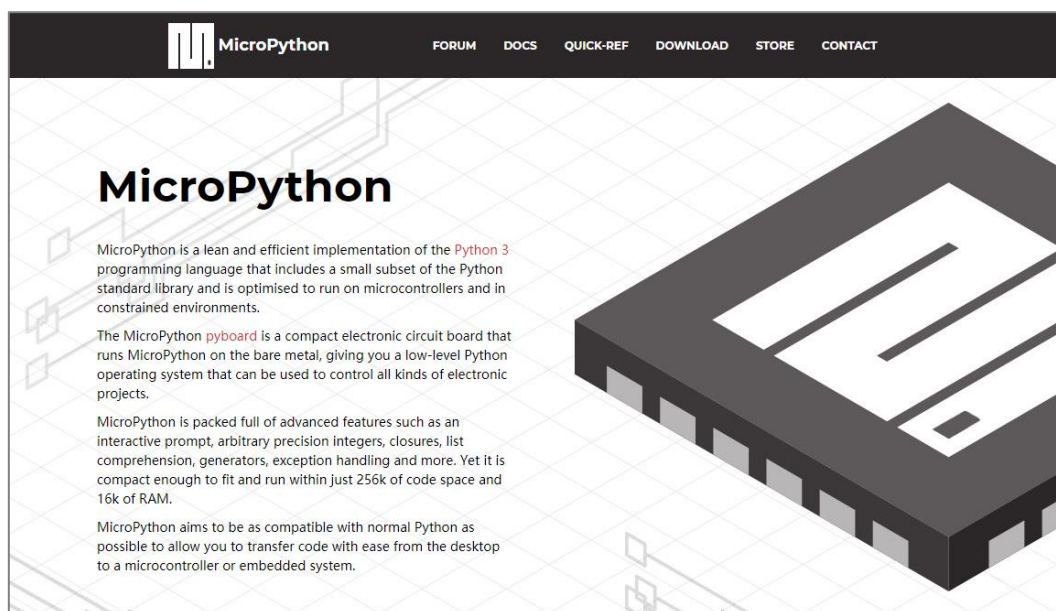


图 1-6 MicroPython 官网

1.4 达芬奇开发套件介绍

为了让广大电子爱好者更方便地学习 MicroPython，01Studio 团队打造了一系列本土化的高性价比学习套件和周边模块。当前已支持 STM32 平台、树莓派 Pico 平台、ESP8266 平台、ESP32 平台、NRF52840 平台、OpenMV 平台、K210 平台以及周边传感器模块和配件外设。我们保留了对 MicroPython 官方的兼容性，同时使开发者可以更好的连接外设，进行更多扩展性实验。

当前本书上的所有例程都是基于本达芬奇 TKM32F499 平台开发的，我们承诺资源会不断更新，保证所有代码程序能直接跑起。毫不夸张地说：你甚至可以将本教材的例程和实践应用在自己的产品研发和项目开发中去。

1.4.1 达芬奇 TKM32F499 开发板

本节只对硬件最基本的介绍, 让用户对达芬奇开发平台上的硬件资源有个简单的认识, 详细的说明可以参考后面实验内容, 每个实验都会有详细讲解。

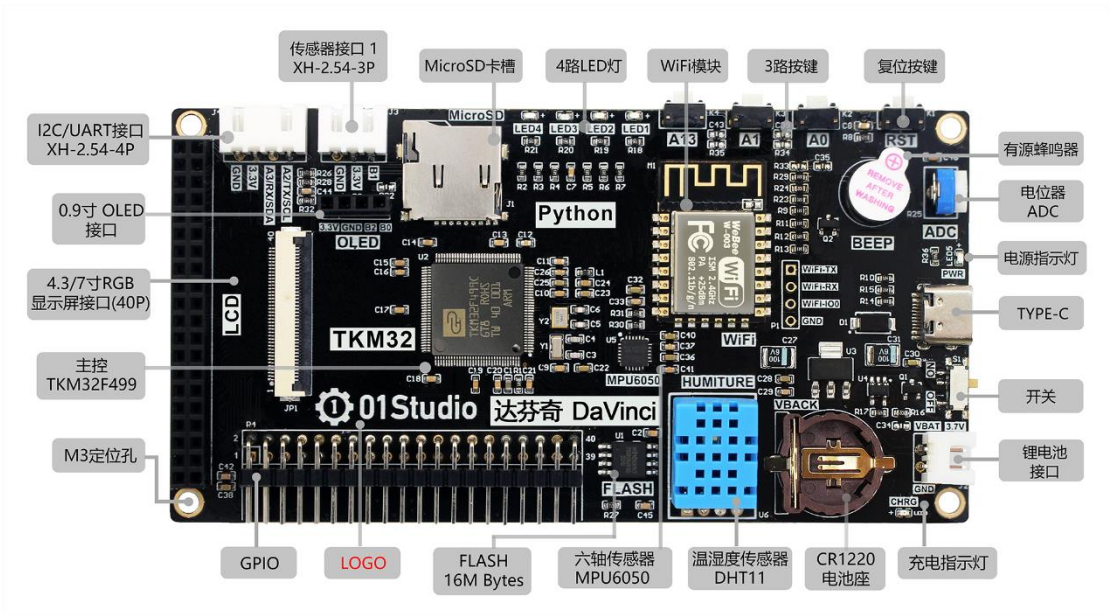


图 1-7 达芬奇开发板资源描述



图 1-8 达芬奇开发板背面

达芬奇开发板资源	
尺寸	12*6.2 cm
主控	TKM32F499CGT8, Cortex M4, 主频: 240 MHz, 内部 RAM:8MBytes, 128Pin
存储	FLASH: 16M Bytes
	SDIO: MicroSD 卡插槽
传感器	温湿度传感器: DHT11
	六轴传感器: MPU6050 (3 轴加速度+3 轴陀螺仪)
	传感器外接: 3P 防呆接口
输入/输出	LED: 4 个
	按键: 4 个 (1 个复位按键+3 个功能按键)
	电位器: ADC 输入
	有源蜂鸣器
	OLED 显示屏接口
	4.3 寸 RGB 显示屏 (电容触摸) 接口
	引出所有 GPIO: 标准 2.54mm 排针
通讯	USB: TYPE-C 接口 (供电、烧写、UART、REPL 调试多合一)
	I2C/UART: 4P 防呆接口
	ESP8266 串口透传 WiFi 模块
电源	USB(TYPE-C)供电: 5V
	锂电池供电: 3.6~4.2V (板载锂电池充电电路)
	RTC 后备电池: CR1220
	3.3V/5V 排针

表 1-1 达芬奇开发板资源

1.4.1.1 主控

达芬奇开发板的主控为 TKM32F499CGT8, 240MHz 主频, 128 个引脚。

芯片主要特点:

- 内置超大 8Mbytes SDRAM, 不再需要外挂 RAM;
- 带 FPU, 支持硬件浮点运算, 在运动算法, 图像变换上有 DSP 功能加持, 如虎添翼;
- 高级 DMA 功能, 单次吞吐量高达 4 亿点;
- 自研高级液晶屏驱动接口 TK80, 较普通 FSMC 快一倍以上;
- 带 RGB888 接口, 硬件双图层及支持软件多图层;
- 采用 USB 方式下载, 彻底摆脱依赖第三方下载软件;
- 带电阻触摸 IP, 为您省芯又省心;
- 丰富的外设和中断源, ADC、IIC、IIS、双 SDIO、QSPI、SPI、UART、Timer;
- 单芯片, 单电源的理想解决方案!

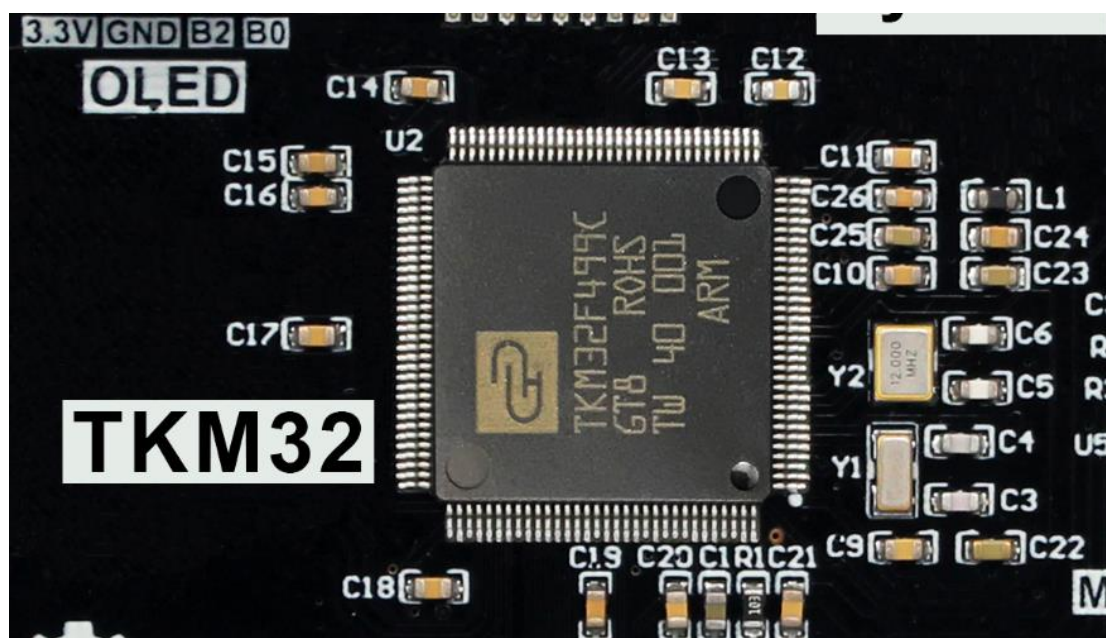


图 1-9 MCU 位置

1.4.1.2 RAM

TKM32F499 芯片自带 8Mbytes RAM，无需外接 RAM。

1.4.1.3 FLASH

TKM32F499 芯片没片内 Flash，因此我们增加了 16M 字节的外扩 Flash。存放 micropython 固件和用作文件系统。

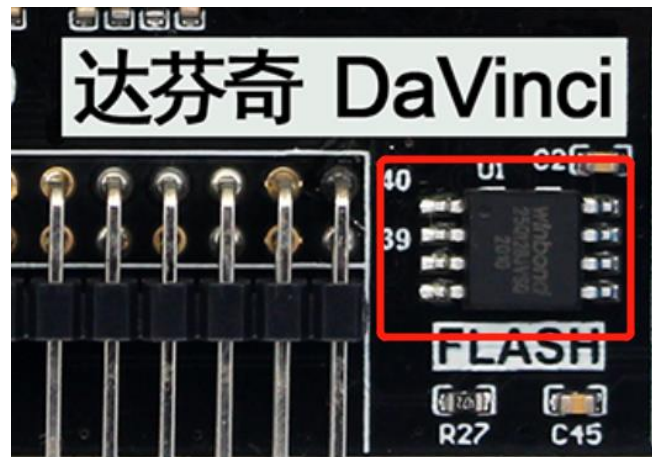


图 1-10 Flash 位置

1.4.1.4 LED

4 路 LED 灯。

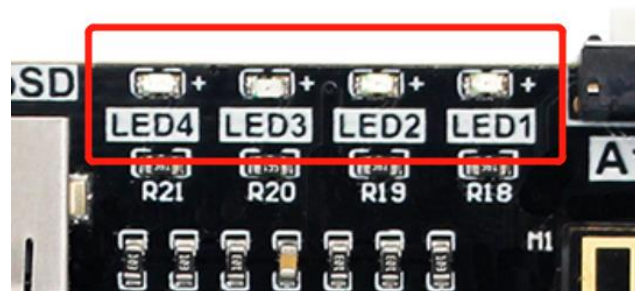


图 1-11

1.4.1.5 按键

1 个复位按键和 3 个功能按键。

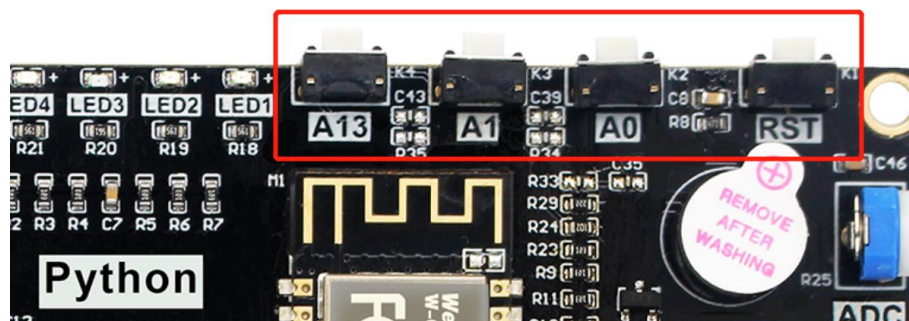


图 1-12

1.4.1.6 温湿度传感器 DHT11

温湿度传感器。

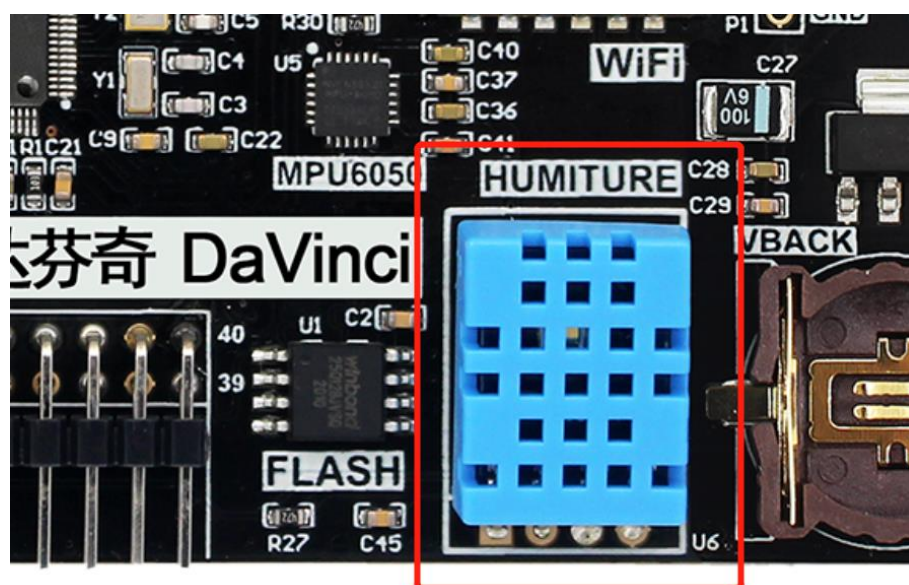


图 1-13

1.4.1.7 MPU6050

板载六轴加速度传感器 MPU6050，可以实现三轴加速度和三轴角速度数据的测量。

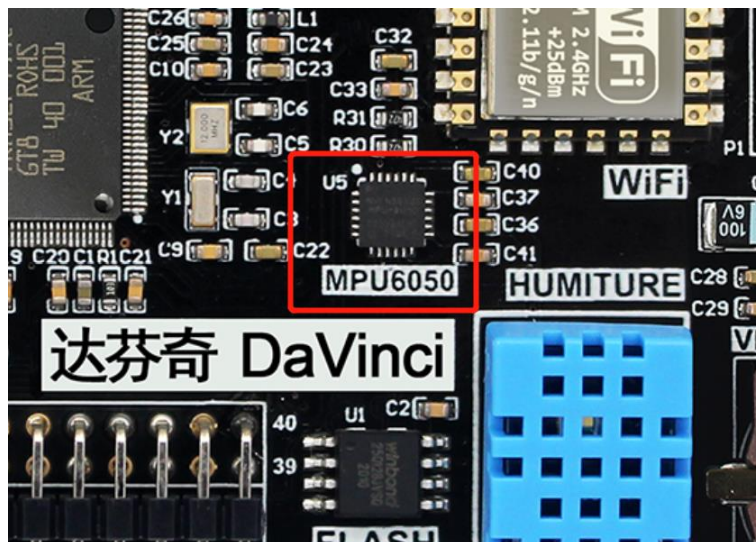


图 1-14 MPU6050

1.4.1.8 可调电阻

通过调节可调电阻可输出不同电压值，用于 ADC（模拟数字转换）实验。



图 1-15 电位器 ADC

1.4.1.9 有源蜂鸣器

有源蜂鸣器用于发出滴滴声音。

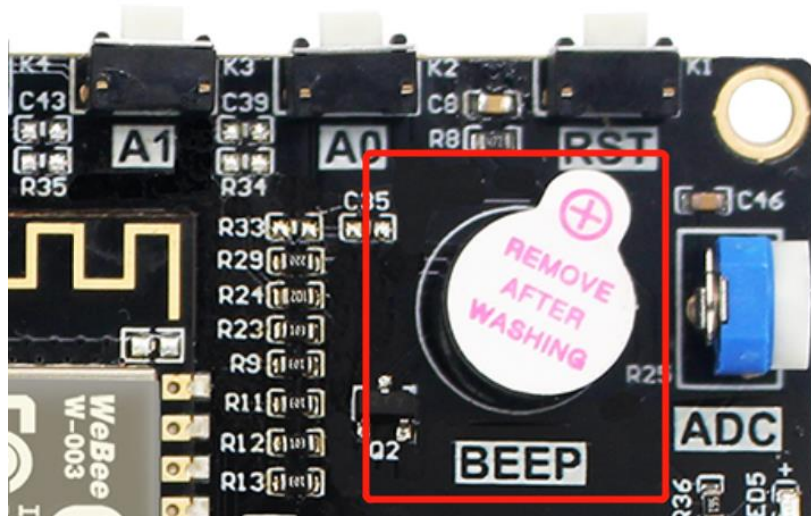


图 1-16

1.4.1.10 纽扣电池

用于 RTC 实时时钟断电后继续运行。

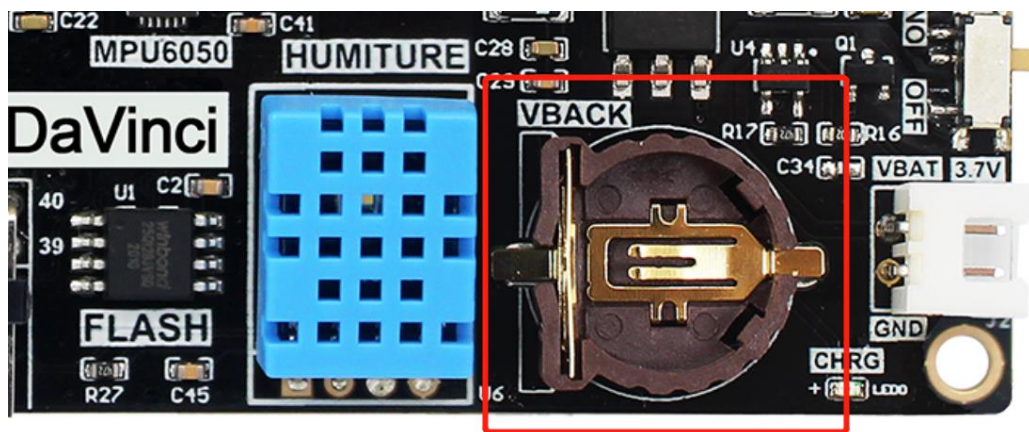


图 1-17

1.4.1.11 SD 卡槽

达芬奇可以通过 microSD 卡获取大容量存储空间，最大支持 32G。

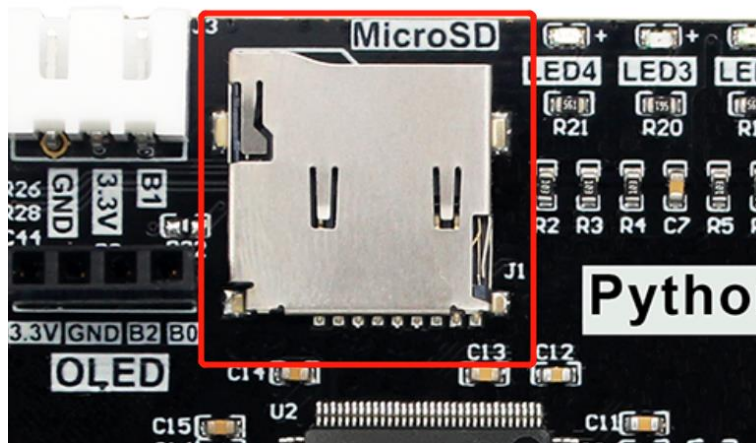


图 1-18

1.4.1.12 ESP8266 串口透传 WiFi 模块

用于连接到网络。

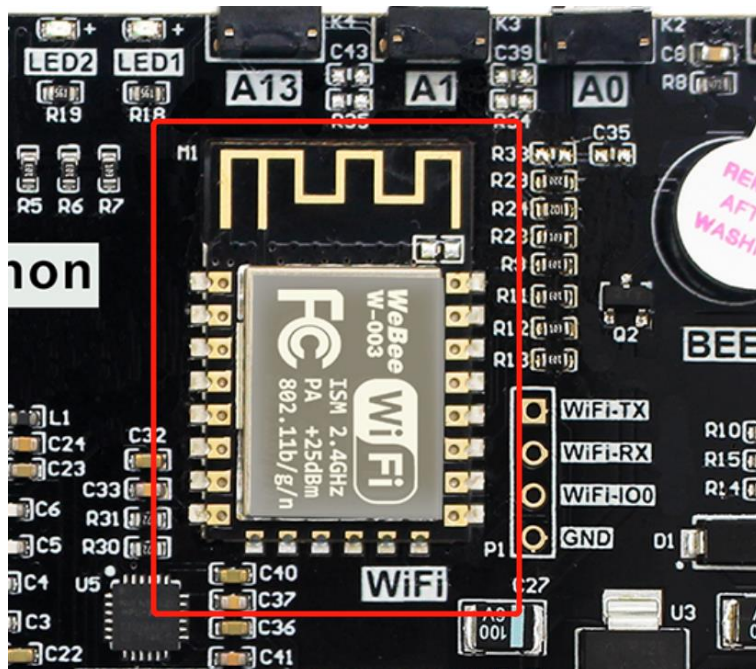


图 1-19

1.4.1.13 3P 传感器扩展接口

用于连接单 GPIO、单总线、ADC 的传感器。

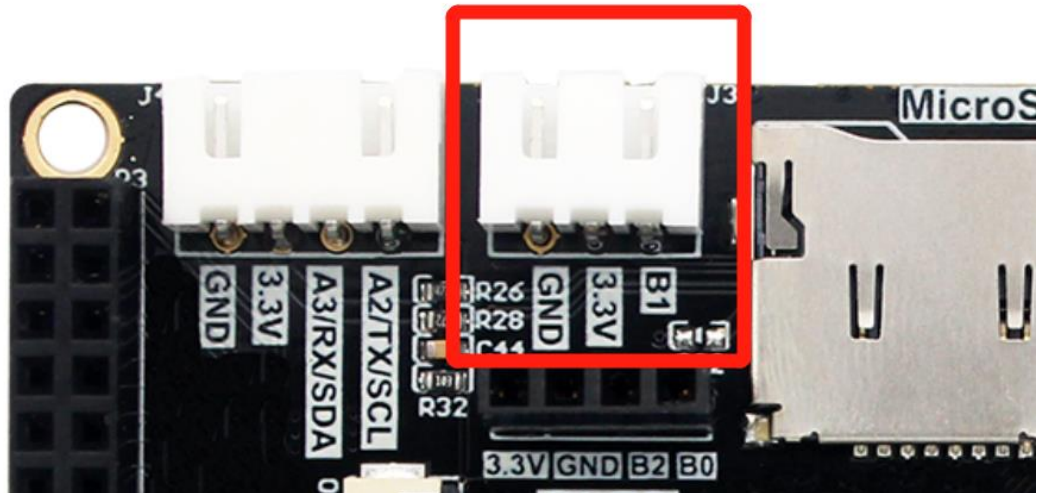


图 1-20

1.4.1.14 4P I2C/UART 扩展接口

用于 I2C、UART 扩展接口。

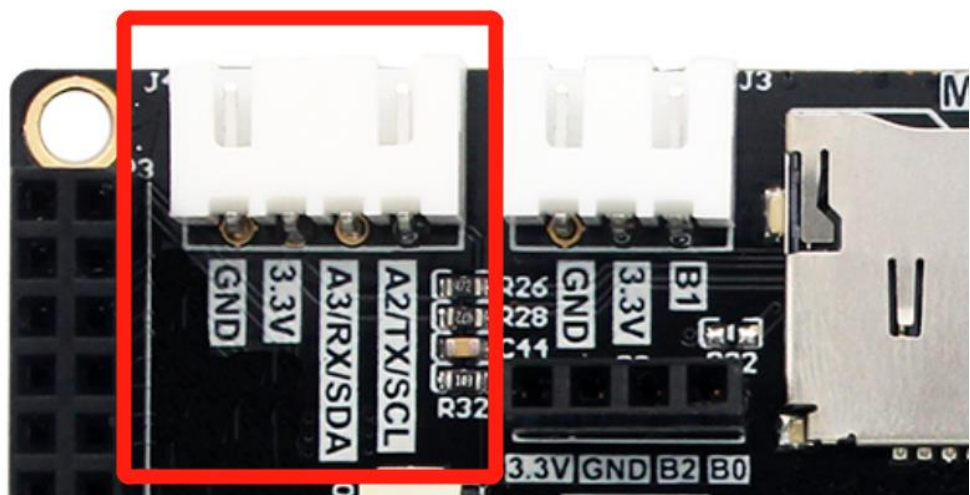


图 1-21

1.4.1.15 LCD 显示屏接口

达芬奇开发板板载 LCD 接口，可支持 RGB888 24 位的 RGB 显示屏（带电容触控）。默认可使用 01Studio 4.3 寸 RGB 显示屏或 7 寸 RGB 显示屏。



图 1-22 LCD 接口 (40P)

1.4.1.16 TYPE-C 接口

使用 TYPE-C 数据线进行开发。支持供电、烧录固件、U 盘拷贝程序、REPL 调试。



图 1-23

1.4.1.17 电源管理

可以使用 TYPE-C 供电（5V），板载 3.7V 锂电池供电和充电电路。同时接入 5V 电源和锂电池即可充电。橙色指示灯亮表示充电，充满熄灭。

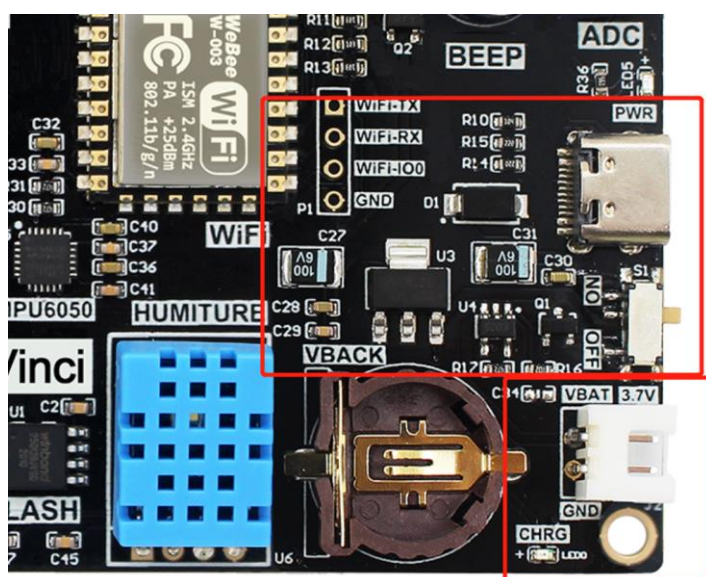


图 1-24

1.4.2 传感器模块

1.4.2.1 人体感应传感器模块

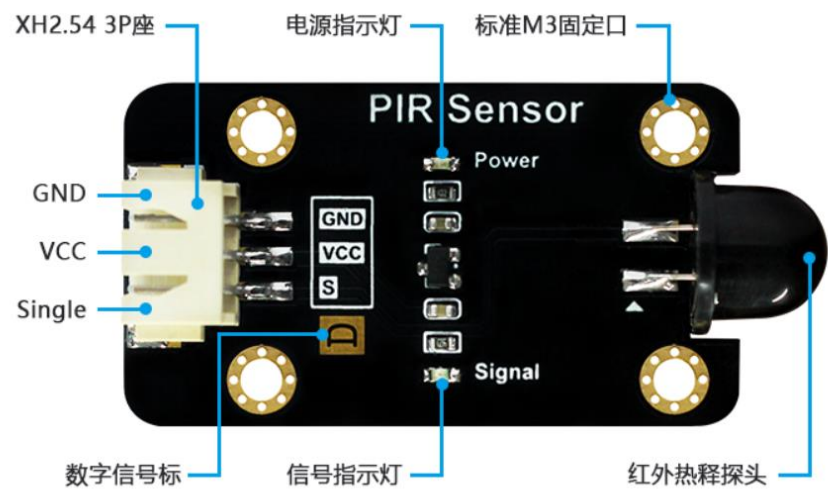


图 1-25 人体感应传感器模块

功能参数	
供电电压	3.3-5V
工作电流	<20 mA
工作温度	-20 至 65℃
接口定义	XH2.54 防呆接口（3Pin）【GND、VCC、Single】
输出信号	数字信号： 检测到人体：高电平 3.3V；持续 3-8 秒 未检测到人体：低电平 0V。
感应角度	100°
感应距离	8 米
模块尺寸	4.5*2.5cm

表 1-2

1.4.2.2 光敏传感器模块

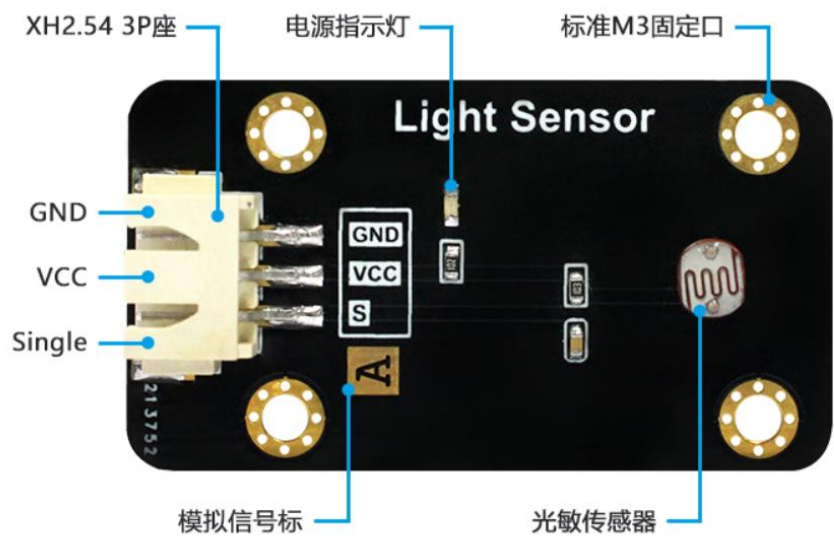


图 1-26 光敏传感器模块

功能参数	
供电电压	3.3-5V
工作电流	<20 mA
工作温度	-20 至 85℃
接口定义	XH2.54 防呆接口（3Pin）【GND、VCC、Single】
输出信号	模拟信号：0-3.3V （VCC=3.3V 时）
模块尺寸	4.5*2.5cm

表 1-3

1.4.2.3 土壤湿度传感器模块

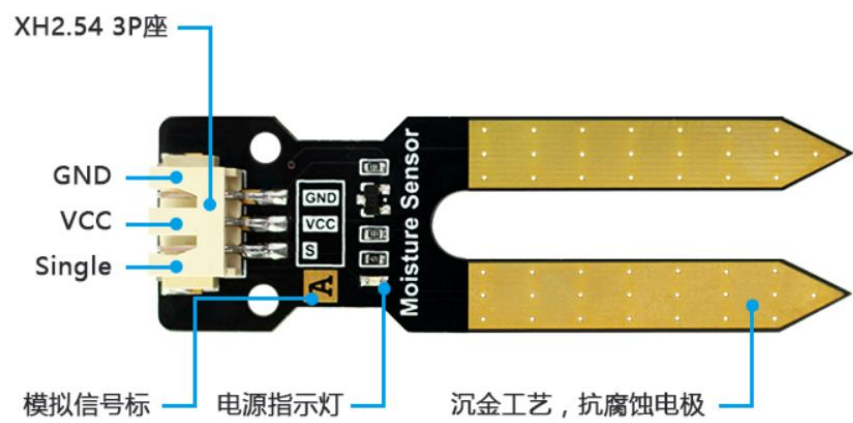


图 1-27 土壤湿度传感器模块

功能参数	
供电电压	3.3-5V
工作电流	<20 mA
工作温度	-40 至 85℃
接口定义	XH2.54 防呆接口（3Pin）【GND、VCC、Single】
输出信号	模拟信号：0-3.3V （VCC=3.3V 时）
模块尺寸	6.6*2.0cm

表 1-4

1.4.2.4 水位传感器模块

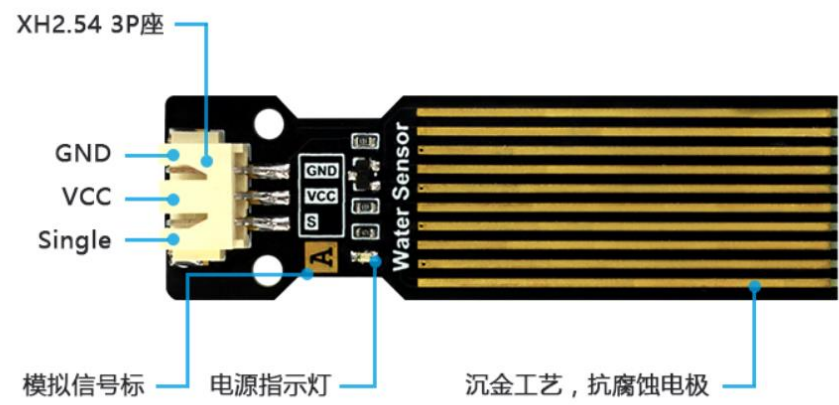


图 1-28 水位传感器模块

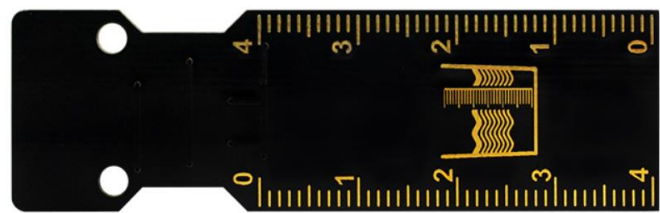


图 1-29 背面

功能参数	
供电电压	3.3-5V
工作电流	<20 mA
工作温度	0 至 60℃
接口定义	XH2.54 防呆接口（3Pin）【GND、VCC、Single】
输出信号	模拟信号：0-3.3V （VCC=3.3V 时）
模块尺寸	6.6*2.0cm
注意事项	水位传感器接口旁元件不能接触水，否则容易短路。所以测量时候要注意液体水位的高度。

表 1-5

1.4.2.5 大气压强传感器模块

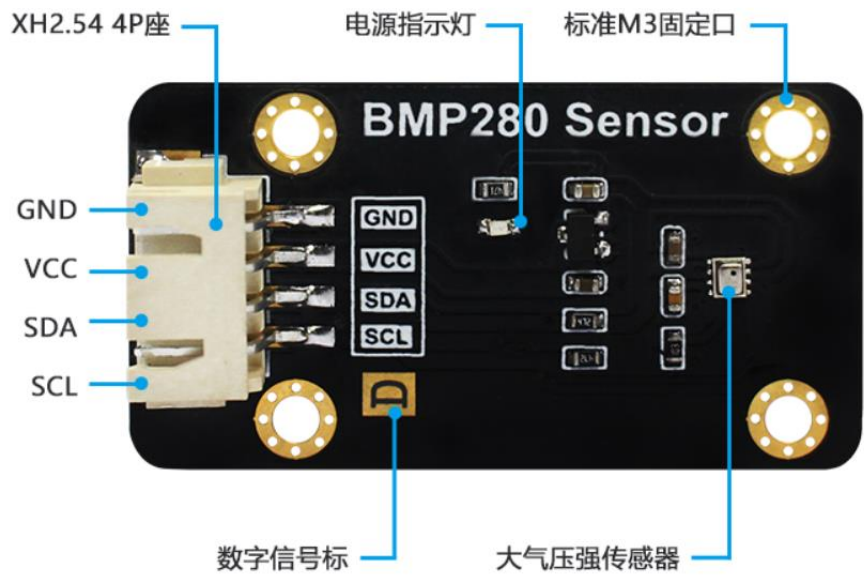


图 1-30 大气压强传感器模块

功能参数	
传感器型号	BMP280
供电电压	3.3-5V
工作电流	<20 mA
工作温度	-20 至 85℃
接口定义	XH2.54 防呆接口（4Pin）【GND、VCC、SDA、SCL】
通讯信号	I2C 总线
I2C 地址	0x76（BMP280 的 SDO 默认下拉）； 当 BMP280 的 SDO 引脚上拉时，I2C 地址为：0x77
模块尺寸	4.5*2.5cm

表 1-6

1.4.2.6 超声波模块

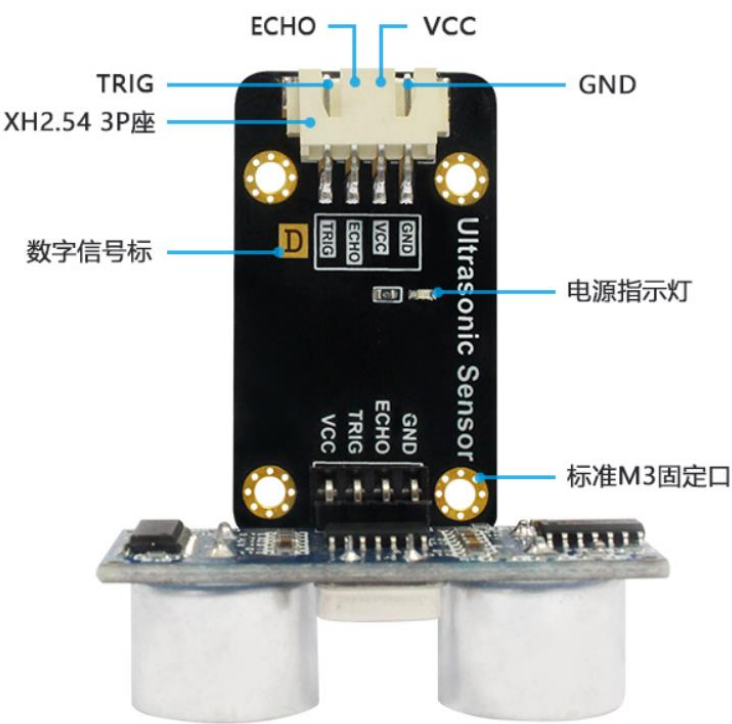


图 1-31 超声波模块

功能参数	
传感器型号	HCSR04
供电电压	3.3-5V
工作电流	<20 mA
工作温度	-20 至 85℃
接口定义	XH2.54 防呆接口（4Pin） 【GND、VCC、ECHO、TRIG】
通讯信号	IO 数字接口
测量距离	2-450 cm
测量精度	0.5 cm
整体尺寸	5.5*4.5*3.0 cm

表 1-7

1.4.2.7 继电器模块



图 1-32 超声波模块

功能参数	
供电电压	3.3V
触发方式	低电平触发
高压连接	最大 250V/3A
接口定义	XH2.54 防呆接口（3Pin） 【GND、VCC、IO】
整体尺寸	4.5*2.5 cm

表 1-8 继电器模块功能参数

1.4.3 IOT/通讯模块

pyIOT 开源项目由 01Studio 发起，旨在为市面上成熟的串口物联网模块开发 MicroPython 库，让用户可以使用 python 编程快速实现各类物联网相关应用。

1.4.3.1 pyIOT-BLE TLS01

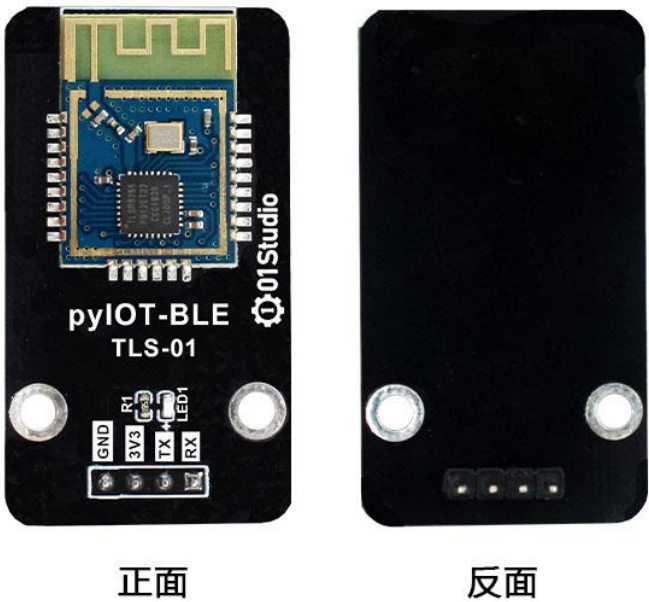


图 1-33

功能参数	
蓝牙主控	TLSR8266
控制方式	UART (串口)
蓝牙版本	BLE 4.0 (从机)
功耗	工作电流：<8.8mA，广播电流：<1mA
引脚	GND, 3.3V, TX, RX
模块尺寸	4.5*2.5cm

表 1-9

1.4.3.2 pyIOT-GPS

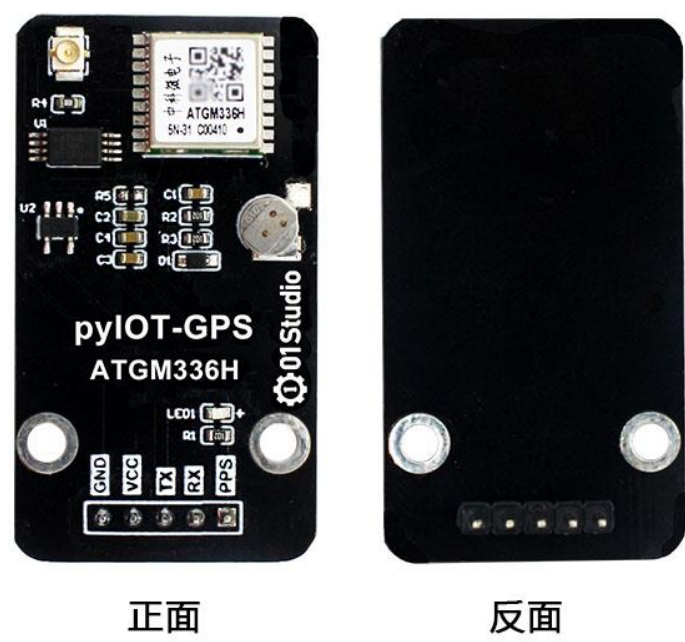


图 1-34

功能参数	
主控芯片	ATGM336H-5N
卫星信号	GPS/BDS/GLONASS
波特率	9600bps
工作电压	3.3V-5V
串口电平	3.3V 或 5V (自适应)
定位精度	2.5m (开阔地点)
功耗	工作：<25mA，待机：<10uA （@3.3V）
模块尺寸	4.2*2.5cm

表 1-10

1.4.3.3 pyIOT-LORA E22

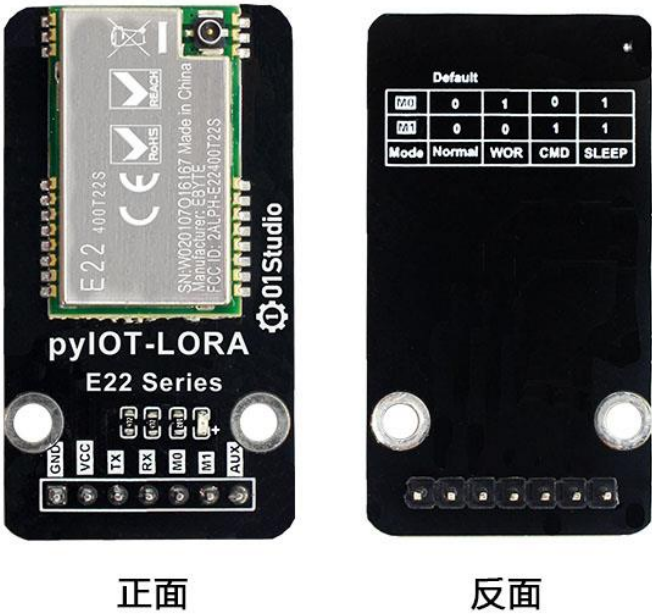


图 1-35

功能参数	
工作频段	410~493MHz (433M)
射频芯片	SX1268
发射功率	22dBm
通讯距离	最大： 5km
通讯接口	UART (串口)
天线	IPEX
发射电流	110mA
供电电压	2.3-5.2V
工作温度	-40℃ - 85℃
波特率	1200 – 115200 bps (默认 9600)
空中速率	0.3k – 42.5kbps
接收长度	1024 字节（自动分包）
模块尺寸	4.2*2.5cm

表 1-11

1.4.4 拓展配件

1.4.4.1 OLED 显示屏



图 1-36 OLED 显示屏

功能参数	
供电电压	3.3V
屏幕尺寸	0.9 寸
颜色参数	黑底白字
通讯方式	I2C 总线
接口定义	2.54mm 排针（4Pin） 【VCC、GND、SCL、SDA】
整体尺寸	2.8*2.8cm

表 1-12

1.4.4.2 4.3 寸 RGB 显示屏（电容触摸）

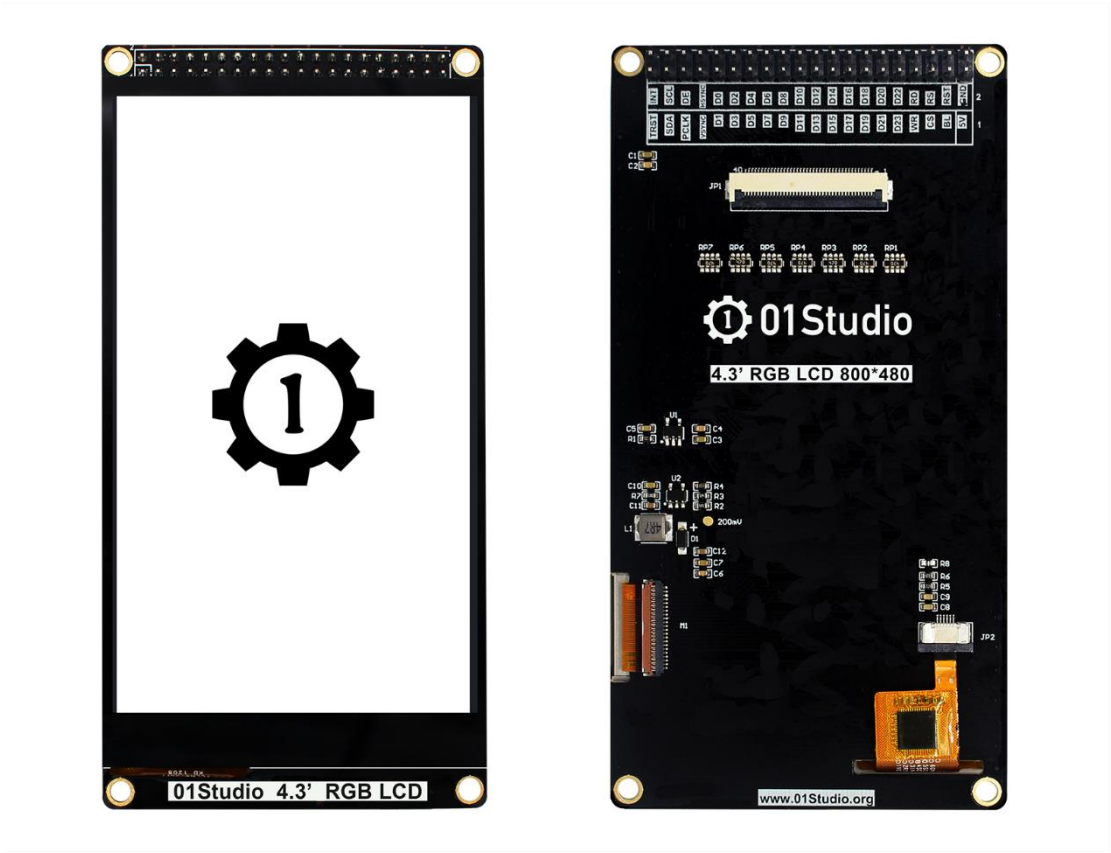


图 1-37 4.3 寸 RGB 显示屏（电阻触摸）

功能参数	
屏幕类型	RGB 屏
屏幕尺寸	4.3 寸
颜色参数	24 位彩色
驱动芯片	LG4572B + FT5436(触摸)
触摸方式	电容触摸（五点触控），I2C 接口
通讯方式	24 位并行(RGB888)
供电电压	背光：3.7~5V；GPIO: 3.3V
接口	40P-2.54mm 排针和 40P-0.5MM FPC 座
尺寸	12*6.2 cm

表 1-13 4.3 寸 RGB 显示屏功能参数

1.4.4.3 7 寸 RGB 显示屏（电容触摸）



图 1-38 7 寸 RGB 显示屏（电阻触摸）

功能参数	
屏幕类型	RGB 屏
屏幕尺寸	7 寸
颜色参数	24 位彩色
驱动芯片	群创 AT070TN92 + GT911(触摸)
触摸方式	电容触摸（五点触控），I2C 接口

通讯方式	24 位并行(RGB888)
供电电压	背光： 3.7~5V； GPIO: 3.3V
接口	40P-2.54mm 排针和 40P-0.5MM FPC 座
尺寸	18*12cm

表 1-14 7 寸 RGB 显示屏功能参数

1.4.4.4 USB 转串口 TTL

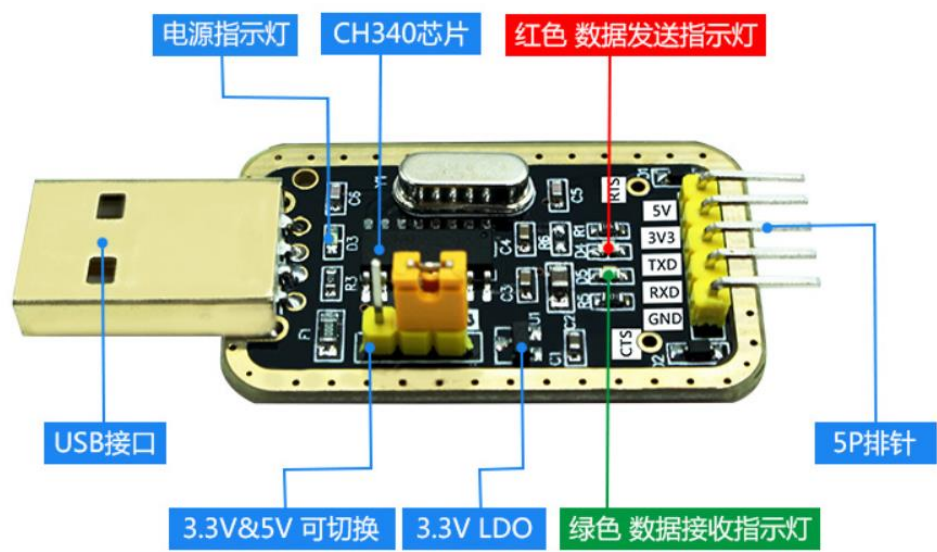


图 1-39 USB 转串口 TTL

功能参数	
驱动芯片	CH340
对外供电	5V 和 3.3V
支持电平	3.3V 和 5V 可切换
引脚接口	2.54MM 排针

表 1-15

第2章 Python 基础知识

由于 python 语言是学习 micropython 的基础，为了照顾没有 python 基础的朋友，我们一直在思考应该给大家提供怎么样的教程。Python 相关的书籍和学习资料相信大家能在网上找到不少，所以这里整理给大家的经典的 python3 快速学习资料，你甚至可以使用它来当作 python 字典查阅！

【原著：Louie Dinh，翻译：Geoff Liu】

```
# 用井字符开头的是单行注释

""" 多行字符串用三个引号
    包裹，也常被用来做多
    行注释
"""
```

2.1 原始数据类型和运算符

```
#####

## 1. 原始数据类型和运算符

#####

# 整数

3 # => 3

# 算术没有什么出乎意料的

1 + 1 # => 2

8 - 1 # => 7

10 * 2 # => 20
```

```
# 但是除法例外，会自动转换成浮点数

35 / 5 # => 7.0

5 / 3 # => 1.6666666666666667


# 整数除法的结果都是向下取整

5 // 3 # => 1

5.0 // 3.0 # => 1.0 # 浮点数也可以

-5 // 3 # => -2

-5.0 // 3.0 # => -2.0


# 浮点数的运算结果也是浮点数

3 * 2.0 # => 6.0


# 模除

7 % 3 # => 1


# x 的 y 次方

2**4 # => 16


# 用括号决定优先级

(1 + 3) * 2 # => 8


# 布尔值

True

False


# 用 not 取非

not True # => False

not False # => True
```

逻辑运算符，注意 and 和 or 都是小写

True and False # => False

False or True # => True

整数也可以当作布尔值

0 and 2 # => 0

-5 or 0 # => -5

0 == False # => True

2 == True # => False

1 == True # => True

用==判断相等

1 == 1 # => True

2 == 1 # => False

用!=判断不等

1 != 1 # => False

2 != 1 # => True

比较大小

1 < 10 # => True

1 > 10 # => False

2 <= 2 # => True

2 >= 2 # => True

大小比较可以连起来！

1 < 2 < 3 # => True

2 < 3 < 2 # => False

```

# 字符串用单引双引都可以
"这是个字符串"
'这也是个字符串'

# 用加号连接字符串
"Hello " + "world!" # => "Hello world!"

# 字符串可以被当作字符列表
"This is a string"[0] # => 'T'

# 用.format 来格式化字符串
"{ } can be { }".format("strings", "interpolated")

# 可以重复参数以节省时间
"{0} be nimble, {0} be quick, {0} jump over the {1}".format("Jack",
" candle stick")
# => "Jack be nimble, Jack be quick, Jack jump over the candle stick"

# 如果不想数参数, 可以用关键字
"{name} wants to eat {food}".format(name="Bob", food="lasagna")
# => "Bob wants to eat lasagna"

# 如果你的 Python3 程序也要在 Python2.5 以下环境运行, 也可以用老式的格式化语法
"%s can be %s the %s way" % ("strings", "interpolated", "old")

# None 是一个对象
None # => None

# 当与 None 进行比较时不要用 ==, 要用 is。is 是用来比较两个变量是否指向同一个对象。

```

```
"etc" is None # => False
```

```
None is None # => True
```

```
# None, 0, 空字符串, 空列表, 空字典都算是 False
```

```
# 所有其他值都是 True
```

```
bool(0) # => False
```

```
bool("") # => False
```

```
bool([]) # => False
```

```
bool({}) # => False
```

2.2 变量和集合

```
#####  
## 2. 变量和集合  
#####  
  
# print 是内置的打印函数  
print("I'm Python. Nice to meet you!")  
  
# 在给变量赋值前不用提前声明  
# 传统的变量命名是小写，用下划线分隔单词  
some_var = 5  
some_var # => 5  
  
# 访问未赋值的变量会抛出异常  
# 参考流程控制一段来学习异常处理  
some_unknown_var # 抛出 NameError  
  
# 用列表(list)储存序列  
li = []  
# 创建列表时也可以同时赋给元素  
other_li = [4, 5, 6]  
  
# 用 append 在列表最后追加元素  
li.append(1) # li 现在是[1]  
li.append(2) # li 现在是[1, 2]  
li.append(4) # li 现在是[1, 2, 4]  
li.append(3) # li 现在是[1, 2, 4, 3]  
# 用 pop 从列表尾部删除
```



```
li.pop()          # => 3 且 li 现在是[1, 2, 4]
```

```
# 把 3 再放回去
```

```
li.append(3)      # li 变回[1, 2, 4, 3]
```

```
# 列表存取跟数组一样
```

```
li[0]  # => 1
```

```
# 取出最后一个元素
```

```
li[-1]  # => 3
```

```
# 越界存取会造成 IndexError
```

```
li[4]  # 抛出 IndexError
```

```
# 列表有切割语法
```

```
li[1:3]  # => [2, 4]
```

```
# 取尾
```

```
li[2:]  # => [4, 3]
```

```
# 取头
```

```
li[:3]  # => [1, 2, 4]
```

```
# 隔一个取一个
```

```
li[::2]  # =>[1, 4]
```

```
# 倒排列表
```

```
li[::-1]  # => [3, 4, 2, 1]
```

```
# 可以用三个参数的任何组合来构建切割
```

```
# li[始:终:步伐]
```

```
# 用 del 删除任何一个元素
```

```
del li[2]  # li is now [1, 2, 3]
```

```
# 列表可以相加
```

```
# 注意: li 和 other_li 的值都不变
```

```
li + other_li    # => [1, 2, 3, 4, 5, 6]

# 用 extend 拼接列表
li.extend(other_li)    # li 现在是[1, 2, 3, 4, 5, 6]

# 用 in 测试列表是否包含值
1 in li    # => True

# 用 len 取列表长度
len(li)    # => 6

# 元组是不可改变的序列
tup = (1, 2, 3)
tup[0]    # => 1
tup[0] = 3 # 抛出 TypeError

# 列表允许的操作元组大都可以
len(tup)    # => 3
tup + (4, 5, 6)    # => (1, 2, 3, 4, 5, 6)
tup[:2]    # => (1, 2)
2 in tup    # => True

# 可以把元组合列表解包，赋值给变量
a, b, c = (1, 2, 3)    # 现在 a 是 1, b 是 2, c 是 3

# 元组周围的括号是可以省略的
d, e, f = 4, 5, 6

# 交换两个变量的值就这么简单
e, d = d, e    # 现在 d 是 5, e 是 4
```

```
# 用字典表达映射关系

empty_dict = {}

# 初始化的字典

filled_dict = {"one": 1, "two": 2, "three": 3}


# 用[]取值

filled_dict["one"] # => 1


# 用 keys 获得所有的键。

# 因为 keys 返回一个可迭代对象，所以在这里把结果包在 list 里。我们下面会详细介绍可迭代。

# 注意：字典键的顺序是不定的，你得到的结果可能和以下不同。

list(filled_dict.keys()) # => ["three", "two", "one"]


# 用 values 获得所有的值。跟 keys 一样，要用 list 包起来，顺序也可能不同。

list(filled_dict.values()) # => [3, 2, 1]


# 用 in 测试一个字典是否包含一个键

"one" in filled_dict # => True

1 in filled_dict # => False


# 访问不存在的键会导致 KeyError

filled_dict["four"] # KeyError


# 用 get 来避免 KeyError

filled_dict.get("one") # => 1
```

```

filled_dict.get("four")    # => None
# 当键不存在的时候 get 方法可以返回默认值
filled_dict.get("one", 4)  # => 1
filled_dict.get("four", 4) # => 4

# setdefault 方法只有当键不存在的时候插入新值
filled_dict.setdefault("five", 5) # filled_dict["five"]设为 5
filled_dict.setdefault("five", 6) # filled_dict["five"]还是 5

# 字典赋值
filled_dict.update({"four":4}) # => {"one": 1, "two": 2, "three": 3,
"four": 4}
filled_dict["four"] = 4 # 另一种赋值方法

# 用 del 删除
del filled_dict["one"] # 从 filled_dict 中把 one 删除

# 用 set 表达集合
empty_set = set()
# 初始化一个集合，语法跟字典相似。
some_set = {1, 1, 2, 2, 3, 4} # some_set 现在是{1, 2, 3, 4}

# 可以把集合赋值于变量
filled_set = some_set

# 为集合添加元素
filled_set.add(5) # filled_set 现在是{1, 2, 3, 4, 5}

# & 取交集

```

```
other_set = {3, 4, 5, 6}
filled_set & other_set    # => {3, 4, 5}

# | 取并集
filled_set | other_set    # => {1, 2, 3, 4, 5, 6}

# - 取补集
{1, 2, 3, 4} - {2, 3, 5}  # => {1, 4}

# in 测试集合是否包含元素
2 in filled_set    # => True
10 in filled_set   # => False
```

2.3 流程控制和迭代器

```
#####  
## 3. 流程控制和迭代器  
#####  
  
# 先随便定义一个变量  
some_var = 5  
  
# 这是个 if 语句。注意缩进在 Python 里是有意义的  
# 印出"some_var 比 10 小"  
if some_var > 10:  
    print("some_var 比 10 大")  
elif some_var < 10:    # elif 句是可选的  
    print("some_var 比 10 小")  
else:                  # else 也是可选的  
    print("some_var 就是 10")  
  
"""  
用 for 循环语句遍历列表  
打印:  
    dog is a mammal  
    cat is a mammal  
    mouse is a mammal  
"""  
for animal in ["dog", "cat", "mouse"]:  
    print("{} is a mammal".format(animal))  
  
"""
```

"range(number)"返回数字列表从 0 到给定的数字

打印:

0

1

2

3

"""

```
for i in range(4):
```

```
    print(i)
```

"""

while 循环直到条件不满足

打印:

0

1

2

3

"""

```
x = 0
```

```
while x < 4:
```

```
    print(x)
```

```
    x += 1 # x = x + 1 的简写
```

用 try/except 块处理异常状况

```
try:
```

```
    # 用 raise 抛出异常
```

```
    raise IndexError("This is an index error")
```

```
except IndexError as e:
```

```
    pass # pass 是无操作，但是应该在这里处理错误
```

```
except (TypeError, NameError):
```



```

    pass    # 可以同时处理不同类的错误
else:    # else 语句是可选的，必须在所有的 except 之后
    print("All good!")    # 只有当 try 运行完没有错误的时候这句才会运行

# Python 提供一个叫做可迭代(iterable)的基本抽象。一个可迭代对象是可以被当作序列
# 的对象。比如说上面 range 返回的对象就是可迭代的。

filled_dict = {"one": 1, "two": 2, "three": 3}
our_iterable = filled_dict.keys()
print(our_iterable) # => dict_keys(['one', 'two', 'three']), 是一个实现可
迭代接口的对象

# 可迭代对象可以遍历
for i in our_iterable:
    print(i)    # 打印 one, two, three

# 但是不可以随机访问
our_iterable[1] # 抛出 TypeError

# 可迭代对象知道怎么生成迭代器
our_iterator = iter(our_iterable)

# 迭代器是一个可以记住遍历的位置的对象
# 用 __next__ 可以取得下一个元素
our_iterator.__next__() # => "one"

# 再一次调取 __next__ 时会记得位置
our_iterator.__next__() # => "two"

```

```
our_iterator.__next__() # => "three"
```

```
# 当迭代器所有元素都取出后，会抛出 StopIteration
```

```
our_iterator.__next__() # 抛出 StopIteration
```

```
# 可以用 list 一次取出迭代器所有的元素
```

```
list(filled_dict.keys()) # => Returns ["one", "two", "three"]
```

2.4 函数

```
#####  
## 4. 函数  
#####  
  
# 用 def 定义新函数  
def add(x, y):  
    print("x is {} and y is {}".format(x, y))  
    return x + y    # 用 return 语句返回  
  
# 调用函数  
add(5, 6)    # => 印出"x is 5 and y is 6"并且返回 11  
  
# 也可以用关键字参数来调用函数  
add(y=6, x=5)    # 关键字参数可以用任何顺序  
  
# 我们可以定义一个可变参数函数  
def varargs(*args):  
    return args  
  
varargs(1, 2, 3)    # => (1, 2, 3)  
  
# 我们也可以定义一个关键字可变参数函数  
def keyword_args(**kwargs):  
    return kwargs
```

```

# 我们来看看结果是什么:

keyword_args(big="foot", loch="ness")  # => {"big": "foot", "loch":
"ness"}


# 这两种可变参数可以混着用

def all_the_args(*args, **kwargs):
    print(args)
    print(kwargs)
"""
all_the_args(1, 2, a=3, b=4) prints:
    (1, 2)
    {"a": 3, "b": 4}
"""


# 调用可变参数函数时可以做跟上面相反的, 用*展开序列, 用**展开字典。

args = (1, 2, 3, 4)
kwargs = {"a": 3, "b": 4}

all_the_args(*args)  # 相当于 foo(1, 2, 3, 4)
all_the_args(**kwargs)  # 相当于 foo(a=3, b=4)
all_the_args(*args, **kwargs)  # 相当于 foo(1, 2, 3, 4, a=3, b=4)


# 函数作用域

x = 5


def setX(num):
    # 局部作用域的 x 和全局域的 x 是不同的
    x = num  # => 43
    print (x)  # => 43

```

```

def setGlobalX(num):
    global x
    print (x) # => 5
    x = num # 现在全局域的 x 被赋值
    print (x) # => 6

setX(43)
setGlobalX(6)

# 函数在 Python 是一等公民
def create_adder(x):
    def adder(y):
        return x + y
    return adder

add_10 = create_adder(10)
add_10(3)    # => 13

# 也有匿名函数
(lambda x: x > 2)(3)    # => True

# 内置的高阶函数
map(add_10, [1, 2, 3])    # => [11, 12, 13]
filter(lambda x: x > 5, [3, 4, 5, 6, 7])    # => [6, 7]

# 用列表推导式可以简化映射和过滤。列表推导式的返回值是另一个列表。
[add_10(i) for i in [1, 2, 3]]    # => [11, 12, 13]
[x for x in [3, 4, 5, 6, 7] if x > 5]    # => [6, 7]

```

2.5 类

```
#####  
## 5. 类  
#####  
  
# 定义一个继承 object 的类  
class Human(object):  
  
    # 类属性，被所有此类的实例共用。  
    species = "H. sapiens"  
  
    # 构造方法，当实例被初始化时被调用。注意名字前后的双下划线，这是表明这个属  
    # 性或方法对 Python 有特殊意义，但是允许用户自行定义。你自己取名时不应该用  
    这  
    # 种格式。  
    def __init__(self, name):  
        # Assign the argument to the instance's name attribute  
        self.name = name  
  
    # 实例方法，第一个参数总是 self，就是这个实例对象  
    def say(self, msg):  
        return "{name}: {message}".format(name=self.name, message=msg)  
  
    # 类方法，被所有此类的实例共用。第一个参数是这个类对象。  
    @classmethod  
    def get_species(cls):  
        return cls.species
```

```

# 静态方法。调用时没有实例或类的绑定。

@staticmethod
def grunt():
    return "*grunt*"

# 构造一个实例
i = Human(name="Ian")
print(i.say("hi"))    # 印出 "Ian: hi"

j = Human("Joel")
print(j.say("hello")) # 印出 "Joel: hello"

# 调用一个类方法
i.get_species()    # => "H. sapiens"

# 改一个共用的类属性
Human.species = "H. neanderthalensis"
i.get_species()    # => "H. neanderthalensis"
j.get_species()    # => "H. neanderthalensis"

# 调用静态方法
Human.grunt()    # => "*grunt*"

```

2.6 模块

```
#####  
## 6. 模块  
#####  
  
# 用 import 导入模块  
import math  
print(math.sqrt(16)) # => 4.0  
  
# 也可以从模块中导入个别值  
from math import ceil, floor  
print(ceil(3.7)) # => 4.0  
print(floor(3.7)) # => 3.0  
  
# 可以导入一个模块中所有值  
# 警告：不建议这么做  
from math import *  
  
# 如此缩写模块名字  
import math as m  
math.sqrt(16) == m.sqrt(16) # => True  
  
# Python 模块其实就是普通的 Python 文件。你可以自己写，然后导入，  
# 模块的名字就是文件的名字。  
  
# 你可以这样列出一个模块里所有的值  
import math  
dir(math)
```


2.7 高级用法

```
#####  
## 7. 高级用法  
#####  
  
# 用生成器(generators)方便地写惰性运算  
def double_numbers(iterable):  
    for i in iterable:  
        yield i + i  
  
# 生成器只有在需要时才计算下一个值。它们每一次循环只生成一个值，而不是把所有的  
# 值全部算好。  
#  
# range 的返回值也是一个生成器，不然一个 1 到 900000000 的列表会花很多时间和内存。  
#  
# 如果你想用一个 Python 的关键字当作变量名，可以加一个下划线来区分。  
range_ = range(1, 900000000)  
# 当找到一个 >=30 的结果就会停  
# 这意味着 `double_numbers` 不会生成大于 30 的数。  
for i in double_numbers(range_):  
    print(i)  
    if i >= 30:  
        break  
  
# 装饰器(decorators)  
# 这个例子中，beg 装饰 say
```

```

# beg 会先调用 say。如果返回的 say_please 为真，beg 会改变返回的字符串。
from functools import wraps

def beg(target_function):
    @wraps(target_function)
    def wrapper(*args, **kwargs):
        msg, say_please = target_function(*args, **kwargs)
        if say_please:
            return "{} {}".format(msg, "Please! I am poor :(")
        return msg

    return wrapper

@beg
def say(say_please=False):
    msg = "Can you buy me a beer?"
    return msg, say_please

print(say()) # Can you buy me a beer?
print(say(say_please=True)) # Can you buy me a beer? Please! I am
                             poor :(

```

第3章 开发环境快速建立

我们以往做嵌入式开发的时候，一般都会用到 IDE 开发软件，比如 KEIL、IAR 等，有些还需要专门的烧录器，这为嵌入式开发增加了门槛。而使用 MicroPython 的话，用户只需要 1 根 USB 数据线（达芬奇使用 type-c 数据线）就可以开发了。

达芬奇板载 16M 字节的 Flash，前 3MB 被分配用于存放固件，还剩 13MB 用于作为开发板的文件系统，所有代码或其它相关文件存放在这个 13MB 空间里面，这使得我们在不同的 PC 平台，Windows/Mac/Linux 上可以直接打开文件系统里面的文件进行编程开发，你甚至可以用自带的文档编辑器来编写你的程序。我们将会对 Windows、MAC、Linux 不同开发环境的搭建进行讲解。

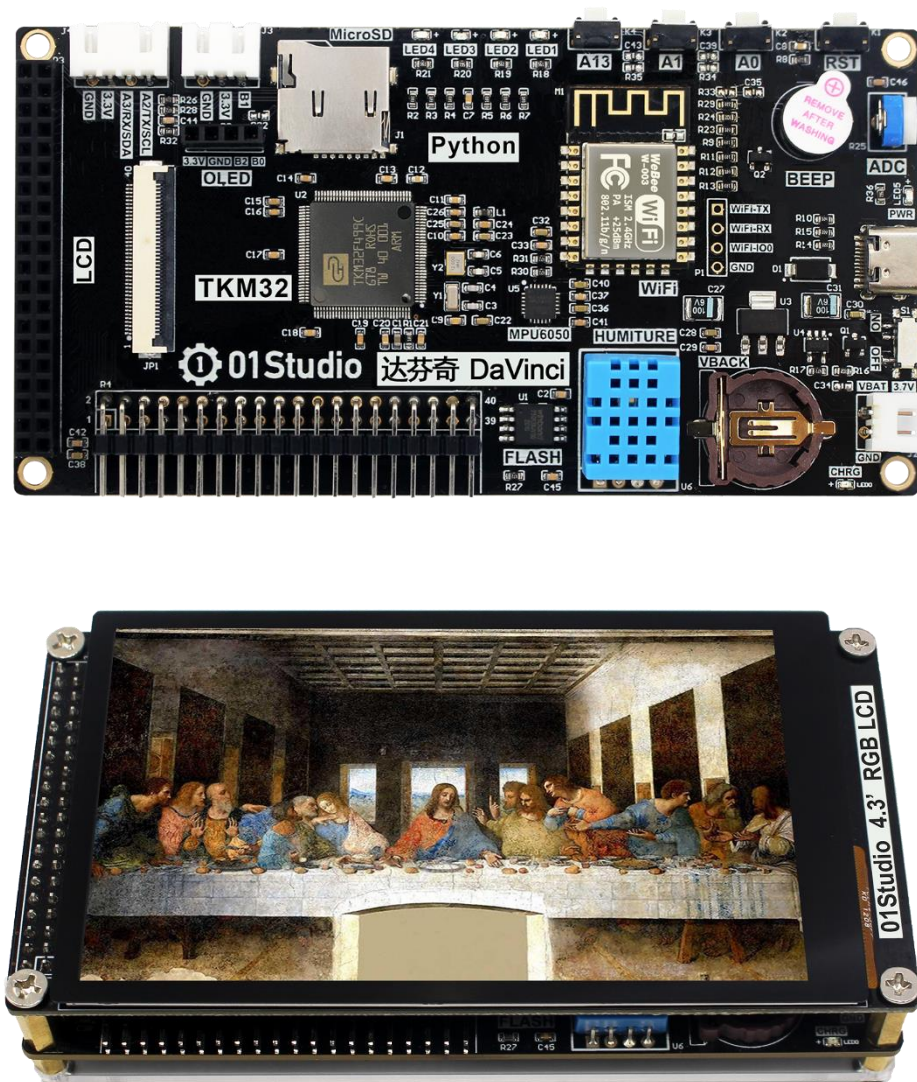


图 3-1 达芬奇开发套件

3.1 基于 Windows

Windows 是大部分开发人员的主流平台，毕竟微软凭借其 windows 操作系统在 PC 时代统治了几十年，大部分软件厂商还是愿意针对 windows 来开发桌面软件，但这也跟乔布斯英年早逝有关。扯远了，所以为了后面省事，我们还是推荐使用 Windows 作为首选的开发环境，推荐使用 Windows 10，微软官方已经宣布停止对 win 7 更新和维护了，win10 功能强大，有些驱动还能自动联网安装，免去人工安装的麻烦。

3.1.1 安装开发软件 Thonny

开发软件是指我们用来写代码的工具，Python 拥有众多的编程器，如果你之前已经熟练掌握 python 或已经使用 python 开发，那么可以直接使用你原来习惯的开发软件来编程。如果你是初学者或者喜欢简单而快速应用，我们使用官方推荐的 Thonny Python IDE。

Thonny Python IDE 是一款开源软件，以极简方式设计，对 MicroPython 的兼容性非常友善。而且支持 Windows、Mac OS、Linux、树莓派。由于开源，所以软件迭代速度非常快，功能日趋成熟。具体安装方法如下：

在 <https://thonny.org/> 下载，选择自己的开发平台进行下载安装即可(这里选择 Windows！)：

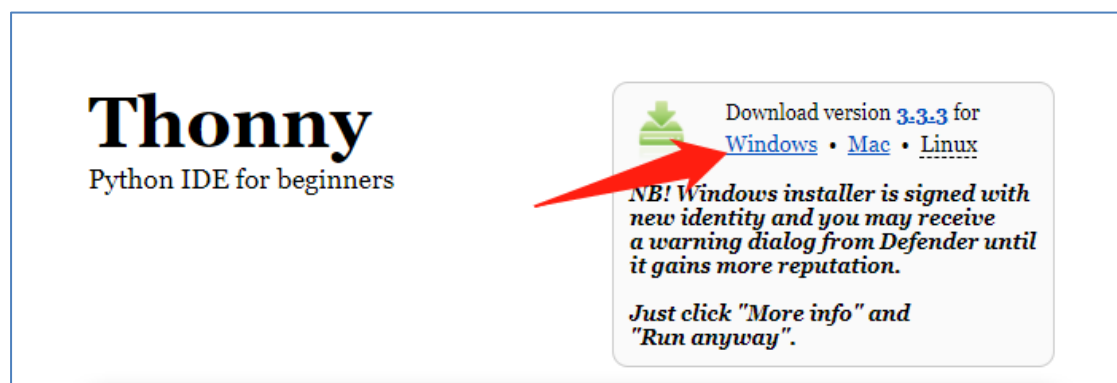


图 3-2 Thonny Python IDE 下载

下载完成后直接双击打开安装即可，安装完成后可以在桌面看到相关图标，打开软件如下：



图 3-3 安装完成

至此，Thonny 安装完成。关于如何在 Thonny 上使用达芬奇，我们在接下来的章节将详细讲解。

3.1.2 开发套件使用

3.1.2.1 驱动安装

我们将达芬奇开发板通过 type-C USB 数据线连接到电脑，将 type-c 座子旁边的拨码开关打到‘ON’端，给开发板上电，蓝色灯亮。



图 3-4 通过 Type-C USB 线连接到电脑

连接电脑后，系统（win10 系统）会自动帮我们安装一个 COM 串口驱动。鼠标右键点击 “我的电脑” —属性—设备管理器：可以看到系统找到了新的 COM 设备（只要找到 **COM** 即可，不确定的可以拔插开发板，找到出现和消失的 **COM** 就是开发板对应的的串口 **COM** 号）

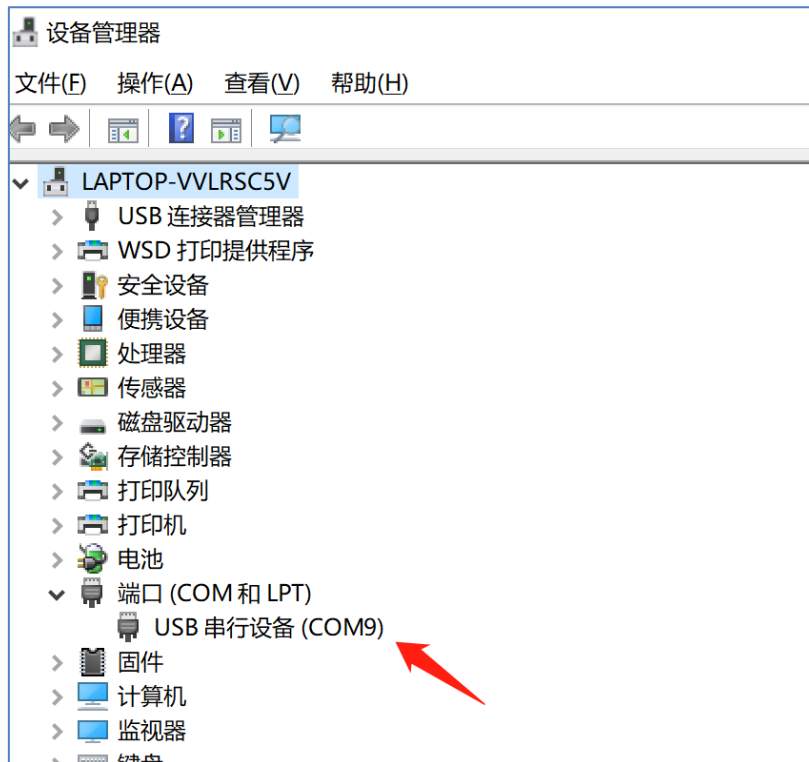


图 3-5 串口 COM 驱动安装成功

注意事项:

部分用户的 Win7 系统由于使用的是 Ghost 版系统，Ghost 系统精简了很多系统文件，有可能导致驱动无法安装。我们不建议开发者安装 Ghost 的系统。这时候建议用户采取以下措施。

- (1) 使用 360 的系统重装功能，实测重装后驱动可以自动安装；
- (2) 升级成 Win10 系统。

我们更推荐方法 (2)，因为微软已经宣布停止对 Win7 进行更新，另外就是我们的教程都会基于 Win10 系统。

3.1.2.2 第一个实验

前面我们已经安装好了 Thonny IDE，接下来我们使用最简单的方式来做一个人点亮 LED4 蓝色灯实验，大家暂时先不用理解代码意思，后面章节会有解释。这里主要是为了让大家了解一下 MicroPython 编程软件 Thonny 的使用方法和原理。具体如下：

我们打开 Thonny，将达芬奇连接到电脑。点击右下角：



图 3-6 选择要连接的设备

在弹出的列表选择：**Configure interpreter**



图 3-7

选择“MicroPython 一般”和达芬奇对应的串口号，点击确认。

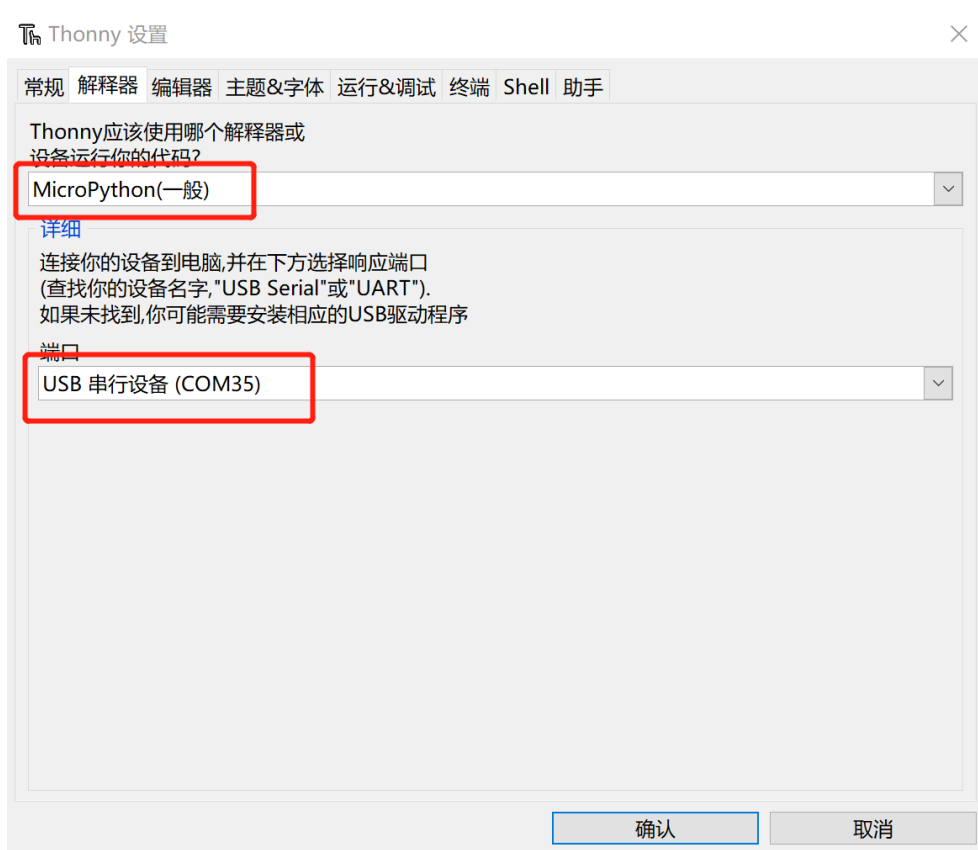


图 3-8

连接成功后可以在 **shell**（串口终端）看到固件的相关信息：

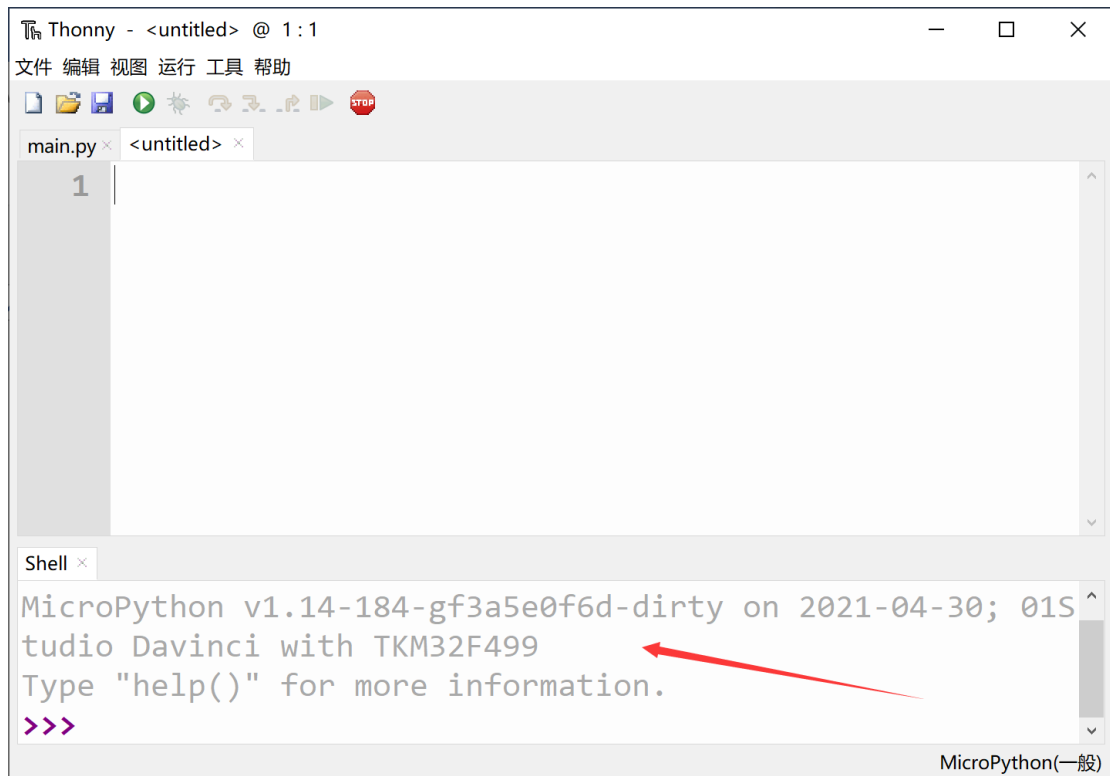


图 3-9 连接成功

接下来我们简单的测试一下代码，这里测试功能为主，更详细的代码讲解将在后面实验章节展开。在 Thonny 中点击打开资料包里面的例程文件：

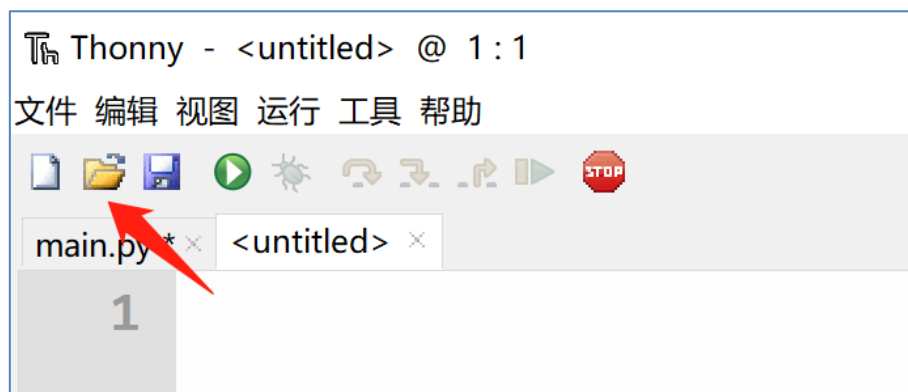


图 3-10

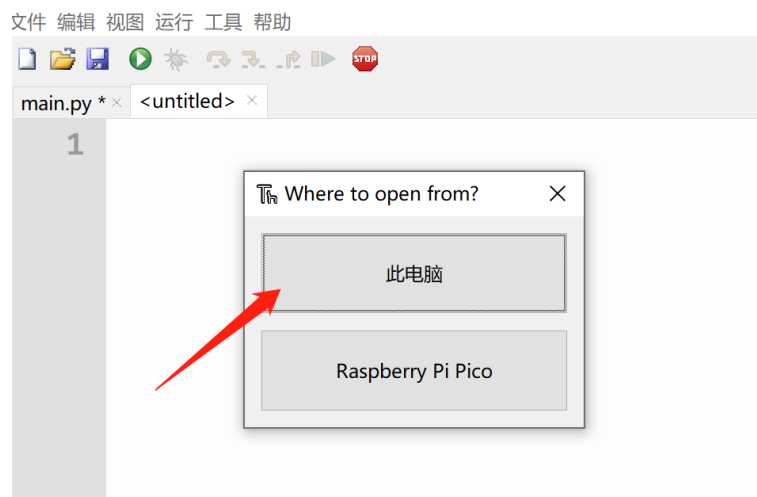


图 3-11 打开本地文件选择此电脑

选择资料包路径 零一科技（01Studio）MicroPython 开发套件（基于达芬奇 TKM32F499 平台）配套资料\02-示例程序\1.基础实验\1.点亮第一个 LED 下的 py 文件。

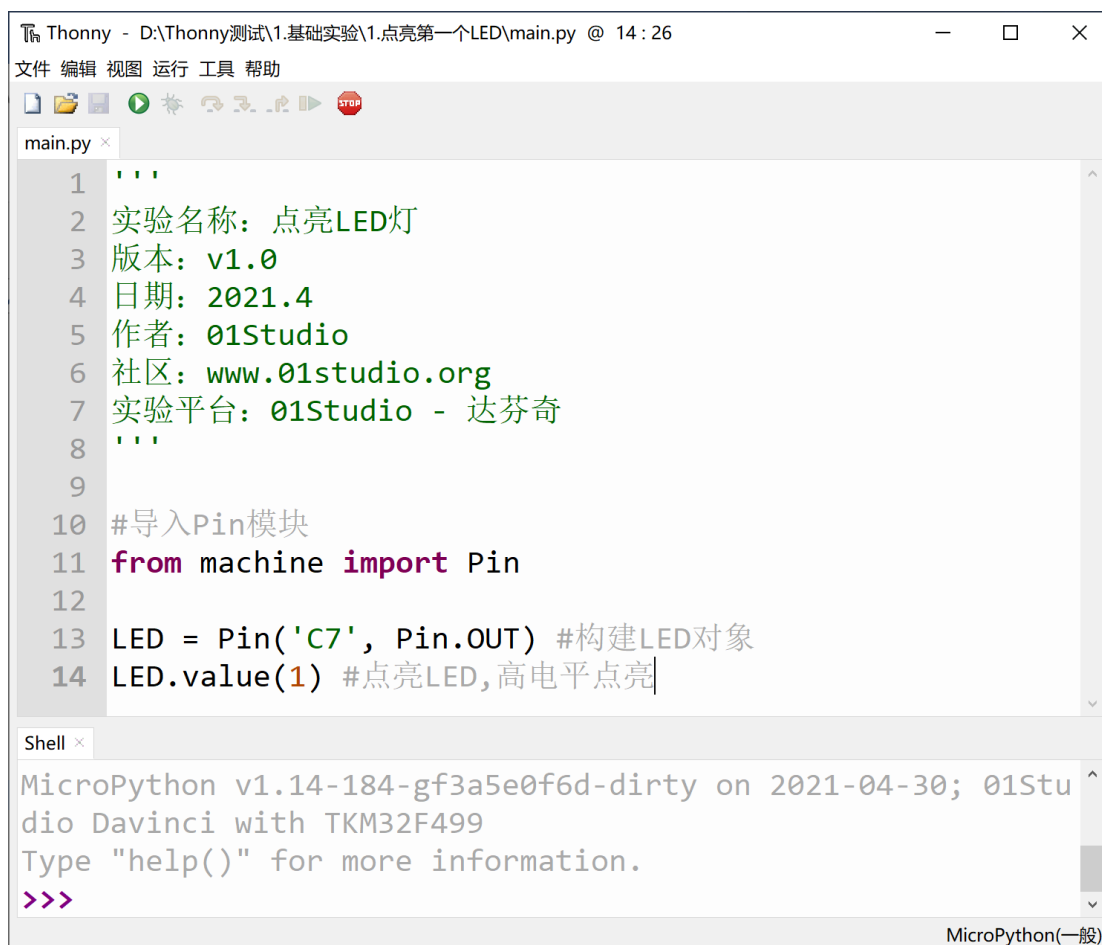


图 3-12 打开本地的 py 文件

然后点击绿色按钮“运行”：

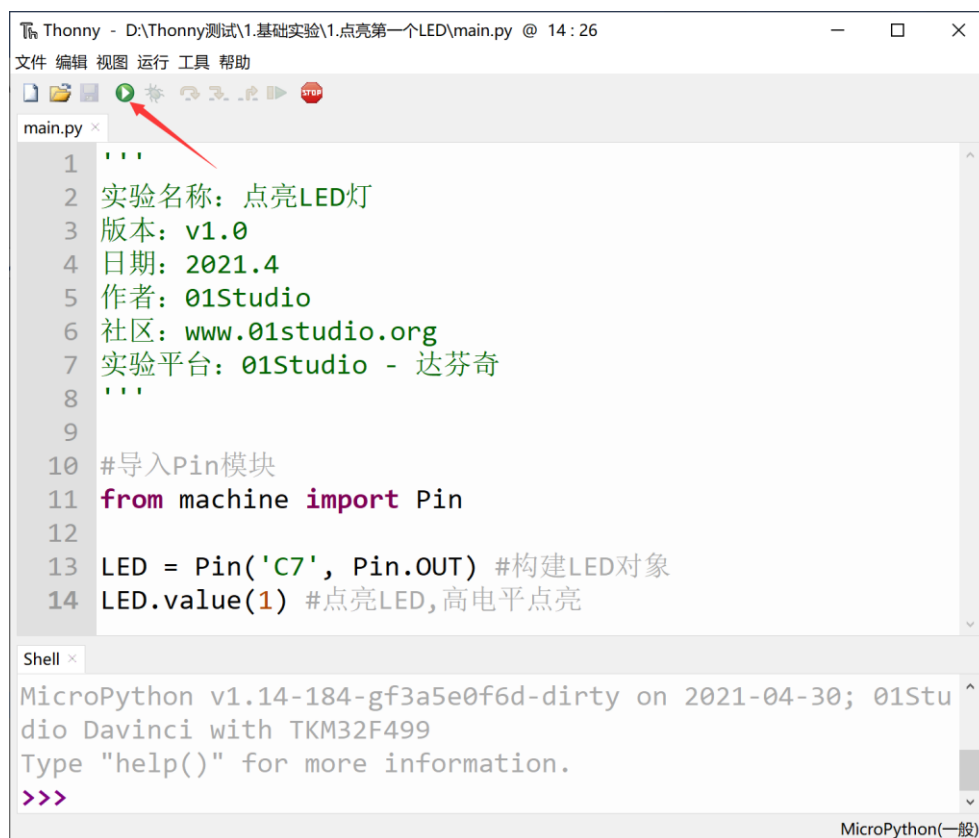


图 3-13

可以看到开发板的 LED 绿灯被点亮：



图 3-14 点亮 LED

3.1.2.3 REPL 串口交互调试

达芬奇的 MicroPython 固件集成了交互解释器 REPL 【读取(Read)-运算(Eval)-输出(Print)-循环(Loop)】，开发者可以直接通过串口终端来调试达芬奇开发板。

在 Thonny 中的 Shell 其实就是达芬奇的 REPL，通过串口交互的，Thonny 连接达芬奇后，我们在 Shell 里面输入 `print("Hello 01Studio!")`，按回车，可以看到打印出 Hello 01Studio 字符：

A screenshot of the Thonny Shell window. The window title is 'Shell'. The text inside shows the prompt 'Type "help()" for more information.' followed by the command '>>> print("Hello 01Studio!")' and its output 'Hello 01Studio!'. A red arrow points to the output. The bottom right corner says 'MicroPython (Raspberry Pi Pico)'.

图 3-15

再输入 `1+1`，按回车：

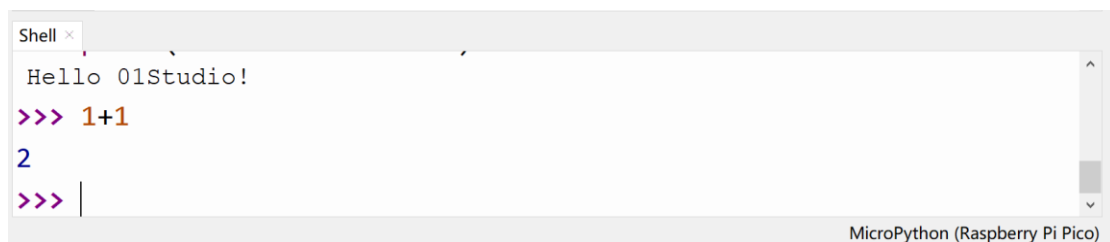
A screenshot of the Thonny Shell window. The window title is 'Shell'. The text inside shows the prompt 'Hello 01Studio!' followed by the command '>>> 1+1' and its output '2'. The bottom right corner says 'MicroPython (Raspberry Pi Pico)'.

图 3-16

接下来我们将上一节的三行代码逐行输入和逐行按回车，可以看到 LED 灯也被点亮：

```
from machine import Pin
LED = Pin('C7', Pin.OUT)
LED.value(1)
```

```
Shell x
>>> from machine import Pin
>>> LED = Pin('C7', Pin.OUT)
>>> LED.value(1)
>>>
```

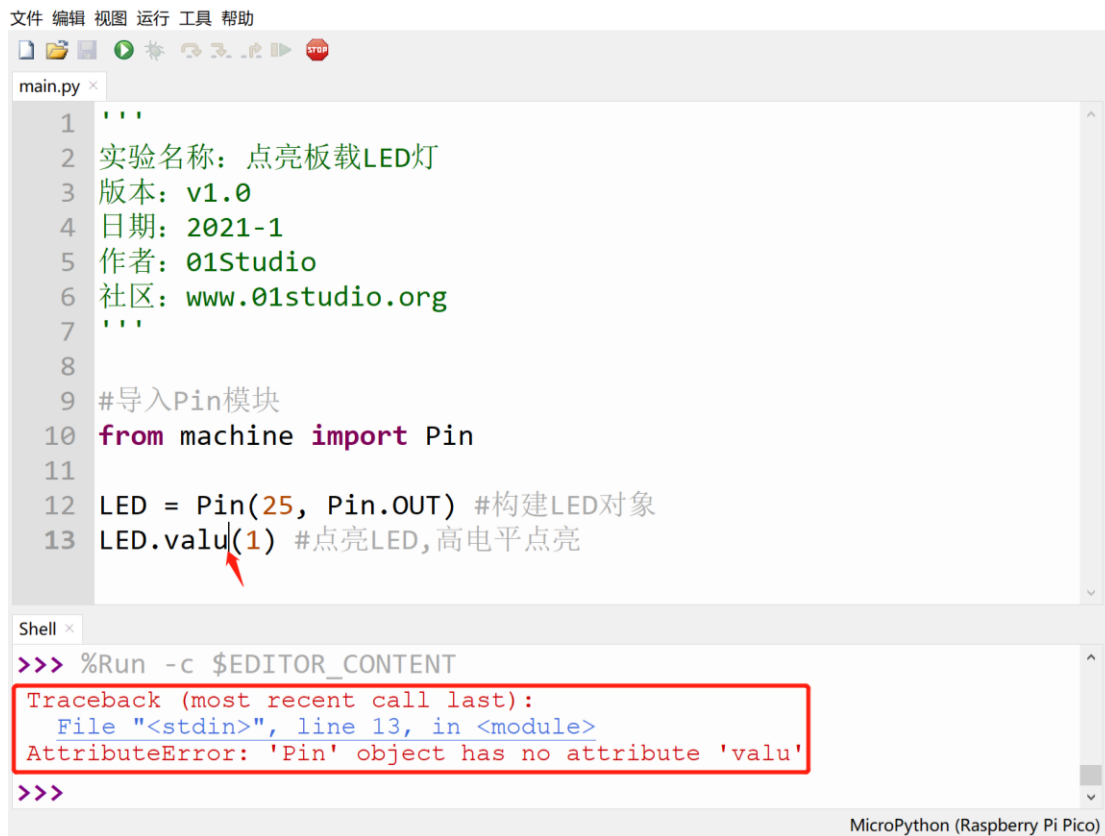
MicroPython(一般)

图 3-17 逐行输入



图 3-18 LED 被点亮

REPL 还有一个强大的功能就是打印错误的代码来调试程序，我们还是用上一节点亮 LED 的代码，将 **value** 改成 **valu**，然后运行代码，可以看到报错。指示出了错误的行数和错误原因，大大提高了编程调试的效率：



The screenshot shows a MicroPython IDE window with a menu bar (文件, 编辑, 视图, 运行, 工具, 帮助) and a toolbar. The main editor displays a file named `main.py` with the following code:

```
1 '''
2 实验名称: 点亮板载LED灯
3 版本: v1.0
4 日期: 2021-1
5 作者: 01Studio
6 社区: www.01studio.org
7 '''
8
9 #导入Pin模块
10 from machine import Pin
11
12 LED = Pin(25, Pin.OUT) #构建LED对象
13 LED.valu(1) #点亮LED,高电平点亮
```

A red arrow points to the `valu` attribute in line 13. Below the editor is a Shell window showing the command `>>> %Run -c $EDITOR_CONTENT` and the resulting error message:

```
Traceback (most recent call last):
  File "<stdin>", line 13, in <module>
AttributeError: 'Pin' object has no attribute 'valu'
>>>
```

The status bar at the bottom right indicates "MicroPython (Raspberry Pi Pico)".

图 3-19 错误打印

REPL 终端常用按键:

Ctrl + C: 打断正在运行的程序 (特别是含 `While True:` 的代码);

Ctrl + D: 软件复位开发板。

3.1.2.4 文件系统

达芬奇主控外挂了 16MBytes 的 Flash 可以用于存放代码、图片等文件。传输的方式是通过 Thonny IDE。IDE 连接达芬奇后，我们打开本地文件，然后点击 IDE 的文件 – 另存为，选择 micropython 设备：

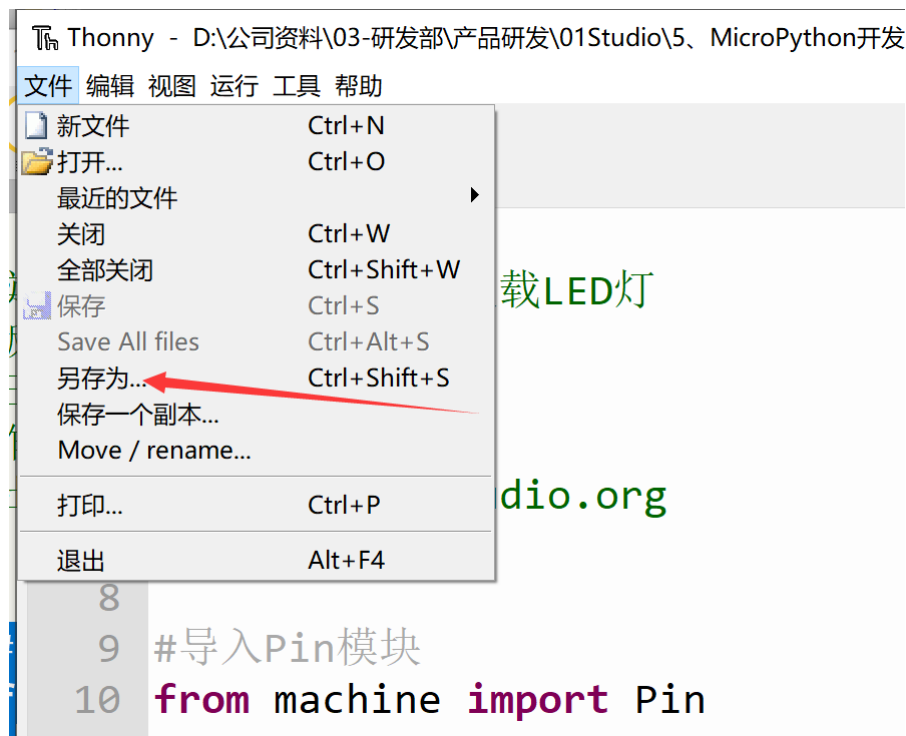


图 3-20

选择“MicroPython 设备”：

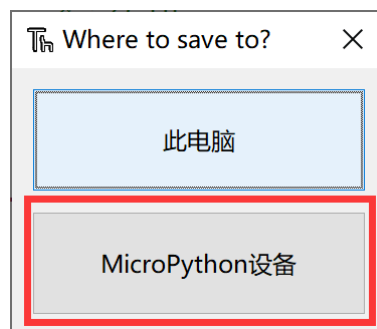


图 3-21

然后输入文件名，务必带 **.py** 结尾，然后点击确认：

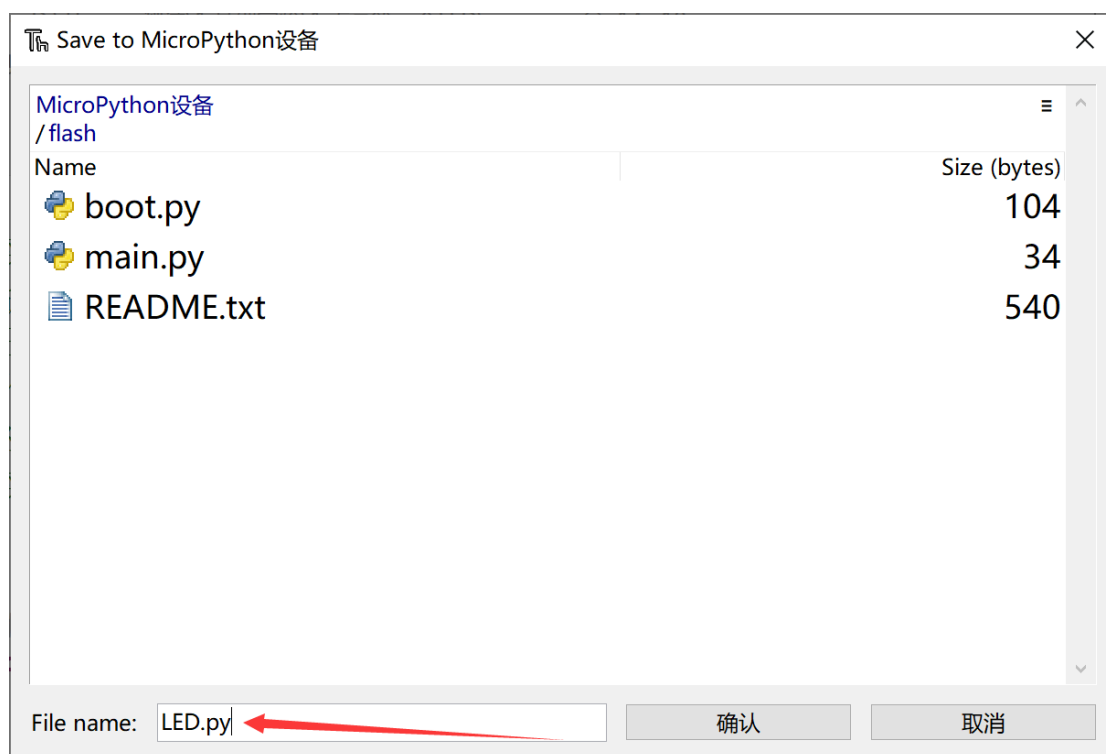


图 3-22

接下来我们点击打开，选择“MicroPython 设备”：



图 3-23

可以看到里面多了个 LED.py 文件，这个文件以后可以直接打开在 Thonny IDE 里面编辑并保存，非常方便：

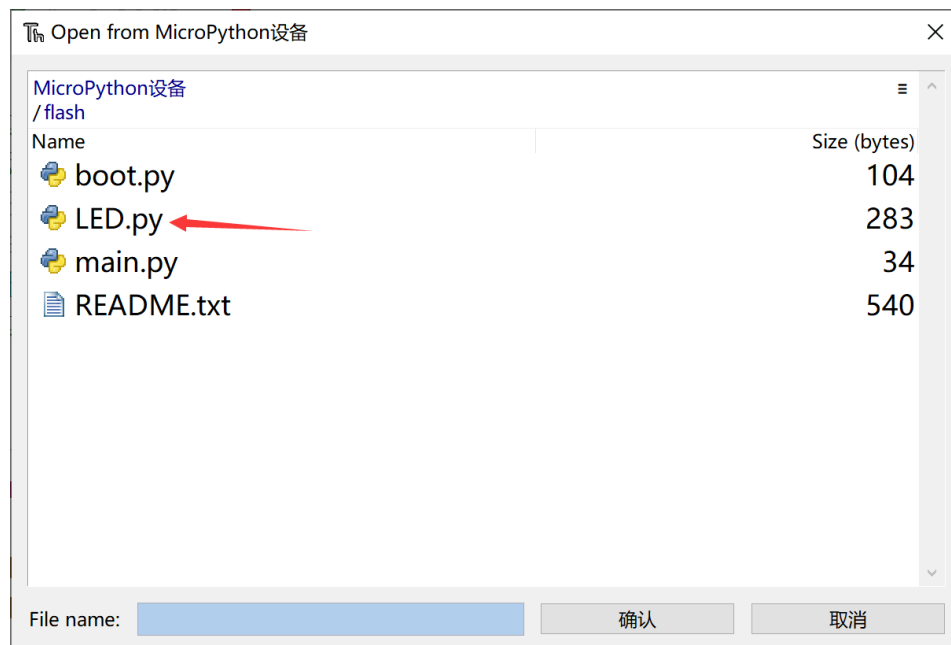


图 3-24

如需删除，可以点击右键，然后点击“删除”的方式删除改文件：

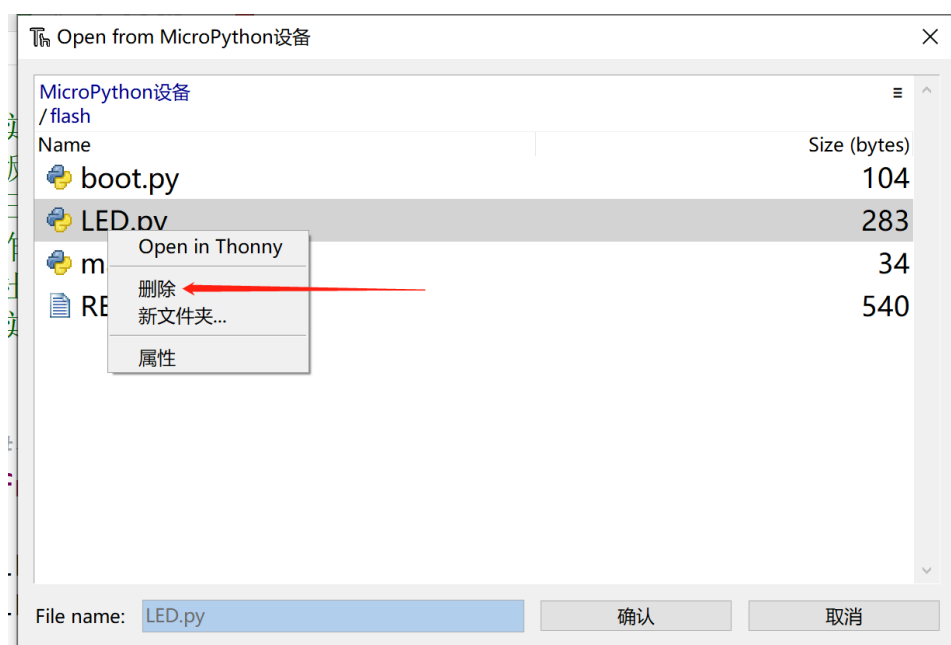


图 3-25

上述方法只能保存.py 格式文件，而且不是很直观接下来我们讲解一下如何传输 bmp、JPG 等其它格式的文件。连接达芬奇开发板，点击 视图--文件：

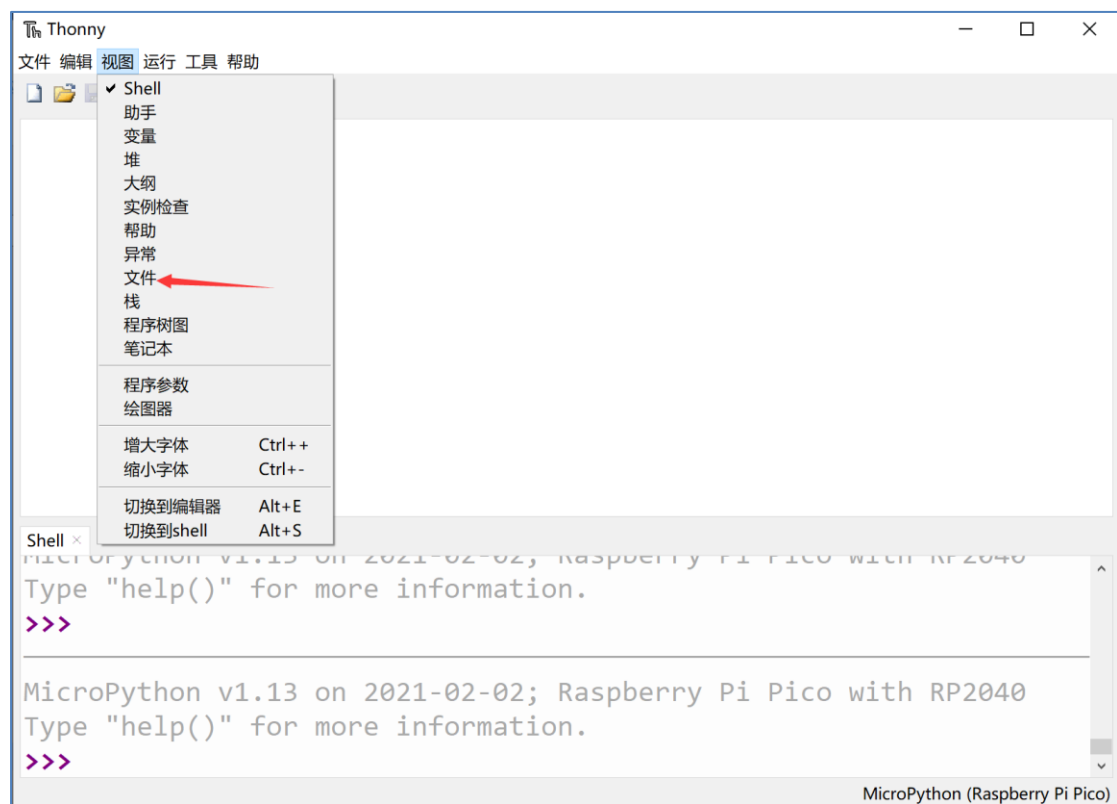


图 3-26

可以看到左边出现本地和开发板的实时文件浏览窗口：

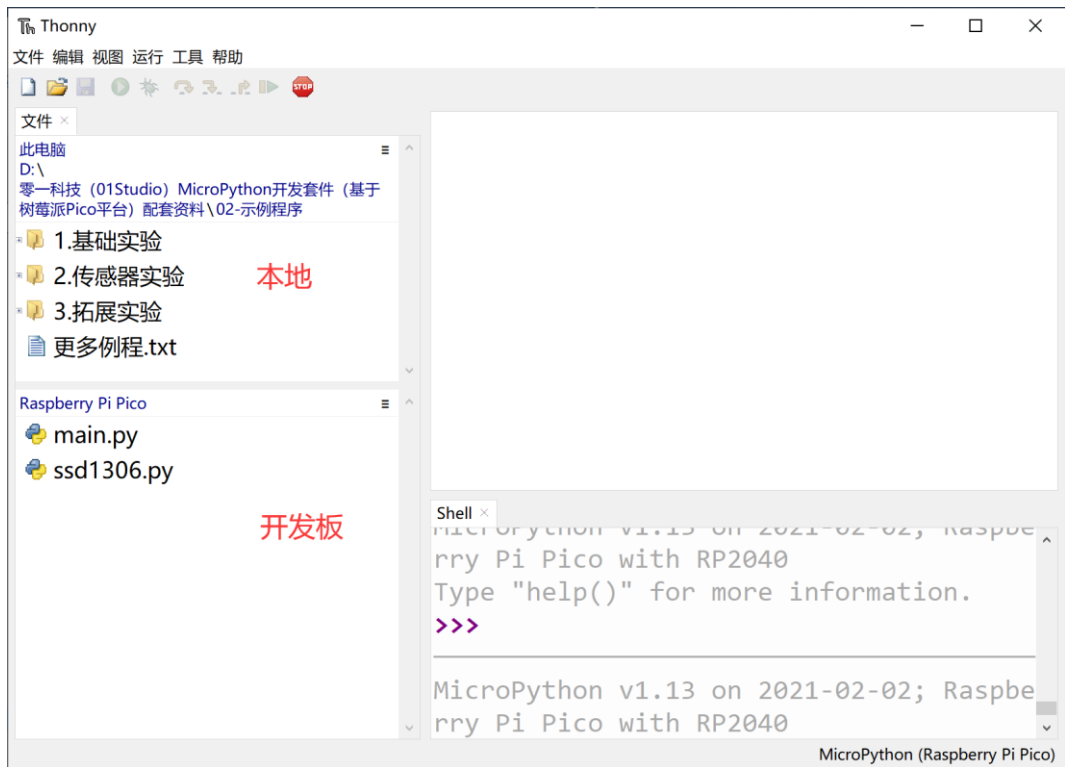


图 3-27

这里测试将 txt 文件发送到开发板，点击文件右键，然后点上载到：

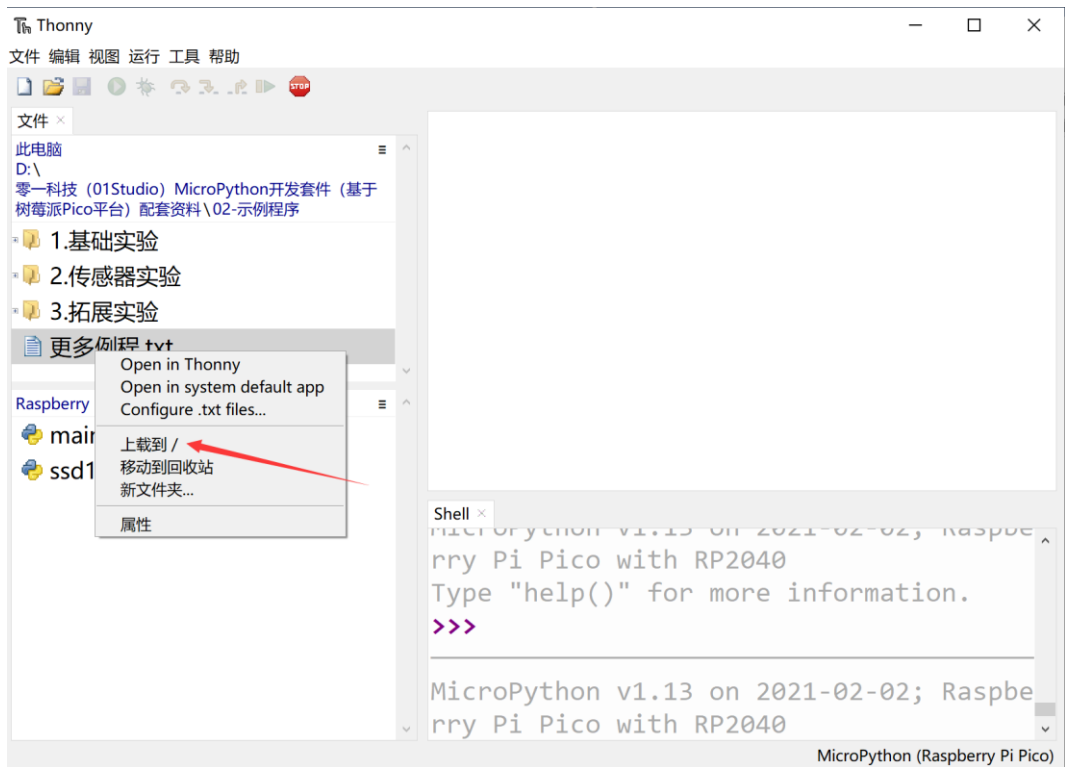


图 3-28

可以看到文件成功发送到开发板：

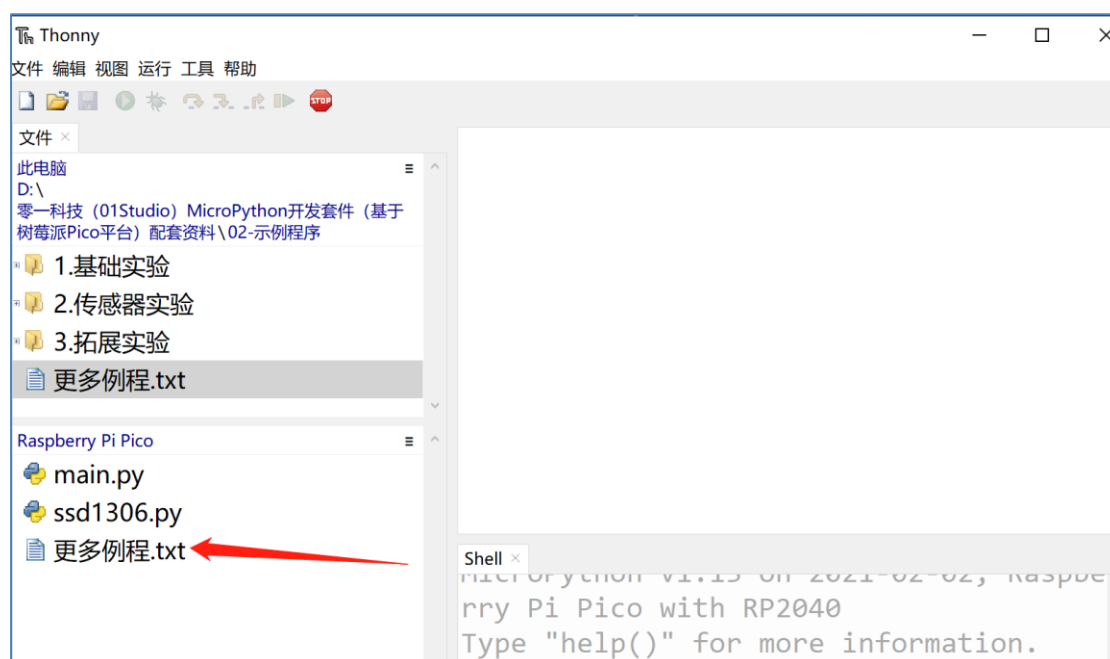


图 3-29

同样可以在这里点击文件右键，执行删除等相关功能。使用起来更方便。

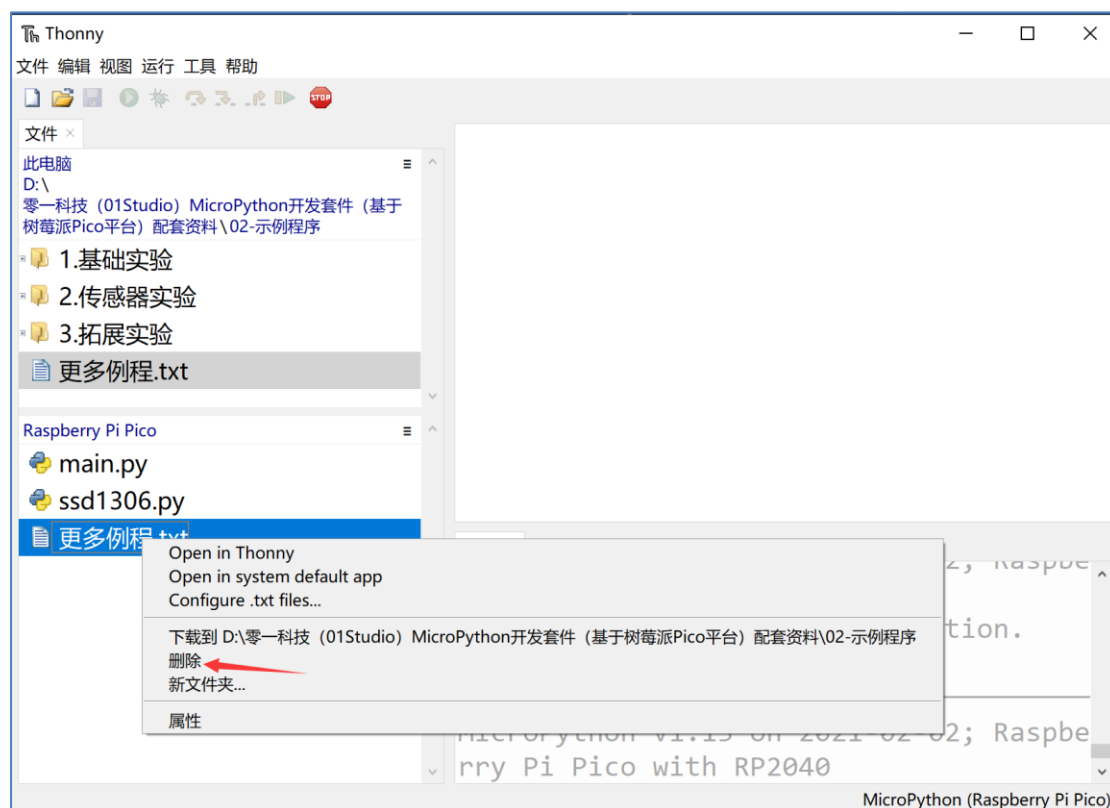


图 3-30

3.1.2.5 恢复出厂设置

当达芬奇开发板文件系统出现异常时，可以通过以下方式修改启动顺序或者恢复出厂设置，方法如下：

按着达芬奇开发板上的 A0 按键不放，再按一下 RST 键，LED 灯会持续交替闪烁，当闪烁达到你想要的模式时候，松开 A0 键，LED 灯会快速闪烁，板子会接着重新启动。

模式 1: 只有绿灯亮，正常模式：先启动``boot.py``然后``main.py``。

模式 2: 只有橙色灯亮，安全模式：启动时候不运行任何脚本。(仅 1 次有效)

模式 3: 绿灯和橙灯同时亮，文件系统重置：文件系统恢复出厂状态，然后以安全模式启动。

如果你的文件系统损坏了，可以通过**模式 3** 启动修复它。



图 3-31 模式 3 启动，恢复出厂设置

重新启动后红色 LED 灯会亮一会儿，待其熄灭后再使用。这时候打开 Thonny 查看文件系统，可见是剩下 3 个原始文件：

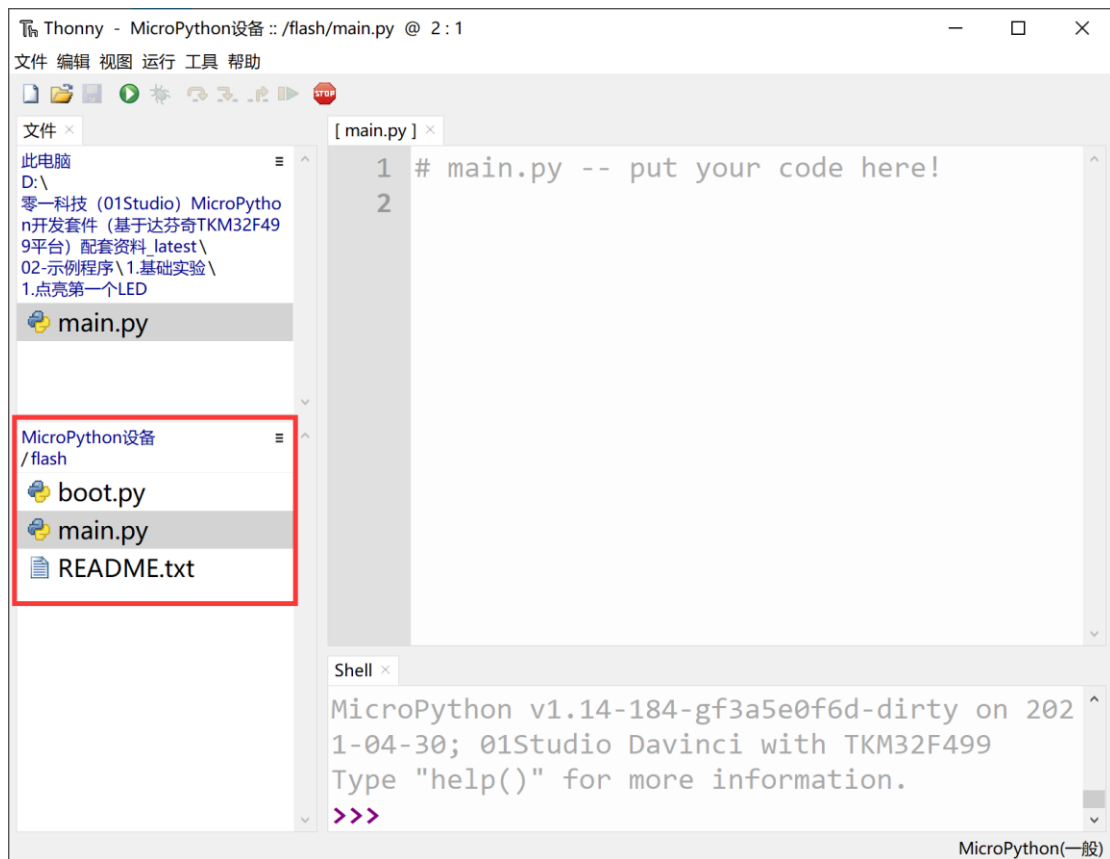


图 3-32 恢复出厂后的文件系统

下面是对各个初始文件的简介：

1. **boot.py**：系统启动文件；
2. **main.py**：主函数代码文件；
3. **README.txt**：说明文件；

系统上电后默认先执行 **boot.py**，再执行 **main.py**。我们一般情况下在 **main.py** 文件下编程即可。

3.1.2.6 固件更新

当达芬奇上的固件意外丢失或者我们希望升级到较新版本固件时候，就需重新烧录开发板的固件。达芬奇烧写固件的方法很简单，通过拖拽文件方式更新。具体方法如下：

按住达芬奇开发板上的 A13 键不放，然后再按一下 RST 复位键，



图 3-33 让开发板进入烧录固件模式

然后可以看到我的电脑弹出一个 TK499 的 U 盘盘符，此时松开 A13 按键。

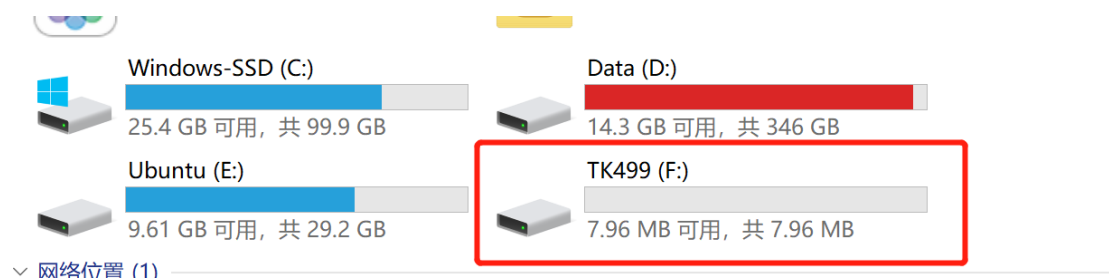


图 3-34

然后将零一科技（01Studio）达芬奇 **MicroPython** 开发套件配套资料 [latest\03-相关固件](#) 目录下的固件文件拷贝到这个 U 盘即可。

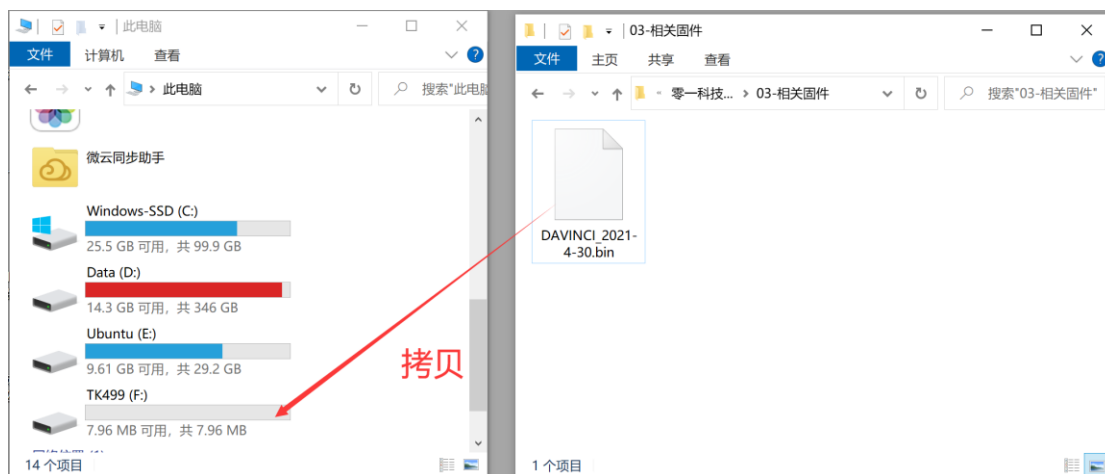


图 3-35 拷贝固件升级

拷贝完成后 TK499 U 盘消失，开发板自动重新启动，在设备管理器看到弹出达芬奇的“COM”串口号说明启动成功。更新固件可能会导致文件系统里面的文件丢失，请务必备份好自己的例程文件再烧录固件！

注意：如过烧录后开发板无法自动启动，请将先开发板断电，然后按着按键 **A13** 同时给开发板上电，重复以上烧写步骤即可！

3.1.2.7 SD 卡使用

达芬奇开发板 SD 卡使用非常方便，只需要将 SD 卡插进卡槽（请断电后拔插），然后通过 TYPE-C 连接到电脑，开发板上电后自动以 SD 卡作为开发板的主要文件系统。优先执行 SD 卡里面的 main.py。

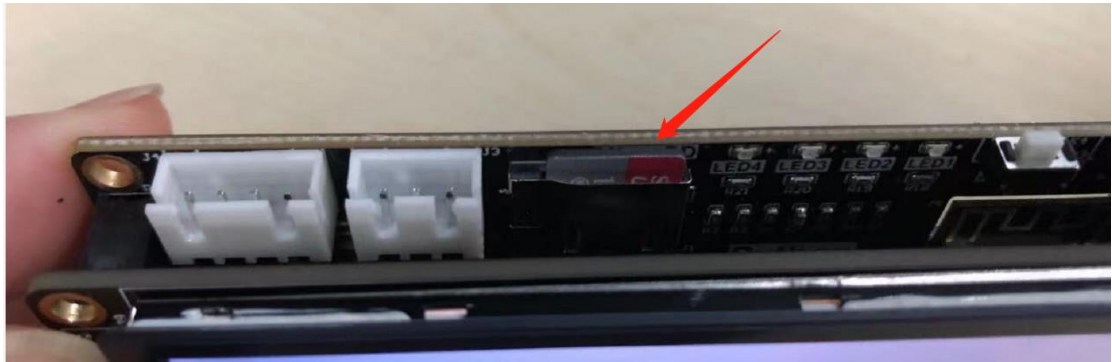


图 3-36 插入 SD 卡

从 Thonny 可以看到文件系统切换到 SD 卡：

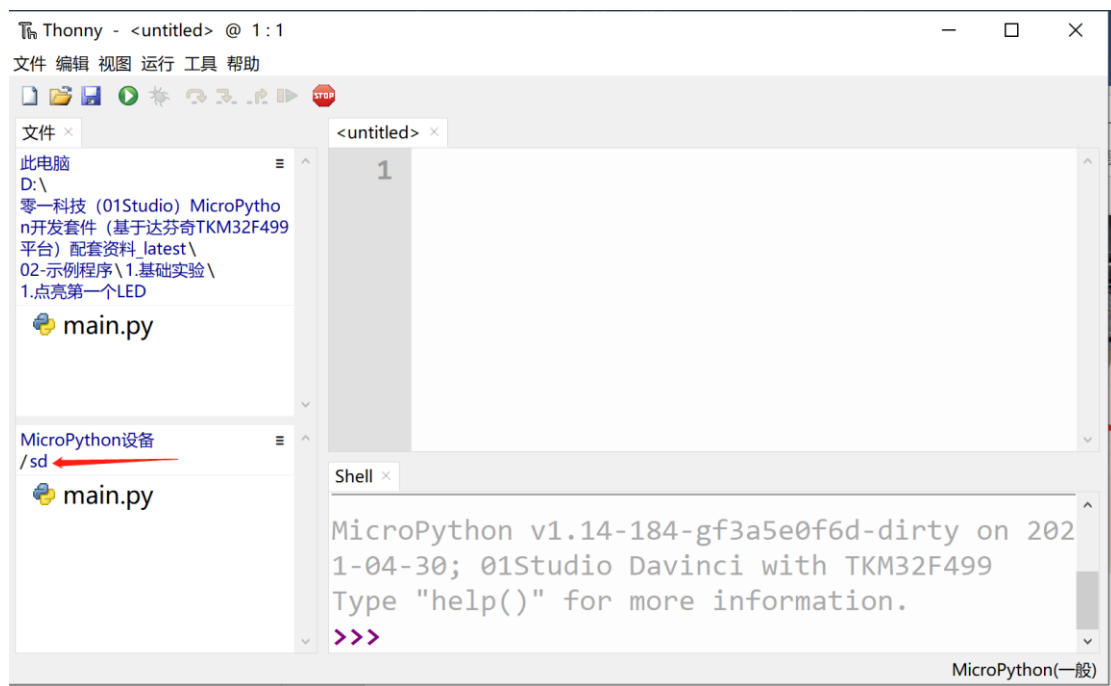


图 3-37 文件系统自动换成 SD 卡

以下是查看 SD 卡容量方法：

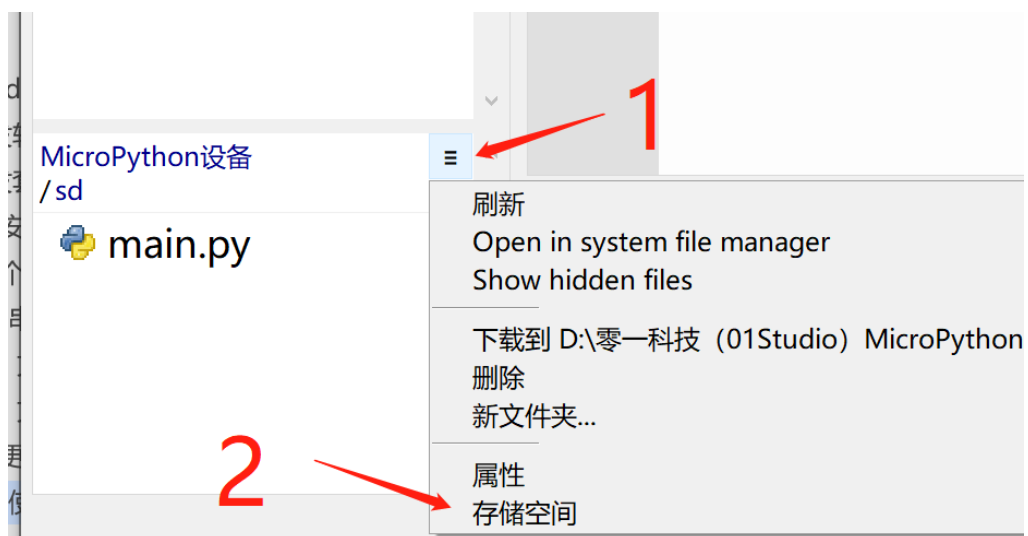


图 3-38

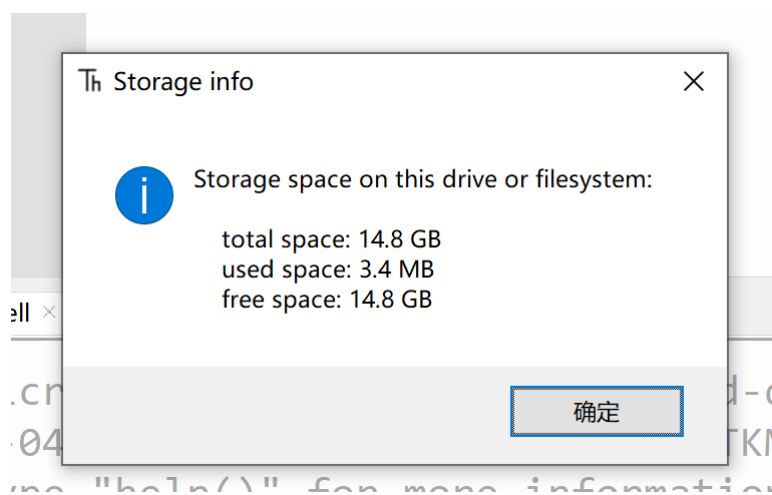


图 3-39 查看 SD 卡容量

3.2 基于 Mac OS

还记得当年乔布斯从大信封里面拿出 macbook 么，苹果和微软在 PC 时代走了两个极端，乔布斯主导下的苹果要求软硬一体化，封闭，务求最佳的用户体验；而微软则主打开发，让操作系统兼容不同的 PC 厂家，使得其 windows 一度占据了 90% 的市场份额。从商业角度，盖茨毫无疑问是赢家，就个人而言，我更认可乔布斯的方式，现在的 win10 和微软自家的笔记本，某程度上都有一体化的思想。我用了半天的 Macbook 和 MAC 系统，如果不是要做硬件发，基本上可以放弃十多年的 windows 了。

3.2.1 安装开发软件 Thonny

在 <https://thonny.org/> 选择 Mac 版本下载：



图 3-40

Thonny IDE 在 MAC OS 环境下的安装和使用方法和 Windows 一样，具体请参考：3.1.1 安装开发软件 Thonny 章节内容，这里不再重复。

3.2.2 开发套件使用

由于 Thonny IDE 基本可以满足开发板的编程、调试功能，因此直接参考 Windows 章节 3.1.2 开发套件使用 内容即可。

3.3 基于 Linux（树莓派）

Linux 系统为大多工程师热衷的开源操作系统，一般内部都集成了 python3 和 IDE，用户直接使用即可。本节教程同样适用于树莓派的 Linux 系统。

3.3.1 安装开发软件 Thonny

在 <https://thonny.org/> 下载 Linux 版本的 Thonny IDE，按提示指令安装即可：

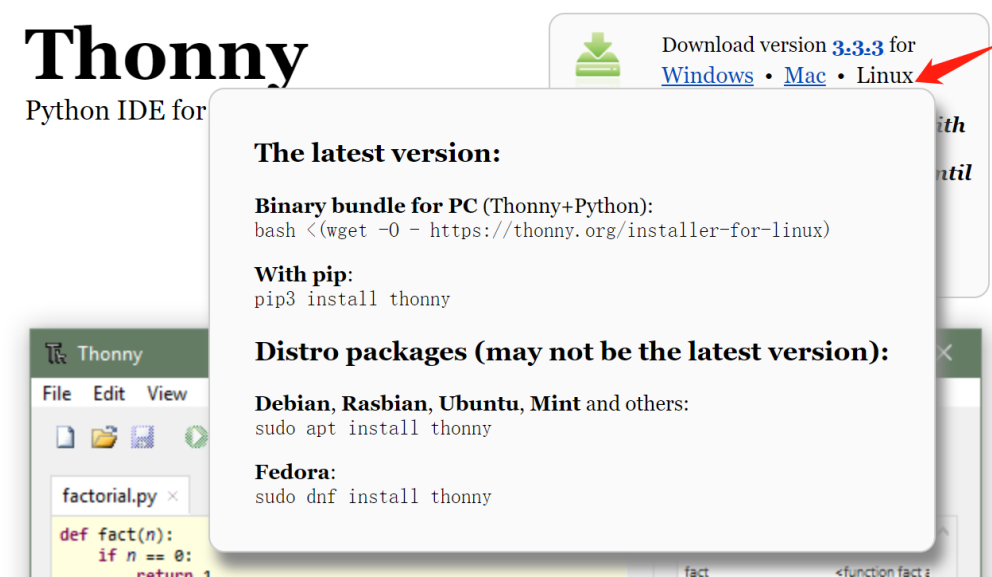


图 3-41

如果你实验树莓派开发板开发，它出厂自带 Thonny IDE，无需额外安装。

3.3.2 开发套件使用

由于 Thonny IDE 基本可以满足开发板的编程、调试功能，因此直接参考 Windows 章节 3.1.2 开发套件使用 内容即可。

第4章 基础实验

MicroPython 更强调的是针对应用的学习，强大的底层库函数让我们可以直接关心功能的实现，也就是说我们只要理解和熟练相关的函数用法，就可以很好的玩转 MicroPython。它让我们可以做到不关心硬件和底层原理（当然有兴趣和能力的小伙伴可以深入研究）而直接跑起硬件。

基础实验是针对一些简单的实验学习讲解，可以让我们更快和更直接感受到 MicroPython 针对嵌入式开发的强大之处，我后面的学习和开发夯实基础。

请先看实验讲解格式预览，每一节我们都会以以下形式讲解，图文并茂，力求达到快速理解和学习的作用：

- (1) **前言：**简单介绍这个实验；
- (2) **实验平台：**实验所用到的开发板、配件和接线说明；
- (3) **实验目的：**本实验要实现的功能；
- (4) **实验讲解：**对函数、代码、编程方法以及实验的详细讲解；
- (5) **实验结果：**记录程序下载到开发板上的图片示例；
- (6) **总结：**对实验进行总结。

4.1 点亮第一个 LED 灯

- **前言：**

相信大部分人开始学习嵌入式单片机编程都会从点亮 LED 开始，我们 MicroPython 的学习也不例外，通过点亮第一个 LED 能让你对编译环境和程序架构有一定的认识，为以后的学习和更大型的程序打下基础，增加信心。

- **实验平台：**

达芬奇 MicroPython 开发套件。



图 4-1 MicroPython 开发套件

- **实验目的：**

点亮 LED4（蓝灯）。

- **实验讲解：**

达芬奇上总共有 4 个 LED，分别是 LED1(红色)、LED2(绿色)、LED3(黄色)、LED4(蓝色)；从下面原理可以看到分别连接到引脚“C2”，“C3”，“C6”，“C7”。

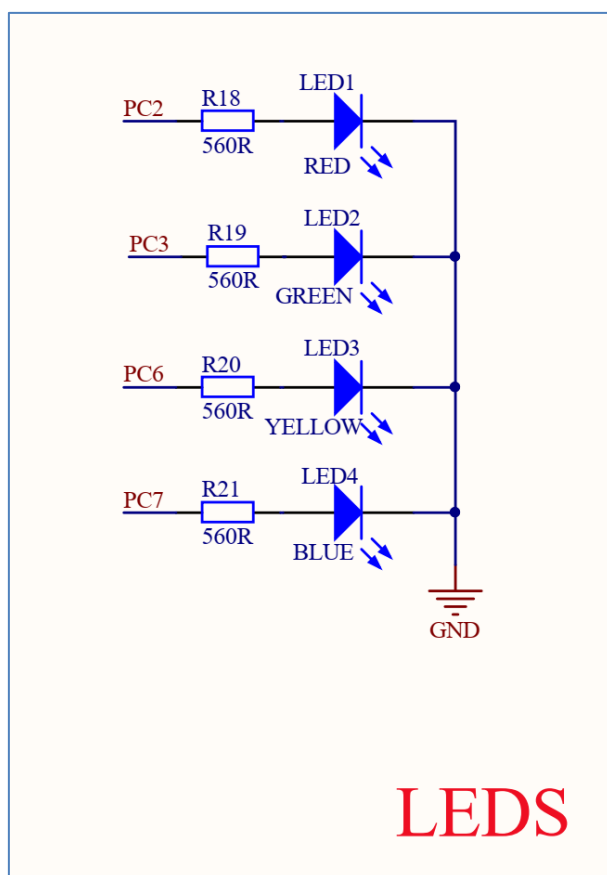


图 4-2 LED 引脚图

控制 LED 使用到 machin 中的 Pin 对象，其构造函数和使用方法如下：

构造函数
<code>led=machine.Pin(id,mode,pull)</code>
构建 led 对象。 【id】：Pico 引脚编号； 【mode】：输入输出方式； Pin.IN: 输入 Pin.OUT: 输出 【pull】：上下拉电阻配置。 None: 无上下拉电阻； Pin.PULL_UP: 上拉电阻； Pin.PULL-DOWN: 下拉电阻。

使用方法
<code>led.value([x])</code>
引脚电平值。输出状态：x=0 表示低电平，x=1 表示高电平；输入状态：无须参数，返回当前引脚值。
<code>led.high()</code>
使引脚输出高电平“1”。
<code>led.low()</code>
使引脚输出低电平“0”。

表 4-1 Pin 对象

上表对 MicroPython 的 machine 中 Pin 对象做了详细的说明，machine 是大模块，Pin 是 machine 下面的其中一个小模块，在 python 编程里有两种方式引用相关模块：

方式 1 是：import machine，然后通过 machine.Pin 来操作；

方式 2 是：from machine import Pin,意思是直接从 machine 中引入 Pin 模块，然后直接通过构建 led 对象来操作。显然方式 2 会显得更直观和方便，本实验也是使用方式 2 来编程。代码编写流程如下：

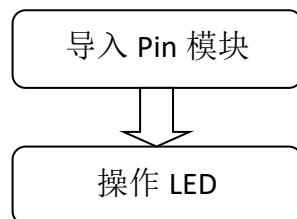


图 4-3 代码编写流程

LED4 蓝灯跟引脚‘C7’相连接，通过输出高电平方式点亮。

以下是本实验示例代码：

```

...

实验名称：点亮 LED 灯

版本：v1.0

日期：2021.4
  
```

作者: 01Studio

社区: www.01studio.org

实验平台: 01Studio - 达芬奇

'''

#导入 Pin 模块

from machine import Pin

LED = Pin('C7', Pin.OUT) #构建 LED 对象

LED.value(1) #点亮 LED,高电平点亮

运行程序有两个方法:

方法一:

编写好代码后点击 Thonny 上方的“运行”按钮,可以直接观察到代码运行情况。这个方法不会将程序代码保存到达芬奇开发板的文件系统(FLASH)里面。这主要是方便调试使用。

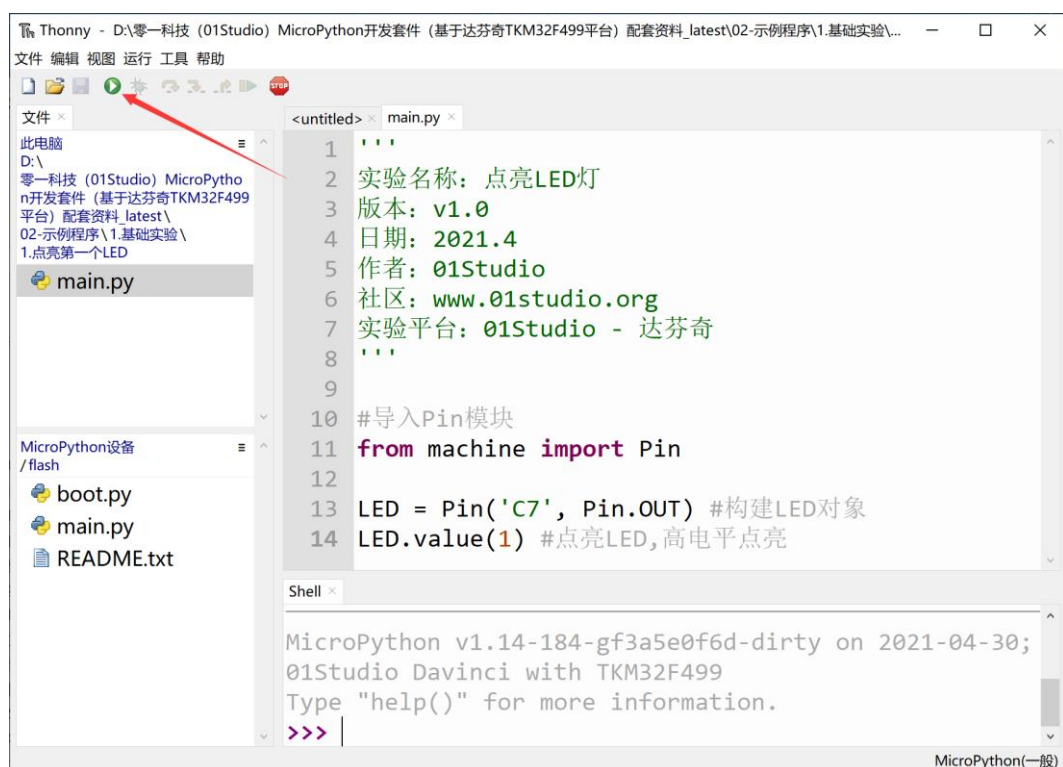


图 4-4 运行仿真

方法二:

使用 Thonny 的文件另存为功能, 将代码文件保存名称为 “main.py” 的 py 文件到达芬奇, 然后按 RST 复位按键, 设备会自动运行相关代码, 这个方式相当于将程序烧录到设备 flash, 可以脱机使用。操作方法参考: [3.1.2.4 文件系统](#) 章节内容。

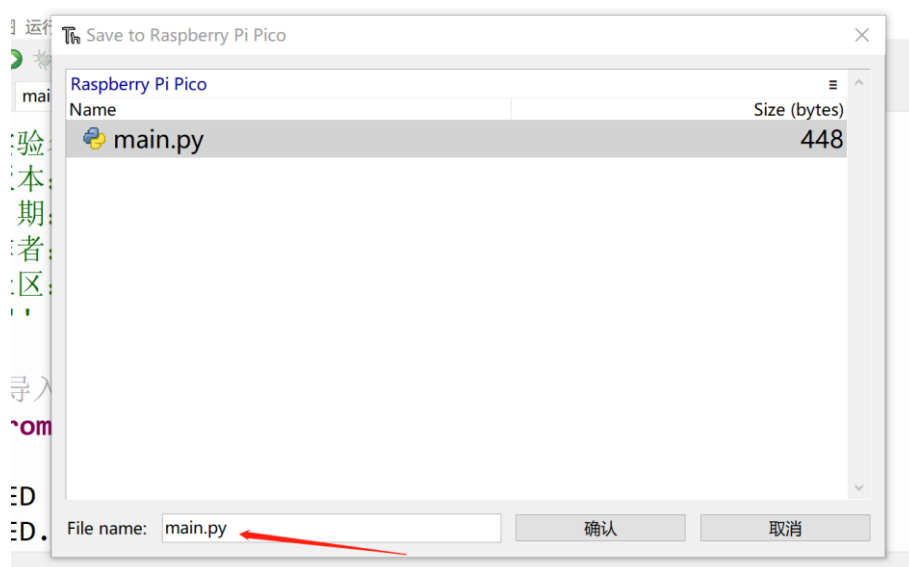


图 4-5 另存文件到设备, 名称为: main.py

- **实验结果：**



图 4-6 LED4 蓝灯被点亮

- **总结：**

从第一个实验我们可以看到，使用 MicroPython 来开发关键是要学会构造函数和其使用方法，便可完成对相关对象的操作，在强大的模块函数支持下，实验只用了简单的两行代码便实现了点亮 LED 灯。

4.2 流水灯

- **前言：**

通过上一节点亮 LED 灯的学习，我们已经对 micropython 的编程有了初步的了解，这一节来做一个功能稍微复杂一点的实验，流水灯。流水灯也叫跑马灯，也就是让几个 LED 来回亮灭，达到好像流水的效果。也是单片机开发学习的典型例子。

- **实验平台：**

达芬奇 MicroPython 开发套件。



图 4-7 MicroPython 开发套件

- **实验目的：**

流水灯。让 LED2-LED4 循环亮灭，达到像流水一样的效果。

- **实验讲解：**

达芬奇上总共有 4 个 LED，分别是 LED1(红色)、LED2(绿色)、LED3(黄色)、LED4(蓝色)；上一章节我们已经学习过 LED 点亮，这里要实现固定时间来亮灭，需要用到 time 延时的函数。具体如下：

LED 的构造函数和使用方法如下表所示：

构造函数
<code>led=machine.Pin(id,mode,pull)</code>
构建 led 对象。 【id】：Pico 引脚编号； 【mode】：输入输出方式； Pin.IN: 输入 Pin.OUT: 输出 【pull】：上下拉电阻配置。 None: 无上下拉电阻； Pin.PULL_UP: 上拉电阻； Pin.PULL-DOWN: 下拉电阻。
使用方法
<code>led.value([x])</code>
引脚电平值。输出状态： x=0 表示低电平， x=1 表示高电平；输入状态：无须参数，返回当前引脚值。
<code>led.high()</code>
使引脚输出高电平“1”。
<code>led.low()</code>
使引脚输出低电平“0”。

表 4-2 Pin 对象

time 延时函数和使用方法如下表所示：

构造函数
<code>import time</code>
直接导入使用。
使用方法
<code>time.sleep(seconds)</code>
秒级延时。 【seconds】 延时秒数。

<code>time.sleep_ms(ms)</code>
毫秒级延时。 【seconds】延时毫秒数。
<code>time.sleep_ms(us)</code>
微秒级延时。 【seconds】延时微秒数。

表 4-3 time 对象

知道了函数的使用方法后，我们可以简单的梳理一下流程，首先导入 Pin 和 time 模块，程序开始先让 LED2-LED4 灭掉，开启循环，依次点亮每个 LED，延时 1 秒，关闭 LED。流程如下：

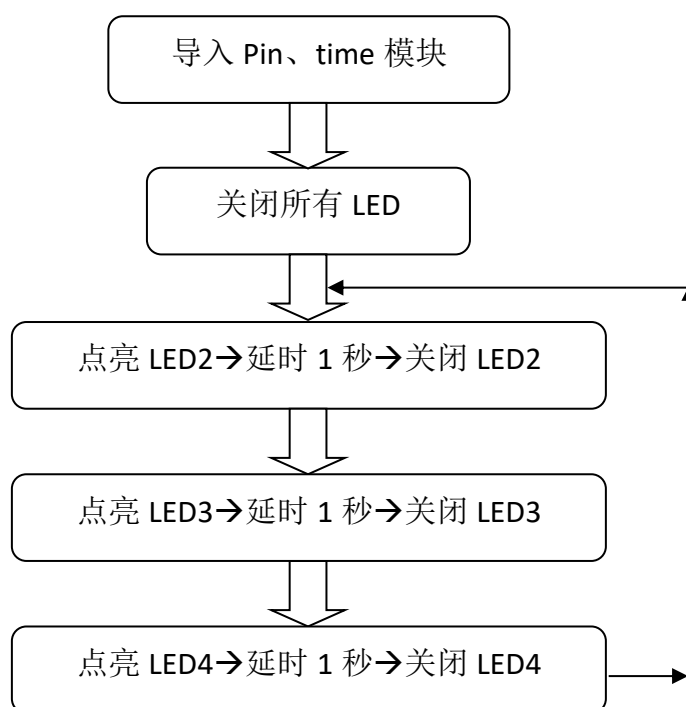


图 4-8 代码编写流程

以下是本实验参考程序代码：

...

实验名称：流水灯

版本：v1.0

日期：2021.4

作者：CaptainJackey

实验平台：01Studio - 达芬奇

...

#导入相关模块

```
from machine import Pin
```

```
import time
```

#构建 LED 对象

```
LED1 = Pin('C3', Pin.OUT)
```

```
LED2 = Pin('C6', Pin.OUT)
```

```
LED3 = Pin('C7', Pin.OUT)
```

#关闭所有 LED

```
LED1.low()
```

```
LED2.low()
```

```
LED3.low()
```

#while True 表示一直循环

```
while True:
```

```
    #LED1 亮 1 秒
```

```
    LED1.high()
```

```
    time.sleep_ms(1000)
```

```
    LED1.low()
```



```

#LED2 亮 1 秒
LED2.value(1)
time.sleep_ms(1000)
LED2.value(0)

#LED3 亮 1 秒
LED3.value(1)
time.sleep_ms(1000)
LED3.value(0)

```

上述代码没错是完整的按照编程思路来编写,但可以见到有很多格式相似的地方,这显得代码非常冗余。我们可以通过 `for` 函数来编写程序,由于是对 3 个 LED 的操作,因此我们可以用 `for i in range(3):` 语句来修改。参考代码如下:

```

...

实验名称: 流水灯
版本: v2.0
日期: 2021.4
作者: CaptainJackey
实验平台: 01Studio - 达芬奇
...

#导入相关模块
from machine import Pin
import time

#构建 LED 对象
LED1 = Pin('C3', Pin.OUT)

```

```

LED2 = Pin('C6', Pin.OUT)
LED3 = Pin('C7', Pin.OUT)

LEDS = [LED1,LED2,LED3]

# 相当于 for i in [0, 1, 2], LED[i].low()执行 3 次，分别是 LED 1, 2, 3
for i in range(3):
    LEDS[i].low()

while True:
    #使用 for 循环
    for i in range(3):
        LEDS[i].high()
        time.sleep_ms(1000) #延时 1000 毫秒，即 1 秒
        LEDS[i].low()

```

修改后的代码一下子变得简约美观。Python 作为高级语言，使用起来非常灵活，常常能通过几行代码就可以表达复杂的逻辑。我们在编程的过程中应该多思考如何优化代码，才能让自己的编程水平得到较快的提升。

（需要注意的是如果在 **main.py** 文件里使用了类似 **while True:** 的死循环函数，开发板的 **REPL** 功能将阻塞，可以在 **REPL** 里面按 **Ctrl+C** 打断程序。）

或者使用 Thonny 软件中断功能：

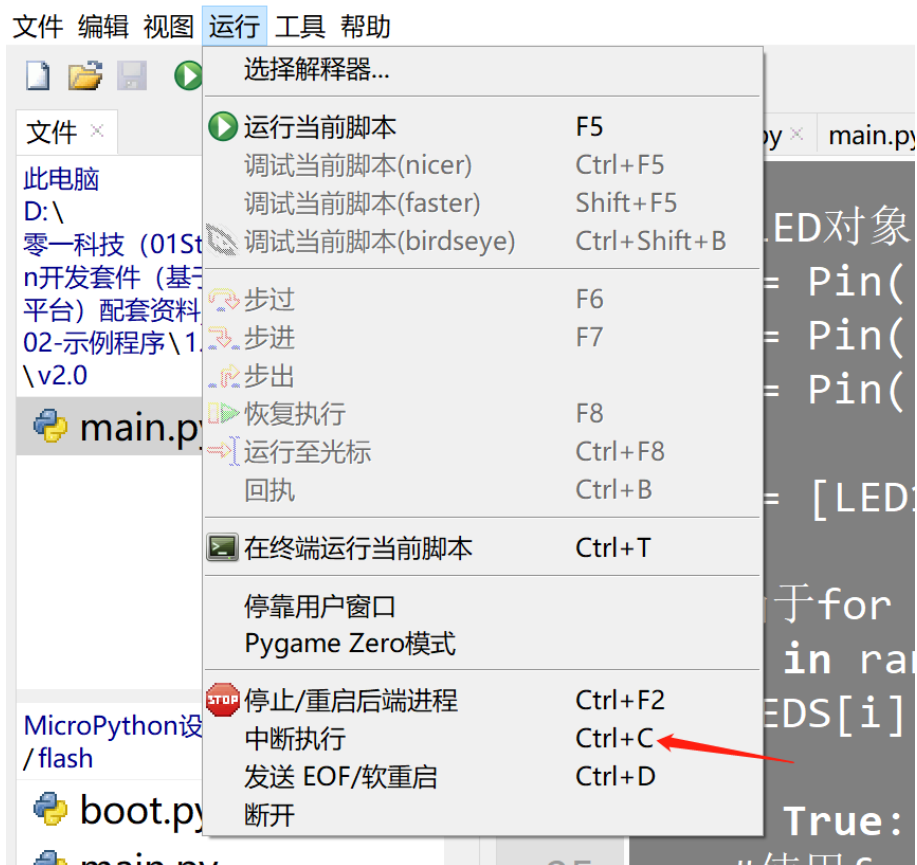


图 4-9 中断 while True

● **实验结果:**

LED2、LED3、LED4 循环点亮，实现流水效果。



图 4-10



图 4-11



图 4-12

● **总结：**

本节实验相对于第一节相比增加了延时函数的应用，可见 MicroPython 编程功能增加就是各类函数的叠加，当开发复杂功能时候，因为底层函数已经封装好，所以面向应用的开发使用起来非常方便。除此之外我们也体验了 Python 代码的简洁和高效，使得程序更好的阅读。

4.3 按键

- **前言：**

按键是最简单也最常见的输入设备，很多产品都离不开按键，包括早期的 iPhone，今天我们就来学习一下如何使用 MicroPython 来编写按键程序。有了按键输入功能，我们就可以做很多好玩的东西了。

- **实验平台：**

达芬奇 MicroPython 开发套件。



图 4-13 MicroPython 开发套件

- **实验目的：**

使用按键功能，通过检测按键被按下后，改变 LED4 蓝灯的亮灭状态。

- **实验讲解：**

达芬奇开发板上除了 RST 复位按键，还有 3 个功能按键，分别是 A0 键、A1 键和 A13 键。

让我们先来搞清楚 MicroPython 里面 Pin 模块实现按键的构造函数和使用方法。

构造函数
<code>KEY=machine.Pin(id,mode,pull)</code>
构建按键对象。 【id】 ：Pico 引脚编号； 【mode】 ：输入输出方式； Pin.IN: 输入 Pin.OUT: 输出 【pull】 ：上下拉电阻配置。 None: 无上下拉电阻； Pin.PULL_UP: 上拉电阻； Pin.PULL-DOWN: 下拉电阻。
使用方法
<code>KEY.value()</code>
引脚电平值。输入状态：无须参数，返回当前引脚值 0 或者 1。

表 4-4 Pin 对象用于按键方法

可以看到跟上一节 LED 一样，按键只是从 LED 输出变成了输入状态的一个改变。从下面原理图可以看到，我们只需要在开发板上电后判断 KEY 引脚的电平，需要注意的是当按键被按下时候引脚为高电平“1”。

本实验我们使用按键 A0。

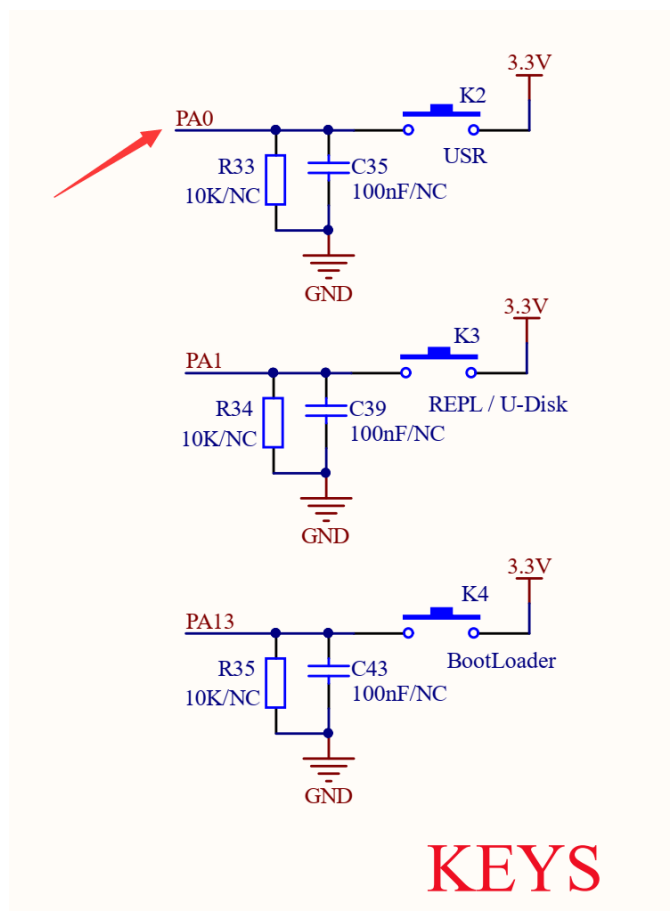


图 4-14 按键原理图

编程流程图如下：

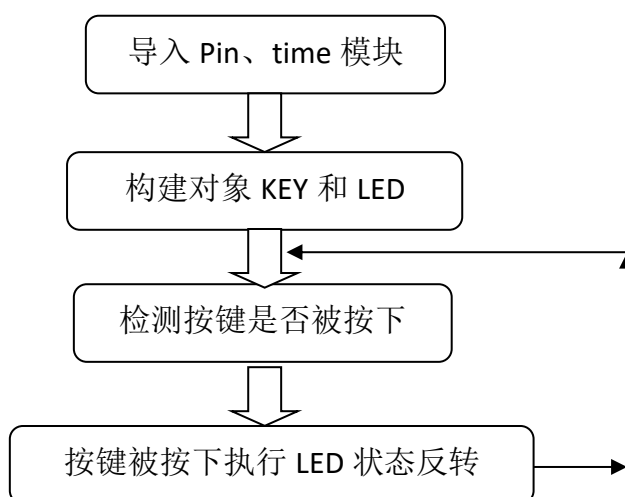


图 4-15 代码编写流程

参考程序代码如下：

```
'''
实验名称：按键(GPIO)
版本： v1.0
日期： 2021-1
作者： 01Studio
实验平台： 01Studio - 达芬奇
社区： www.01studio.org
'''

from machine import Pin
import time

#构建 LED 对象
led=Pin('C7', Pin.OUT)

#配置按键
key = Pin('A0', Pin.IN, Pin.PULL_DOWN)

while True:

    if key.value()==1: #KEY 被按下拉高

        led.high()    #点亮 LED

    else:

        led.low()      #关闭 LED
```


从上面代码可以看到，初始化各个对象后，进入循环，当检测到 KEY 的值为 1（按键被按下）时候，点亮 LED；

● **实验结果：**

按下 A1 按键和松开时候，LED4 蓝灯状态相应变化。



图 4-16 按键实验

● **总结：**

按键作为我们学习的第一个输入设备，有了输入设备我们就可以跟硬件做人机交互了，这对后面的学习非常有意义。按键在 MicroPython 下开发也显得很简单，我们同样可以根据自己的理解和经验来优化代码。

4.4 外部中断

- **前言：**

前面我们在做普通的 GPIO 时候，虽然能实现 IO 口输入输出功能，但代码是一直在检测 IO 输入口的变化，因此效率不高，特别是在一些特定的场合，比如某个按键，可能 1 天才按下一次去执行相关功能，这样我们就浪费大量时间来实时检测按键的情况。

为了解决这样的问题，我们引入外部中断概念，顾名思义，就是当按键被按下(产生中断)时，我们才去执行相关功能。中断的应用非常普遍，而 pyboard 上基本每个 IO 口都具备中断功能。

- **实验平台：**

达芬奇 MicroPython 开发套件。



图 4-17 MicroPython 开发套件

- **实验目的：**

利用中断方式来检查 A0 按键状态，按键被按下（产生外部中断）后使 LED4 蓝灯的状态反转。

- **实验讲解：**

外部中断也是通过 machine 模块的 Pin 子模块来配置，我们先来看看其配构造函数和使用方法：

构造函数
pyb.ExtInt(pin,mode,pull,callback)
【Pin】 Pin 脚，如 X1,Y1 【mode】 ExtInt.IRQ_RISING: 上升沿触发 ExtInt.IRQ_FALLING: 下降沿触发 ExtInt.IRQ_RISING_FALLING: 上升或下降沿触发 【pull】 Pin.PULL_NONE-没有上下拉电阻； Pin.PULL_UP-启用上拉电阻；
使用方法
产生中断后自动执行回调函数，将需要执行的代码定义在 callback 函数即可。

表 4-5 外部中断对象

A0 的电路原理图如下：

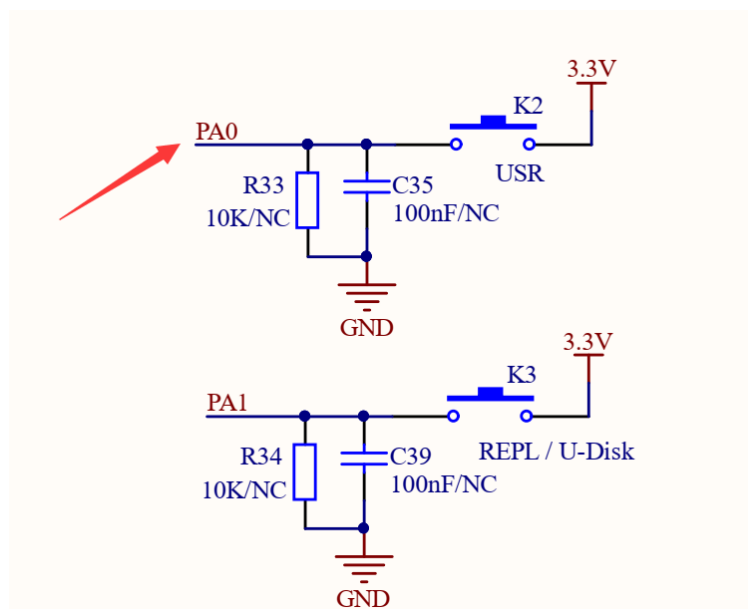


图 4-18

我们先来了解一下上升沿和下降沿的概念，由于“A0”引脚是通过按键接到 3.3V，也就是我们所说的高电平“1”，所以当按键被按下再松开时，引脚先获得上升沿，再获得下降沿，如下图所示：

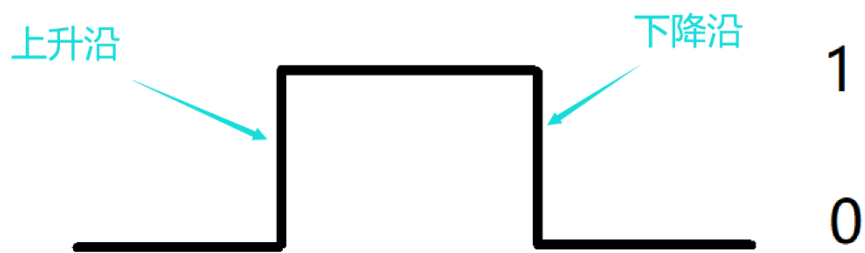


图 4-19 上升沿和下降沿

由此可见，我们可以选择上升沿方式触发外部中断，也就是当按键被按下的时候立即产生中断。

编程思路中断跟按键章节类似，在初始化中断后，当系统检测到外部终端时候，执行 LED（4）状态反转的代码即可。

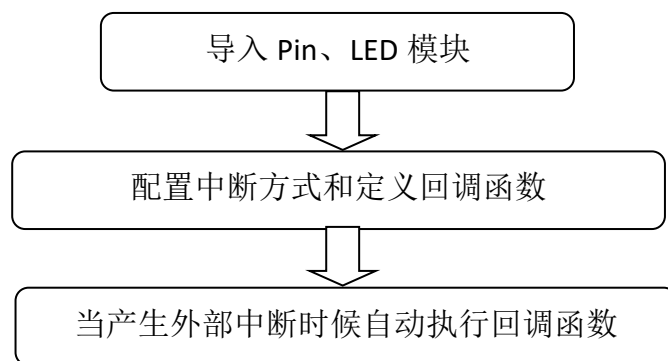


图 4-20 代码编写流程

参考程序代码如下：

```
...  
实验名称：外部中断  
版本：v1.0  
日期：2021-1  
作者：01Studio  
实验平台：达芬奇
```

```

社区: www.01studio.org
'''

from machine import Pin
import time

#构建 LED 对象
led=Pin('C7', Pin.OUT)

#配置按键
key = Pin('A0', Pin.IN, Pin.PULL_DOWN)

state=0 #LED 引脚状态

#LED 状态翻转函数
def fun(key):

    global state
    time.sleep_ms(10) #消除抖动
    if key.value()==1: #确认按键被按下,开发板按下拉高
        state = not state
        led.value(state)

key.irq(fun,Pin.IRQ_RISING) #定义中断, 下降沿触发

```

- 实验结果:



图 4-21 中断方式控制 LED4 蓝灯

- 总结:

中断相对于使用 `while True` 实时检测按键的方法来看,代码的效率大大增强。外部中断的应用非常广,除了普通的按键输入和电平检测外,很大一部分输入设备,比如传感器也是通过外部中断方式来实时检测,这个在后面的章节会讲述。

4.5 定时器

- **前言：**

定时器，顾名思义就是用来计时的，我们常常会设定计时或闹钟，然后时间到了就告诉我们要做什么了。单片机也是这样，通过定时器可以完成各种预设好的任务。

- **实验平台：**

达芬奇 MicroPython 开发套件。



图 4-22 达芬奇 MicroPython 开发套件

- **实验目的：**

通过定时器让 LED4 蓝灯周期性每秒闪烁 1 次。

- **实验讲解：**

达芬奇的 micropython 固件内置了定时器，在 machine 的 Timer 模块中。通过 MicroPython 可以轻松编程使用。我们也是只需要了解其构造对象函数和使用方法即可！

构造函数
<code>tim=machine.Timer(id)</code>
构建定时器对象。 【id】-1 为虚拟定时器
使用方法
<code>tim.init(period,mode,callback)</code>
定时器初始化。 【period】：单位为 ms； 【mode】：2 种工作模式，Timer.ONE_SHOT（执行一次）、Timer.PERIODIC（周期性）； 【callback】：定时器中断后的回调函数。 更多说明请参考官方文档： http://docs.micropython.01studio.org/zh_CN/latest/library/machine.Timer.html#machine-timer

表 4-6 定时器对象

定时器到了预设指定时间后，也会产生中断，因此跟外部中断的编程方式类似，代码编程流程图如下：

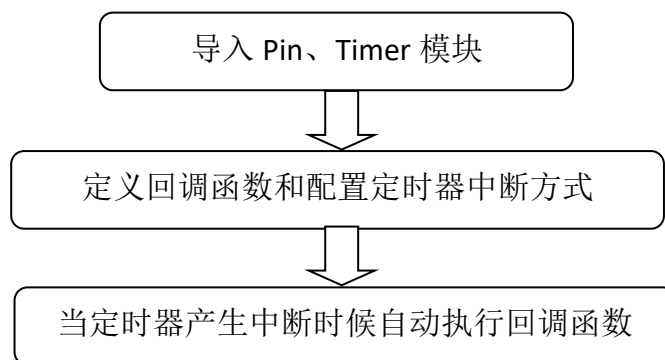


图 4-23 代码编写流程

参考代码如下：

```
'''
实验名称：定时器
版本：v1.0
日期：2021.1
作者：01Studio
实验平台：达芬奇
说明：通过定时器让 LED 周期性每秒闪烁 1 次
'''

from machine import Pin,Timer

#蓝灯
led=Pin('C7', Pin.OUT)

state=0 #LED 引脚状态

#LED 状态翻转函数
def fun(tim):

    global state

    state = not state

    led.value(state)

#构建软件定时器，编号-1
tim = Timer(-1)
tim.init(period=1000, mode=Timer.PERIODIC,callback=fun) #周期为 1000ms
```

- **实验结果:**



图 4-24 LED 以周期 1 秒的间隔闪烁

- **总结:**

本节实验介绍了 RTOS 定时器的使用方式，有用户可能会认为使用延时延时也可以实现这个功能，但相比于延时函数，定时器的好处就是不占用 CPU 资源。

4.6 蜂鸣器

- **前言：**

日常生活中我们不少电子设备在遇到故障时都会报警，而声音比指示灯往往更容易引起人们的注意，本节我们来学习一下蜂鸣器应用。

- **实验平台：**

达芬奇 MicroPython 开发套件。

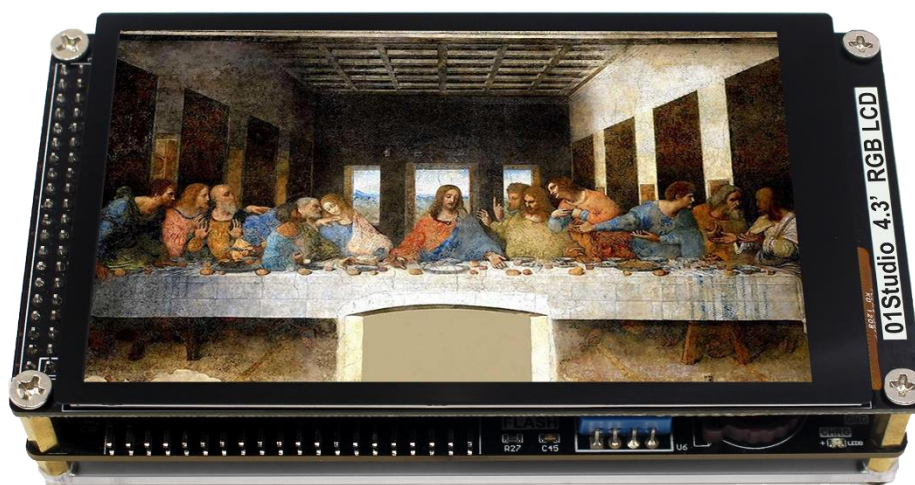


图 4-25

- **实验目的：**

通过编程实现蜂鸣器发出滴滴响声。

- **实验讲解：**

蜂鸣器分有源蜂鸣器和无源蜂鸣器，有源蜂鸣器的使用方式非常简单，只需要接上电源，蜂鸣器就发声，断开电源就停止发声。无源蜂鸣器，是需要给定指定的频率，才能发声的，而且可以通过改变频率来改变蜂鸣器的发声音色。达芬奇上配备的是有源蜂鸣器，因此控制方式非常简单。

我们从下面原理图可以看到，蜂鸣器跟达芬奇 TKM32 主控的“PB13”引脚连接，可以通过控制该引脚输出低电平来让蜂鸣器发出响声（IO 输出 0）。

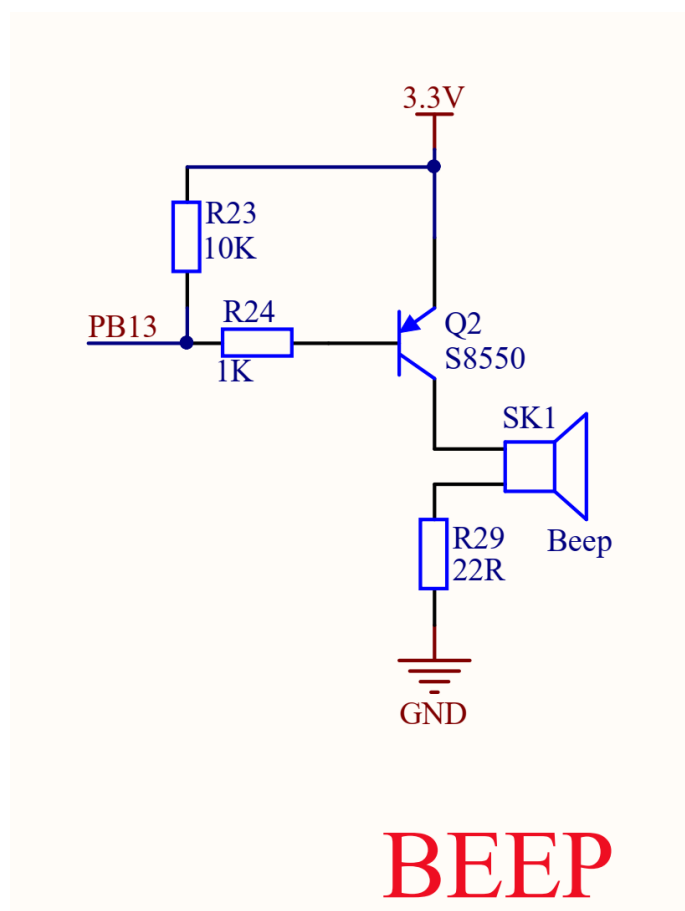


图 4-26 蜂鸣器电路图

GPIO 对象我们前面章节学习过了，非常简单，具体构造函数和使用方法如下：

构造函数
<pre>beep=machine.Pin(id,mode,pull)</pre>
构建 led 对象。 【id】：Pico 引脚编号； 【mode】：输入输出方式； Pin.IN: 输入 Pin.OUT: 输出 【pull】：上下拉电阻配置。 None: 无上下拉电阻； Pin.PULL_UP: 上拉电阻；

Pin.PULL-DOWN: 下拉电阻。
使用方法
beep.value([x])
引脚电平值。输出状态: x=0 表示低电平, x=1 表示高电平; 输入状态: 无须参数, 返回当前引脚值。
beep.high()
使引脚输出高电平 “1”。
beep.low()
使引脚输出低电平 “0”。

表 4-7 Pin 对象

可以看到有源蜂鸣器的使用跟之前的 LED 很像, 就是高低电平控制。除此之外我们还需要一个延时函数来实现滴滴的效果。这里学习一个新的延时模块 **time**。说明如下:

构造函数
import time
直接使用
使用方法
time.sleep(seconds)
秒级延时。例 <code>time.sleep(1)</code> 表示延时 1 秒;
time.sleep_ms(ms)
毫秒级延时。例 <code>time.sleep_ms(100)</code> 表示延时 100 毫秒;
time.sleep_us(us)
微秒级延时。例 <code>time.sleep_us(10)</code> 表示延时 10 微秒;

表 4-8

代码编写流程如下:

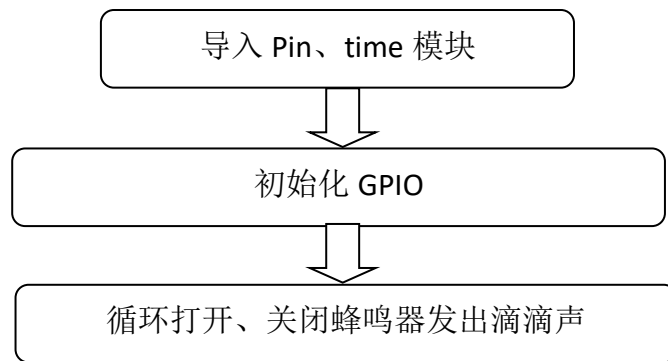


图 4-27 代码编写流程

这里需要注意的是达芬奇的蜂鸣器设计成低电平打开。参考代码如下：

```
'''
实验名称：蜂鸣器
版本：v1.0
日期：2021-4
作者：01Studio
实验平台：达芬奇
实验说明：编写实现有源蜂鸣器发出滴滴响声。
'''

from machine import Pin
import time

beep = Pin('B13', Pin.OUT) # 蜂鸣器对应引脚是 13,

while True:

    beep.value(0)      #低电平发出响声
    time.sleep_ms(500) #延时 500 毫秒
    beep.value(1)      #关闭蜂鸣器
    time.sleep_ms(500)
```

- **实验结果:**

运行代码，可以听到达芬奇开发板上的蜂鸣器发出滴滴响声。



图 4-28

如果觉得声音小，可以将蜂鸣器表面的贴纸撕开，那要声音就会大得多。

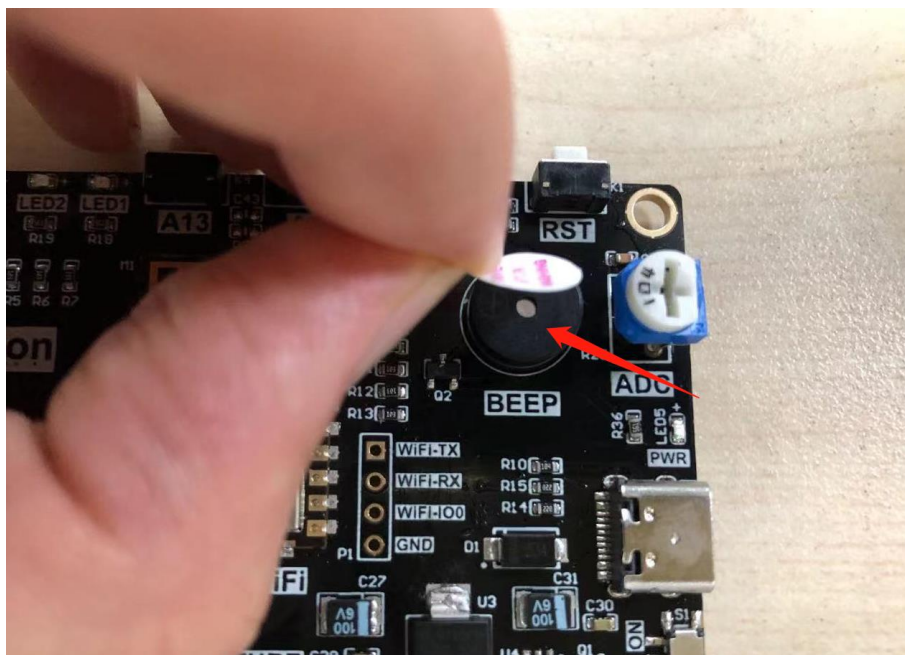


图 4-29 撕开蜂鸣器贴纸

- **总结：**

到了这一节，我们发现实验中 `python` 对象函数使用方法非常简单。这是好事，让我们可以将更多精力放在应用上，做出更多好玩的创意。而不需要过多的关注复杂的底层代码开发。

4.7 LCD 显示屏

在前面学习了按键输入设备后，我们这一节先来学习输出设备 LCD 显示屏，其实之前的 LED 灯也算是输出设备，因为它们确切地告诉了我们硬件的状态。只是相对于只有亮灭的 LED 而言，LCD 显示屏可以显示更多的信息，体验更好。

我们把它放在了前面来学习，是因为学会了 LCD 显示屏的使用，那么在后面的实验中可玩性就更强了。

LCD 液晶显示屏是非常常见的一个外接显示设备，我们看到的手持设备、小型电器，很多都用到 LCD，部分配合触摸屏应用，能实现非常多的功能。

这里主要针对 4.3 寸和 7 寸 RGB 屏的讲解，学习完后你会发现，得益于 micropython 强大的移植性，他们的使用方法几乎完全一样。

4.7.1 4.3 寸 RGB 屏

- **前言:**

本节讲解达芬奇 + 4.3 寸 RGB 屏的使用方法。

- **实验平台:**

达芬奇 MicroPython 开发套件、4.3 寸 RGB 显示屏（电容触摸）。



图 4-30 开发板+4.3 寸 RGB 屏

- **实验目的:**

通过 MicorPython 编程方式实现 LCD 的各种显示功能, 包括画点、线、矩形、圆形、显示英文、显示图片等。

- **实验讲解:**

我们先来看看实验所使用 LCD 的参数介绍:



图 4-31 4.3 寸 LCD 显示屏（电容触摸）

功能参数	
屏幕类型	RGB 屏
屏幕尺寸	4.3 寸
颜色参数	24 位彩色
驱动芯片	LG4572B + FT5436(触摸)
触摸方式	电容触摸（五点触控），I2C 接口
通讯方式	24 位并行(RGB888)
供电电压	背光：3.7~5V；GPIO: 3.3V
接口	40P-2.54mm 排针和 40P-0.5MM FPC 座
尺寸	12*6.2 cm

表 4-9 4.3 寸 RGB 显示屏功能参数

实验用的 LCD 是 4.3 寸,驱动是的 LG4572B,使用 24 位 RGB888 并口跟 TKM32 通信,按以往嵌入式 C 语言开发,我们需要对 LG4572B 进行编程实现驱动,然后再建立各种描点、划线、以及显示图片函数。

使用 MicroPython 其实也需要做以上工作,但由于可读性和移植性强的特点,我们只需要搞清各个对象函数使如何使用即可。总的来说和前面实验一样,有构造函数和功能函数。构造函数解决的是初始化问题,告诉达芬奇开发板该外设是怎么接线,初始化参数如何,而功能函数解决的则是使用问题,我们基于自己的需求直接调用相关功能函数,实现自己的功能即可!

我们管这些函数的集合叫驱动,驱动可以是预先在固件里面,也可以通过.py 文件存放在开发板文件系统。也就是说工程师已经将复杂的底层代码封装好,我们顶层直接使用 python 开发即可,人生苦短。我们来看看达芬奇开发板 4.3 寸 RGB LCD 的构造函数和使用方法。

构造函数
<code>tftlcd.LCD43R(portrait=1)</code> 构建 4.3 寸 RGB LCD 对象。 【portrait】 设置屏幕方向: ● 1 - 横屏, 800*480 , 默认 ● 2 - 竖屏, 480*800 , 1 基础上顺时针旋转 90° ● 3 - 横屏, 800*480 , 1 基础上顺时针旋转 180° ● 4 - 竖屏, 480*800 , 1 基础上顺时针旋转 270°
使用方法
<code>LCD43R.fill(color)</code> 【color】 RGB 颜色数据; 如(255,0,0)表示红色。
<code>LCD43R.drawPixel(x,y,color)</code> 画点。 【x】:横坐标, 【y】:纵坐标, 【color】:颜色。
<code>LCD43R.drawLine(x0,y0,x1,y1,color)</code>

画线段。

【x0,y0】:起始坐标,

【x1,y1】:终点坐标,

【color】:颜色

LCD43R.drawRect(x,y,width,height,color,border=1,fillcolor=None)

画矩形。

【x,y】:起始坐标,

【width】:宽度,

【height】:高度,

【color】:颜色,

【border】:边宽,

【fillcolor】:填充颜色, 默认 None 为不填充

LCD43R.drawCircle(x,y,radius,color,border=1,fillcolor=None)

画圆。

【x,y】:圆心,

【radius】:半径,

【color】:颜色,

【border】:边宽,

【fillcolor】:填充颜色, 默认 None 为不填充

LCD43R.printStr(text,x,y,color,backcolor=None,size=2)

写字符。

【text】:字符,

【x,y】:起始坐标,

【color】:字体颜色;

【backcolor】:字体背景颜色;

【size】:字体尺寸 (1-小号, 2-标准, 3-中号, 4-大号)

LCD43R.Picture(x,y,filename,cached=True)

显示图片。支持图片格式类型: jpg、bmp

【x,y】:起始坐标。

【filename】：图片路径+名称，如："/flash/cat.jpg"、"/sd/cat.jpg"（flash 表示开发板的板载 flash, sd 表示 sd 卡）。 【cached】：选择是否使用缓存文件，默认 True 表示使用。使用缓存文件能提高图片显示速度。
LCD43R.CachePicture(filename,path=filename+'.cache',replace=False)
制作图片缓存文件，支持图片格式：jpg/bmp，生成.cache 文件。最大尺寸 800*480； 【cached】：图片路径+名称，如："/flash/cat.jpg"、"/sd/dog.bmp"（flash 表示开发板的板载 flash, sd 表示 sd 卡）。 【path】：缓存文件保存路径，默认 filename + '.cache' 表示和图片文件路径一致。文件格式为 '.cache'； 【replace】：是否覆盖已经存在的缓存文件，默认 replace=False 表示不覆盖。

表 4-10 4.3 寸 LCD 对象

有了上面的对象构造函数和使用说明，编程可以说是信手拈来了，我们在使用中将以上功能都跑一遍先看看编程流程图：

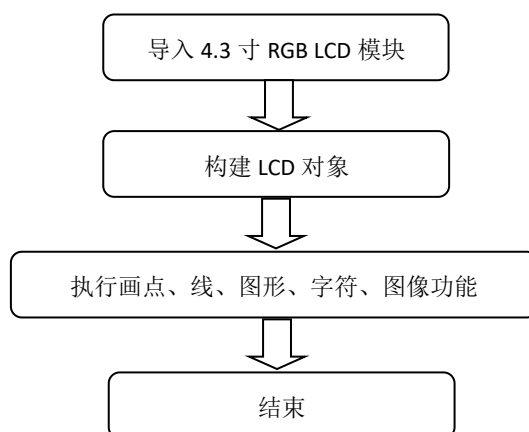


图 4-32 编程流程图

参考 main.py 函数代码如下：

```

'''
实验名称：4.3 寸 LCD 液晶显示屏
版本：v1.0
日期：2021.4
作者：01Studio
社区：www.01studio.org
说明：通过编程实现 LCD 的各种显示功能，包括填充、画点、线、矩形、圆形、显示英文、显示图片等。
'''

from tftlcd import LCD43R
import time

#定义常用颜色
RED = (255,0,0)
GREEN = (0,255,0)
BLUE = (0,0,255)
BLACK = (0,0,0)
WHITE = (0,0,0)

#####
# 构建 4.3 寸 RGB LCD 对象并初始化
#####
d = LCD43R(portrait=1) #默认方向,横屏 800*480

#填充白色
d.fill((255,255,255))

#画点
d.drawPixel(5, 20, RED)

```

```
#画线段
d.drawLine(5, 50, 300, 50, RED)

#画矩形
d.drawRect(5, 100, 300, 100, RED, border=5)

#画圆
d.drawCircle(150, 300, 50, RED, border=10)

#写字符,4 种尺寸
d.printStr('Hello 01Studio', 400, 20, RED, size=1)
d.printStr('Hello 01Studio', 400, 100, GREEN, size=2)
d.printStr('Hello 01Studio', 400, 200, BLUE, size=3)
d.printStr('Hello 01Studio', 400, 300, BLACK, size=4)

time.sleep(3) #等待 3 秒

#显示图片
d.Picture(0,0,"/flash/picture/supper.jpg")
```

- **实验结果:**

将示例程序的素材文件上传到达芬奇开发板。(也可以只上传单张图片, 注意修改代码中文件的路径即可。)

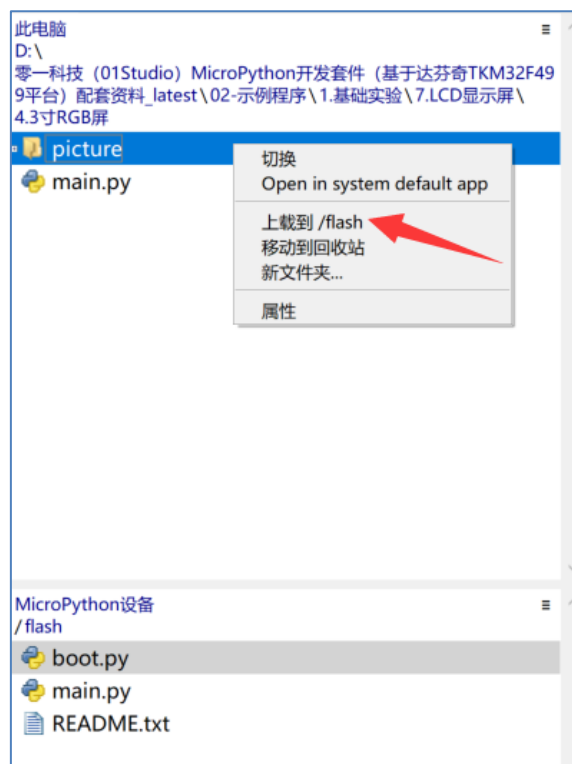


图 4-33 上传图片文件夹到 Flash

运行程序，可以看到 LCD 依次显示相关内容。



图 4-34



图 4-35 显示图片

● 总结:

通过本实验我们体验到了 LCD 使用 micropython 开发的简单和灵活性。我们轻松实现了 LCD 的各种常规操作, 让用户将精力放在应用。相信随着 micropython 库函数的日益成熟, 其性能和可玩性将变得更强大!

4.7.2 7 寸 RGB 屏

- **前言：**

本节讲解达芬奇 + 7 寸 RGB 屏的使用方法。

- **实验平台：**

达芬奇 MicroPython 开发套件、7 寸 RGB 显示屏（电容触摸）。



图 4-36 达芬奇开发板+7 寸 LCD（替换）

- **实验目的：**

通过 MicorPython 编程方式实现 LCD 的各种显示功能，包括画点、线、矩形、圆形、显示英文、显示图片等。

- **实验讲解：**

我们先来看看实验所使用 LCD 的参数介绍：

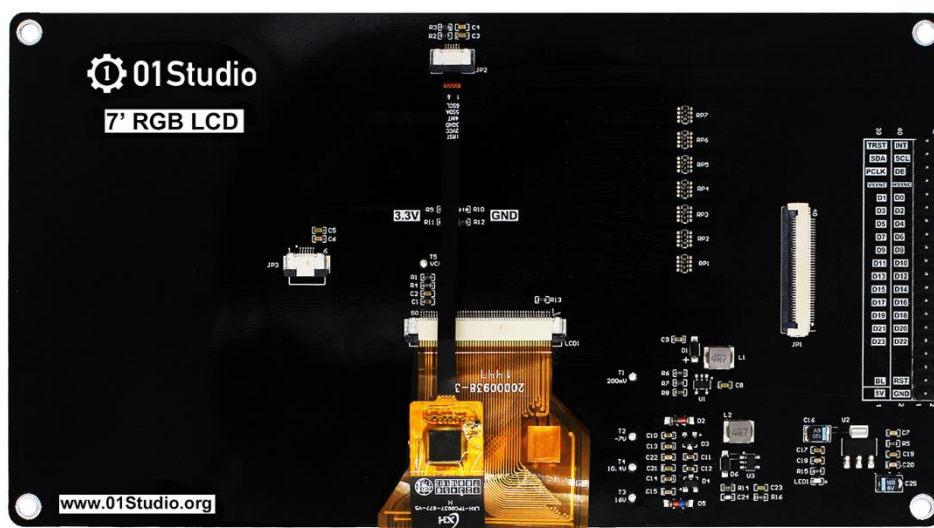
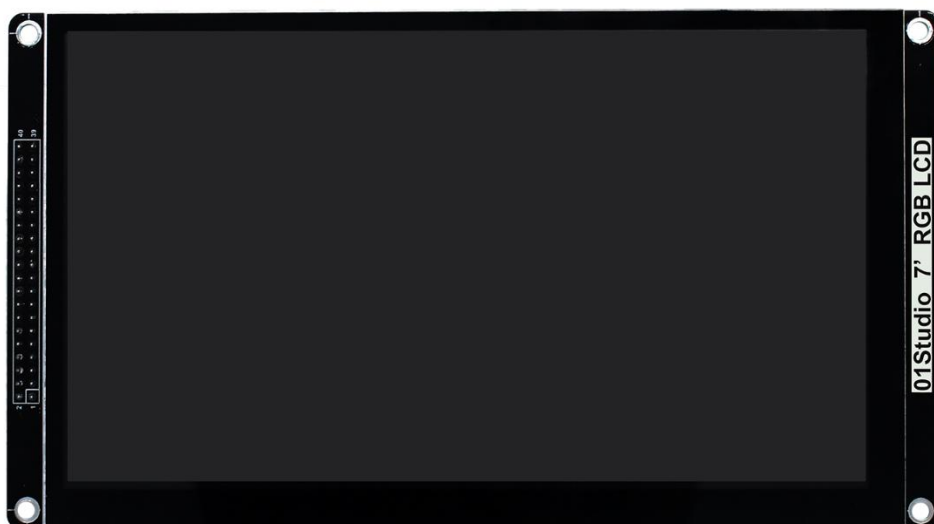


图 4-37 7 寸 RGB 显示屏（电容触摸）替换

功能参数	
屏幕类型	RGB 屏
屏幕尺寸	7 寸
颜色参数	24 位彩色
驱动芯片	群创 AT070TN92 + GT911(触摸)
触摸方式	电容触摸（五点触控），I2C 接口
通讯方式	24 位并行(RGB888)

供电电压	背光：3.7~5V；GPIO: 3.3V
接口	40P-2.54mm 排针和 40P-0.5MM FPC 座
尺寸	18*12cm

表 4-11 7 寸 RGB 显示屏功能参数

实验用的 LCD 是 7 寸，型号是群创 AT070TN92，使用 24 位 RGB888 并口跟 TMK32 通信，按以往嵌入式 C 语言开发，我们需要对其进行编程实现驱动，然后再建立各种描点、划线、以及显示图片函数。

使用 MicroPython 其实也需要做以上工作，但由于可读性和移植性强的特点，我们只需要搞清各个对象函数使如何使用即可。总的来说和前面实验一样，有构造函数和功能函数。构造函数解决的是初始化问题，告诉达芬奇开发板该外设是怎么接线，初始化参数如何，而功能函数解决的则是使用问题，我们基于自己的需求直接调用相关功能函数，实现自己的功能即可！

我们管这些函数的集合叫驱动，驱动可以是预先在固件里面，也可以通过.py 文件存放在开发板文件系统。也就是说工程师已经将复杂的底层代码封装好，我们顶层直接使用 python 开发即可，人生苦短。我们来看看达芬奇开发板 7 寸 RGB LCD 的构造函数和使用方法。

构造函数
<code>tftlcd.LCD7R(portrait=1)</code>
构建 7 寸 RGB LCD 对象。
【portrait】 设置屏幕方向： ● 1- 横屏，800*480 ， 默认 ● 2- 竖屏，480*800 ， 1 基础上顺时针旋转 90° ● 3- 横屏，800*480 ， 1 基础上顺时针旋转 180° ● 4- 竖屏，480*800 ， 1 基础上顺时针旋转 270°
使用方法
<code>LCD7R.fill(color)</code>
【color】 RGB 颜色数据；如(255,0,0)表示红色。
<code>LCD7R.drawPixel(x,y,color)</code>
画点。

【x】:横坐标, 【y】:纵坐标, 【color】:颜色。
LCD7R.drawLine(x0,y0,x1,y1,color)
画线段。 【x0,y0】:起始坐标, 【x1,y1】:终点坐标, 【color】:颜色
LCD7R.drawRect(x,y,width,height,color,border=1,fillcolor=None)
画矩形。 【x,y】:起始坐标, 【width】:宽度, 【height】:高度, 【color】:颜色, 【border】:边宽, 【fillcolor】:填充颜色, 默认 None 为不填充
LCD7R.drawCircle(x,y,radius,color,border=1,fillcolor=None)
画圆。 【x,y】:圆心, 【radius】:半径, 【color】:颜色, 【border】:边宽, 【fillcolor】:填充颜色, 默认 None 为不填充
LCD7R.printStr(text,x,y,color,backcolor=None,size=2)
写字符。 【text】:字符, 【x,y】:起始坐标, 【color】:字体颜色; 【backcolor】:字体背景颜色;

【size】 :字体尺寸（1-小号，2-标准，3-中号，4-大号）
LCD7R.Picture(x,y,filename,cached=True)
显示图片。支持图片格式类型：jpg、bmp 【x,y】 :起始坐标。 【filename】 : 图片路径+名称，如："/flash/cat.jpg"、"/sd/cat.jpg" （flash 表示开发板的板载 flash, sd 表示 sd 卡）。 【cached】 : 选择是否使用缓存文件，默认 True 表示使用。使用缓存文件能提高图片显示速度。
LCD7R.CachePicture(filename,path=filename+'.cache',replace=False)
制作图片缓存文件，支持图片格式：jpg/bmp，生成.cache 文件。最大尺寸 800*480； 【cached】 : 图片路径+名称，如："/flash/cat.jpg"、"/sd/dog.bmp" （flash 表示开发板的板载 flash, sd 表示 sd 卡）。 【path】 : 缓存文件保存路径，默认 filename + '.cache'表示和图片文件路径一致。文件格式为 '.cache'； 【replace】 : 是否覆盖已经存在的缓存文件，默认 replace=False 表示不覆盖。

表 4-12 4.3 寸 LCD 对象

有了上面的对象构造函数和使用说明，编程可以说是信手拈来了，我们在使用中将以上功能都跑一遍先看看编程流程图：

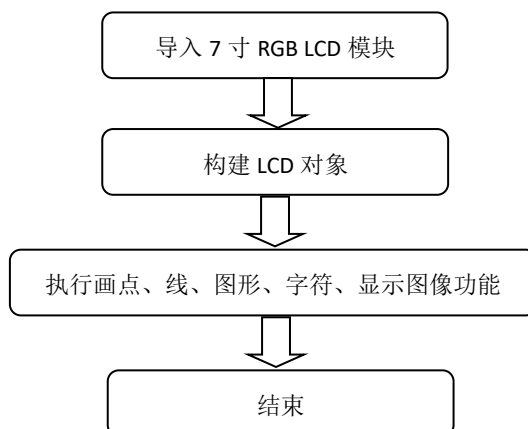


图 4-38 编程流程图

参考 main.py 函数代码如下：

```
'''
实验名称：4.3 寸 LCD 液晶显示屏
版本：v1.0
日期：2021.4
作者：01Studio
社区：www.01studio.org
说明：通过编程实现 LCD 的各种显示功能，包括填充、画点、线、矩形、圆形、显示英文、显示图片等。
'''

from tftlcd import LCD43R
import time

#定义常用颜色
RED = (255,0,0)
GREEN = (0,255,0)
BLUE = (0,0,255)
BLACK = (0,0,0)
WHITE = (0,0,0)

#####
# 构建 4.3 寸 RGB LCD 对象并初始化
#####
d = LCD43R(portrait=1) #默认方向,横屏 800*480

#填充白色
d.fill((255,255,255))
```



```
#画点
d.drawPixel(5, 20, RED)

#画线段
d.drawLine(5, 50, 300, 50, RED)

#画矩形
d.drawRect(5, 100, 300, 100, RED, border=5)

#画圆
d.drawCircle(150, 300, 50, RED, border=10)

#写字符,4 种尺寸
d.printStr('Hello 01Studio', 400, 20, RED, size=1)
d.printStr('Hello 01Studio', 400, 100, GREEN, size=2)
d.printStr('Hello 01Studio', 400, 200, BLUE, size=3)
d.printStr('Hello 01Studio', 400, 300, BLACK, size=4)

time.sleep(3) #等待 3 秒

#显示图片
d.Picture(0,0,"/flash/picture/supper.jpg")
```

● 实验结果:

将示例程序的素材文件上传到达芬奇开发板。(也可以只上传单张图片, 注意修改代码中文件的路径即可。)

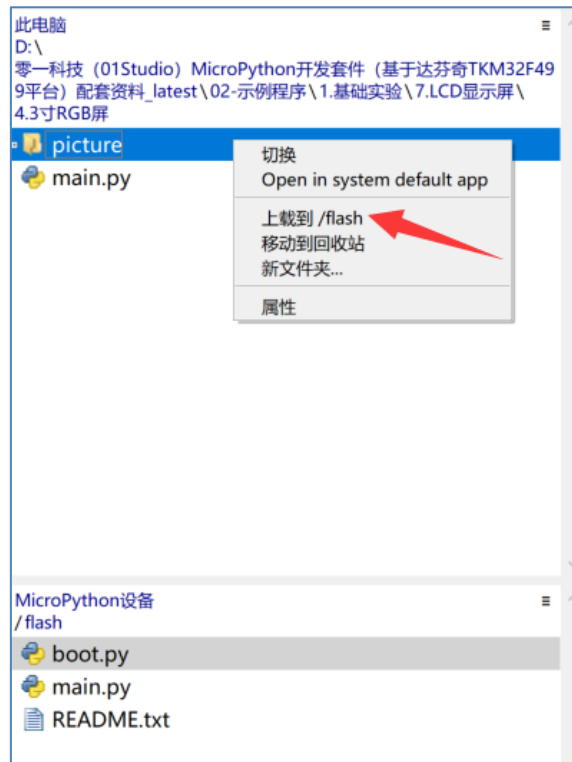


图 4-39 上传图片文件夹到 Flash

运行程序，可以看到 LCD 依次显示相关内容。

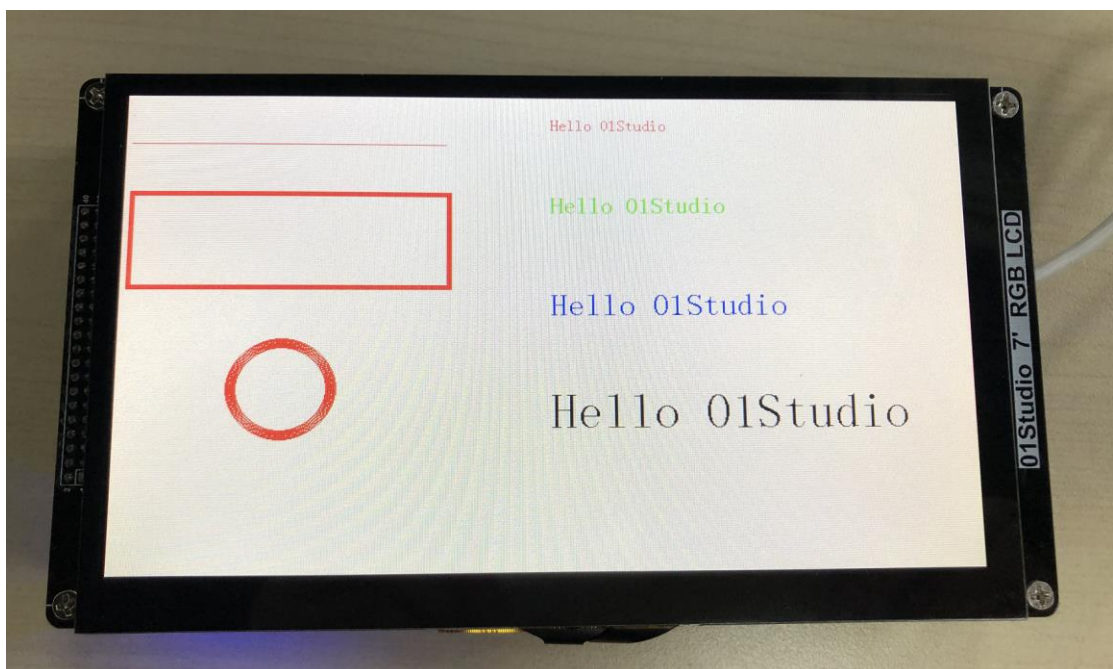


图 4-40



图 4-41 显示图片

● **总结:**

从 4.3 寸到 7 寸，使用 micropython 编程只需要修改简单的初始化构建对象并可轻松切换，通过本实验我们体验到了 LCD 使用 micropython 开发的简单和灵活性。我们轻松实现了 LCD 的各种常规操作，让用户将精力放在应用。相信随着 micropython 库函数的日益成熟，其性能和可玩性将变得更强大！

4.7.3 图片缓存文件制作

- **前言：**

相信做过前面 LCD 显示实验的小伙伴都会发现一个问题，就是 LCD 显示图片的速度比较慢。那是因为 TKM32 通过软件解码 JPG 方式生产字节数据再显示，这样就需要大量的世界在处理解码，导致显示图片时间变长。

那么我们能否将图片预解码并保存为缓存文件，TKM32 直接调用该文件来显示？答案是可以的。按这个思路，我们编写了用于生成图片解码后缓存文件的 MicroPython 库，大大提高了图片的显示速度！

- **实验平台：**

达芬奇 MicroPython 开发套件、4.3 寸 RGB 显示屏（电容触摸）。



图 4-42 4.3 寸 LCD

- **实验目的：**

通过制作图片缓存文件，用于图片显示，提高显示速度。

- **实验讲解：**

在前面 LCD 显示屏实验中我们学会了 4.3 寸 RGB 屏的使用，这里我们重温一下其对象：

构造函数
tftlcd.LCD43R(portrait=1) 构建 4.3 寸 RGB LCD 对象。 【portrait】 设置屏幕方向： ● 1 - 横屏，800*480 ， 默认 ● 2 - 竖屏，480*800 ， 1 基础上顺时针旋转 90° ● 3 - 横屏，800*480 ， 1 基础上顺时针旋转 180° ● 4 - 竖屏，480*800 ， 1 基础上顺时针旋转 270°
使用方法
LCD43R.fill(color) 【color】 RGB 颜色数据； 如(255,0,0)表示红色。
LCD43R.drawPixel(x,y,color) 画点。 【x】 :横坐标， 【y】 :纵坐标， 【color】 :颜色。
LCD43R.drawLine(x0,y0,x1,y1,color) 画线段。 【x0,y0】 :起始坐标， 【x1,y1】 :终点坐标， 【color】 :颜色
LCD43R.drawRect(x,y,width,height,color,border=1,fillcolor=None) 画矩形。 【x,y】 :起始坐标， 【width】 :宽度， 【height】 :高度， 【color】 :颜色， 【border】 :边宽， 【fillcolor】 :填充颜色，默认 None 为不填充
LCD43R.drawCircle(x,y,radius,color,border=1,fillcolor=None) 画圆。

【x,y】:圆心, 【radius】:半径, 【color】:颜色, 【border】:边宽, 【fillcolor】:填充颜色, 默认 None 为不填充
LCD43R.printStr(text,x,y,color,backcolor=None,size=2)
写字符。 【text】:字符, 【x,y】:起始坐标, 【color】:字体颜色; 【backcolor】:字体背景颜色; 【size】:字体尺寸 (1-小号, 2-标准, 3-中号, 4-大号)
LCD43R.Picture(x,y,filename,cached=True)
显示图片。支持图片格式类型: jpg、bmp 【x,y】:起始坐标。 【filename】: 图片路径+名称, 如: "/flash/cat.jpg"、 "/sd/cat.jpg" (flash 表示开发板的板载 flash, sd 表示 sd 卡)。 【cached】: 选择是否使用缓存文件, 默认 True 表示使用。使用缓存文件能提高图片显示速度。
LCD43R.CachePicture(filename,path=filename+'.cache',replace=False)
制作图片缓存文件, 支持图片格式: jpg/bmp, 生成.cache 文件。最大尺寸 800*480; 【cached】: 图片路径+名称, 如: "/flash/cat.jpg"、 "/sd/dog.bmp" (flash 表示开发板的板载 flash, sd 表示 sd 卡)。 【path】: 缓存文件保存路径, 默认 filename + '.cache'表示和图片文件路径一致。文件格式为 '.cache'; 【replace】: 是否覆盖已经存在的缓存文件, 默认 replace=False 表示不覆盖。

表 4-13 4.3 寸 LCD 对象

从上表可以看到，LCD 对象带了【CachePicture】功能，我们只需要预先制作好缓存文件，然后再显示图片即可，下面是编程流程图：

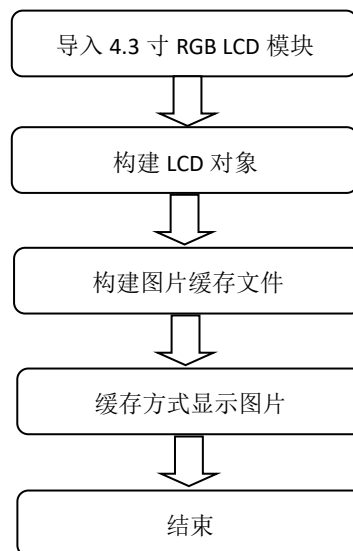


图 4-43 编程流程图

参考 main.py 函数代码如下：

```
'''
实验名称：图片缓存文件制作
版本：v1.0
日期：2021.4
作者：01Studio
社区：www.01studio.org
说明：制作缓存文件用于显示。
实验平台：01Studio-达芬奇
'''

from tftlcd import LCD43R
import time
```

```

#定义常用颜色

RED = (255,0,0)

GREEN = (0,255,0)

BLUE = (0,0,255)

BLACK = (0,0,0)

WHITE = (0,0,0)


#####

# 构建 4.3 寸 RGB LCD 对象并初始化

#####

d = LCD43R(portrait=1) #默认方向,横屏 800*480


#填充白色

d.fill((255,255,255))


#制作缓存文件,使用板载 Flash 制作较慢,使用 SD 卡制作速度非常快。

d.CachePicture("/flash/picture/supper.jpg",replace=False)


d.fill((255,255,255)) #清屏用于观察


#####

## 带计时功能的显示 ##

#####


start=time.ticks_ms() #起始时间


#缓存文件方式显示图片,可以改成 cached=False 对比速度

d.Picture(0,0,"/flash/picture/supper.jpg",cached=True)


end=time.ticks_ms() #结束时间

```



```
print('Total use: '+str(end-start)+' ms') #打印耗时
```

● 实验结果:

将示例程序的素材文件上传到达芬奇开发板（也可以只上传单张图片，注意修改代码中文件的路径即可）。建议使用 SD 卡，制作速度会提升很多。

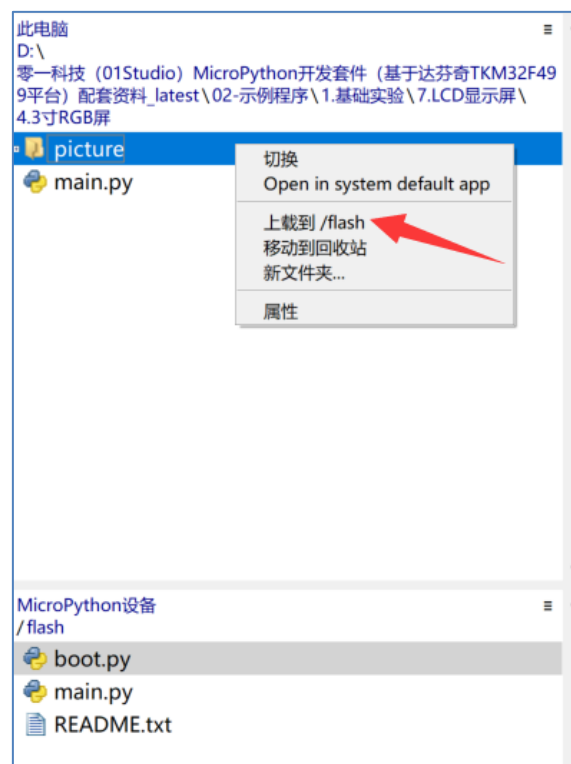


图 4-44 上传图片文件夹到 Flash

运行程序，可以看到 LCD 在执行制作缓存文件，LCD 左上角和 REPL 终端会实时提示进度：

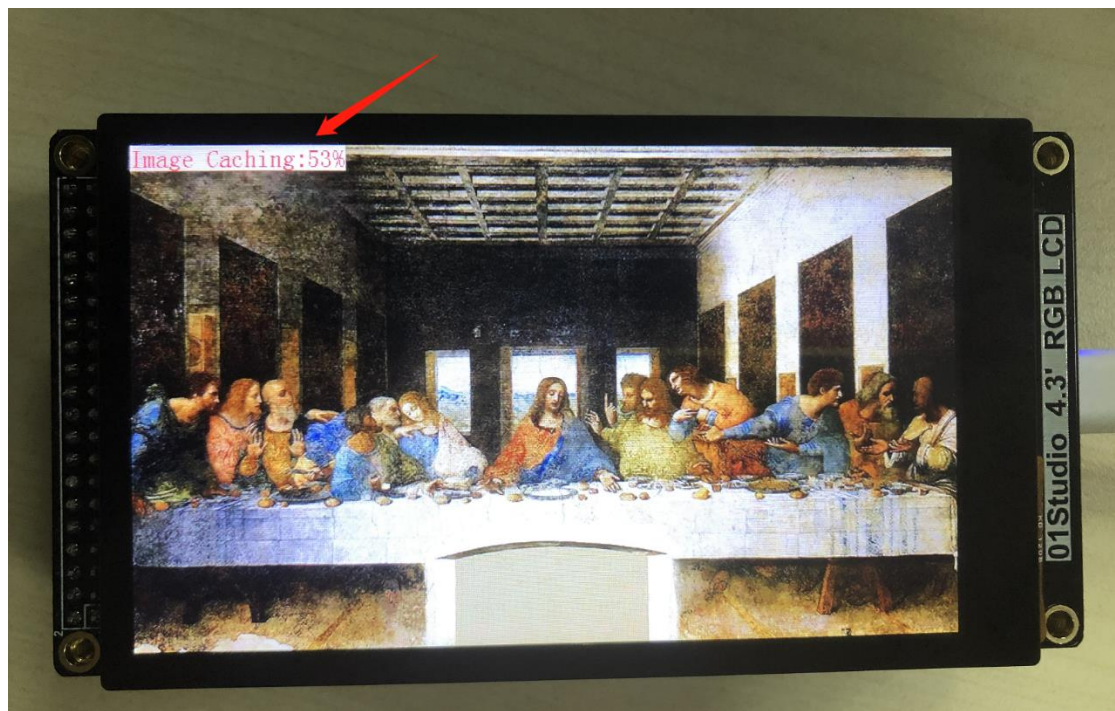


图 4-45 LCD 进度提示

```
Shell x
start loading->picture/supper.jpg
start create cache->picture/supper.jpg.cache
loading:0%
loading:10%
loading:20%
loading:30%
loading:40%
loading:50%
loading:60%
```

MicroPython(一般)

图 4-46 REPL 进度提示

制作完成后可以看到文件系统里多了.cache 缓存文件（没有的话刷新或者复位一下开发板）：

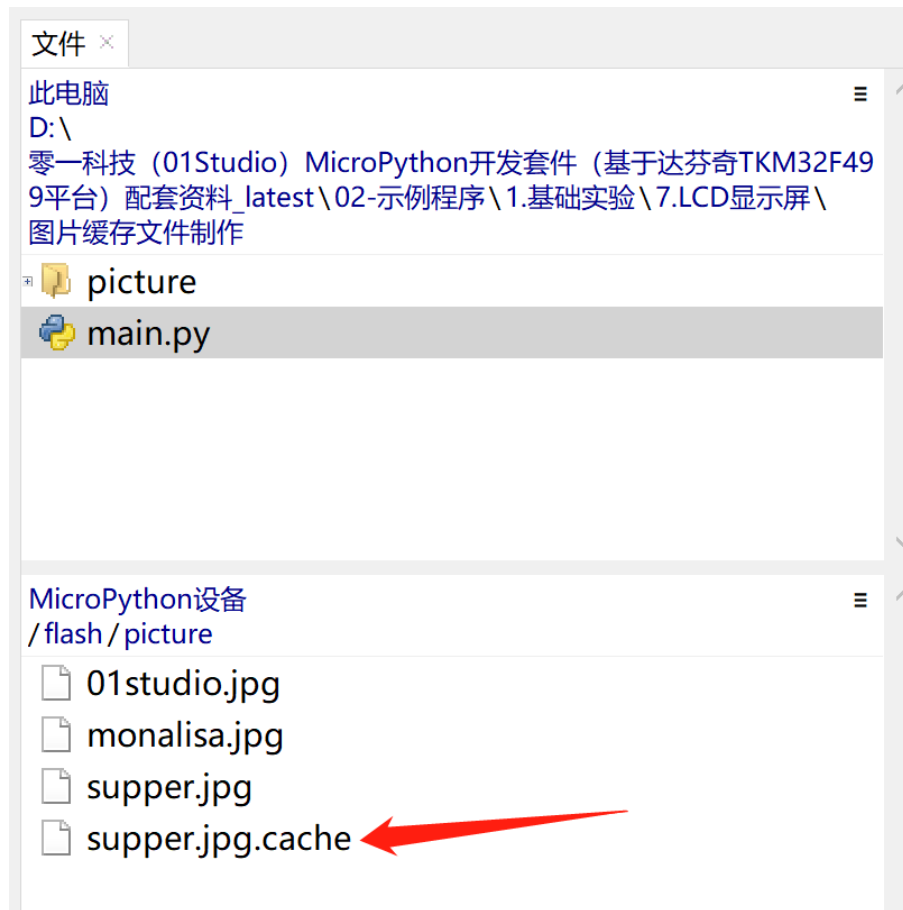


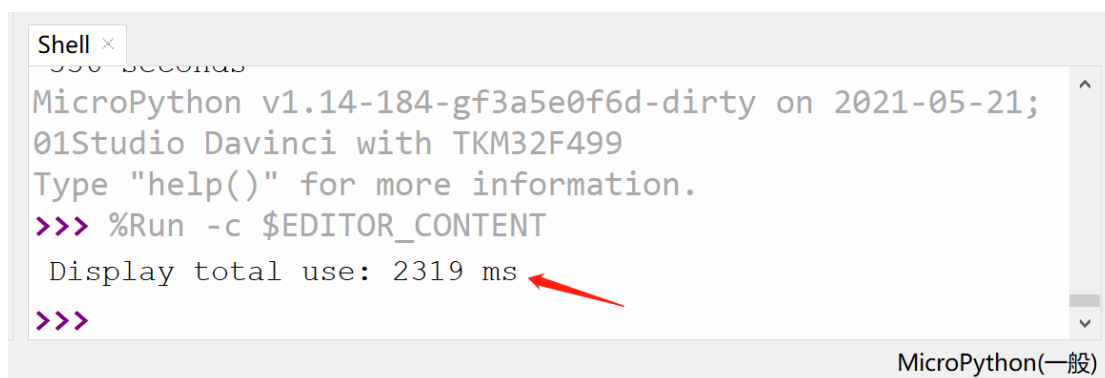
图 4-47 制作好的 cache 图片缓存文件

执行显示图片，使用缓存文件显示：



图 4-48 显示图片

我们可以在 REPL 看到显示耗时约为 2 秒多：

A screenshot of a MicroPython REPL window. The window title is "Shell x". The text inside shows the MicroPython version and environment: "MicroPython v1.14-184-gf3a5e0f6d-dirty on 2021-05-21; 01Studio Davinci with TKM32F499". It prompts the user to type "help()" for more information. Then, it shows a command prompt ">>> %Run -c \$EDITOR_CONTENT" followed by the output "Display total use: 2319 ms". A red arrow points to the "2319 ms" value. The bottom right corner of the window says "MicroPython(一般)".

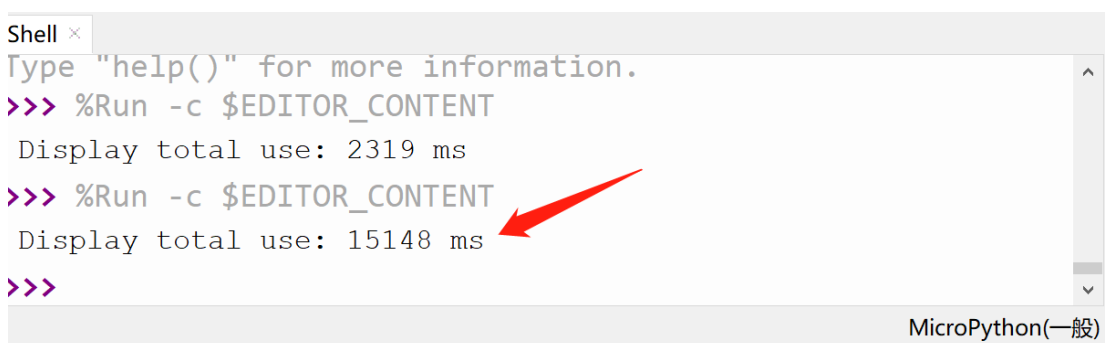
```
Shell x
MicroPython v1.14-184-gf3a5e0f6d-dirty on 2021-05-21;
01Studio Davinci with TKM32F499
Type "help()" for more information.
>>> %Run -c $EDITOR_CONTENT
Display total use: 2319 ms
>>>
```

图 4-49

接下来我们将缓存参数改成不使用缓存文件显示：`cached=False`，即原始的解码显示：

```
#缓存文件方式显示图片，可以改成 cached=False 对比速度
d.Picture(0,0,"/flash/picture/supper.jpg",cached=False)
```

这次可以看到耗时为 15 秒，可见缓存文件显示方式大大提升了刷屏速度。

A screenshot of a MicroPython REPL window. The window title is "Shell x". The text inside shows the same MicroPython version and environment as the previous screenshot. It prompts the user to type "help()" for more information. Then, it shows a command prompt ">>> %Run -c \$EDITOR_CONTENT" followed by the output "Display total use: 2319 ms". Then, it shows another command prompt ">>> %Run -c \$EDITOR_CONTENT" followed by the output "Display total use: 15148 ms". A red arrow points to the "15148 ms" value. The bottom right corner of the window says "MicroPython(一般)".

```
Shell x
Type "help()" for more information.
>>> %Run -c $EDITOR_CONTENT
Display total use: 2319 ms
>>> %Run -c $EDITOR_CONTENT
Display total use: 15148 ms
>>>
```

图 4-50 原始方式显示 JPG 图片耗时

● 总结：

通过图片缓存文件制作并显示，大大提升了刷屏速度，使用 SD 卡效果更好，而且 4.3 寸和 7 寸制作的文件完全通用，只要分辨率一致即可。

4.8 电容触摸屏

上一节我们学习了 LCD 实验，但 LCD 只能显示相关内容，跟人是缺乏交互的。好比我们的智能手机，如果只有显示不能触碰，那么就没有可玩性了。因此本节学习一下触摸屏的使用。

4.8.1 4.3 寸 RGB 屏触摸

● 前言：

本节讲解达芬奇 + 4.3 寸 RGB 屏的电容触摸屏使用方法。

● 实验平台：

达芬奇 MicroPython 开发套件、4.3 寸 RGB 显示屏（电容触摸）。



图 4-51 4.3 寸 LCD(电容触摸)

● 实验目的：

获取触摸屏的坐标并画点标记。

● 实验讲解：

01Studio 达芬奇开发板配套的 4.3 寸 RGB LCD 上带 1 个电容触摸屏,驱动芯片为 FT5436。当手指按下时候，通过简单的编程即可返回一个坐标，我们来看看其 micropython 构造函数和使用方法：

构造函数
touch.FT5436(portrait=1)
构建触摸屏对象。 FT5436 表示驱动芯片型号。
【portrait】 设置屏幕方向：

● 1 - 横屏，800*480 ， 默认 ● 2 - 竖屏，480*800 ， 1 基础上顺时针旋转 90° ● 3 - 横屏，800*480 ， 1 基础上顺时针旋转 180° ● 4 - 竖屏，480*800 ， 1 基础上顺时针旋转 270°
使用方法
FT5436.tick_inc()
刷新触摸。
FT5436.read()
读取触摸屏数据，返回（states,x,y） 【states】-当前触摸状态：0：按下；1：移动；2：松开。 【x】：触摸横坐标 【y】：触摸纵坐标

表 4-14 FT5436 触摸对象

学会了触摸对象用法后，我们可以编程实现触摸后屏幕打点表示，然后左上角显示当前触摸的坐标。另外再加入一个按键，按下清空屏幕。编程流程图如下：

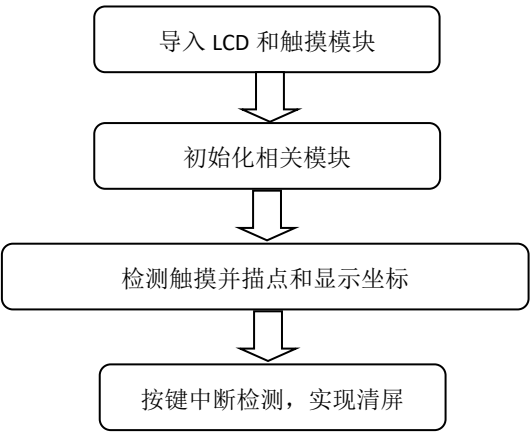


图 4-52 编程流程图

参考代码如下：

```
...
实验名称：电容触摸屏
```

版本: v1.0

日期: 2021.4

作者: 01Studio

社区: www.01studio.org

说明: 电容触摸屏采集触摸信息

...

```
from touch import FT5436
```

```
from tftlcd import LCD43R
```

```
from machine import Pin
```

```
import time
```

```
#定义颜色
```

```
BLACK = (0,0,0)
```

```
WHITE = (255,255,255)
```

```
RED=(255,0,0)
```

```
BLUE=(0,0,255)
```

```
#LCD 初始化
```

```
d = LCD43R(portrait=1)
```

```
d.fill(WHITE)#填充白色
```

```
#电容触摸屏初始化，方向和 LCD 一致
```

```
t = FT5436(portrait=1)
```

```
#配置按键
```

```
key = Pin('A0', Pin.IN, Pin.PULL_DOWN)
```

```
#清屏函数
```

```
def fun(key):
```



```

d.fill(WHITE)

key.irq(fun,Pin.IRQ_RISING) #定义中断，下降沿触发

while True:

    data = t.read() #获取触摸屏坐标

    #当产生触摸时

    if data[0]!=2: #0: 按下; 1: 移动; 2: 松开

        print(data) #REPL 打印坐标信息

        #触摸坐标画圆

        d.drawCircle(data[1], data[2], 10, BLACK, fillcolor=BLACK)

        d.printStr('(X:'+str(data[1])+ ' Y:'+str(data[2])+')  ',
                    10,10,RED,size=3)

    time.sleep_ms(20) #间隔 20ms

```

- **实验结果:**

运行程序，用手指触摸屏幕或者在屏幕上滑动，可以看到描点并在 LCD 左上角显示当前坐标。

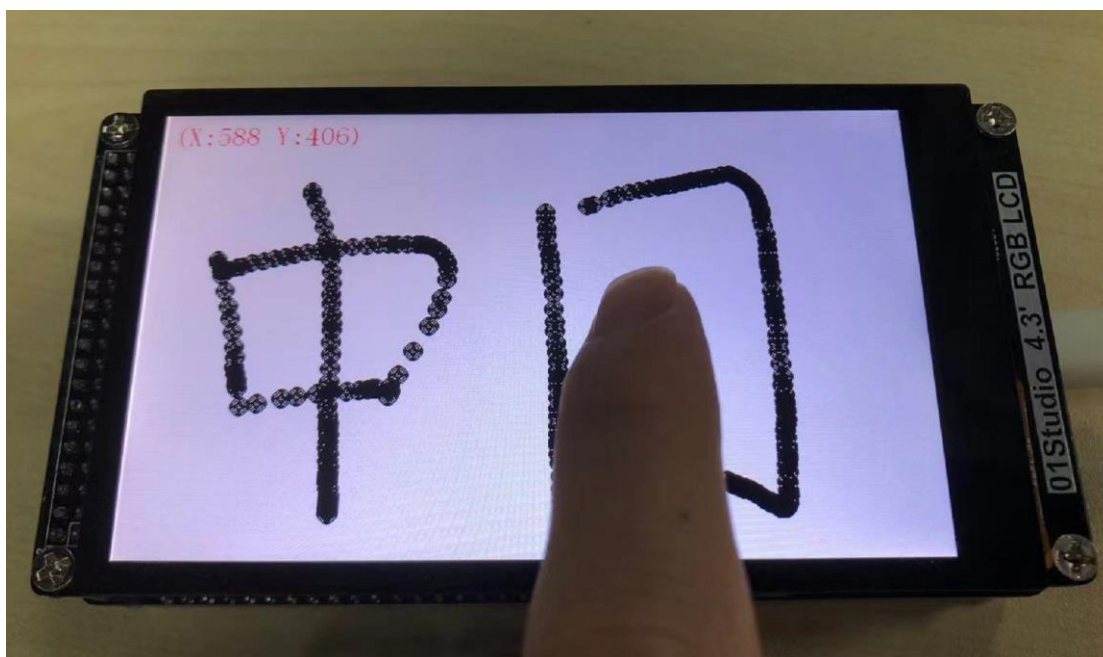


图 4-53 触摸实验

按下 A0 按键清空屏幕内容：



图 4-54 清屏

● **总结：**

没有触摸屏的 LCD 就失去了灵魂，有了触摸屏，跟开发板的交互就变得有意思了。

4.8.2 7 寸 RGB 屏触摸

- **前言：**

本节讲解达芬奇 + 7 寸 RGB 屏的电容触摸屏使用方法。

- **实验平台：**

达芬奇 MicroPython 开发套件、7 寸 RGB 显示屏（电容触摸）。



图 4-55 4.3 寸 LCD(电容触摸) 替换

- **实验目的：**

获取触摸屏的坐标并画点标记。

- **实验讲解：**

01Studio 达芬奇开发板配套的 7 寸 RGB 上带 1 个电容触摸屏，驱动芯片为 GT911。当手指按下时候，通过简单的编程即可返回一个坐标，我们来看看其 micropython 构造函数和使用方法：

构造函数
touch.GT911(portrait=1)
构建触摸屏对象。 GT911 表示驱动芯片型号。
【portrait】 设置屏幕方向： ● 1 - 横屏，800*480 ， 默认 ● 2 - 竖屏，480*800 ， 1 基础上顺时针旋转 90° ● 3 - 横屏，800*480 ， 1 基础上顺时针旋转 180° ● 4 - 竖屏，480*800 ， 1 基础上顺时针旋转 270°
使用方法
GT911.tick_inc()
刷新触摸。
GT911.read()
读取触摸屏数据，返回（states,x,y） 【states】 - 当前触摸状态：0： 按下；1： 移动；2： 松开。 【x】 : 触摸横坐标 【y】 : 触摸纵坐标

表 4-15 GT911 触摸对象

学会了触摸对象用法后，我们可以编程实现触摸后屏幕打点表示，然后左上角显示当前触摸的坐标。另外再加入一个按键，按下清空屏幕。编程流程图如下：

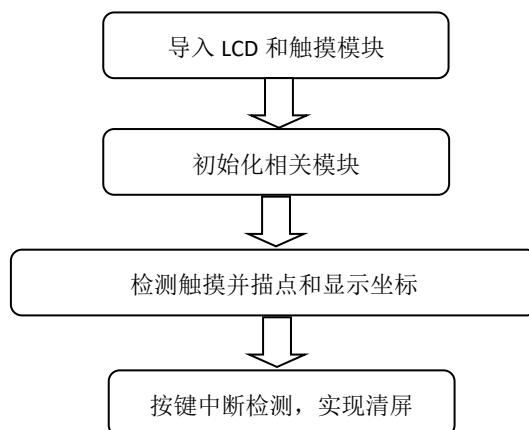


图 4-56 编程流程图

参考代码如下：

```
'''
实验名称：7 寸电容触摸屏
版本：v1.0
日期：2021.4
作者：01Studio
社区：www.01studio.org
说明：电容触摸屏采集触摸信息
'''

from touch import GT911
from tftlcd import LCD7R
from machine import Pin
import time

#定义颜色
BLACK = (0,0,0)
WHITE = (255,255,255)
RED=(255,0,0)
BLUE=(0,0,255)

#LCD 初始化
d = LCD7R(portrait=1)
d.fill(WHITE)#填充白色

#电容触摸屏初始化，方向和 LCD 一致
t = GT911(portrait=1)

#配置按键
```

```

key = Pin('A0', Pin.IN, Pin.PULL_DOWN)

#清屏函数
def fun(key):

    d.fill(WHITE)

key.irq(fun,Pin.IRQ_RISING) #定义中断，下降沿触发

while True:

    data = t.read() #获取触摸屏坐标

    #当产生触摸时
    if data[0]!=2: #0: 按下; 1: 移动; 2: 松开

        print(data) #REPL 打印坐标信息

        #触摸坐标画圆
        d.drawCircle(data[1], data[2], 10, BLACK, fillcolor=BLACK)
        d.printStr('(X:'+str(data[1])+ ' Y:'+str(data[2])+')  ',
                    10,10,RED,size=3)

    time.sleep_ms(20) #间隔 20ms

```

● 实验结果:

运行程序，用手指触摸屏幕或者在屏幕上滑动，可以看到描点并在 LCD 左上角显示当前坐标。

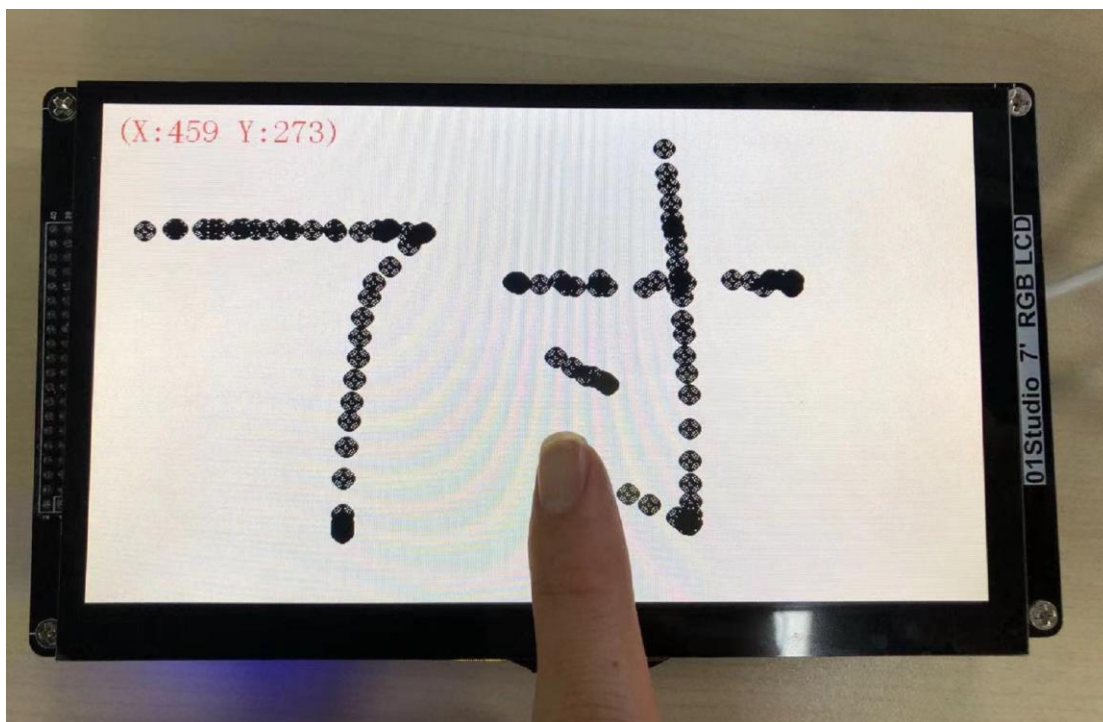


图 4-57 触摸实验

按下 A0 按键清空屏幕内容：



图 4-58 清屏

● **总结：**

没有触摸屏的 LCD 就失去了灵魂，有了触摸屏，跟开发板的交互就变得有意思了。

4.9 触摸屏按钮

- **前言：**

上两节我们分别学习了 LCD 显示和触摸屏操作，这一节我们就来整合一下，做一个好玩的东西，那就是生成触摸按钮。由于达芬奇开发板上只有 3 个功能按键，而使用 LCD 和触摸屏则可以创建非常多的按钮来执行我们的任务。而且操作更直观。

- **实验平台：**

达芬奇 MicroPython 开发套件、4.3 寸 LCD 显示屏（电容触摸）。

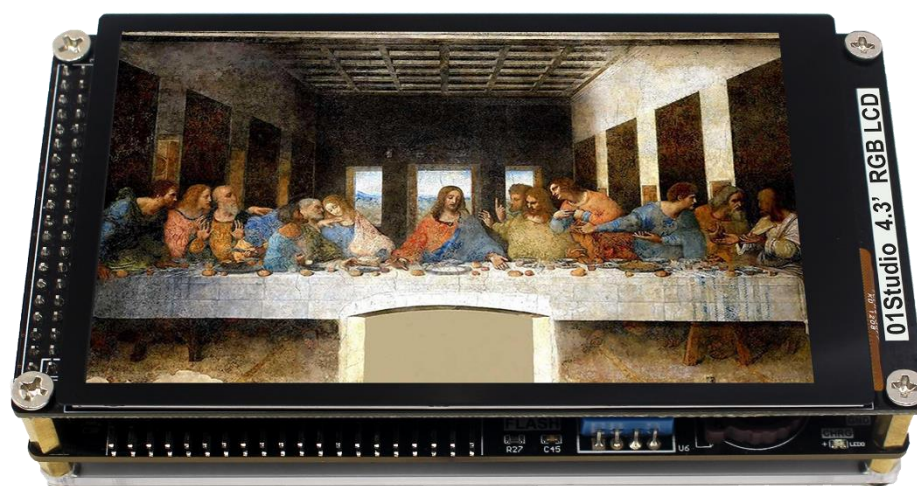


图 4-59 4.3 寸 LCD(电容触摸)

- **实验目的：**

生成触摸按钮，实现触摸控制 LED 灯状态变换。

- **实验讲解：**

思考一下学习到现在你，会用什么方法来实现这个功能呢？你或许会想在 LCD 上画一个填充矩形，然后获取触摸坐标，跟这个矩形位置对上后就判断这个按钮被按下了。

没错，这个原理是可行的，看似很简单，但我们还需考虑很多问题，比如触摸后用什么方式执行任务效率最高？如果区分不同按钮以及它们之间是否存在

冲突。当你这么思考的时候，就是在开始构建一个简单 GUI（图形用户界面）。
还是那句，人生苦短，01studio 已经将底层封装好，用户只需要直接学会对象的使用即可。我们来看看触摸按钮对象：

构造函数
gui.TouchButton(x,y,width,height,color,label,label_color,callback)
构建触摸对象；定义在 gui 模块下。 【x】按钮起始横坐标； 【y】按钮起始纵坐标； 【width】按钮宽度； 【height】按钮高度； 【color】按钮颜色； 【label】按钮标签； 【label_color】按钮标签颜色； 【callback】按钮触发的回调函数，按下松开后触发。 注意：当前最多定义 20 个，软件复位不会释放按键编号。
使用方法
gui.task_handler()
执行所有任务。也就是执行按钮回调函数里面的代码。
TouchButton.ID()
获取当前触摸 ID 的编号。

表 4-16 触摸按钮对象

从上表可以看到只需要简单的语句便实现了触摸按钮的构建，我们可以构建 4 个按钮控制 4 路 LED，编程思路如下：

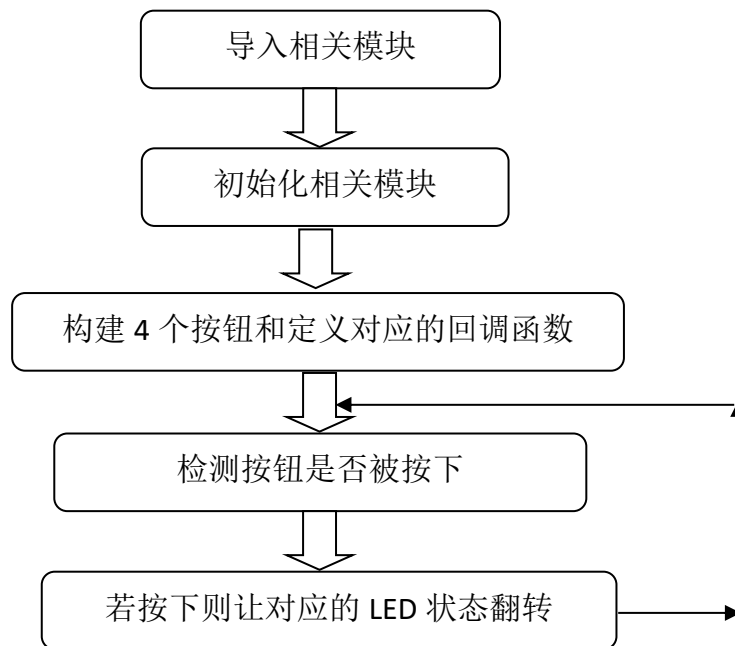


图 4-60 代码编写流程

参考代码如下：

```
'''
实验名称：触摸按钮
版本：v1.0
日期：2021.4
作者：01Studio
社区：www.01studio.org
说明：编程实现触摸按钮控制 LED。
实验平台：01Studio 达芬奇
'''

#导入相关模块
from tftlcd import LCD43R
from touch import FT5436
from machine import LED,Timer
import gui,time
```

```

#定义常用颜色

BLACK = (0,0,0)

WHITE = (255,255,255)

RED = (255,0,0)

GREEN = (0,255,0)

BLUE = (0,0,255)

ORANGE =(0xFF,0x7F,0x00) #橙色


#LCD 初始化

d = LCD43R() #默认方向

d.fill(WHITE) #填充白色


#触摸屏初始化

t = FT5436() #默认方向


#####

#定义 4 个按键和回调函数

#####

def fun1(B1):

    LED(1).toggle() #LED3 状态翻转


def fun2(B2):

    LED(2).toggle() #LED4 状态翻转


def fun3(B3):

    LED(3).toggle() #LED3 状态翻转


def fun4(B4):

    LED(4).toggle() #LED4 状态翻转

```

```

B4 = gui.TouchButton(40,50,150,80,BLUE,'LED4',WHITE,fun4)
B3 = gui.TouchButton(230,50,150,80,ORANGE,'LED3',WHITE,fun3)
B2 = gui.TouchButton(420,50,150,80,GREEN,'LED2',WHITE,fun2)
B1 = gui.TouchButton(610,50,150,80,RED,'LED1',WHITE,fun1)

#####

#### 定时器用于扫描按钮触发事件 ##

#####

tim_flag = 0

def count(tim):
    global tim_flag
    tim_flag = 1

#构建软件定时器，编号-1
tim = Timer(-1)
tim.init(period=20, mode=Timer.PERIODIC,callback=count) #周期为 20ms

while True:

    #执行按钮触发的任务

    if tim_flag == 1:

        t.tick_inc()
        gui.task_handler()
        tim_flag = 0

```

提示：Micropython 所有中断回调函数都不支持新的内存分配语句，否则会报错，因此可以通过定义全局变量（**global**），在回调函数修改，然后在主循环函数判断然后做出相应的控制。

- **实验结果:**

运行代码, 可以看到 LCD 显示屏上生成了 4 个按钮, 分别点击每个按钮, 可以看到对应的 LED 状态发生翻转。



图 4-61 触摸按钮

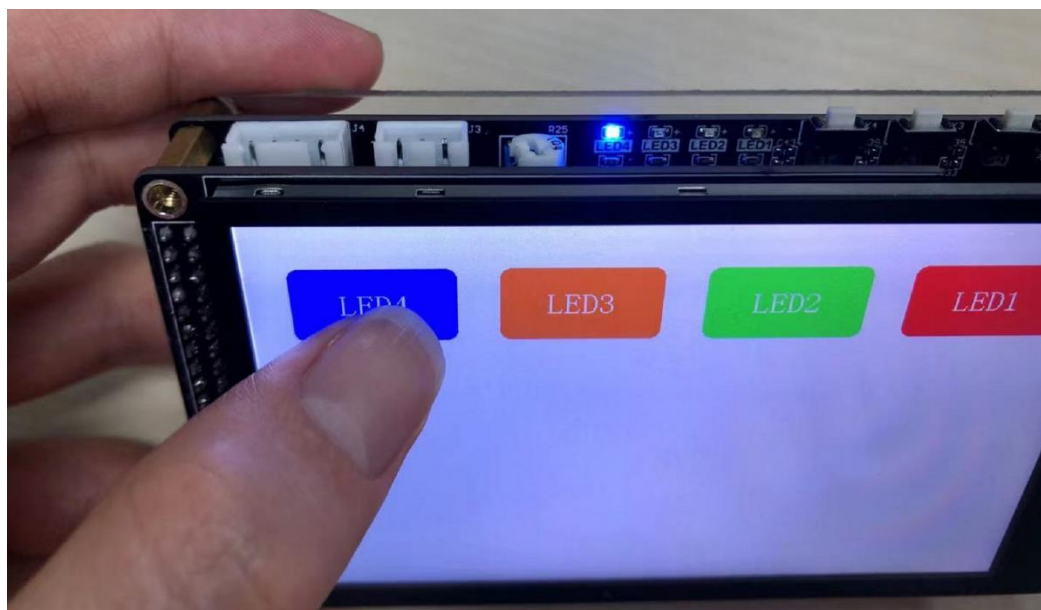


图 4-62 控制 LED

- **总结:**

触摸按钮实现简单但却是用途非常广泛的功能, 有了自定义按键, 我们就可以通过触摸按键来实现所有外设设备的交互。让设备的控制变得更简单有趣。当前颜色、尺寸、标签等都为可修改项, 大大提高了用户编程的灵活性。

4.10 ADC

- **前言:**

ADC(analog to digital conversion) 模拟数字转换。意思就是将模拟信号转化成数字信号, 由于单片机只能识别数字信号, 所以外界模拟信号常常会通过 ADC 转换成其可以识别的数字信息。常见的应用就是将变化的电压转成数字信号。

- **实验平台:**

达芬奇 MicroPython 开发套件和 4.3 寸 RGB LCD 显示屏。



图 4-63

达芬奇 MicroPython 开发套件

- **实验目的:**

通过编程调用 MicroPython 的内置 ADC 函数, 实现测量 0-3.3V 电压, 并显示到 LCD 屏幕上。

- **实验讲解:**

从原理图可以看到, 达芬奇主控的 ADC_4 引脚连接到了电位器, 通过电位器的调节可以使得 ADC_4 引脚上的电压变化范围实现从 0-3.3V。

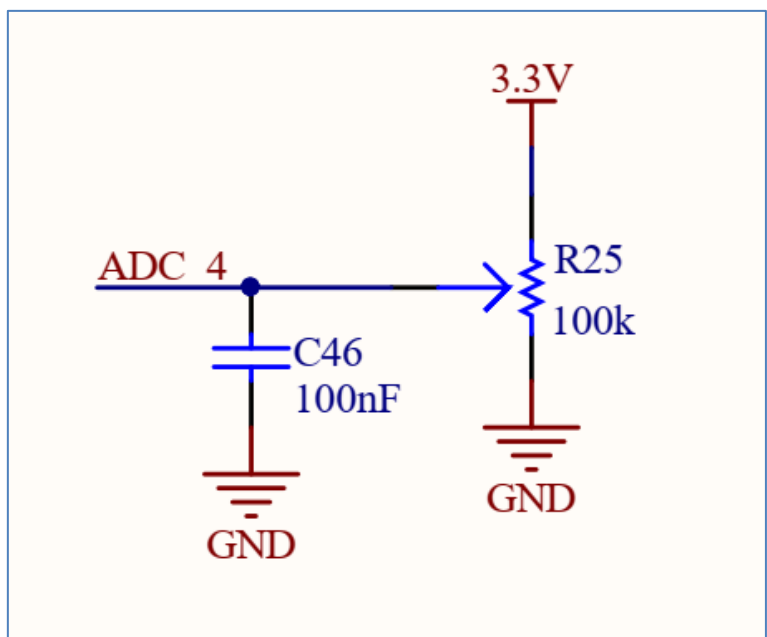


图 4-64 电位器原理图

在了解了原理图后，我们来看看 micropython 内置 ADC 模块的函数使用方法。

构造函数
<code>adc=machine.ADC(id)</code>
构建 ADC 对象； 【id】TKM32F499 的 ADC 引脚编号；一共有 6 路: ADC_4、ADC_5、PB0、PB1、PB2、PB3
使用方法
<code>adc.read_16()</code>
获取 ADC 值。测量精度是 12 位(注意返回的是 0-65535)（表示 0-3.3V）。

表 4-17 ADC 对象

你没看错，就这么简单。两句函数就可以获得 AD 数值，让我们来理顺一下编程逻辑。先导入相关模块，然后初始化模块。在循环中不断读取 ADC 的值，转化成电压值后在 LCD 上面显示，每隔 0.5 秒读取一次，具体如下：

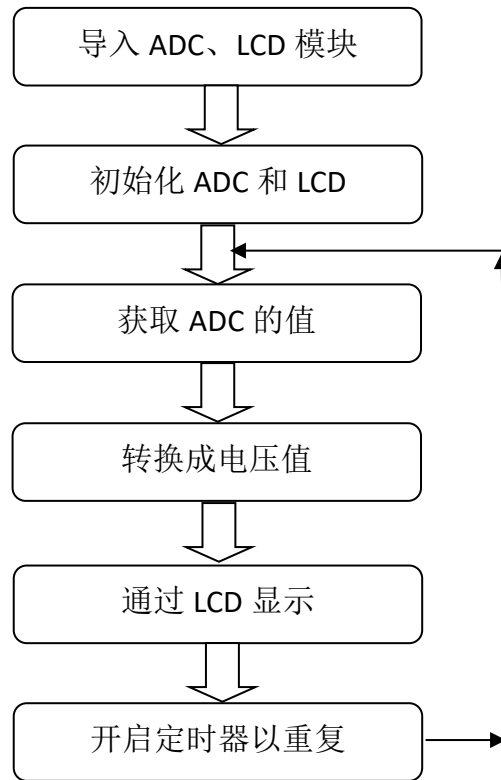


图 4-65 代码编写流程

实验参考代码：

```
'''
实验名称：ADC-电压测量
版本：v1.0
日期：2021.4
作者：01Studio
说明：通过对 ADC 数据采集，转化成电压在显示屏上显示。
      ADC 精度 12 位，电压 0-3.3V。
'''

#导入相关模块
from machine import ADC
from tftlcd import LCD43R
import time
```



```

#定义常用颜色

RED = (255,0,0)

GREEN = (0,255,0)

BLUE = (0,0,255)

BLACK = (0,0,0)

WHITE = (255,255,255)

#####

# 构建 4.3 寸 LCD 对象并初始化

#####

d = LCD43R(portrait=1) #默认方向


d.fill(WHITE)#填充白色


#构建 ADC 对象, 引脚 PA5

adc = ADC('ADC_4')


#显示标题

d.printStr('01Studio ADC', 100, 10, BLUE, size=4)


while True:


    #电压采集

    value = adc.read_u16() #原始值

    vol = str('%.2f'%(value/4095*3.3)) #电压值, 0-3.3V


    #显示测量值和电压值

    d.printStr('Vol: '+vol+" V", 10, 100, BLACK, size=4)

    d.printStr('Value: '+str(value)+"   ", 10, 200, BLACK, size=4)

```

```
d.printStr("(4095)", 300, 200, BLACK, size=4)
print(value)

time.sleep_ms(500) #延时 500ms
```

● 实验结果:

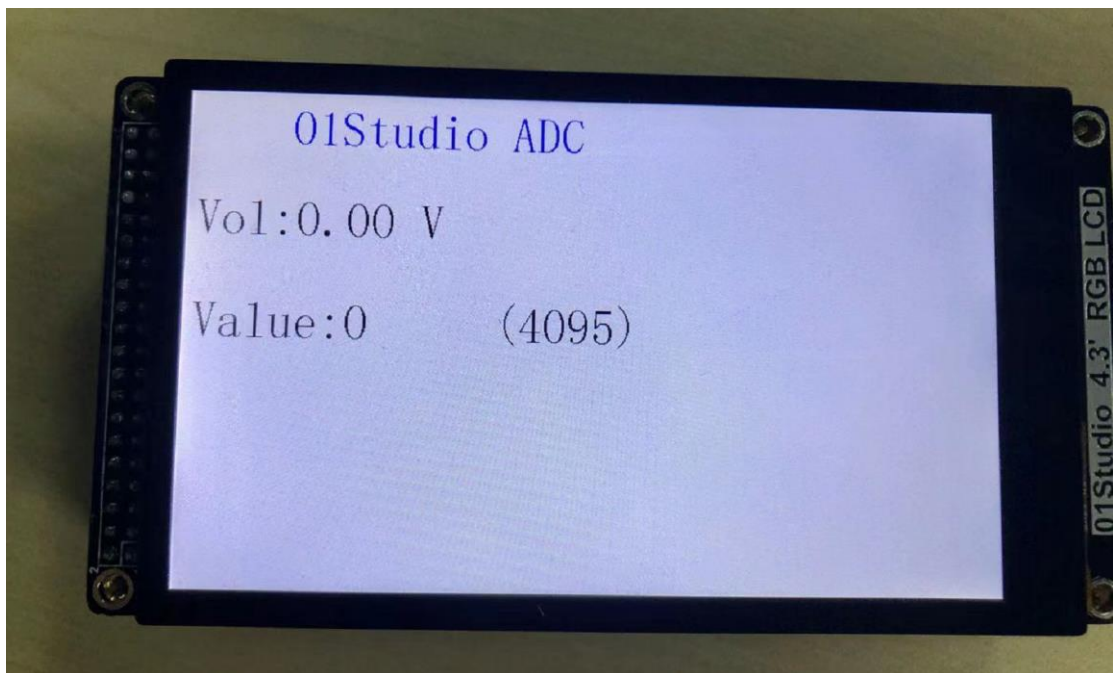


图 4-66 电位器顺时针拧到尽头是 0V

通过调节电位器，可以发现电压在不断变化。



图 4-67 调节电位器来改变电压输入

● **总结：**

这一节我们学习了 ADC 的应用。主要用于电压的检测，达芬奇上有很多 ADC 引脚，这意味着可以实现多路同时检测。有兴趣的小伙伴可以自行研究。

4.11 RTC 实时时钟

- **前言：**

时钟可以说我们日常最常用的东西了，手表、电脑、手机等等无时无刻不显示当前的时间。可以说每一个电子爱好者心中都希望拥有属于自己制作的一个电子时钟，接下来我们就用达芬奇开发板来制作一个属于自己的电子时钟。

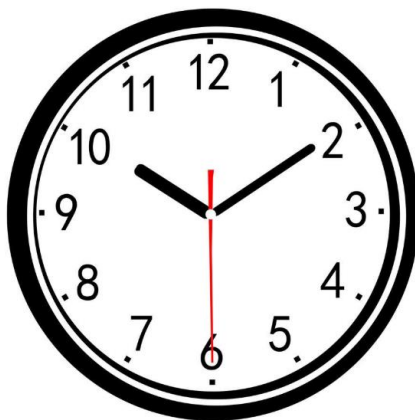


图 4-68 时钟

- **实验平台：**

达芬奇 MicroPython 开发套件。



图 4-69 MicroPython 开发套件

- **实验目的：**

学习 RTC 编程和制作电子时钟，使用 LCD 显示。

- **实验讲解：**

实验的原理是读取 RTC 数据，然后通过 OLED 显示。毫无疑问，强大的 micropython 已经集成了内置时钟函数模块。位于 machine 的 RTC 模块中，具体如下：

构造函数
<code>rtc=machine.RTC()</code>
构建 RTC 对象。
使用方法
<code>rtc.datetime((2021, 4, 1, 3, 0, 0, 0, 0))</code>
设置日期和时间。按顺序分别是：（年，月，日，星期，时，分，秒，微秒），其中星期使用 0-6 表示周一至周日。
<code>rtc.datetime()</code>
获取当前日期和时间。

表 4-18 RTC 实时时钟对象

从上表可以看到 RTC() 的使用方法，我们需要做的就是先设定时间，然后再获取当前芯片里的时间，通过 LCD 显示屏显示，如此循环。在循环里，如果一直获取日期时间数据会造成资源浪费，所以可以每隔第一段时间获取一次数据，又由于肉眼需要看到至少每秒刷新一次即可，这里每隔 300ms 获取一次数据，所以具体流程如下：

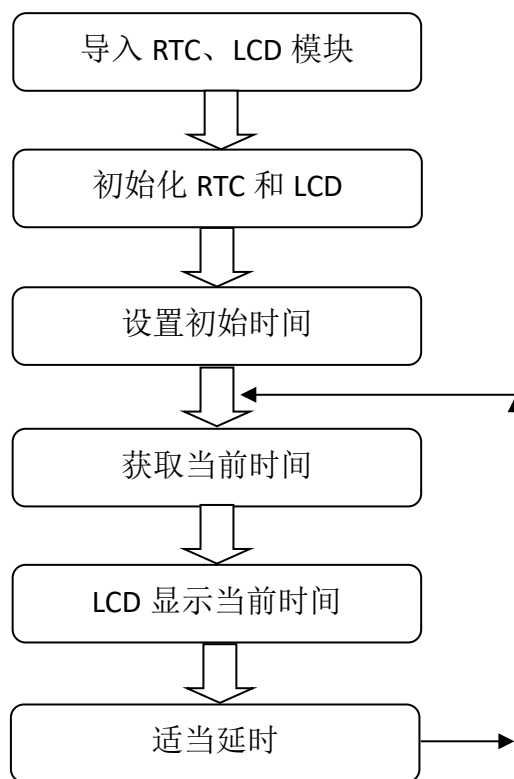


图 4-70 代码编写流程

参考代码如下：

```
...  
  
实验名称：RTC 实时时钟  
版本：v1.0  
日期：2021.5  
作者：01Studio  
说明：在 LCD 上显示时间(使用 Thonny IDE 连接开发板会自动同步时间！)  
社区：www.01studio.org  
实验平台：01Studio 达芬奇  
...  
  
#导入相关模块  
from machine import RTC  
from tftlcd import LCD43R  
import time
```

```

#定义常用颜色

RED = (255,0,0)

GREEN = (0,255,0)

BLUE = (0,0,255)

BLACK = (0,0,0)

WHITE = (255,255,255)

#####

# 构建 4.3 寸 LCD 对象并初始化

#####

d = LCD43R(portrait=1) #默认方向

d.fill(WHITE)#填充白色


#初始化 RTC

rtc = RTC()


# 定义星期和时间（时分秒）显示字符列表

week = ['Mon', 'Tues', 'Wed', 'Thur', 'Fri', 'Sat', 'Sun']

Time = ['', '', '']


#显示标题

d.printStr('01Studio RTC', 100, 10, BLUE, size=4)


# 首次上电配置时间，按顺序分别是：年，月，日，星期，时，分，秒，微秒；

# 这里做了一个简单的判断，检查到当前年份不对就修改当前时间，开发者可以根据自己

# 实际情况来修改。

# thonny 连接开发板后会自动更新开发板时间。

```

```

# if rtc.datetime()[0] != 2021:
#     rtc.datetime((2021, 4, 1, 4, 0, 0, 0, 0))

while True:

    datetime = rtc.datetime() # 获取当前时间

    # 显示日期，字符串可以直接用“+”来连接
    d.printStr(str(datetime[0]) + '-' + str(datetime[1]) + '-' +
               str(datetime[2]) + ' ' + week[(datetime[3] - 1)], 10,
               100, BLACK, size=4)

    # 显示时间需要判断时、分、秒的值否小于 10，如果小于 10，则在显示前面补“0”
    # 以到较佳的显示效果
    for i in range(4, 7):
        if datetime[i] < 10:
            Time[i - 4] = "0"
        else:
            Time[i - 4] = ""

    # 显示时间
    d.printStr(Time[0] + str(datetime[4]) + ':' + Time[1] +
               str(datetime[5]) + ':' + Time[2] + str(datetime[6]), 10,
               200, BLACK, size=4)

    time.sleep_ms(300) # 延时 300 毫秒

```


- **实验结果:**

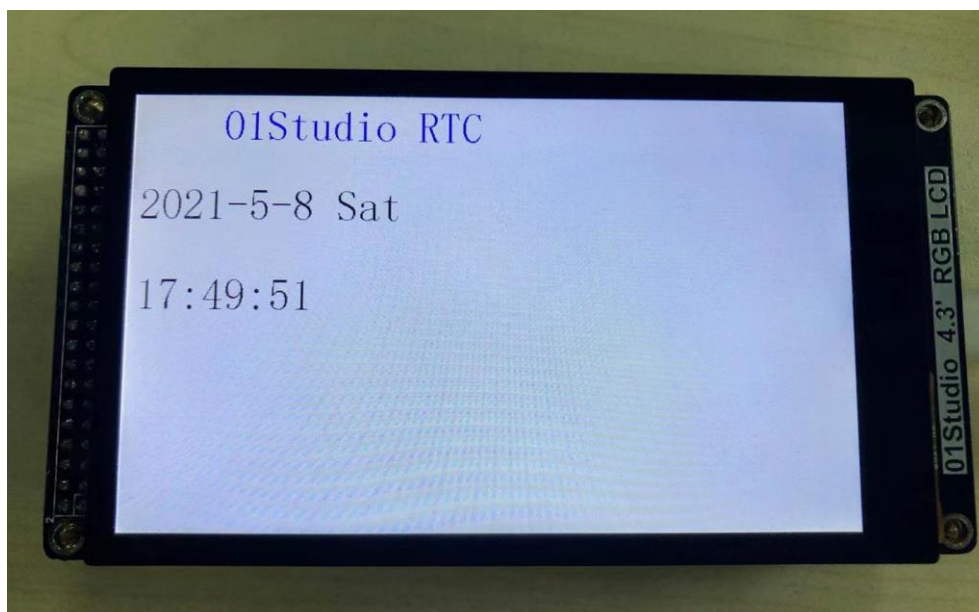


图 4-71 RTC 实时时钟实验

细心的用户或者会发现开发板的正下方有个小的纽扣电池, 这个小电池的作用是后备电池, 装上后开发板断电后时钟会继续跑, 重新上电后时间是最新的。如果没有这电池, 那么开发板断电后时钟无法继续运行。

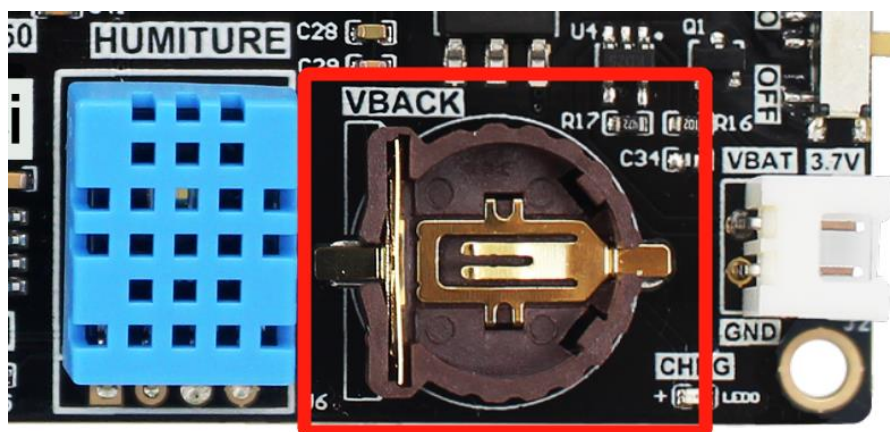


图 4-72 后备电池 (VBACK)

- **总结:**

RTC 实时时钟的可玩性很强, 我们还可以根据自己的风格来设定数字显示位置, 以及加上一些属于自己的字符标识。打造自己的电子时钟。

4.12 OLED 显示屏

- **前言:**

除了之前学习的 4.3 寸 LCD 大屏，今天我们来学习一下小的 OLED 显示屏。本节的 OLED 显示屏学习，实际上是在使用 I2C 的总线接口，达芬奇主控是通过 I2C 总线与 OLED 显示屏通讯的。学完本节，你就多了一个显示屏选择了。

- **实验平台:**

达芬奇 MicroPython 开发套件+0.9 寸 OLED 显示屏。

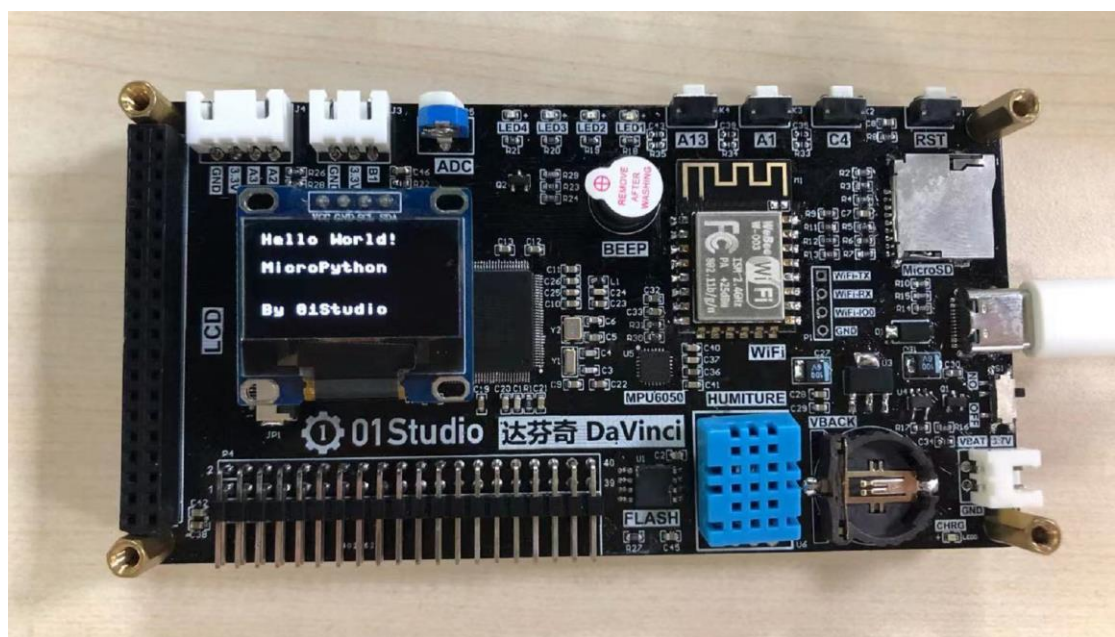


图 4-73 达芬奇 MicroPython 开发套件

- **实验目的:**

学习使用 MicroPython 的 I2C 总线通讯编程和 OLED 显示屏的使用。

- **实验讲解:**

什么是 I2C?

I2C 是用于设备之间通信的双线协议，在物理层面，它由 2 条线组成：SCL 和 SDA，分别是时钟线和数据线。也就是说不通设备间通过这两根线就可以进行通信。

什么是 OLED 显示屏？

OLED 的特性是自己发光，不像 TFT LCD 需要背光，因此可视度和亮度均高，

其次是电压需求低且省电效率高，加上反应快、重量轻、厚度薄，构造简单，成本低等特点。简单来说跟传统液晶的区别就是里面像素的材料是由一个个发光二极管组成，因为密度不高导致像素分辨率低，所以早期一般用作户外 LED 广告牌。随着技术的成熟，使得集成度越来越高。小屏也可以制作出较高的分辨率。



图 4-74 I2C 接口的 OLED

在了解完 I2C 和 OLED 显示屏后，我们先来看看开发板的原理图，也就是 MicroPython 上的 OLED 接口是如何连线的。

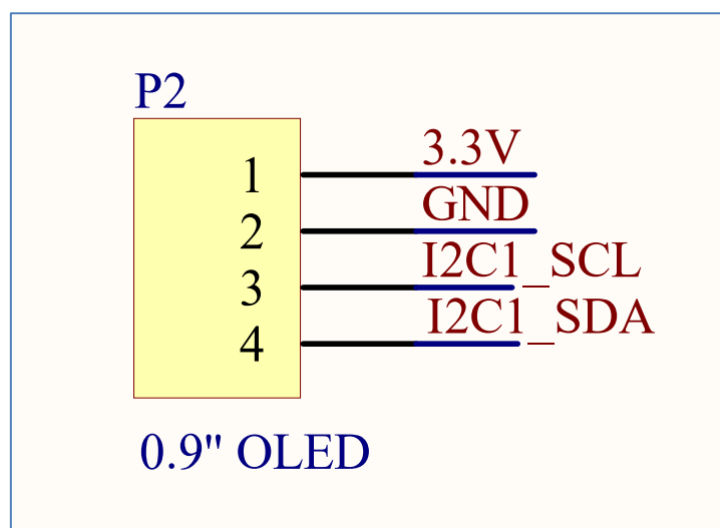


图 4-75 OLED 接口原理图

	HSYNC	PB6	24	PB6/TIM4_CH1/HSYNC
	PCLK	PB5	25	PB5/TIM4_CH2/PCLK/CAN2_TX
	DE	PB4	26	PB4/TIM4_CH3/DE/CAN2_RX
I2C1 SDA	T SDA	PB0	27	IIC1_SDA/SPI1_MOSI/Y-/PB0
		PB1	28	SPI1_MISO/X-/PB1
I2C1 SCL	T SCL	PB2	29	IIC1_SCK/SPI1_SCK/Y+/PB2
	T RST	PB3	30	SPI1_CS/X+/PB3
	GND		31	VSS
			32	

图 4-76

从上图可见连接 OLED 的别是 PB0→SDA 和 PB2→SCL。我们可以通过软件模拟 I2C 方式来实现 OLED 的数据通信和使用。本例程将使用 MicroPython 的 Machine 模块来定义 Pin 口和 I2C 初始化。具体如下：

先来看看 Machine 模块里面 Pin 的用法：

构造函数
machine.Pin(id,.....)
构造 Pin 对象； 【id】引脚编码；
使用方法
直接使用。如 machine.Pin("B0")，B0 表示引脚 PB0

表 4-19 Machine 的 Pin 对象

Machine 模块下 I2C 的用法：

构造函数
machine.I2C(SDA,SCL,
自定义 I2C 接口。
使用方法
machine.I2C.scan()
返回扫描到的 I2C 设备地址，从 0x08 至 0x77 之间
machine.I2C.write(buf)
写数据到 I2C 设备。

定义好 I2C 后，还需要驱动一下 OLED。这里我们已经写好了 OLED 的库函数，在 `ssd1306.py` 文件里面。开发者只需要拷贝到达芬奇开发板的文件系统里面，然后在 `main.py` 里面调用函数即可。人生苦短，我们学会调用函数即可，也就是注重顶层的应用，想深入的小伙伴也可以自行研究 `ssd1306.py` 文件代码。OLED 显示屏的使用方法如下：

构造函数
<code>SSD1306_I2C(width,height,i2c,addr,.....)</code>
OELD 初始化： 【width】：屏幕宽像素； 【height】：屏幕高像素； 【i2c】：已经定义的 I2C 总线对象 【addr】：I2C 设备地址
使用方法
<code>SSD1306_I2C.text("string",width,height)</code>
将 “string”在写在指定位置 【string】：写入的内容 【width】横坐标 【height】纵坐标
<code>SSD1306_I2C.show()</code>
显示内容
<code>SSD1306_I2C.fill(RGB)</code>
清屏：RGB=0 表示黑色，RGB=1 表示白色

表 4-21 OLED 显示屏对象

学习了 Pin、I2C、OLED 对象用法后我们通过流程图来理顺一下思路：

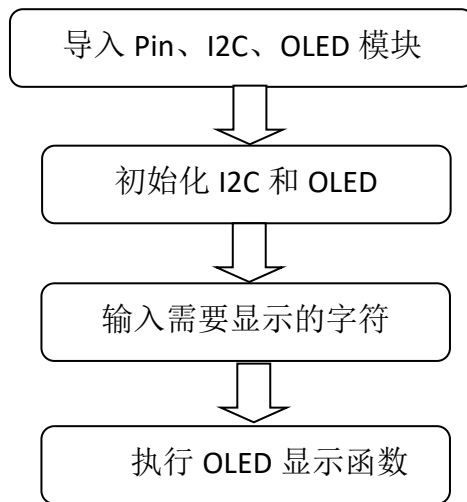


图 4-77 代码编写流程

参考代码如下：

```
'''
实验名称：OLED 显示屏（I2C 总线）
版本：v1.0
日期：2020.12
作者：01Studio
'''

from machine import I2C,Pin          #从 machine 模块导入 I2C、Pin 子模块
from ssd1306 import SSD1306_I2C      #从 ssd1306 模块中导入 SSD1306_I2C 子模
块

i2c = I2C(sda=Pin("F11"), scl=Pin("B0"))  #pyBoard I2C 初始化：sda-->
Y8, scl --> Y6
oled = SSD1306_I2C(128, 64, i2c, addr=0x3c) #OLED 显示屏初始化：128*64 分
辨率,OLED 的 I2C 地址是 0x3c

oled.text("Hello World!", 0, 0)        #写入第 1 行内容
oled.text("MicroPython", 0, 20)        #写入第 2 行内容
oled.text("By 01Studio", 0, 50)        #写入第 3 行内容
```

```
oled.show()    #OLED 执行显示
```

上述代码中 OLED 的 I2C 地址是 0x3C,不同厂家的产品地址可能预设不一样,具体参考厂家的说明书。或者也可以通过 I2C.scan()来获取设备地址。

另外记得将我们提供的示例代码中的 `ssd1306.py` 文件拷贝到达芬奇文件系统下,跟 `main.py` 保持同一个路径。



图 4-78 上传 `ssd1306.py` 文件到达芬奇文件新系统

● 实验结果:

将 0.9 尺寸 I2C 接口的 OLED 插到 OLED 接口,注意 OLED 引脚顺序需要和接口顺序一致。

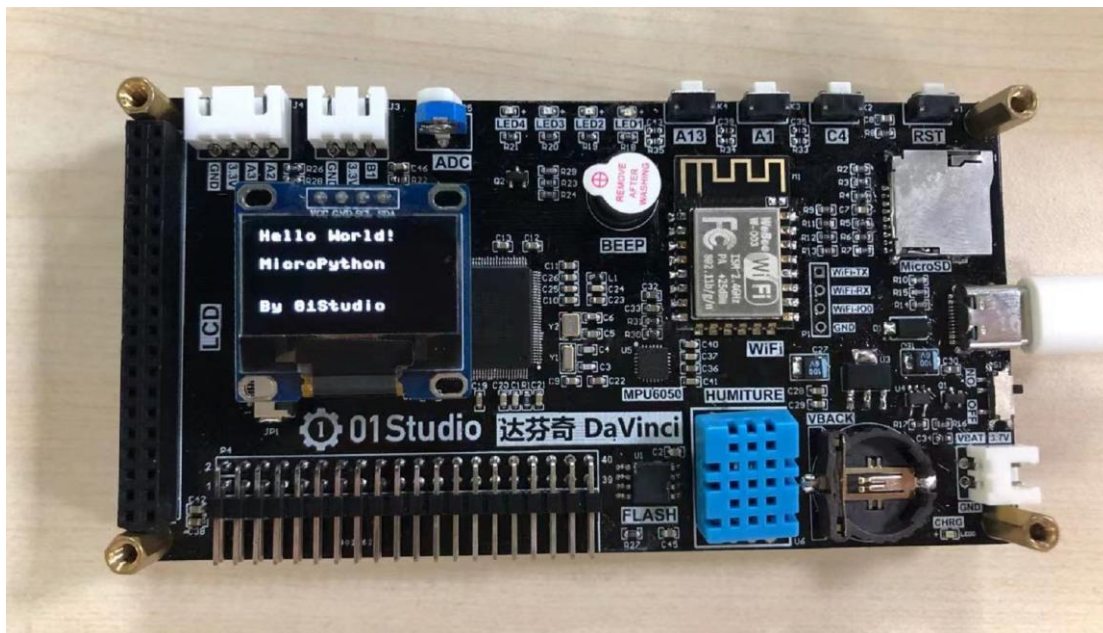


图 4-79 OLED 显示实验

● **总结:**

这一节我们学会了驱动 OLED 显示屏，换着以往如果从使用单片机从 0 开发的话你需要了解 I2C 总线原理，了解 OLED 显示屏的使用手册，编程 I2C 代码，有经验的嵌入式工程师搞不好也要弄个几天。现在基本半个小时解决问题。当然前提是别人已经给你搭好桥了，有了强大的底层驱动代码支持，我们只做好应用就好。

4.13 thread（线程）

- **前言：**

我们看到前面的编程都是一个循环来完成，但当我们需分时分完成不同任务时候，线程编程就派上用场了。这有点像 RTOS(实时操作系统)，今天我们就来学习一下如何通过 MicroPython 编程实现多线程。

- **实验平台：**

达芬奇 MicroPython 开发套件。



图 4-80

- **实验目的：**

编程实现多线程同时运行任务。

- **实验讲解：**

达芬奇的 MicroPython 固件已经集成了_thread 线程模块。我们直接调用即可。该模块衍生于 python3，属于低级线程，详情可以看官网介绍：

<https://docs.python.org/3.5/library/thread.html#module-thread>

编程流程如下：

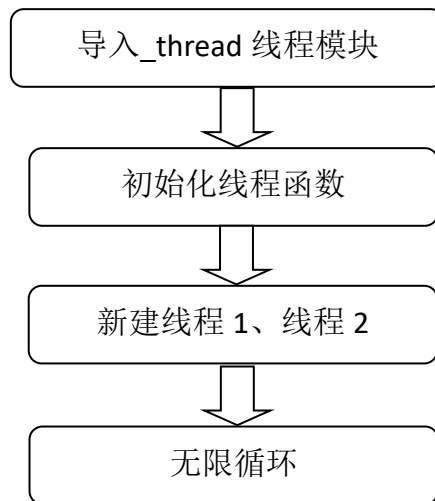


图 4-81 代码编写流程

参考代码如下：

```
'''
实验名称：thread（多线程）
版本： v1.0
日期： 2021.5
翻译和注释： 01Studio
说明：通过编程实现多线程。
'''

import _thread #导入线程模块
import time

#线程函数
def func(name):
    while True:
        print("thread {}".format(name))
        time.sleep(1)

_thread.start_new_thread(func, ("1",)) #开启线程 1, 参数必须是元组
```

```
_thread.start_new_thread(func, ("2",)) #开启线程 2, 参数必须是元组

while True:
    pass
```

- **实验结果:**

运行代码, 可以看到串口终端重复执行 2 个线程。

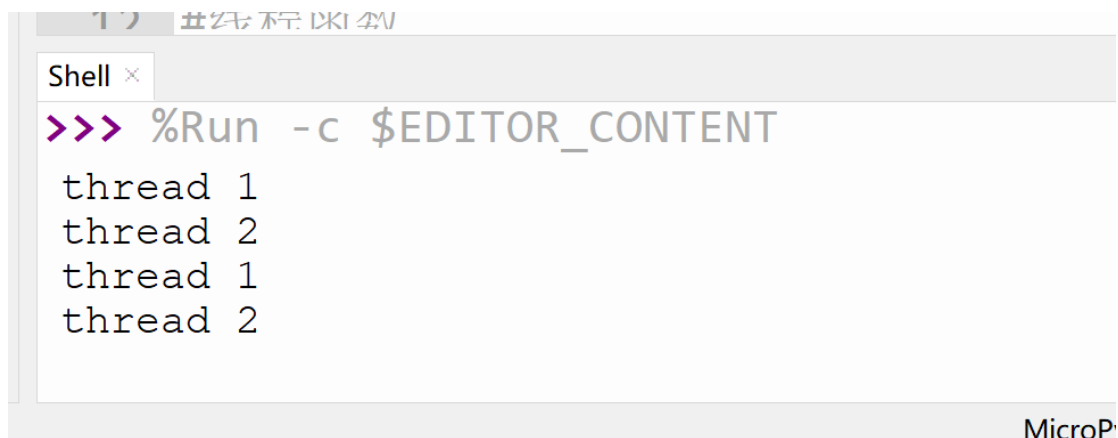


图 4-82 实验结果

- **总结:**

本章我们学习了基于 MicroPython 的线程编程, 实际是 python3 的低级线程编程。线程的引入让我们在处理多任务时候变得简单, 大大增加了程序编写的灵活性。

第5章 传感器实验

基础实验让我们对 **MicroPython** 的操作和编程有了比较全面的了解，但其魅力绝不限于此。日常生活中我们会用到各式各样的外设或者传感器，还是那句，一个有经验的嵌入式开发工程师驱动一款未接触过的传感器的一般流程是：了解传感器原理、设计电路图、信号时序分析和编程。没个几天折腾不出来。

生活中有很多传感器已经是非常通用了，前人已经做好封装函数模块，我们直接调用函数即可。我们不需要将时间花在“怎么用”上，而更多的是考虑“用到什么地方”！

本章实验以最常见的传感器出发，讲述是如何通过 **MicroPython** 编程实现传感器应用的。实验还是基于我们的达芬奇 **MicroPython** 开发板。而且实验例程将持续更新。

5.1 温湿度传感器 DHT11

● 前言:

温湿度也是我们日常非常常见的指标，我们使用的是 DHT11 数字温湿度传感器。这是一款含有已校准数字信号输出的温湿度复合传感器，它应用专用的数字模块采集技术和温湿度传感技术，确保产品具有极高的可靠性和卓越的长期稳定性。

DHT11 具有小体积、极低的功耗，信号传输距离可达 20 米以上，使其成为给类应用甚至最为苛刻的应用场合的最佳选择。产品为 4 针单排引脚封装，连接方便。

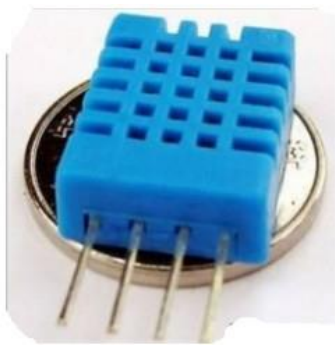


图 5-1 DHT11 温湿度传感器

● 实验平台:

达芬奇 MicroPython 开发套件。

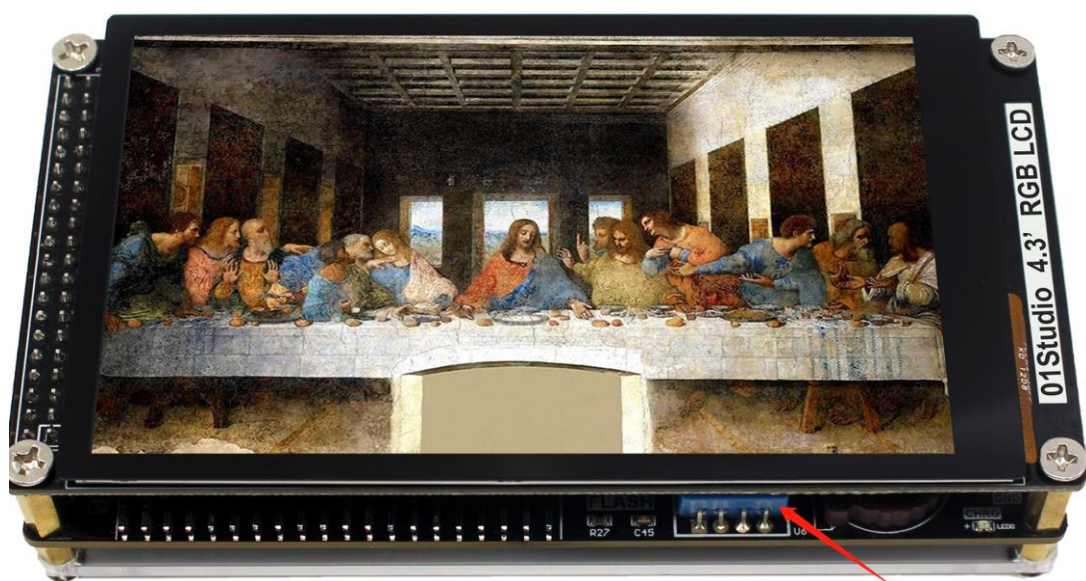


图 5-2 DHT11 传感器

- **实验目的：**

通过编程采集温湿度数据，并在 LCD 上显示。

- **实验讲解：**

DHT11 虽然有 4 个引脚，但其中第 3 个引脚是悬空的，也就是说 DHT11 也是单总线的传感器，只占用 1 个 IO 口。

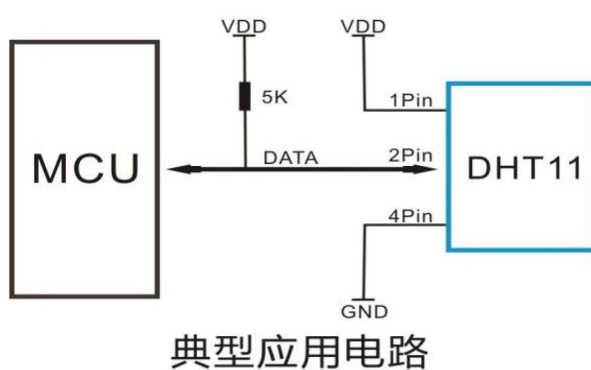


图 5-3 DHT11 应用

我们来看看 DHT11 在开发板上的接线图：

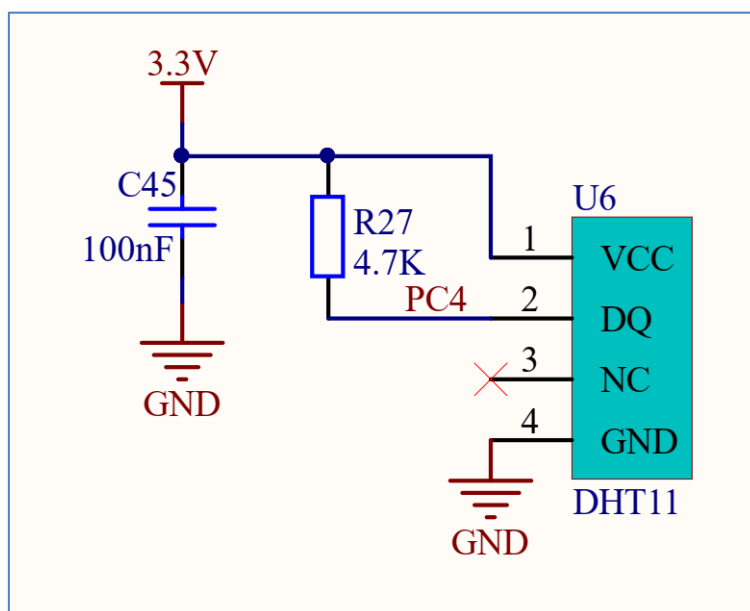


图 5-4 DHT11 接线图

可以看到 DHT11 连接到达芬奇主控的 ‘PC4’ 引脚，可以针对该引脚编程来

驱动 DHT11 传感器，达芬奇 micropython 固件已经集成了 dht 模块，用户可以轻松驱动 DHT11，函数模块说明如下：

构造函数
<code>dht.DHT11(Pin)</code>
指定连接 DHT11 的引脚
使用方法
<code>DHT11.measure()</code>
温湿度数据测量
<code>DTH11.temperature()</code>
获取温度值
<code>DTH11.humidity()</code>
获取湿度值

表 5-1 DHT11 对象

建议上电先延时 2 秒，让 DHT11 稳定后再开设读取。另外 DTH11 的采集周，官方建议是 2~3 秒以上，否则可能会出错。本实验我们采集周期设置为 2 秒。代码编写流程如下：

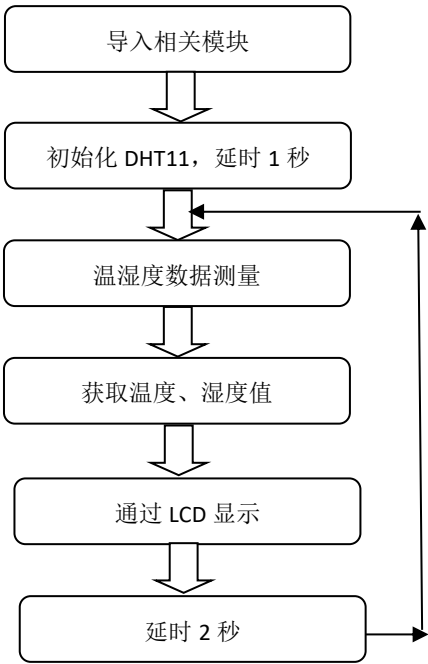


图 5-5 代码编程流程

参考代码如下：

```
'''
实验名称：温湿度传感器实验 DTH11
版本： v1.0
日期： 2021.1
作者： 01Studio
实验平台： 达芬奇
'''

#引入相关模块

from machine import Pin
from dht import DHT11
from tftlcd import LCD43R
import time

#定义常用颜色
WHITE=(255,255,255)
BLACK = (0,0,0)
BLUE = (0,0,255)

#初始化 LCD
d=LCD43R()
d.fill(WHITE) #填充白色

#显示标题
d.printStr('01Studio DHT11', 40, 10, BLUE, size=4)

#创建 DTH11 对象 dt
dt = DHT11(Pin('A0'))
```



```
# 延时 2 秒等待 DHT11 稳定
time.sleep(2)

while True:

    try: #异常处理

        dt.measure()

        te=dt.temperature() #获取温度值

        dh=dt.humidity()    #获取湿度值


        #REPL 打印

        print(str(dt.temperature())+' C')
        print(str(dt.humidity())+' %')


        #实时显示温湿度值

        d.printStr('Temp: '+str(te)+' C',10,100,BLACK,size=4)
        d.printStr('Humi: '+str(dh)+' %',10,200,BLACK,size=4)


    except Exception as e: #异常提示

        print('Time Out!')


    time.sleep(2) #采集周期 2 秒
```

- **实验结果:**

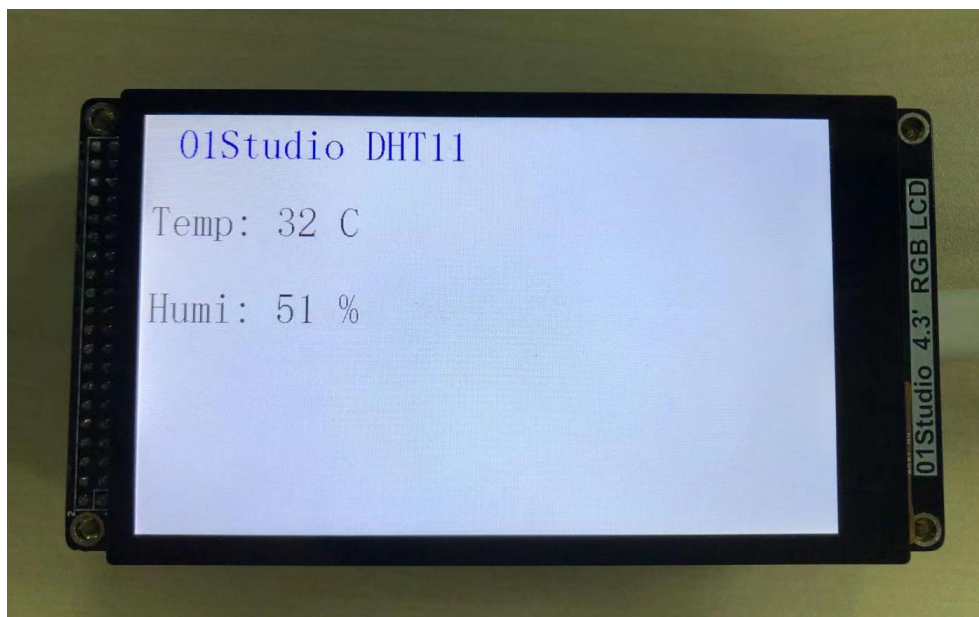


图 5-6 DHT11 实验结果

- **总结:**

通过本节学习我们学会了使用 MicroPython 来驱动 DTH11 温湿度传感器，DHT11 性价比比较高，是很适合学习使用的，但精度和响应速度有点低，需要更高要求应用的用户可以使用 DHT22 或者其他更高级的传感器。

5.2 MPU6050 六轴传感器

- **前言：**

MPU6050 是一款高性能的 6 轴（三轴加速度+三轴陀螺仪）传感器模块，用于测试物体的运动姿态，I2C 接口，本节我们就通过编程来学习读取 MPU6050 传感器的原始数据。

- **实验平台：**

达芬奇 MicroPython 开发板。



图 5-7 达芬奇开发板

- **实验目的：**

通过编程实现 MPU6050 六轴传感器数据获取，并显示在 LCD 上。

- **实验讲解：**

MPU6050 传感器位于达芬奇开发板的中间位置：

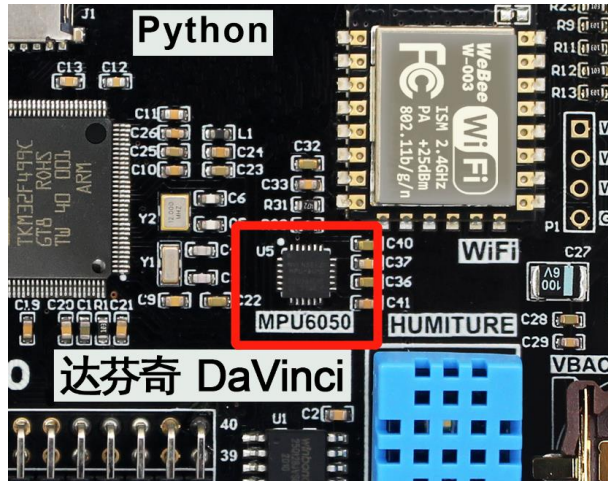


图 5-8 MPU6050 传感器

从原理图可以看到连接到了达芬奇主控的 I2C3 接口：

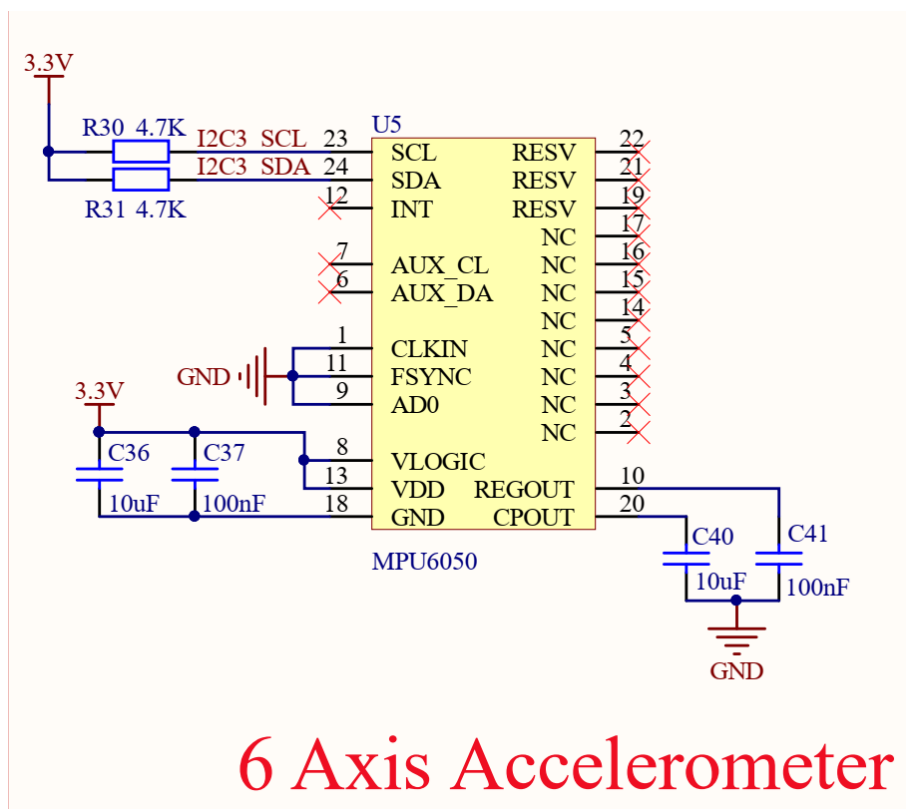


图 5-9

我们来看看 MPU6050 传感器的详细参数：

功能参数	
芯片型号	MPU6050
供电电压	3.3V
控制方式	I2C 总线（默认地址：0x68）
测量维度	加速度：3 维 陀螺仪：3 维
加速度测量范围	$\pm 2/\pm 4/\pm 8/\pm 16g$
陀螺仪测量范围	$\pm 250/\pm 500/\pm 1000/\pm 2000^{\circ}/s$
ADC 位数	16 位
温度传感器	测量范围： $-40^{\circ}C \sim 85^{\circ}C$ （精度： $\pm 1^{\circ}C$ ）

表 5-2 MPU6050 功能参数

上面介绍可以看到，MPU6050 是一款通过 I2C 接口驱动传感器。连接到开发板主控的 I2C3 接口。我们通过前面学习的 I2C 接口使用的方式，即可以对该模块实现数据通讯。Micropython 模块已经写好，位于例程文件夹里面的 mpu6050.py 文件，我们来看看其构造函数和使用方法：

构造函数
<code>mpu6050.accel(i2c)</code>
构建 MPU6050 对象； 【i2c】i2c 对象
使用方法
<code>accelerometer.get_values()</code>
获取测量数据。返回元组如下： vals["AcX"]：加速度 X 轴值； vals ["AcY"] 加速度 Y 轴值； vals ["AcZ"] 加速度 Z 轴值； vals ["GyX"] 陀螺仪 X 轴值； vals ["GyY"] 陀螺仪 Y 轴值； vals ["GyZ"] 陀螺仪 Z 轴值；

表 5-3 MPU6050 对象

代码编写流程如下：

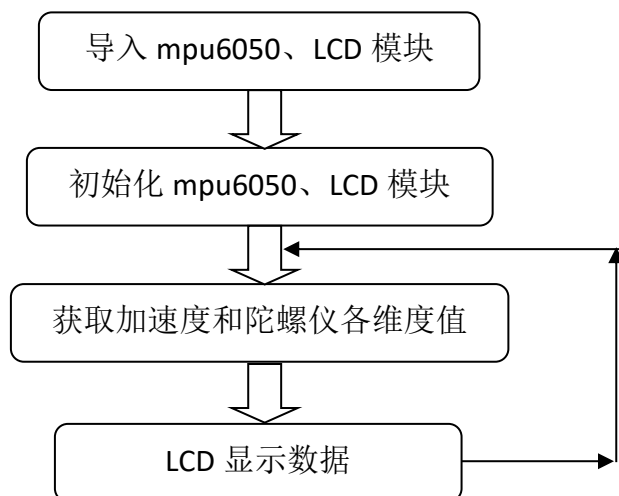


图 5-10 代码编写流程

参考代码如下：

```
...  
  
实验名称：MPU6050 六轴传感器  
版本：v1.0  
日期：2021.5  
作者：01Studio  
说明：通过编程获 MPU6050 的 6 轴（3 轴加速度+3 轴陀螺仪）和温度数据，并在 LCD 上  
显示。  
实验平台：01Studio 达芬奇  
...  
  
#导入相关模块  
  
import mpu6050,time  
  
from machine import SoftI2C,Pin  
  
from tftlcd import LCD43R  
  
  
  
#定义常用颜色
```

```

WHITE=(255,255,255)

BLACK = (0,0,0)

BLUE = (0,0,255)

#初始化 LCD

d=LCD43R()

d.fill(WHITE)#填充白色


#MPU6050 初始化, 这里使用软件 I2C

i2c = SoftI2C(sda=Pin("C0"), scl=Pin("C1"))

accelerometer = mpu6050.accel(i2c)


#显示标题

d.printStr('01Studio MPU6050', 40, 10, BLUE, size=4)


while True:


    #获取传感器信息

    value=accelerometer.get_values()

    print(value)


    #显示加速度数据

    d.printStr('Ac-X: '+str(value["AcX"]), 10, 100, BLACK, size=4)
    d.printStr('Ac-Y: '+str(value["AcY"]), 10, 150, BLACK, size=4)
    d.printStr('Ac-Z: '+str(value["AcZ"]), 10, 200, BLACK, size=4)


    #显示陀螺仪数据

    d.printStr('Gy-X: '+str(value["GyX"]), 10, 300, BLACK, size=4)
    d.printStr('Gy-Y: '+str(value["GyY"]), 10, 350, BLACK, size=4)
    d.printStr('Gy-Z: '+str(value["GyZ"]), 10, 400, BLACK, size=4)

```

```
time.sleep_ms(300) #延时 300 毫秒
```

● **实验结果:**

运行程序，可以看到 LCD 显示采集到的 MPU6050 六维数据值。

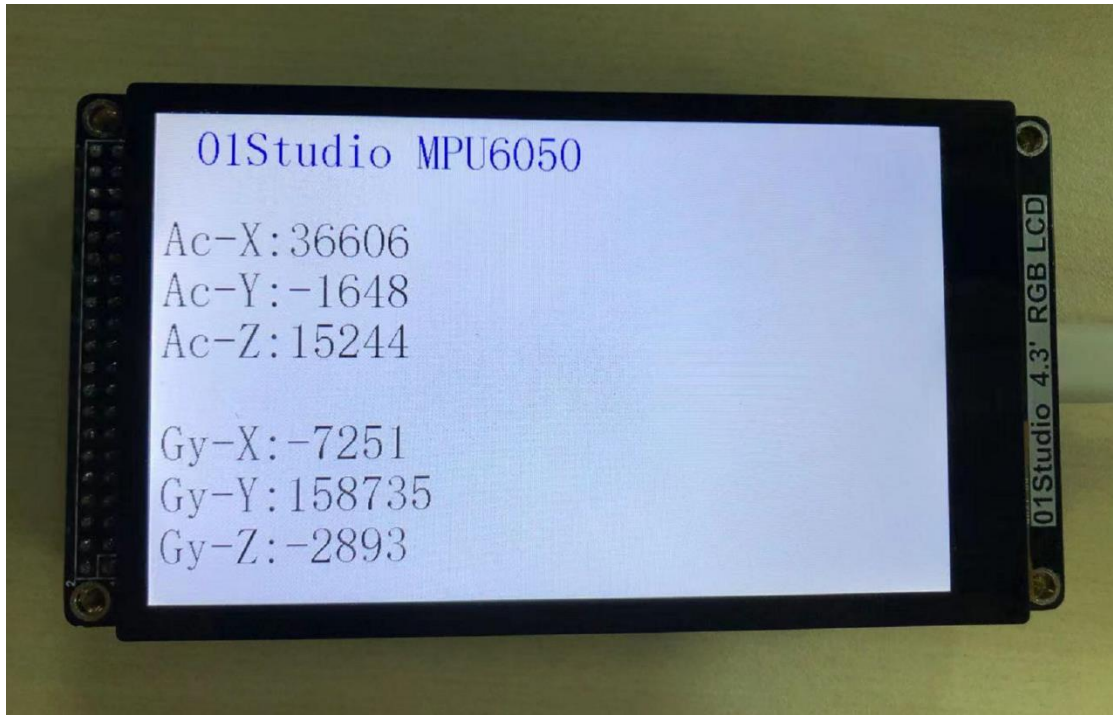


图 5-11

拿起开发板并往各个方向转动，可以看到相关数值在变化：

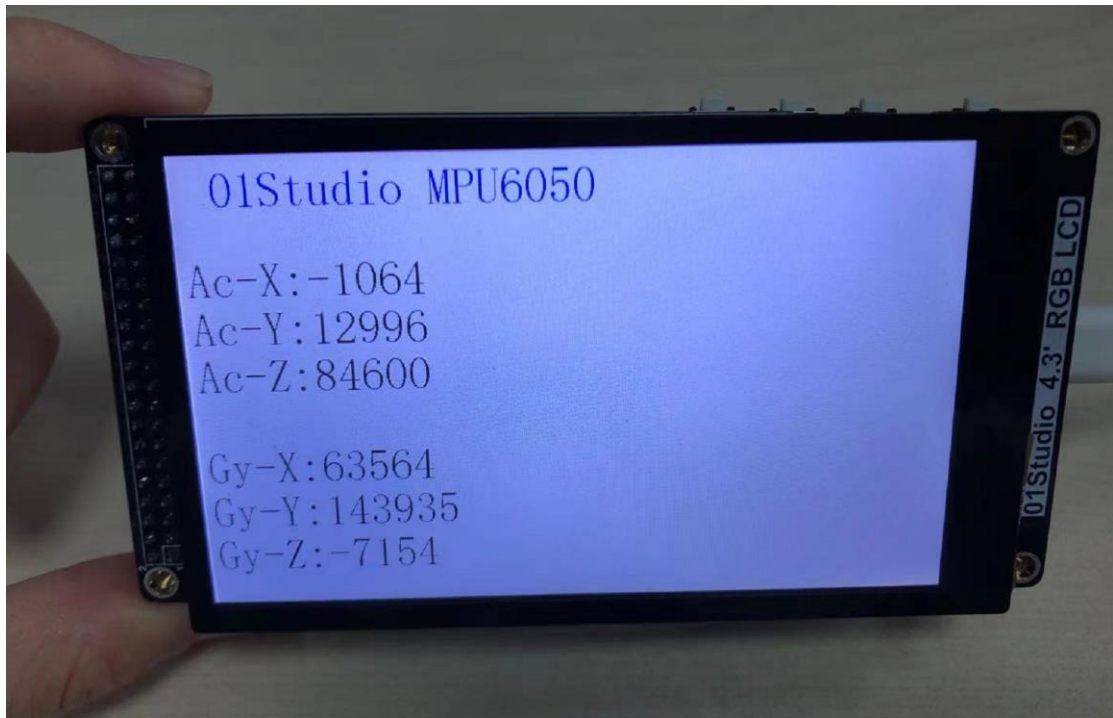


图 5-12

● **总结：**

本节通过 micropython 编程轻松实现了对 MPU6050 六轴传感器数据采集，但发现采集的是原始数据，因此我们可以用这些数据来进行简单的方向识别。类似于手机的重力感应来旋转屏幕相关的应用。

另外可以看到数据的跳动变化是非常快的，实际应用可以加入数据滤波处理，有兴趣的小伙伴可以自行研究。

5.3 人体感应传感器

- **前言：**

人体感应传感器，在室内安防应用非常普遍，其原理是由探测元件将探测到人体的红外辐射转变成微弱的电压信号，经过放大后输出。为了提高探测器的探测灵敏度以增大探测距离，一般在探测器的前方装设一个塑料的菲涅尔透镜，它和放大电路相配合，可将信号放大 70dB 以上，这样就可以测出 5~10 米范围内人的行动。

- **实验平台：**

达芬奇 MicroPython 开发套件和人体感应传感器模块。

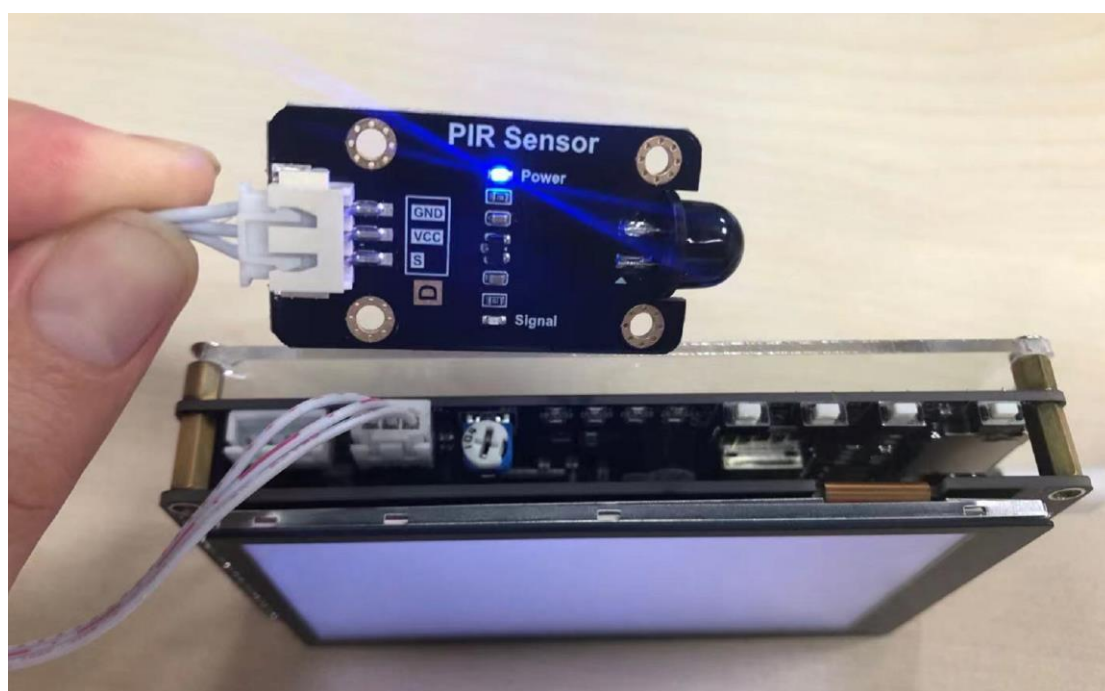


图 5-13 开发套件和传感器接线

- **实验目的：**

通过外部中断编程来检测人体感应模块，当有人出现时候 LCD 通过 “Get People!!!” 闪烁提示。

- **实验讲解：**

我们先来看看人体感应传感器的介绍：

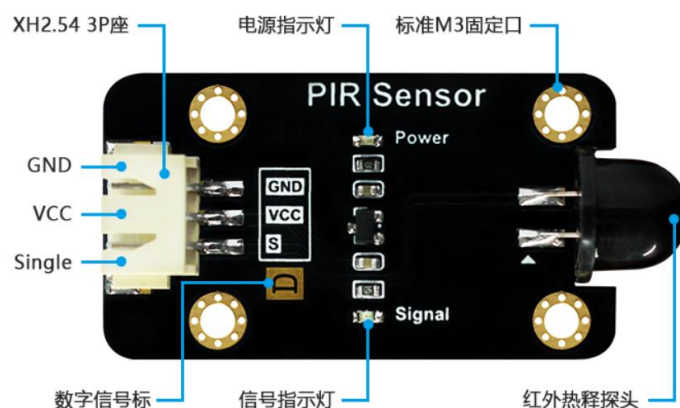


图 5-14 人体感应传感器模块

功能参数	
供电电压	3.3-5V
工作电流	<20 mA
工作温度	-20 至 65℃
接口定义	XH2.54 防呆接口（3Pin）【GND、VCC、Single】
输出信号	数字信号： 检测到人体：高电平 3.3V；持续 3-8 秒 未检测到人体：低电平 0V。
感应角度	100°
感应距离	8 米
模块尺寸	4.5*2.5cm

表 5-4 人体感应传感器模块参数说明

从上表可以知，当检测到人体红外时候，传感器的输出信号为高电平并持续 3-5 秒。如下图所示：

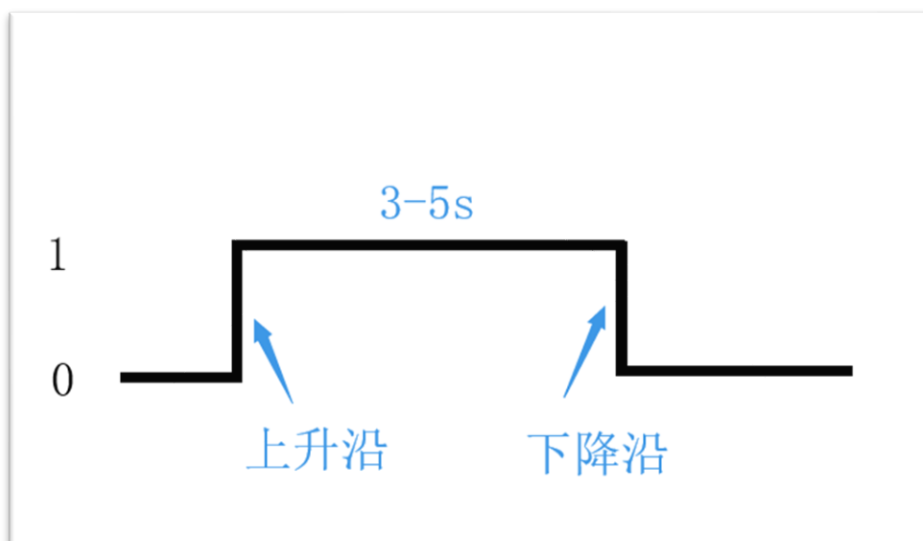


图 5-15 人体感应传感器电平变化

由此可见，可以使用外部中断结合上升沿的出发方式来编程实现相关功能。传感器的输出引脚连接 **Sensor1** 接口，即连接到达芬奇的“**PB1**”引脚。编程方法可以是当“**PB1**”引脚产生中断时候，说明传感器检测到人体红外线，此时可以在 **LCD** 上显示相关信息。

关于外部中断的编程方法可以参考基础实验的外部中断章节，这里不再重复，具体的编程流程图如下：

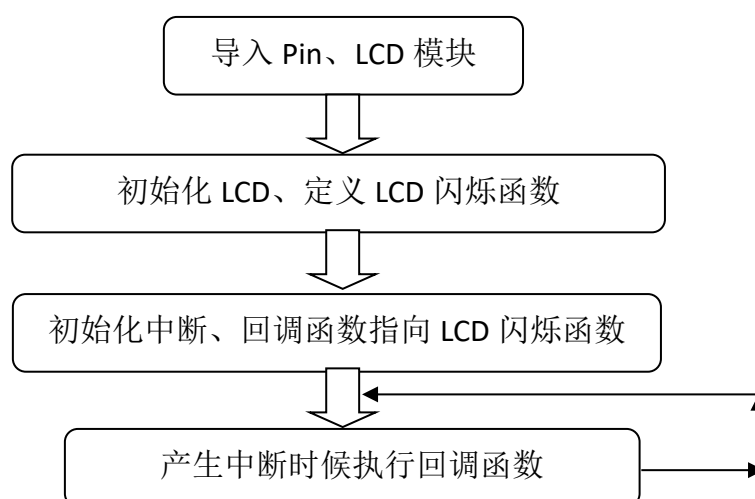


图 5-16 代码编写流程

参考例程代码如下：

```
'''
实验名称：人体感应传感器
版本： v1.0
日期： 2021.5
作者： 01Studio
实验平台： 01Studio-达芬奇
社区： www.01studio.org
'''

import time

from machine import SoftI2C,Pin    #从 machine 模块导入 I2C、Pin 子模块
from tftlcd import LCD43R

#定义常用颜色
WHITE=(255,255,255)
BLACK = (0,0,0)
RED=(255,0,0)

#初始化 LCD
d=LCD43R()
d.fill(WHITE)#填充白色

#配置按键
human = Pin('B1', Pin.IN, Pin.PULL_UP)

def Display(human): #Get People 闪烁 5 次效果!

    for i in range(5):

        #显示标题

        d.printStr('Get People!!!', 80, 300, RED, size=4)
```

```
time.sleep_ms(500)

d.fill(WHITE)#填充白色

time.sleep_ms(500)

human.irq(Display,Pin.IRQ_RISING) #定义中断，下降沿触发
```

● 实验结果:

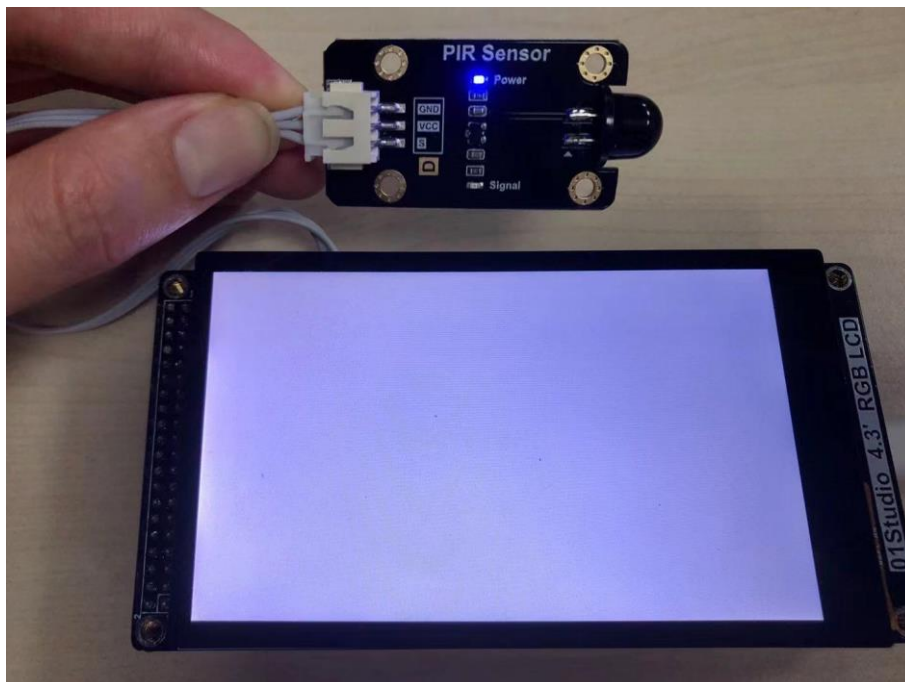


图 5-17 没有检测到到人体



图 5-18 检测到人体，LCD 字符闪烁提示

- **总结：**

本节通过简单的中断方式便实现了对人体感应传感器的检测，人体感应传感器的应用非常广泛，特别是在安防领域，结合其它硬件模块可以实现当发现有人入侵时，执行发出警报、实现远程提醒等功能。

5.4 光敏传感器

- **前言:**

光敏传感器实际是一个款可以检测光照强度的传感器,可以应用于我们日常生活中植物光照强度、室内光线检测、以及某些场合的亮度检测。传感器的原理就是将外界模拟变化的信号转变成数字信号(电压值)让单片机出来,处理方式就是常用的 ADC(模拟数字转换)。

- **实验平台:**

达芬奇 MicroPython 开发套件和光敏传感器模块。



图 5-19 开发套件和光敏传感器接线

- **实验目的:**

采集当前环境的光照强度并在 LCD 显示,显示方式为: Bright-强, Normal-中等, Weak-弱。

- **实验讲解:**

我们先来看看光敏传感器模块的介绍:

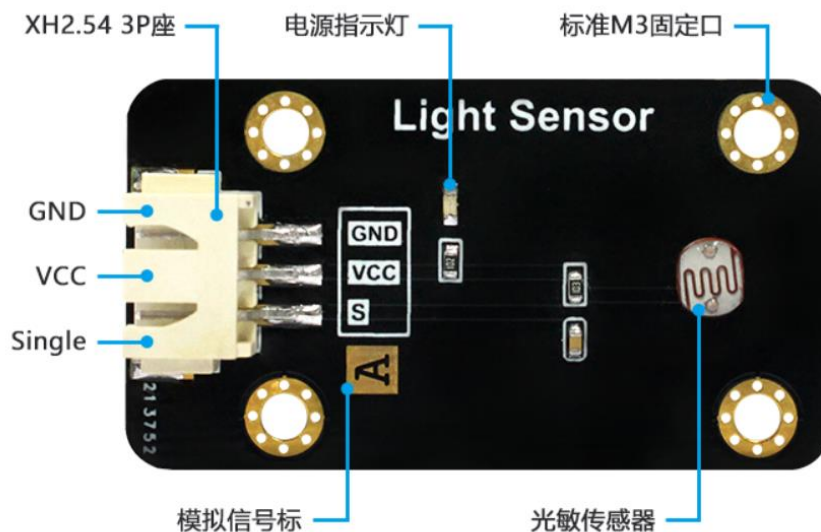


图 5-20 光敏传感器模块

功能参数	
供电电压	3.3-5V
工作电流	<20 mA
工作温度	-20 至 85℃
接口定义	XH2.54 防呆接口（3Pin）【GND、VCC、Single】
输出信号	模拟信号：0-3.3V （VCC=3.3V 时）
模块尺寸	4.5*2.5 cm

表 5-5

从上表可以看到，光敏传感器输出的是模拟信号：0-3.3V，这代表了外界的光照强度。接近 0V 时光线最强，接近 3.3V 时，光线最弱。因此我们可以使用基础实验-ADC 学习过的内容来编程。

光敏传感器接在传感器 3P 接口，对应的引脚是“PB1”。ADC 使用 12bit 精度，即最大值为 $2^{12}-1=4095$ 。然后将检测到的数值 0-4095 分成三段，分别代表光线强：【0-1365】，中等：【1365-2730】，弱：【2730-4095】。当然开发者可以根据自己实际情况来调整数值。编程流程图如下：

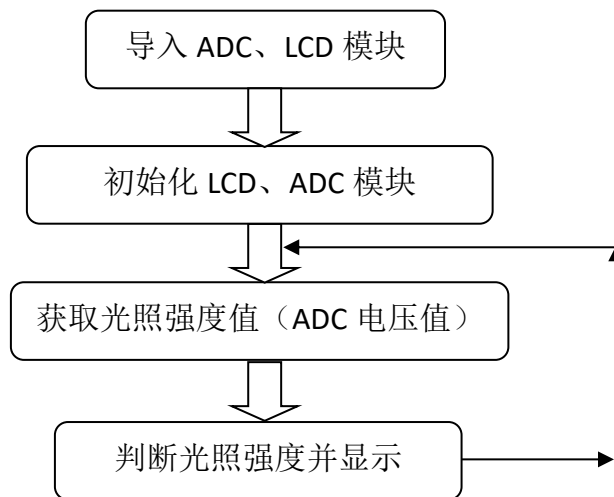


图 5-21 代码编写流程

参考程序代码如下：

```

...

实验名称：光敏传感器

版本：v1.0

日期：2021.4

作者：01Studio 【www.01Studio.org】

实验平台：达芬奇

说明：通过光敏传感器对外界环境光照强度测量并显示。

...

#导入相关模块

from machine import Pin,SoftI2C,ADC

from tftlcd import LCD43R

import time

#定义常用颜色

WHITE=(255,255,255)

BLACK = (0,0,0)

RED=(255,0,0)

BLUE=(0,0,255)

```

```

#初始化 LCD

d=LCD43R()

d.fill(WHITE)#填充白色


#初始化 ADC,Pin=PB1
Light = ADC('B1')


#显示标题
d.printStr('01Studio Light Test', 10, 10, BLUE, size=4)


while True:


    value=Light.read_u16() #获取 ADC 数值


    #LCD 显示
    d.printStr('Vol: '+str('%.2f'%(value/4095*3.3))+" V", 10, 100, BLACK,
               size=4)
    d.printStr('Value: '+str(value)+"   ", 10, 200, BLACK, size=4)
    d.printStr("(4095)", 300, 200, BLACK, size=4)


    #判断光照强度, 分 3 档位显示
    if 0 <= value <=1365:
        d.printStr('Bright', 10, 300, BLACK, size=4)


    if 1365 < value <=2730:
        d.printStr('Normal', 10, 300, BLACK, size=4)


    if 2730 < value <=4095:
        d.printStr('Weak   ', 10, 300, BLACK, size=4)

```

```
time.sleep_ms(1000)
```

- **实验结果:**

用手遮挡住光敏传感器，可以见到 LCD 显示光线强度是弱 Weak:



图 5-22 光线强度弱

打开手机手电筒，照在光敏传感器上，可以见到光线强度是强 Bright:

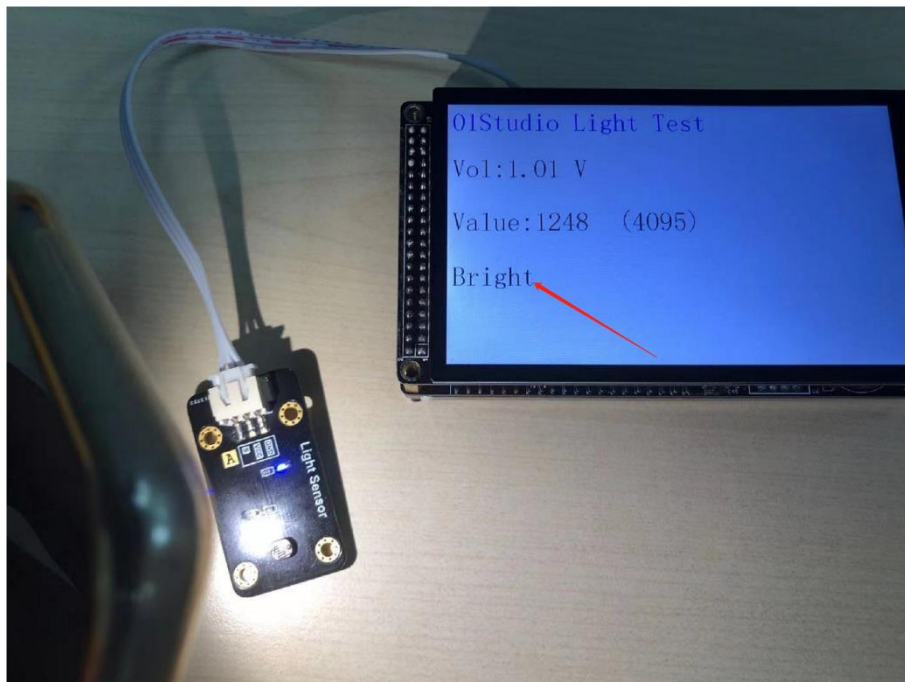


图 5-23 光线强度强

- **总结：**

从实验可以看到，光敏传感器背后的原理是对 ADC 的应用，实现了改功能后。我们可以自行扩展深入，制作自己喜欢的电子产品。

5.5 土壤湿度传感器

- **前言：**

土壤湿度传感器用于检测盆栽泥土的湿度，当泥土干枯时候，我们就需要给植物浇水了。这个用途非常广泛，如自动灌溉。

- **实验平台：**

达芬奇 MicroPython 开发套件和土壤湿度传感器模块。



图 5-24 开发套件和传感器连接图

- **实验目的：**

采集盆栽土壤的光照强度并在 LCD 显示，显示方式为：Dry-干，Normal-中等，Wet-湿。

- **实验讲解：**

我们先来看看土壤湿度传感器模块的介绍：

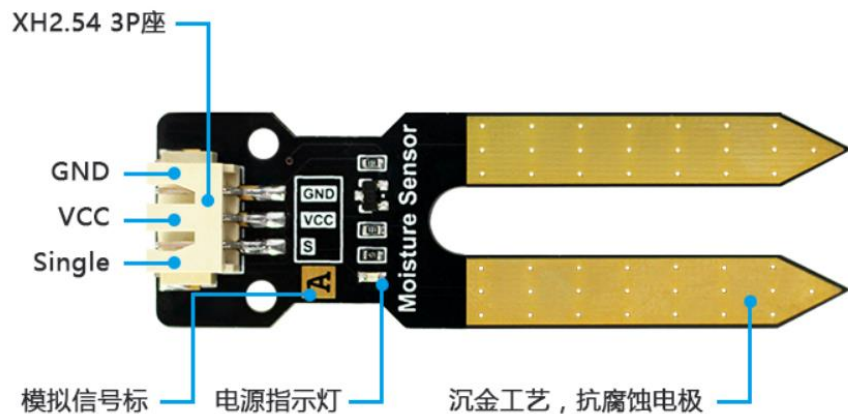


图 5-25 土壤湿度传感器模块

功能参数	
供电电压	3.3-5V
工作电流	<20 mA
工作温度	-40 至 85℃
接口定义	XH2.54 防呆接口（3Pin）【GND、VCC、Single】
输出信号	模拟信号：0-3.3V （VCC=3.3V 时）
模块尺寸	6.6*2.0cm

表 5-6

从上表可以看到，土壤湿度传感器输出的是模拟信号：0-3.3V，这代表土壤的湿度情况。接近 0V 时湿度为干燥，接近 3.3V 时，湿度情况为湿润。因此我们可以使用基础实验-ADC 学习过的内容来编程。

土壤湿度传感器接在 3P 传感器接口，对应的引脚是“PB1”。ADC 使用 12bit 精度，即最大值为 $2^{12}-1=4095$ 。然后我们根据实际测试数据将检测到的数值 0-4095 分成三段，分别代表土壤干燥：【0-1247】，中等：【1247-2238】，湿润：【2238-4095】。当然开发者可以根据自己实际情况来调整数值。编程流程图如下：

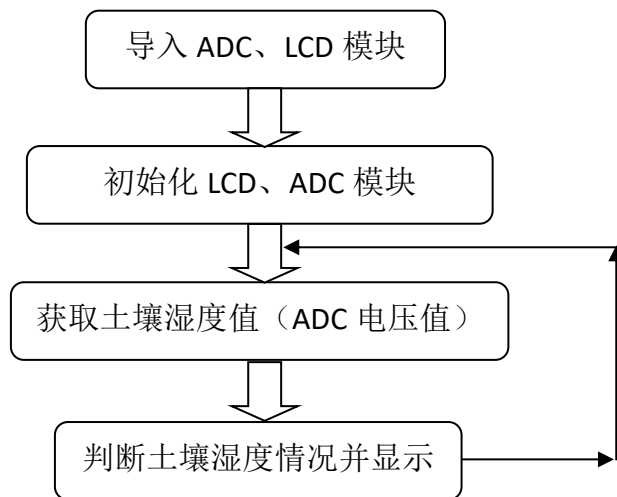


图 5-26 代码编写流程

参考程序代码如下：

```

...

实验名称：土壤湿度传感器

版本：v1.0

日期：2021.5

作者：01Studio 【www.01Studio.org】

实验平台：01Studio - 达芬奇

说明：通过土壤湿度传感器对土壤湿度测量并显示。

...

#导入相关模块

from machine import Pin,ADC

from tftlcd import LCD43R

import time

#定义常用颜色

WHITE=(255,255,255)

BLACK = (0,0,0)

BLUE=(0,0,255)

```



```

#初始化 LCD

d=LCD43R()

d.fill(WHITE)#填充白色


#初始化 ADC

Soil = ADC('B1')


#显示标题

d.printStr('01Studio Soil Test', 10, 10, BLUE, size=4)

while True:


    value=Soil.read_u16() #获取 ADC 数值


    #LCD 显示

    d.printStr('Vol: '+str('%02f'%(value/4095*3.3))+" V", 10, 100, BLACK,
               size=4)

    d.printStr('Value: '+str(value)+"    ", 10, 200, BLACK, size=4)

    d.printStr("(4095)", 300, 200, BLACK, size=4)


    #判断土壤湿度，分 3 档位显示

    if 0 <= value <=1247:

        d.printStr('Dry    ', 10, 300, BLACK, size=4)


    if 1247 < value <=2238:

        d.printStr('Normal', 10, 300, BLACK, size=4)


    if 2238 < value <=4095:

        d.printStr('Wet    ', 10, 300, BLACK, size=4)


    time.sleep(1)#延时 1 秒

```

- 实验结果:

将传感器插到干燥的土壤中，可以见到 OLED 显示干燥-Normal:

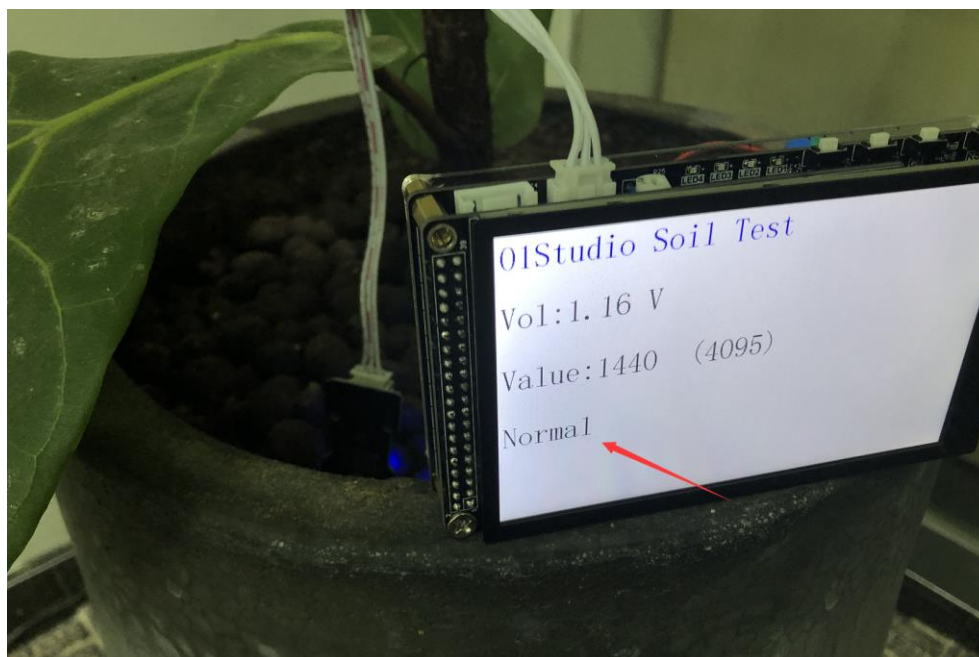


图 5-27 干燥的土壤

从传感器侧面往土壤浇水，让土壤变得湿润。**注意不要浇到传感器电路上！**
可以看到土壤湿度测量值发生变化。



图 5-28 浇水

- **总结:**

从实验可以看到，土壤湿度传感器背后的原理是对 ADC 的应用，实现了该功能后。我们可以自行扩展深入，制作自己喜欢的电子产品。

5.6 水位传感器

- **前言：**

水位（液位）传感器是一款简单易用、小巧轻便、水位/水滴识别检测传感器，传感器通过一系列的暴露的平行导线线迹测量其水滴/水量大小使得传感器输出的模拟信号产生相应的变化，轻松完成水量到模拟信号的转换，从而判断水位。

- **实验平台：**

达芬奇 MicroPython 开发套件和水位传感器模块。



图 5-29 开发套件和水位传感器接线图

- **实验目的：**

测量水位高度并在 LCD 显示，显示方式为：0-4cm。

- **实验讲解：**

我们先来看看水位传感器的介绍：

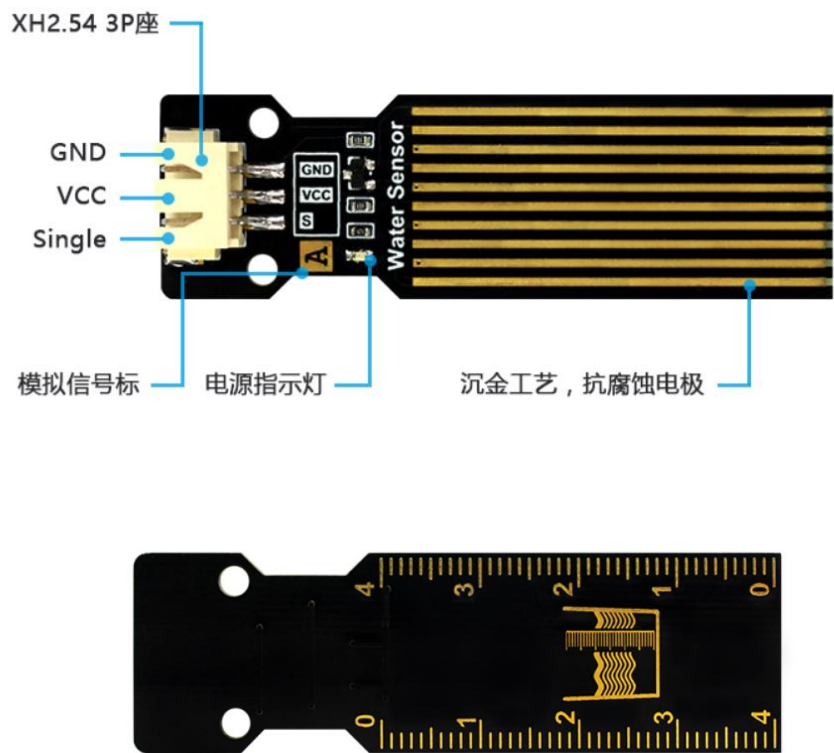


图 5-30 水位传感器模块正反面

功能参数	
供电电压	3.3-5V
工作电流	<20 mA
工作温度	0 至 60℃
接口定义	XH2.54 防呆接口（3Pin）【GND、VCC、Single】
输出信号	模拟信号：0-3.3V （VCC=3.3V 时）
模块尺寸	6.6*2.0cm
注意事项	水位传感器接口旁元件不能接触水，否则容易短路。所以测量时候要注意液体水位的高度。

表 5-7

从上表可以看到，水位传感器的量程为 4cm，输出的是模拟信号：0-3.3V，这代表水位高度情况。接近 0V 时湿度为 0cm 或没有放进液体，电压升高，意味着水位升高。因此我们可以使用基础实验-ADC 学习过的内容来编程。

水位传感器接在 3P 传感器接口，对应的引脚是“PB1”。ADC 使用 12bit 精度，即最大值为 $2^{12}-1=4095$ 。我们使用的是普通桶装水，根据实际测试数据将检测到的数值 0-4095 分成 5 档。分别代表水位高度 0cm:【0-600】，1cm:【600-900】，2cm:【900-1200】，3cm:【1200-1300】，4cm:【>1300】。

当然开发者可以根据自己测量液体不同或其它情况来调整数值。编程流程图如下：

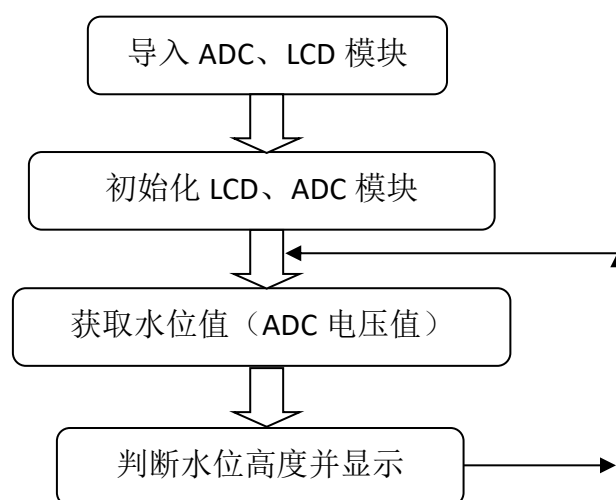


图 5-31 代码编写流程

参考程序代码如下：

```
'''
实验名称：水位传感器
版本：v1.0
日期：2021.5
作者：01Studio 【www.01Studio.org】
开发平台：01Studio 达芬奇
说明：通过水位传感器对水位测量并显示。
'''

#导入相关模块

from machine import Pin, ADC
from tftlcd import LCD43R
```

```

import time

#定义常用颜色
WHITE=(255,255,255)
BLACK = (0,0,0)
BLUE=(0,0,255)

#初始化 LCD
d=LCD43R()
d.fill(WHITE)#填充白色

#初始化 ADC
Water_level = ADC('B1')

#显示标题
d.printStr('01Studio Water Level Test', 10, 10, BLUE, size=4)

while True:

    value=Water_level.read_u16() #获取 ADC 数值

    #LCD 显示
    d.printStr('Vol: '+str('%.2f'%(value/4095*3.3))+ " V", 10, 100, BLACK,
size=4)
    d.printStr('Value: '+str(value)+"   ", 10, 200, BLACK, size=4)
    d.printStr("(4095)", 300, 200, BLACK, size=4)

    #判断水位, 分 5 档
    if 0 <= value <=600:
        d.printStr('0cm', 10, 300, BLACK, size=4)

```

```
if 600 < value <=900:
    d.printStr('1cm', 10, 300, BLACK, size=4)

if 900 < value <=1200:
    d.printStr('2cm', 10, 300, BLACK, size=4)

if 1200 < value <=1300:
    d.printStr('3cm  ', 10, 300, BLACK, size=4)

if 1300 < value:
    d.printStr('4cm  ', 10, 300, BLACK, size=4)

time.sleep(1)#延时 1 秒
```

- **实验结果:**

当没插入水中时，OLED 显示当前水位为 0cm:



图 5-32

将水位传感器模块慢慢放入水中，可以看到 OLED 显示从 1cm-4cm 变化。
(注意量程是 4cm，请勿将超出的电路部分放置水中以免发生短路。)

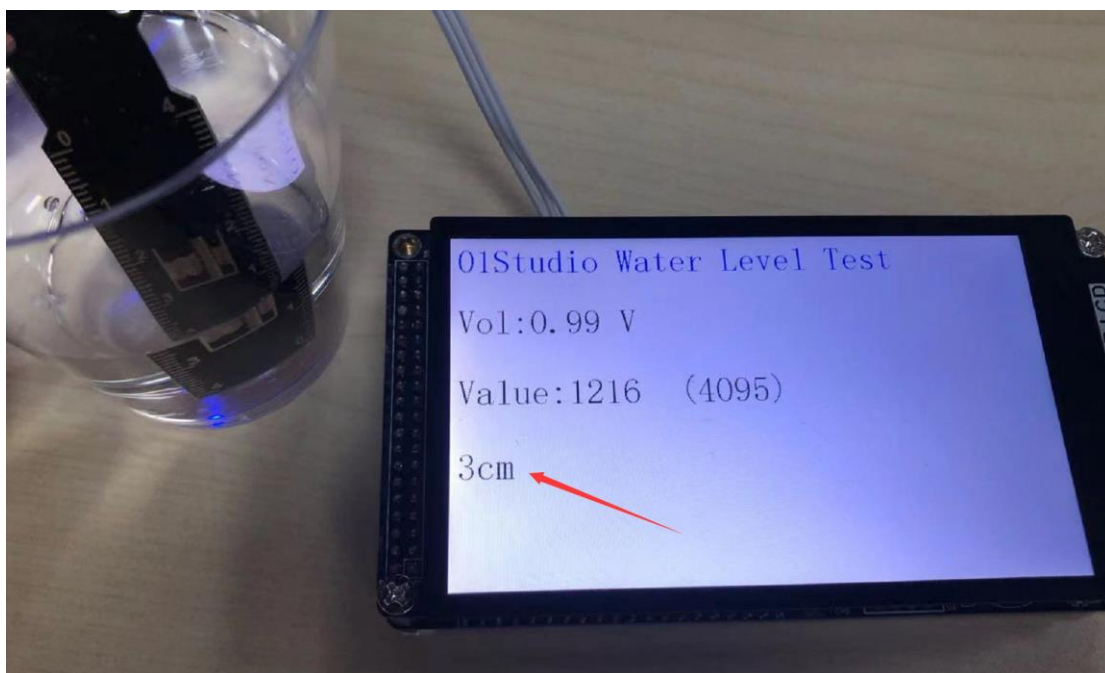


图 5-33 水位值变化

- **总结：**

从实验可以见到，这款水位传感器除了可以用来测量水位（液位）高度外，还可以用来测量室外下雨情况和家里的淹水情况。有兴趣的小伙伴可以动手制作一下！

5.7 大气压强传感器

● 前言：

本实验中大气压强传感器模块使用的是 **BMP280**，这是一款专为移动应用而设计的高精度大气压传感器，传感器模块采用极其紧凑的封装，其小尺寸和低功耗可以应用在手机、GPS 模块、手表的电池供电设备中。**BMP280** 传感器除了可以测量大气压强，还可以测量温度（温度精度不高）以及通过计算公式来换算出海拔高度。

● 实验平台：

达芬奇 **MicroPython** 开发套件和大气压强传感器模块，传感器连接到 I2C/UART 扩展接口。



图 5-34 开发套件和大气压强传感器接线方法

● 实验目的：

编程测量当前环境的大气压强、温度信息，将大气压强通过公式计算出当前海拔高度，并在 LCD 显示。打造自己的气压计！

● 实验讲解：

我们先来看看大气压强传感器模块的介绍：

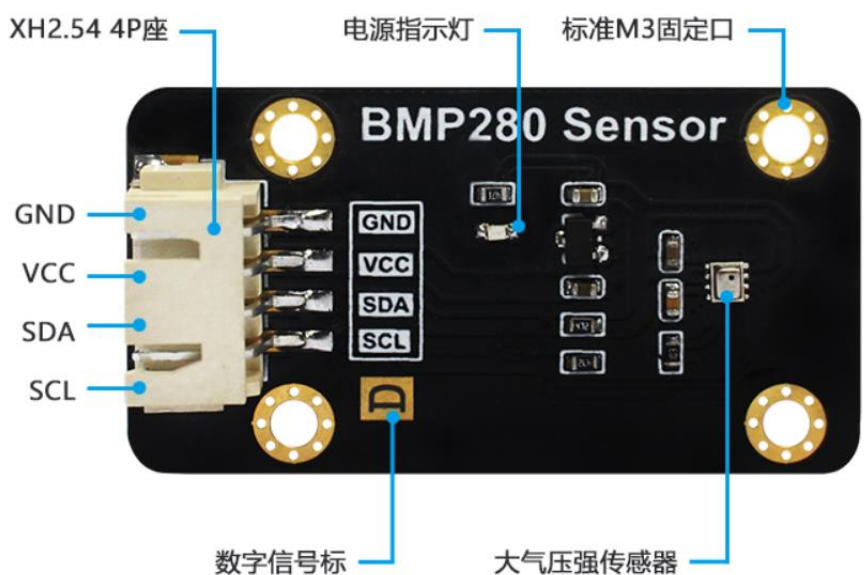


图 5-35 大气压强传感器模块

功能参数	
传感器型号	BMP280
供电电压	3.3-5V
工作电流	<20 mA
工作温度	-20 至 85℃
接口定义	XH2.54 防呆接口（4Pin）【GND、VCC、SDA、SCL】
通讯信号	I2C 总线
I2C 地址	0x76（BMP280 的 SDO 默认下拉）； 当 BMP280 的 SDO 引脚上拉时，I2C 地址为：0x77
模块尺寸	4.5*2.5cm

表 5-8

从上面介绍可以看到，BMP280 是一款通过 I2C 接口驱动的传感器。连接到我们的 I2C/4P 外扩接口上。我们通过前面学习的 I2C 接口使用的方式，即可以对该模块实现数据通讯。

标准大气压是指把温度为 0℃、纬度 45 度海平面（海拔为 0 米）上的气压称为 1 个大气压，其数值为 101325 帕斯卡（Pa）。

大气压同海拔高度的关系： $P=P_0 \times (1-H/44300)^{5.256}$

因此计算高度公式为： $H=44300 \times (1 - (P/P_0)^{1/5.256})$

式中：H 为海拔高度， P_0 =大气压（0℃，101325Pa）

从上面公式可以看到，高度是通过大气压强换算出来的，从物理学的角度我们可以知道，高度越高的地方，空气越稀薄，大气压强越低。通过气压的变化我们就可以计算出海拔高度；但是这存在特定条件，那就是温度为 0℃ 的时候，而温度越高的地方，空气越稀薄，大气压强就越低。因此高度数据理论上需要做温度补偿，因此本实验的高度值换算存在轻微误差。有兴趣的小伙伴可以自行深入研究。

MicroPython 的强大之处是其有丰富的模块和函数库，一旦模块建立了，那么后来者使用起来就非常简单，无须再去做底层驱动的开发。从而实现面向对象的编程，而当有需要的时候又可以去改底层代码，可以说是进可攻退可守，非常灵活。在这里我们直接调用已经编写好的驱动文件 `bmp280.py`，该文件实现了对大气压、温度、高度的测量和计算。用户直接使用即可，具体如下：

构造函数
<code>bmp280.BMP280(I2C)</code>
定义传感器的 I2C 接口。
使用方法
<code>getTemp()</code>
获取温度数据
<code>getPress()</code>
获取气压数据
<code>getAltitude()</code>
获取高度数据

5-9 大气压强传感器 BMP280 对象

打开示例程序的 `bmp280.py` 文件，找到 `getAltitude()` 的定义函数，可以看

到计算公式跟前面分析的一样，这里保留了 2 位小数：

```
# Calculating absolute altitude  
def getAltitude(self):  
    return '%.2f'%(44330*(1-(self.getPress()/101325)**(1/5.256)))
```

理解了传感器原理和模块使用方法后，我们可以整理出编程思路，流程图如下：

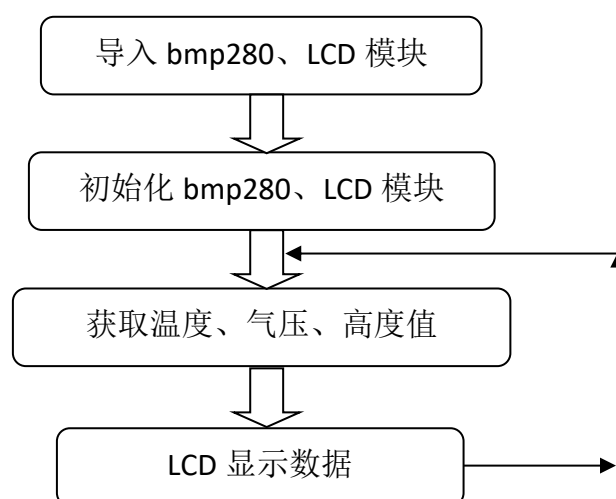


图 5-36 代码编写流程

参考程序代码如下：

```
...  
实验名称：大气压强传感器  
版本：v1.0  
日期：2020.12  
作者：01Studio 【www.01Studio.org】  
说明：测量 BMP280 温度、气压和计算海拔值，并在 OLED 上显示。。  
...  
  
#导入相关模块
```

```

import pyb,bmp280

from machine import Pin,I2C

from tftlcd import LCD43M

#定义常用颜色

WHITE=(255,255,255)

BLACK = (0,0,0)

#初始化 LCD

d=LCD43M()

d.fill(WHITE)#填充白色

#初始化 BMP280, I2C 接口 2

BMP = bmp280.BMP280(I2C(2))

#显示标题

d.printStr('01Studio BMP280', 40, 10, BLACK, size=4)

while True:

    #采集温度、压强、高度信息数据并用 LCD 显示:

    d.printStr('Temp: ' + str(BMP.getTemp()) + ' C', 10, 100, BLACK,
              size=4)

    d.printStr('Press: ' + str(BMP.getPress()) + ' Pa', 10, 200, BLACK,
              size=4)

    d.printStr('Altitude: ' + str(BMP.getAltitude()) + ' M', 10, 300,
              BLACK, size=4)

    pyb.delay(1000) #延时 1 秒

```

- **实验结果:**

程序烧写完复位后可以见到实验结果。测量出温度、大气压和计算得到的海拔高度。



图 5-37 BMP280 传感器测量

开发板接上锂电池供电，尝试上下楼，会发现气压和海拔数值相应的变化。

- **总结:**

本节实现了大气压强传感器模块 **BMP280** 的应用，而且实验结果的精度很高，有兴趣的小伙伴可以结合自己情况，打造一个属于自己的气压计，然后到户外暴走！

5.8 超声波传感器

- **前言：**

超声波传感器是一款测量距离的传感器。其原理是利用声波在遇到障碍物反射接收结合声波在空气中传播的速度计算的得出。在测量、避障小车，无人驾驶等领域都有相关应用。

- **实验平台：**

达芬奇 MicroPython 开发套件和超声波传感器模块，传感器模块连接到 I2C/UART 扩展接口。

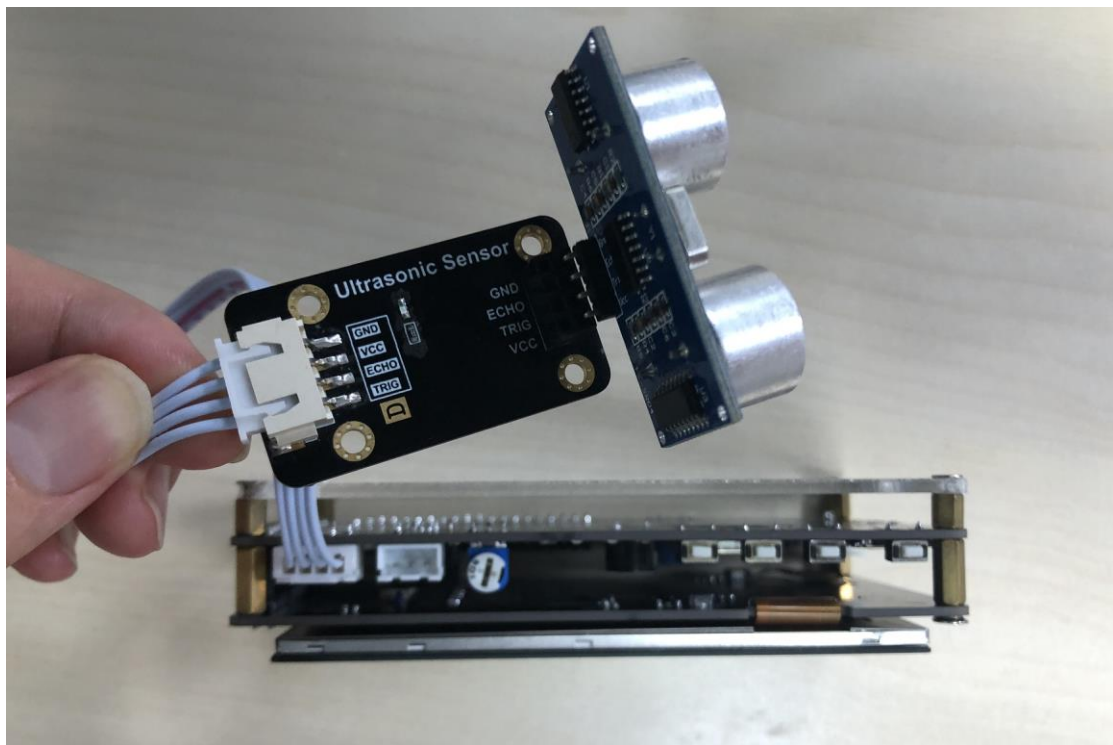


图 5-38 开发套件与传感器连接

- **实验目的：**

测量距离并在 LCD 上显示相关数据。

- **实验讲解：**

我们先来看看超声波传感器模块的介绍：

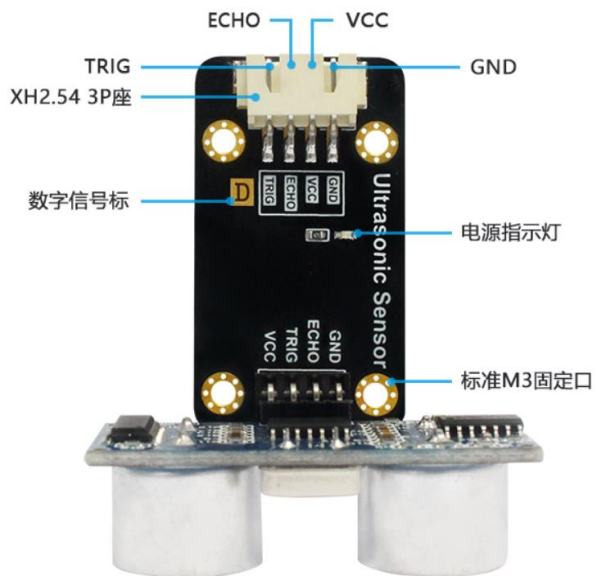


图 5-39 超声波模块

功能参数	
传感器型号	HCSR04
供电电压	3.3-5V
工作电流	<20 mA
工作温度	-20 至 85℃
接口定义	XH2.54 防呆接口（4Pin） 【GND、VCC、ECHO、TRIG】
通讯信号	IO 数字接口
测量距离	2-450 cm
测量精度	0.5 cm
整体尺寸	5.5*4.5*3.0 cm

表 5-10

超声波传感器模块使用两个 IO 口分别控制超声波发送和接收，工作原理如下：

1. 给超声波模块接入电源和地；
2. 给脉冲触发引脚（trig）输入一个长为 20us 的高电平方波；

- 3. 输入方波后，模块会自动发射 8 个 40KHz 的声波，与此同时回波引脚（echo）端的电平会由 0 变为 1；（此时应该启动定时器计时）
- 4. 当超声波返回被模块接收到时，回波引脚端的电平会由 1 变为 0；（此时应该停止定时器计数），定时器记下的这个时间即为超声波由发射到返回的总时长；
- 5. 根据声音在空气中的速度为 340 米/秒，即可计算出所测的距离。

要学习和应用传感器，学会看懂传感器的时序图是很关键的，所以我们来看一下超声波传感器 HCSR04 的时序触发图。

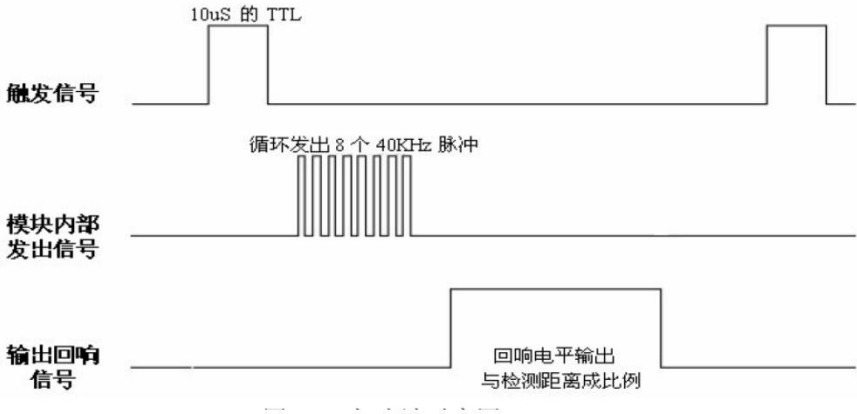


图 5-40 超声波传感器 HCSR04 时序图

以上普及了超声波传感器的原理，我们已经将其集成在 HCSR04.py 文件，如想了解代码原理可以打开 HCSR04.py 文件查看代码实现原理。使用 MicroPython 开发的用户只需要直接使用即可。使用方法如下：

构造函数
HCSR04.HCSR04(trig,echo)
定义连接传感器的 trig 和 echo 引脚
使用方法
getDistance()
获取距离数据

表 5-11 超声波传感器 HCSR04 对象

从上表看到，超声波传感器的使用方法非常简单，使用 2 个代码就完成对距离信息的采集，代码编写流程如下：

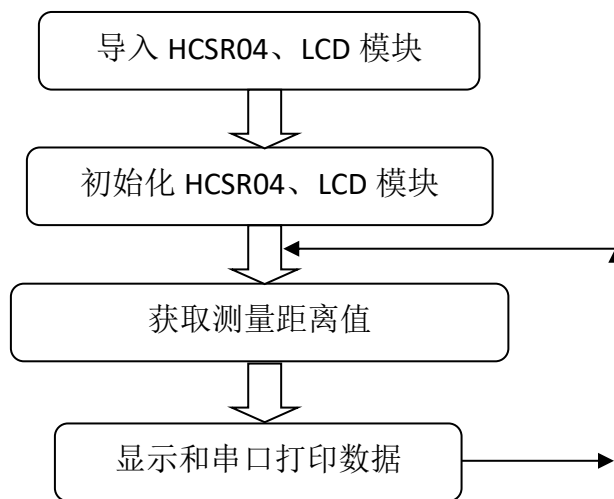


图 5-41 代码编写流程

主程序 main.py 参考代码如下：

```
'''
实验名称：超声波传感器
版本：v1.0
日期：2021.5
作者：01Studio 【www.01Studio.org】
说明：通过超声波传感器测距，并在 OLED 上显示。
'''

#导入相关模块
import time
from machine import Pin
from HCSR04 import HCSR04
from tftlcd import LCD43R

#定义常用颜色
WHITE=(255,255,255)
BLACK = (0,0,0)
```

```

BLUE = (0,0,255)

#初始化 LCD
d=LCD43R()
d.fill(WHITE)#填充白色

#初始化接口 trig='PA2',echo='PA3'
trig = Pin('A2',Pin.OUT_PP)
echo = Pin('A3',Pin.IN)
HC=HCSR04(trig,echo)

#显示标题
d.printStr('01Studio Distance', 40, 10, BLUE, size=4)

while True:

    Distance = HC.getDistance() #测量距离

    #采集超声波距离数据并用 LCD 显示:
    d.printStr(str(Distance) + ' CM ', 10, 100, BLACK, size=4)

    print(str(Distance) + ' CM')

    time.sleep(1) #延时 1 秒

```

● **实验结果:**

在超声波传感器 20cm 外放置障碍物, 可以看到 LCD 显示屏实时显示距离数据约为 20cm。



图 5-42 障碍物距离为 20cm

● **总结：**

除去 LCD 显示代码，我们实际上只用了 2 行代码：初始化和调用测量函数就实现了对超声波传感器测距的应用。这让我们再一次感受到了 MicroPython 的魅力。赶快动手制作自己的避障小车和其他好玩的创作吧。

5.9 继电器

- **前言：**

我们知道我们的开发板 GPIO 输出的电平是 3.3V 的，这是不能直接控制一些高电压的设备，比如电灯（220V）。这时候就可以使用我们常用的低压控制高压元件—继电器。

- **实验平台：**

达芬奇 MicroPython 开发套件和继电器模块。



图 5-43 开发套件与传感器连接

- **实验目的：**

使用按键控制继电器通断。

- **实验讲解：**

我们先来看看继电器模块的介绍：



图 5-44 超声波模块

功能参数	
供电电压	3.3V
触发方式	低电平触发
高压连接	最大 250V/3A
接口定义	XH2.54 防呆接口（3Pin） 【GND、VCC、IO】
整体尺寸	4.5*2.5 cm

表 5-12 继电器模块功能参数

继电器可以理解成是一个开关，相当于我们平时家里面的电灯开关面板一样，只是现在使用单片机的 GPIO 来控制。继电器低压控制高压电器一个比较典型的接线图如下图所示：

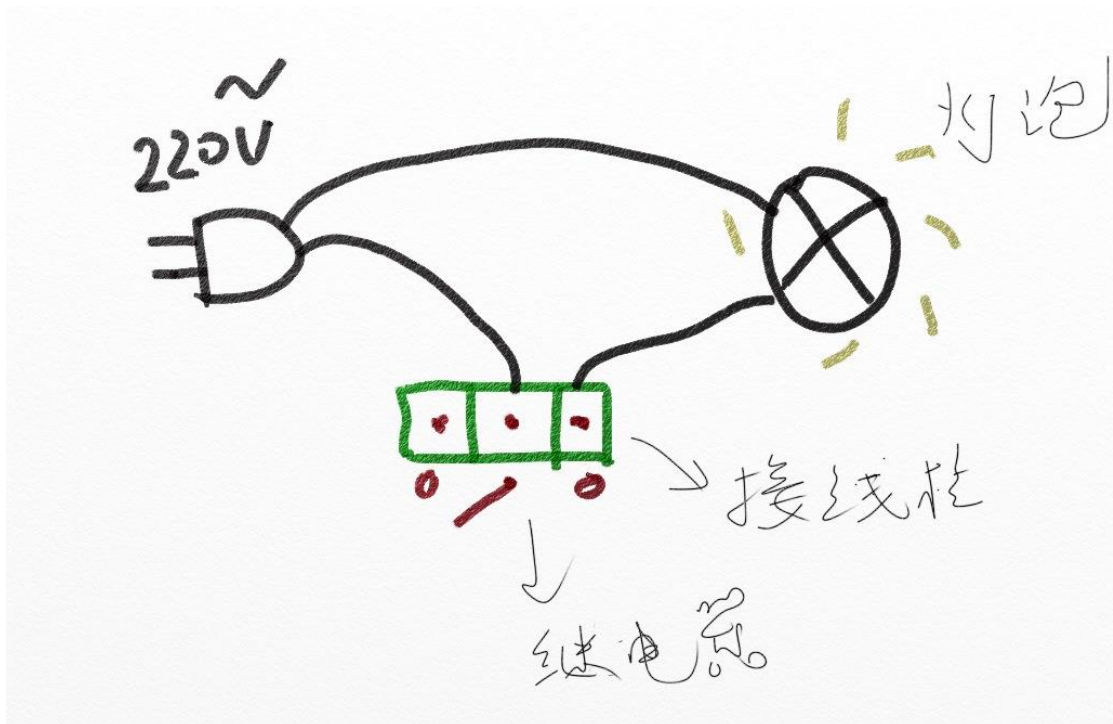


图 5-45 继电器接线

继电器的控制原理非常简单，跟 LED 控制方式一样，只是使用低电平 ‘0’ 表示继电器开，高电平 ‘1’ 表示继电器关。

我们可以参考基础实验—按键外部中断实验例程来使用继电器，请参考：[4.4 外部中断](#) 章节内容，这里不再重复！

继电器连接到达芬奇开发板的 ‘B1’ 引脚，我们使用 ‘A0’ 按键来控制，编程流程图如下：

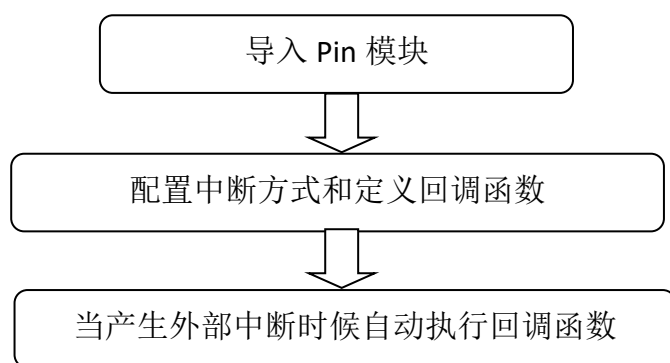


图 5-46 代码编写流程

主程序 main.py 参考代码如下：

```

'''
实验名称：继电器模块
版本： v1.0
日期： 2021-4
作者： 01Studio
实验平台： 达芬奇
社区： www.01studio.org
'''

from machine import Pin
import time

#构建继电器对象
relay=Pin('B1', Pin.OUT,value=1)

#配置按键
key = Pin('A0', Pin.IN, Pin.PULL_DOWN)

state=0 #LED 引脚状态

#继电器通断
def fun(key):

    global state
    time.sleep_ms(10) #消除抖动
    if key.value()==1: #确认按键被按下,开发板按下拉高
        state = not state
        relay.value(state)

key.irq(fun,Pin.IRQ_RISING) #定义中断，下降沿触发

```

- **实验结果:**

通过按键 A0 来控制继电器通断:



图 5-47 障碍物距离为 20cm

- **总结:**

继电器的控制方式非常简单，用途非常广。只需要一个简单的 GPIO 高低电平即可实现控制。

第6章 通讯实验

很多时候我们需要将信息往外传输，比如采集到了温湿度数据希望传给电脑、传给云服务器等。这便是通讯，通讯可以是外设发送数据给达芬奇开发板，也可以是开发板主动往外发数据，但前提是需要有双方约定的协议才能保证通讯的有效性，本章我们将学习常见的通讯实验，如 UART 串口、网络通讯、485、CAN 等等。

作为万能胶水的 python 编程语言，可以说让复杂的通讯协议变得非常简单易懂。事不宜迟，马上开始我们的学习。

6.1 UART（串口通信）

● 前言：

串口是非常常用的通信接口，有很多工控产品、无线透传模块都是使用串口来收发指令和传输数据，这样用户就可以在无须考虑底层实现原理的前提下将各类串口功能模块灵活应用起来。

● 实验平台：

达芬奇开发板，需要使用 USB 转 TTL 工具。



图 6-1 达芬奇开发板

● 实验目的：

编程实现串口收发数据。

● 实验讲解：

达芬奇开发板一共有 5 个串口，编号是 1-5，其中串口 5 用于 REPL 调试，因此并没有开放使用。其余串口号和引脚关系如下：

UART(1)	TX	PA9
	RX	PA10

UART(2)	TX	PA2
	RX	PA3
UART(3)	TX	PB10
	RX	PB11
UART(4)	TX	PD7
	RX	PD6

表 6-1 达芬奇串口引脚资源

我们来了解一下串口对象的构造函数和使用方法：

构造函数
uart=machine.UART(id,baudrate, bits=8, parity=None, stop=1,...)
创建 UART 对象。 【id】 1-4（5 已用于 REPL，不能使用） 【baudrate】 波特率，常用 115200、9600 【bits】 数据位 【parity】 校验；默认 None, 0(偶校验)，1(奇校验) 【stop】 停止位，默认 1 ...
使用方法
uart.deinit()
关闭串口
uart.any()
返回等待读取的字节数据，0 表示没有
uart.read([nbytes])
【nbytes】 读取字节数
UART.readline()
读行
UART.write(buf)
【buf】 串口 TX 写数据

*更多使用说明请阅读官方文档：

http://docs.micropython.01studio.org/zh_CN/latest/library/machine.UART.html

表 6-2 machine 的 UART 对象

我们可以用一个 USB 转 TTL 工具，配合电脑上位机串口助手来跟 MicroPython 开发板模拟通信。

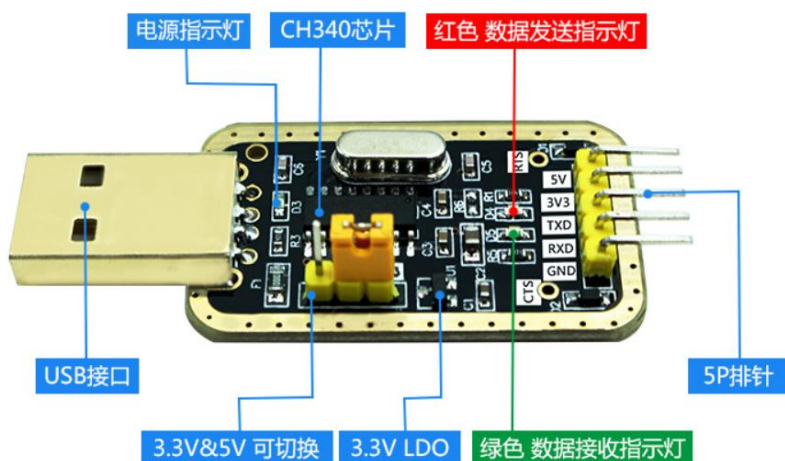


图 6-2 常用 USB 转串口工具

注意要使用 3.3V 电平的 USB 转串口 TTL 工具，本实验我们使用串口 2，也就是 PB10（TX）和 PB11（RX），接线示意图如下：



图 6-3 串口接线图

在本实验中我们可以先初始化串口，然后给串口发去一条信息，这样 PC 机的串口助手就会在接收区显示出来，然后进入循环，当检测到有数据可以接收时候就将数据接收并打印，并通过 REPL 打印显示。代码编写流程图如下：

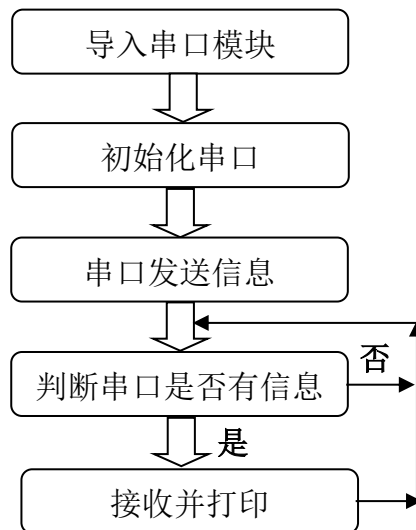


图 6-4 代码编程流程图

参考代码：

```

...

实验名称：串口通信
版本：v1.0
日期：2021.1
作者：01Studio
实验平台：达芬奇
说明：通过编程实现串口通信，跟电脑串口助手实现数据收发。
...

#导入串口模块
from machine import UART
import time

uart=UART(2, 115200) #设置串口号 1 和波特率，MCU_TX--A2，MCU_RX--A3
time.sleep_ms(100) #等待串口初始化稳定
uart.write('Hello 01Studio!')#发送一条数据

```



```
while True:

    #判断有无收到信息

    if uart.any():

        text=uart.read(128) #接收 128 个字符

        print(text) #通过 REPL 打印串口 3 接收的数据
```

● **实验结果:**

我们按照上述方式将 USB 转 TTL 的 TX 接到开发板的 RX (A3)，USB 转 TTL 的 RX 接到开发板的 TX (A2)。GND 接一起，3.3V 可以选择接或不接。



图 6-5 将达芬奇和 usb ttl 工具同时接入电脑

这时候打开电脑的设备管理器，能看到 2 个 COM。写着 CH340 的是串口工具，另外一个则是达芬奇的串口 REPL。如果 CH340 驱动没安装，则需要手动安装，驱动在：配套资料包→开发工具→windows→串口终端→CH340 文件夹下。

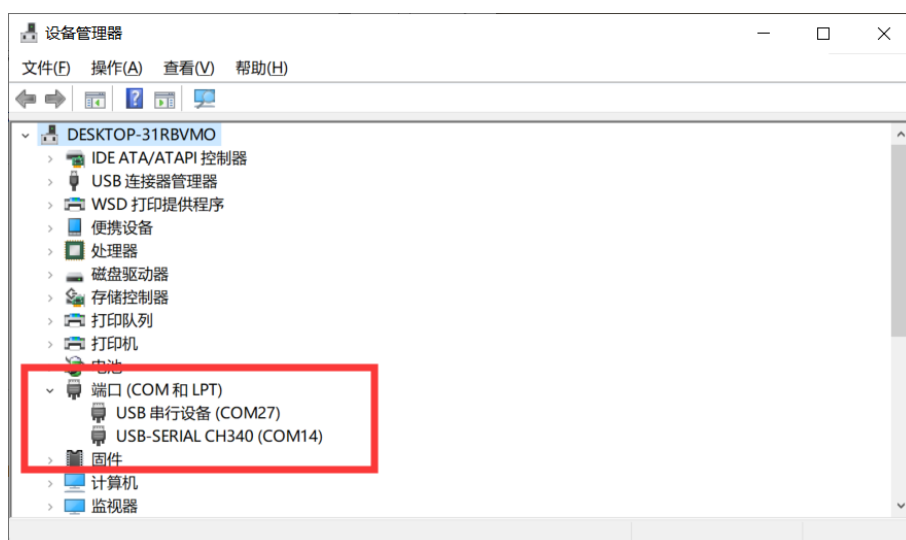


图 6-6

本实验要用到串口助手，打开配套资料包→开发工具→windows→串口终端工具下的【UartAssist.exe】软件。

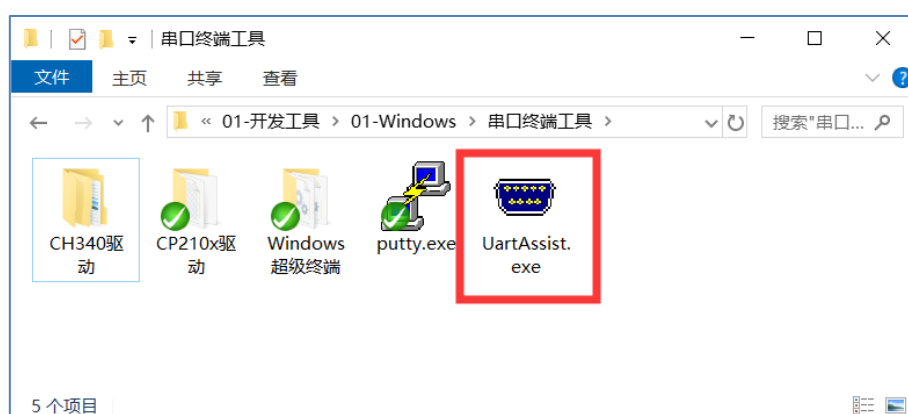


图 6-7

根据上图设备管理器里面的信息，将串口工具配置成 COM14，REPL 串口配置成 COM27（根据自己的串口号调整）。波特率 115200。运行程序，可以看到一开始串口助手收到达芬奇上电发来的信息“Hello 01Studio!”。我们在串口助手的发送端输入“<http://www.01studio.org>”，点击发送，可以看到达芬奇在接收到该信息后在 REPL 里面打印了出来。如下图所示：（达芬奇首次发送给电脑的字符前可能会出现一个乱码，可以忽略。）

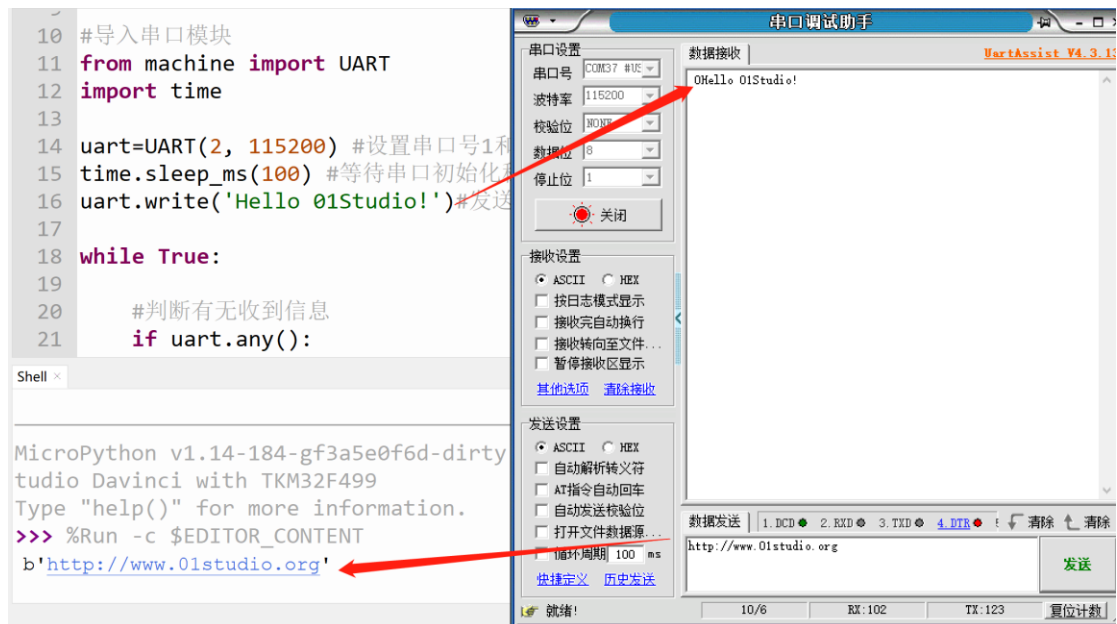


图 6-8 串口收发实验

● 总结:

通过本节我们学会了串口收发应用，达芬奇的串口资源非常丰富，因此可以接非常多的串口外设。从而实验更多的功能。

6.2 网络通讯

达芬奇开发板板载一个 ESP8266 串口透传模块，固件使用上海乐鑫官方透传固件。该 WiFi 通过串口（连接达芬奇的 UART1）跟达芬奇通讯，达芬奇实现连接无线路由器，从而实现 Socket、MQTT、等相关无线和物联网应用。

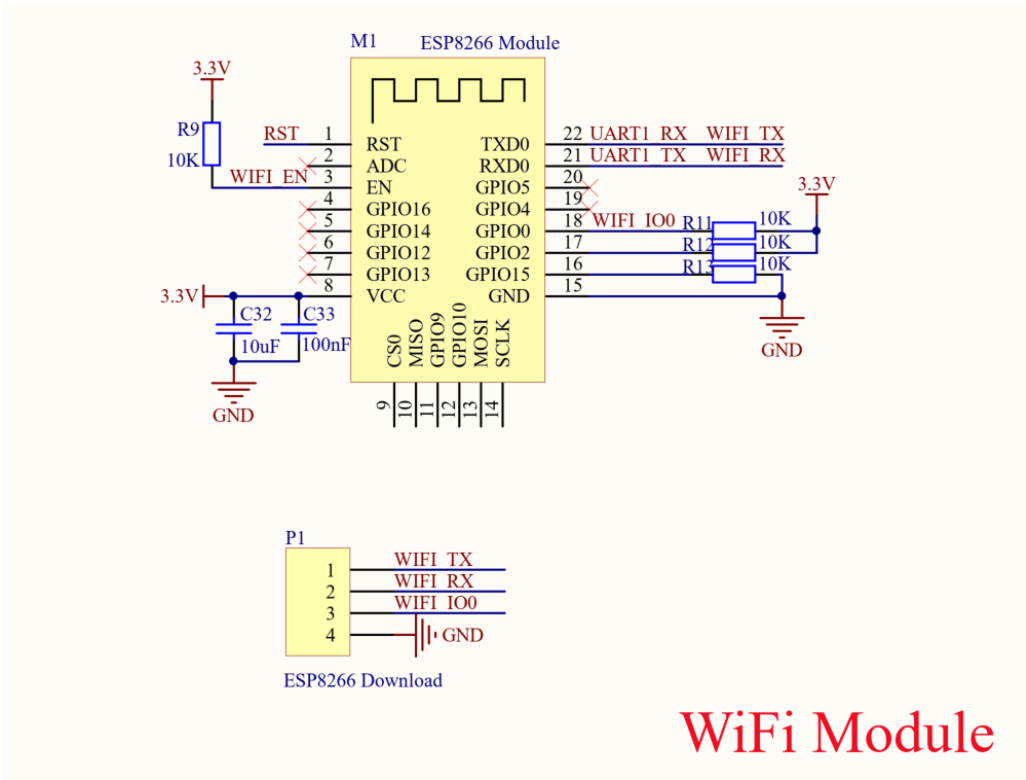


图 6-9 WiFi 模块原理图

功能参数	
WiFi 主控	ESP8266
控制方式	UART（串口）
WiFi 协议	802.11 b/g/n
加密类型	WPA/WPA2
工作电流	80mA (3.3V)

表 6-3 ESP8266 WiFi 模块参数

6.2.1 连接无线路由器

● 前言：

WIFI 是物联网中非常重要的角色，现在基本上家家户户都有 WIFI 网络了，通过 WIFI 接入到互联网，成了智能家居产品普遍的选择。而要想上网，首先需要连接上无线路由器。这一节我们就来学习如何通过 WiFi 扩展模块让达芬奇开发板通过 MicroPython 编程实现连上路由器。

● 实验平台：

达芬奇开发套件。



图 6-10 达芬奇开发套件

● 实验目的：

编程实现连接路由器，将 IP 地址等相关信息通过串口终端打印（只支持 2.4G 网络）。

● 实验讲解：

连接路由器上网是我们每天都做的事情，日常生活中我们只需要知道路由器的账号和密码，就能使用电脑或者手机连接到无线路由器，然后上网冲浪。

达芬奇开发板的 MicroPython 固件已经集成了 network 模块，开发者使用内置的 network 模块函数可以非常方便地连接上路由器。

我们先来看看 network 基于 WiFi（WLAN 模块）的构造函数和使用方法。

构造函数
<code>wlan = network.ESP8266 (uart)</code>
构建 WIFI 模块连接对象。ESP8286 表示模块型号。 【uart】串口对象
使用方法
<code>wlan.scan ()</code>
扫描允许访问的 SSID。
<code>wlan.isconnected()</code>
检查设备是否已经连接上。返回 Ture:已连接; False: 未连接。
<code>wlan.connect(ssid,password)</code>
WIFI 连接。 【ssid】账号; 【password】密码;
<code>wlan.ifconfig()</code>
查看 wifi 连接信息。
<code>wlan.disconnect()</code>
断开连接。
<code>wlan.enable_ap(ssid,key,ch1=5,ecn=3)</code>
AP 热点使能; 【ssid】热点名称; 【key】密码; 【ch1】wifi 信号的信道; 【ecn】加密方法
<code>wlan.disable_ap()</code>
关闭热点。
*更多参数说明请阅读官方文档: 文档链接: https://maixpy.sipeed.com/zh/libs/machine/network.html

表 6-4 K210 network 对象

从上表可以看到 MicroPython 通过模块封装，让 WIFI 联网变得非常简单。模块包含热点 AP 模式和客户端 STA 模式，热点 AP 是指电脑端直接连接 K210 发出的热点实现连接，但这样你的电脑就不能上网了，因此我们一般情况下都是使用 STA 模式。也就是电脑和设备同时连接到相同网段的路由器上。

代码编写流程如下：

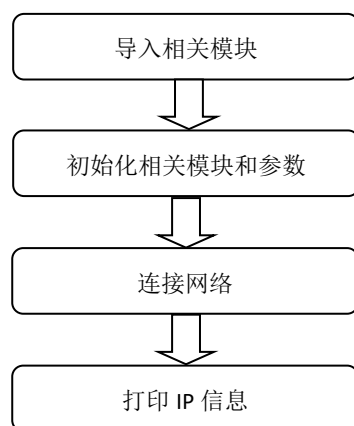


图 6-11 代码编写流程图

实验参考代码如下：

```
'''
实验名称：WiFi 无线连接实验（串口 WiFi 模块）
版本：v1.0
日期：2021.5
作者：01Studio
实验平台：01Studio-达芬奇
说明：pyAI-K210 通过 WiFi 拓展模块连接无线路由器
'''

import network, time
from machine import UART, Pin
from tftlcd import LCD43R
```

```

#定义颜色

BLACK = (0,0,0)

WHITE = (255,255,255)

RED=(255,0,0)

GREEN=(0,255,0)

BLUE=(0,0,255)


#LCD 初始化

d = LCD43R(portrait=1)

d.fill(WHITE)


SSID='WeBee_office_2.4G' # WiFi 账号
KEY='webee0123456789' # WiFi 密码


##### WiFi 模块初始化 #####

uart = UART(1,115200)

wlan = network.ESP8266(uart)


WIFI_LED=Pin('C7', Pin.OUT) #初始化 WIFI 指示灯,LED4 蓝灯


#显示标题

d.printStr('01Studio WiFi Connect:', 40, 10, BLUE, size=4)


#WIFI 连接函数

def WIFI_Connect():

    print('Connecting to network...')#正在连接提示打印

    d.printStr('Connecting... ', 10, 100, BLACK, size=3)

    wlan.connect(SSID, KEY) #输入 WIFI 账号密码

```



```

if wlan.isconnected(): #连接成功

    #LED 点亮
    WIFI_LED.value(1)

    info=wlan.ifconfig()

    #串口打印信息
    print('IP information:')
    print(wlan.ifconfig())

    #;CD 显示 IP 信息
    d.printStr('IP: ' + wlan.ifconfig()[0], 10, 100, BLACK, size=3)
    d.printStr('Subnet: ' + wlan.ifconfig()[1], 10, 150, BLACK,
               size=3)
    d.printStr('Gateway: ' + wlan.ifconfig()[2], 10, 200, BLACK,
               size=3)

else: #连接失败

    print('Connect Fail!')
    d.printStr('Connect Fail!' + nic.ifconfig()[0], 10, 100, BLACK,
               size=3)

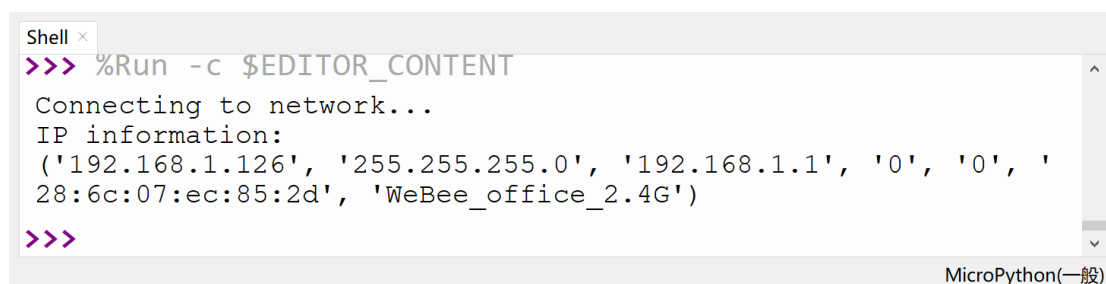
#执行 WIFI 连接函数
WIFI_Connect()

```

上面代码在 `main.py` 文件中，我们也可以新建一个 `WIFI.py` 文件，然后通过模块引用方式来供 `main.py` 调用，以提高程序的灵活性。

- **实验结果：**

运行程序，可以观察到 `wifi` 连接成功后串口终端打印 IP 等信息。



```
Shell x
>>> %Run -c $EDITOR_CONTENT
Connecting to network...
IP information:
('192.168.1.126', '255.255.255.0', '192.168.1.1', '0', '0', '
28:6c:07:ec:85:2d', 'WeBee_office_2.4G')
>>>
```

MicroPython(一般)

图 6-12 串口 REPL 打印 IP 信息

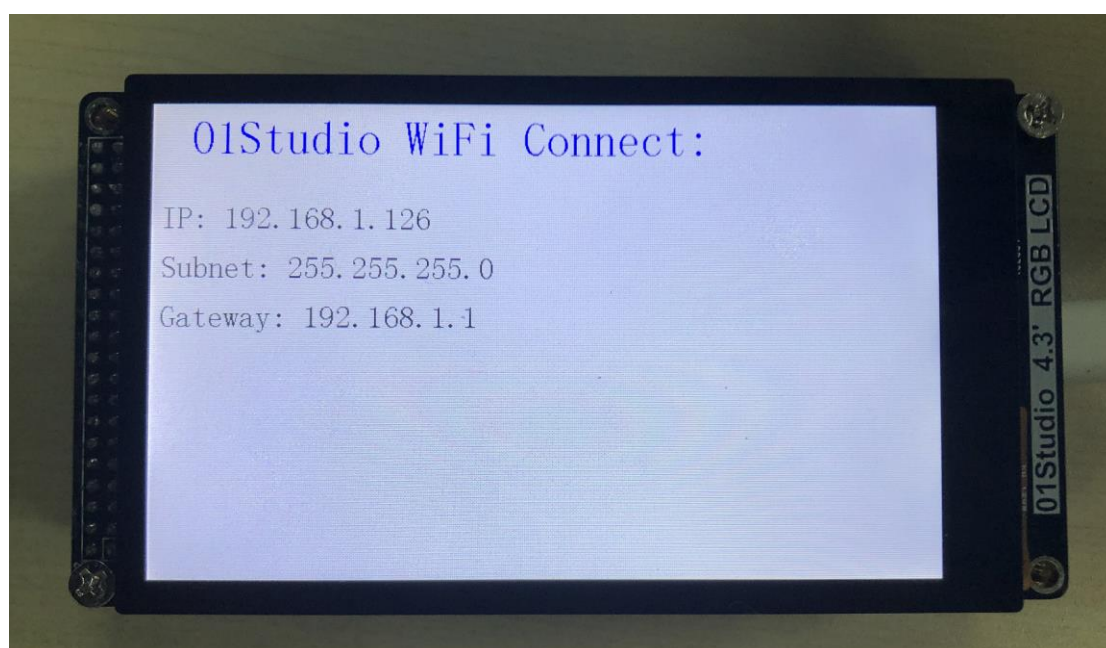


图 6-13 显示屏显示 IP 信息

- **总结：**

本节是 `WIFI` 应用的基础，成功连接到无线路由器的实验后，后面就可以做 `socket` 和 `MQTT` 等相关网络通信的应用了。

6.2.2 Socket 通信

- **前言：**

上一节我们学习了如何通过 MicroPython 编程实现 pyAI-K210 结合 WiFi 拓展模块连接到无线路由器。这一节我们则来学习一下 Socket 通信实验。Socket 几乎是整个互联网通信的基础。

- **实验平台：**

达芬奇开发板。



图 6-14 达芬奇开发套件

- **实验目的：**

通过 Socket 编程实现达芬奇开发板与电脑服务器助手建立连接，相互收发数据。

- **实验讲解：**

Socket 我们听得非常多了，但由于网络工程是一门系统工程，涉及的知识非常广，概念也很多，任何一个知识点都能找出一堆厚厚的书，因此我们经常会混淆。在这里，我们尝试以最容易理解的方式来讲述 Socket，如果需要全面了解，可以自行查阅相关资料学习。

我们先来看看网络层级模型图，这是构成网络通信的基础：

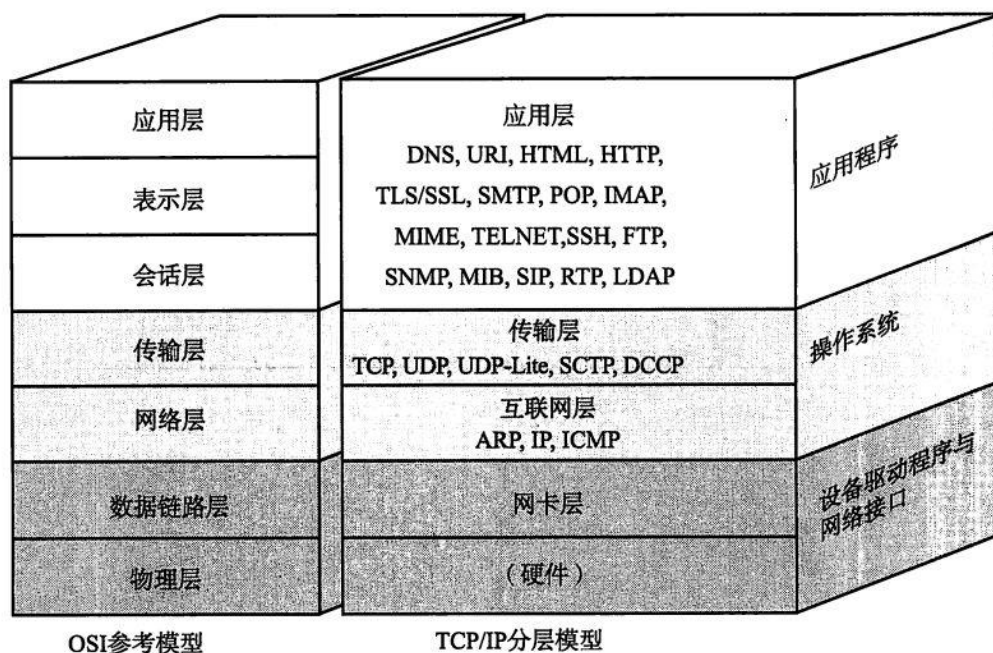


图 6-15 网络层级模型

我们看看 TCP/IP 模型的传输层和应用层，传输层比较熟悉的概念是 TCP 和 UDP，UDP 协议基本就没有对 IP 层的数据进行任何的处理了。而 TCP 协议还加入了更加复杂的传输控制，比如滑动的数据发送窗口（Slice Window），以及接收确认和重发机制，以达到数据的可靠传送。应用层中网页常用的则是 HTTP。那么我们先来解析一下这 TCP 和 HTTP 两者的关系。

我们知道网络通信是最基础是依赖于 IP 和端口的，HTTP 一般情况下默认使用端口 80。举个简单的例子：我们逛淘宝，浏览器会向淘宝网的网址（本质是 IP）和端口发起请求，而淘宝网收到请求后响应，向我们手机返回相关网页数据信息，实现了网页交互的过程。而这里就会引出一个多人连接的问题，很多人访问淘宝网，实际上接收到网页信息后就断开连接，否则淘宝网的服务器是无法支撑这么多人长时间的连接的，哪怕能支持，也非常占资源。

也就是应用层的 HTTP 通过传输层进行数据通信时，TCP 会遇到同时为多个应用程序进程提供并发服务的问题。多个 TCP 连接或多个应用程序进程可能需要通过同一个 TCP 协议端口传输数据。为了区别不同的应用程序进程和连接，许多计算机操作系统为应用程序与 TCP / IP 协议交互提供了套接字(Socket)接口。应用层可以和传输层通过 Socket 接口，区分来自不同应用程序进程或网络连接的通信，实

现数据传输的并发服务。

简单来说，Socket 抽象层介于传输层和应用层之间，跟 TCP/IP 并没有必然的联系。Socket 编程接口在设计的时候，就希望也能适应其他的网络协议。

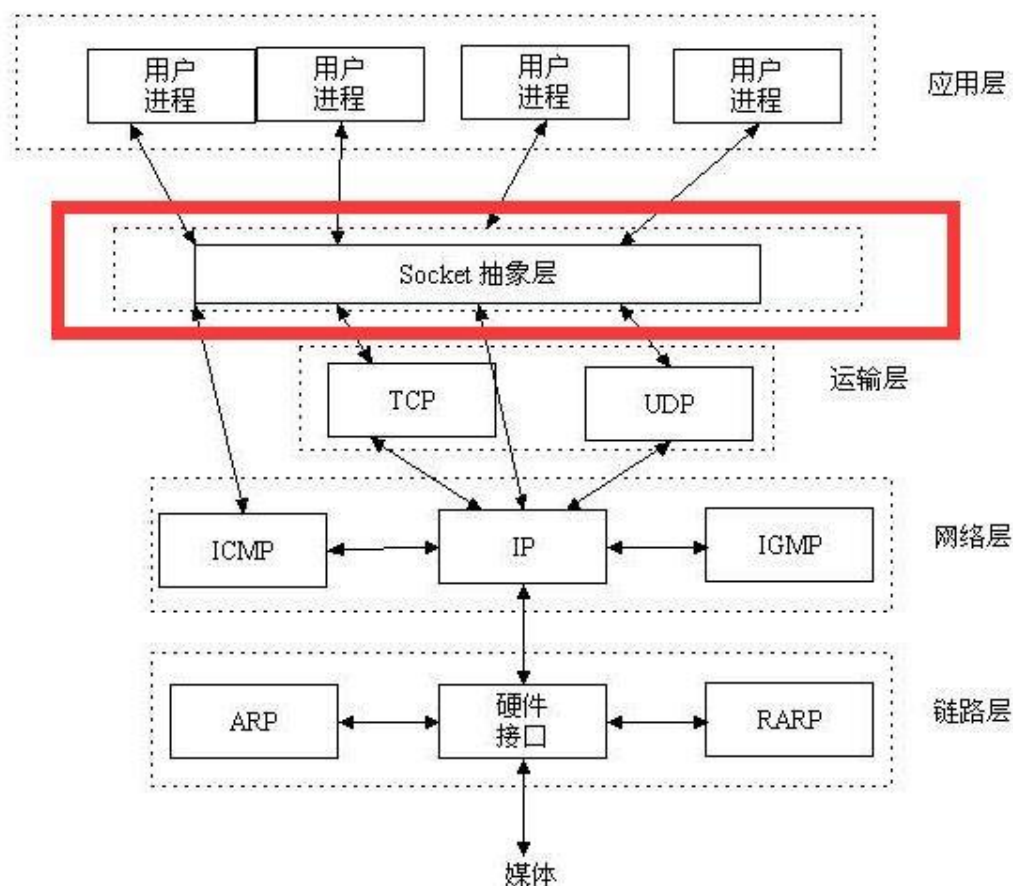


图 6-16 Socket 抽象层

套接字 (socket) 是通信的基石，是支持 TCP/IP 协议的网络通信的基本操作单元。它是网络通信过程中端点的抽象表示，包含进行网络通信必须的五种信息：连接使用的协议（通常是 TCP 或 UDP），本地主机的 IP 地址，本地进程的协议端口，远地主机的 IP 地址，远地进程的协议端口。

所以，socket 的出现只是可以更方便的使用 TCP/IP 协议栈而已，简单理解就是其对 TCP/IP 进行了抽象，形成了几个最基本的函数接口。比如 create, listen, accept, connect, read 和 write 等等。以下是通讯流程：

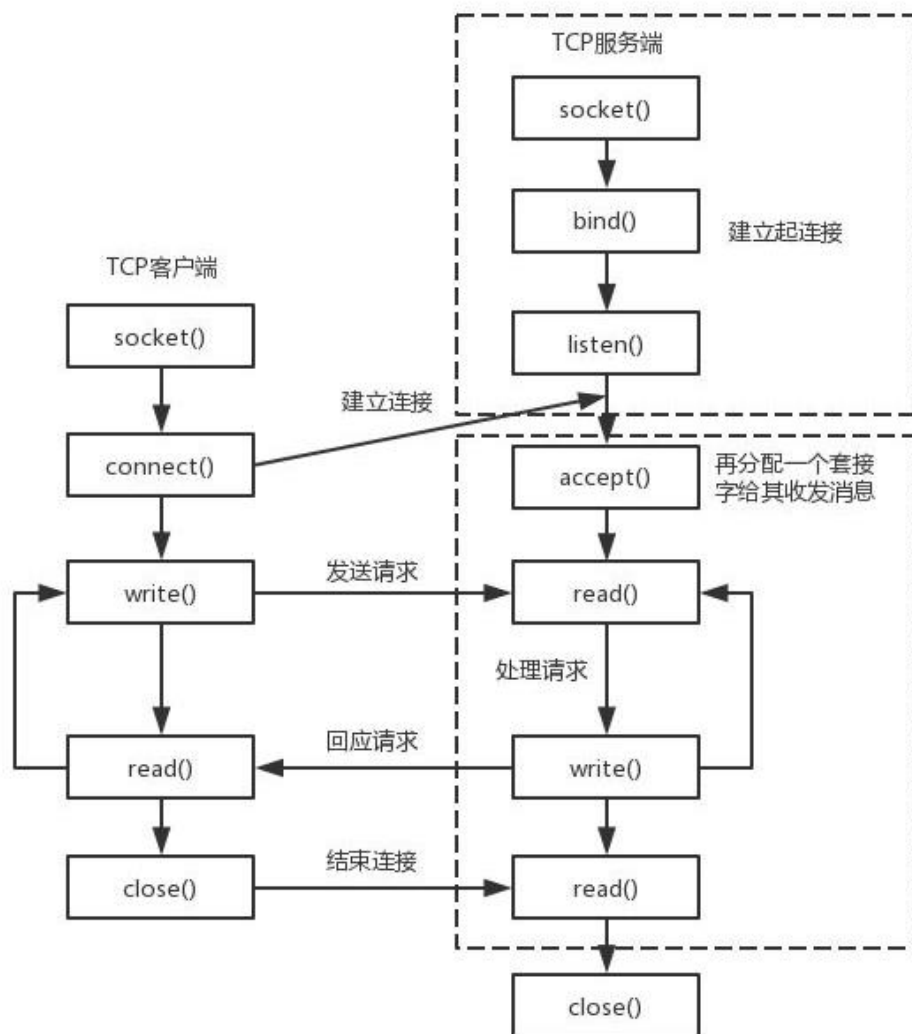


图 6-17 Socket 通信过程

从上图可以看到，建了 Socket 通信需要一个服务器端和一个客户端，以本实验为例，pyAI-K210 作为客户端，电脑使用网络调试助手作为服务器端，双方使用 TCP 协议传输。对于客户端，则需要知道电脑端的 IP 和端口即可建立连接。（端口可以自定义，范围在 0~65535，注意不占用常用的 80 等端口即可。）

以上的内容，简单来说就是如果用户面向应用来说，那么 K210 只需要知道通讯协议是 TCP 或 UDP、服务器的 IP 和端口号这 3 个信息，即可向服务器发起连接和发送信息。就这么简单。

MicroPython 已经封装好相关模块 `usocket`,跟传统的 `socket` 大部分兼容，两

者均可使用，本实验使用 `usocket`，对象如下介绍：

构造函数
<code>s=usocket.socket(af=AF_INET, type=SOCK_STREAM, proto=IPPROTO_TCP)</code>
构建 <code>usocket</code> 对象。 af: AF_INET→IPV4, AF_INET6 → IPV6; type: SOCK_STREAM→TCP, SOCK_DGRAM→UDP; proto: IPPROTO_TCP→TCP 协议, IPPROTO_UDP→UDP 协议。 (如果要构建 TCP 连接, 可以使用默认参数配置, 即不输入任何参数。)
使用方法
<code>addr=usocket.getaddrinfo('www.01studio.org', 80)[0][-1]</code>
获取 Socket 通信格式地址。返回: ('47.91.208.161', 80)
<code>s.connect(address)</code>
创建连接。address: 地址格式为 IP+端口。例: ('192.168.1.115', 10000)
<code>s.send(bytes)</code>
发送。bytes: 发送内容格式为字节
<code>s.recv(bufsize)</code>
接收数据。bufsize: 单次最大接收字节个数。
<code>s.bind(address)</code>
绑定, 用于服务器角色
<code>s.listen([backlog])</code>
监听, 用于服务器角色。backlog: 允许连接个数, 必须大于 0。
<code>s.accept()</code>
阻止套接字操作设置超时。value 参数可以是表示秒的非负浮点数, 也可以是 None。本例程使用 0.1;
<code>s.settimeout()</code>
接受连接, 用于服务器角色。
*其它更多用法请阅读 MicroPython 文档: (搜索: usocket) 文档链接: http://docs.openmv.io/library/usocket.html

表 6-5 Socket 对象

本实验中达芬奇开发板属于客户端，因此只用到客户端的函数即可。实验代码编写流程如下：

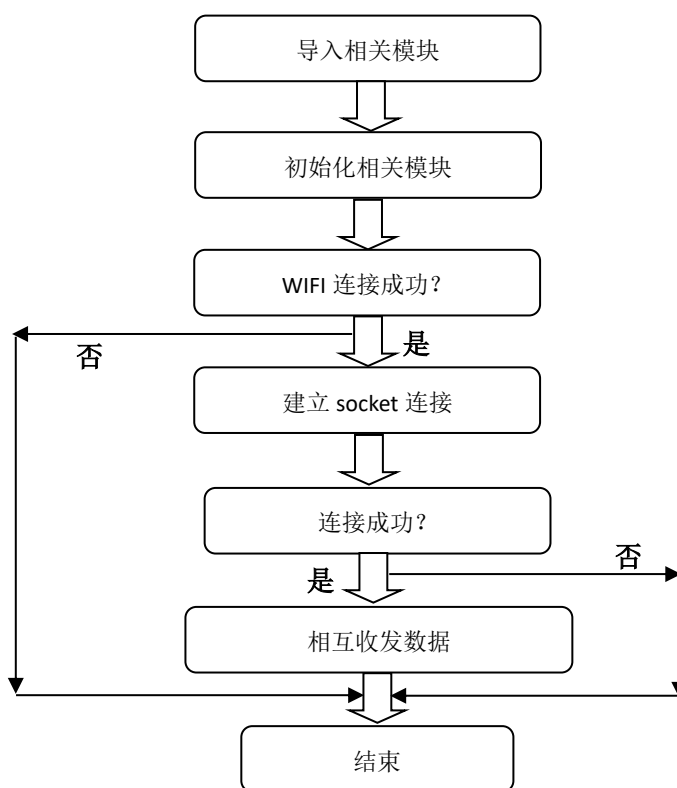


图 6-18 代码编写流程图

实验参考代码：

```
...  
实验名称：TCP 客户端（Socket 通信） 实验  
版本：v1.0  
日期：2021.5  
作者：01Studio  
实验平台：01Studio-达芬奇  
说明：通过 WiFi 模块编程实现达芬奇的 Socket 通信，数据收发。  
...  
  
import network, usocket, time  
from machine import UART, Pin, Timer
```



```

from tftlcd import LCD43R

#定义颜色

BLACK = (0,0,0)
WHITE = (255,255,255)
RED=(255,0,0)
GREEN=(0,255,0)
BLUE=(0,0,255)

#LCD 初始化

d = LCD43R(portrait=1)
d.fill(WHITE)

#wifi 信息

SSID='WeBee_office_2.4G' # WiFi 账号
KEY='webee0123456789' # WiFi 密码

##### WiFi 模块初始化 #####

uart = UART(1,115200)
wlan = network.ESP8266(uart)

#socket 数据接收中断标志位

socket_node = 0

WIFI_LED=Pin('C7', Pin.OUT) #初始化 WIFI 指示灯,LED4 蓝灯

#显示标题

d.printStr('01Studio WiFi Connect:', 40, 10, BLUE, size=4)

#WIFI 连接函数

```

```

def WIFI_Connect():

    print('Connecting to network...')#正在连接提示打印
    d.printStr('Connecting... ', 10, 100, BLACK, size=3)
    wlan.connect(SSID, KEY) #输入 WIFI 账号密码

    if wlan.isconnected(): #连接成功

        #LED 点亮
        WIFI_LED.value(1)

        info=wlan.ifconfig()

        #串口打印信息
        print('IP information:')
        print(wlan.ifconfig())

        #;CD 显示 IP 信息
        d.printStr('IP: ' + wlan.ifconfig()[0], 10, 100, BLACK, size=3)
        d.printStr('Subnet: ' + wlan.ifconfig()[1], 10, 150, BLACK,
size=3)
        d.printStr('Gateway: ' + wlan.ifconfig()[2], 10, 200, BLACK,
size=3)

    else: #连接失败

        print('Connect Fail!')
        d.printStr('Connect Fail!' + nic.ifconfig()[0], 10, 100, BLACK,
size=3)

```

```

#执行 WIFI 连接函数
WIFI_Connect()

#创建 socket 连接，连接成功后发送“Hello 01Studio!”给服务器。
client=usocket.socket()
addr=('192.168.1.116',10000) #服务器 IP 和端口
client.connect(addr)
client.send('Hello 01Studio!')
client.settimeout(0.1) #必须加上

#定时器回调函数
def fun(tim):
    global socket_node
    socket_node = 1 #改变 socket 标志位

#构建软件定时器，编号-1
tim = Timer(-1)
tim.init(period=100, mode=Timer.PERIODIC,callback=fun) #周期为 100ms

while True:
    if socket_node:

        try:
            data = client.recv(256)

        except OSError:
            data = None

        if data:
            print("rcv:", len(data),data)

```

```
socket_node = 0

else: pass
```

WIFI 连接代码在上一节已经讲解，这里不再重复，程序在连接成功后建了 Socket 连接，连接成功发送 ‘Hello 01Studio!’ 信息到服务器。另外 K210 定时器设定了每 100ms 处理从服务器接收到的数据。将接收到数据通过串口打印和重新发送给服务器。

● 实验结果:

先在电脑端打开网络调试助手并建立服务器，软件在 零一科技（01Studio）MicroPython 开发套件配套资料\01-开发工具\01-Windows\网络调试助手 下的 NetAssist.exe ，直接双击打开即可！

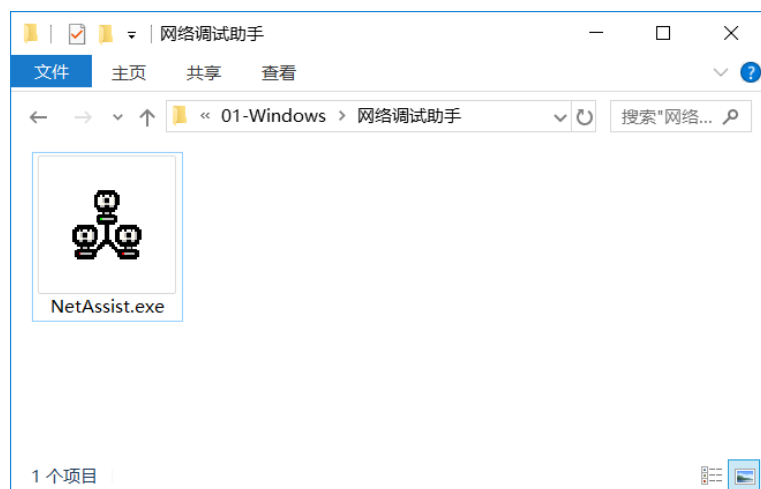


图 6-19 网络调试助手

以下是新建服务器的方法，打开网络调试助手后在左上角协议类型选择 TCP Server；中间的本地 IP 地址是自动识别的，不要修改，这个就是服务器的 IP 地址。然后端口写 10000（0-65535 都可以。），点击连接，成功后红点亮。如下图：



图 6-20 TCP 服务器配置

在时候服务器已经在监听状态！用户需要根据自己的实际情况自己输入 WIFI 信息和服务器 IP 地址+端口。即修改上面的代码以下部分内容。（服务器 IP 和端口可以在网络调试助手找到。）

WiFi 信息

SSID='01Studio' # Network SSID

KEY='88888888' # Network key

addr=('192.168.1.115',10000) #服务器 IP 和端口

运行程序，开发板成功连接 WIFI 后，发起了 socket 连接，连接成功可以可以看到网络调试助手收到了开发板发来的信息。在下方列表多了一个连接对象，点击选中：

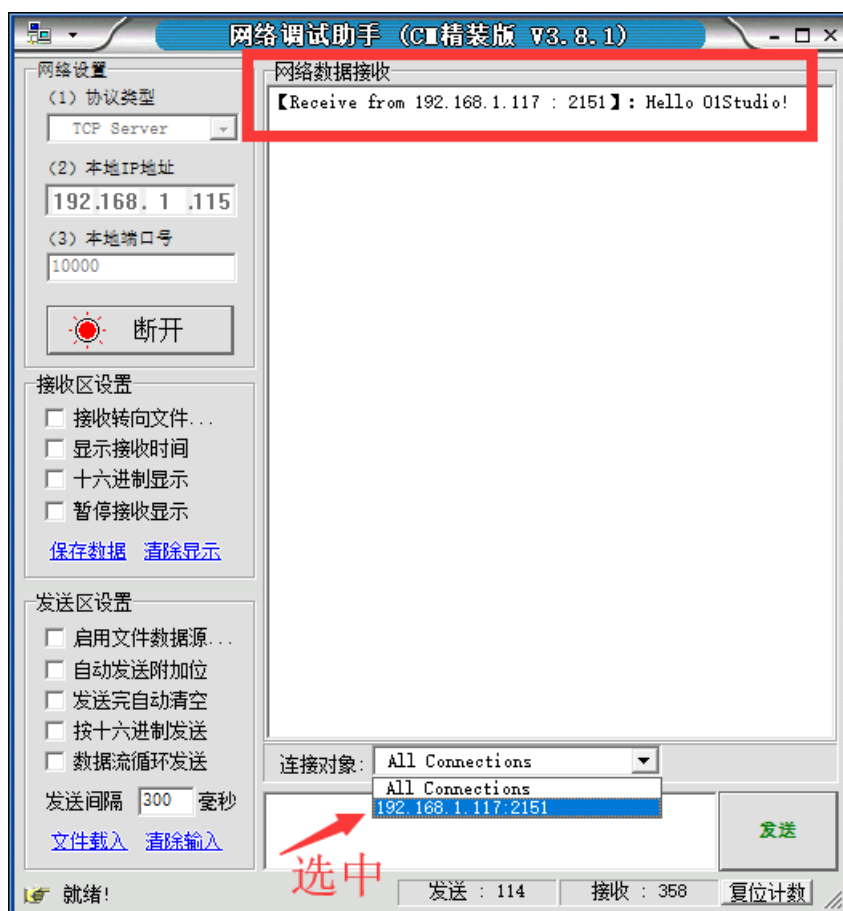


图 6-21 选中连接对象

选中后我们在发送框输入信息“<http://www.01studio.org>”，点击发送，可以看到开发板的串行终端打印出来该信息，类型为字节数据。

```

2F499
Type "help()" for more information.
>>> %Run -c $EDITOR_CONTENT
Connecting to network...
IP information:
('192.168.1.106', '255.255.255.0', '192.168.1.1', '0', '0', '28:6c:07:ec:85:2d', 'WeBe
e_office_2.4G')
rcv: 23 b'http://www.01studio.org'

```

图 6-22

● 总结:

通过本节学习,我们了解了 socket 通信原理以及使用 MicroPython 进行 socket 编程并且通信的实验。得益于优秀的封装,让我们可以直接面向 socket 对象编程就可以快速实现 socket 通信,从而开发更多的网络应用,例如将前面采集到的传感器数据发送到服务器。

6.2.3 MQTT 通信

- **前言：**

上一节，我们学习了 **Socket** 通信，当服务器和客户端建立起连接时，就可以相互通信了。在互联网应用大多使用 **WebSocket** 接口来传输数据。而在物联网应用中，常常出现这样的情况：海量的传感器，需要时刻保持在线，传输数据量非常低，有着大量用户使用。如果仍然使用 **socket** 作为通信，那么服务器的压力和通讯框架的设计随着数量的上升将变得异常复杂！

那么有无一个框架协议来解决这个问题呢，答案是有的。那就是 **MQTT**(消息队列遥测传输)。

- **实验平台：**

达芬奇开发板。

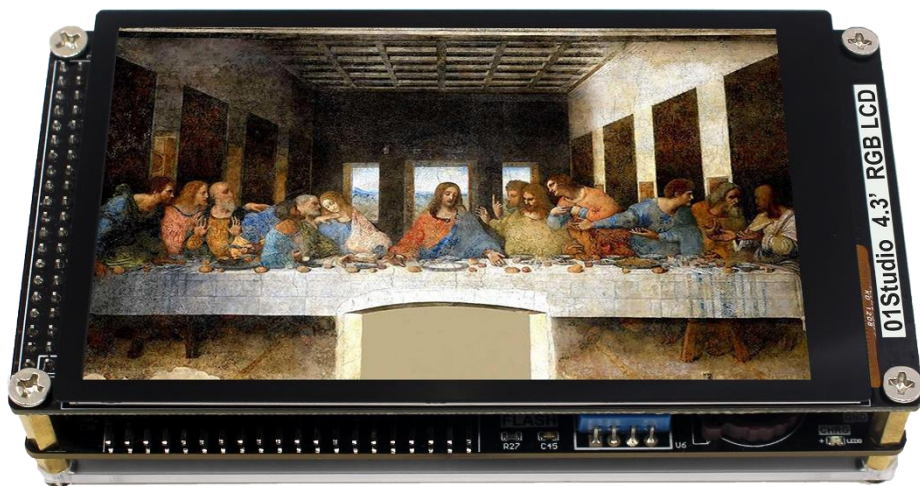


图 6-23 达芬奇开发板

- **实验目的：**

通过编程让达芬奇开发板实现 **MQTT** 协议信息的发布和订阅（接收）。

- **实验讲解：**

MQTT 是 **IBM** 于 1999 年提出的，和 **HTTP** 一样属于应用层，它工作在 **TCP/IP** 协议族上，通常还会调用 **socket** 接口。是一个基于客户端-服务器的消息发布/订

阅传输协议。其特点是协议是轻量、简单、开放和易于实现的，这些特点使它适用范围非常广泛。在很多情况下，包括受限的环境中，如：机器与机器（M2M）通信和物联网（IoT）。其在，通过卫星链路通信传感器、偶尔拨号的医疗设备、智能家居、及一些小型化设备中已广泛使用。

总结下来 MQTT 有如下特性/优势：

- 异步消息协议
- 面向长连接
- 双向数据传输
- 协议轻量级
- 被动数据获取

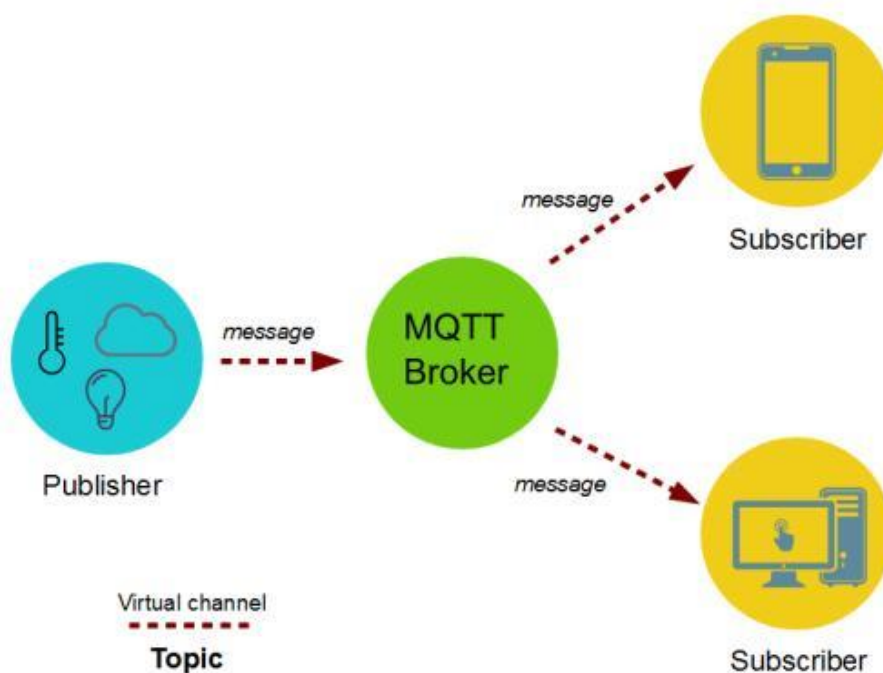


图 6-24 MQTT 通信流程

从上图可以看到，MQTT 通信的角色有两个，分别是服务器和客户端。服务器只负责中转数据，不做存储；客户端可以是信息发送者或订阅者，也可以同时是两者。具体如下图：

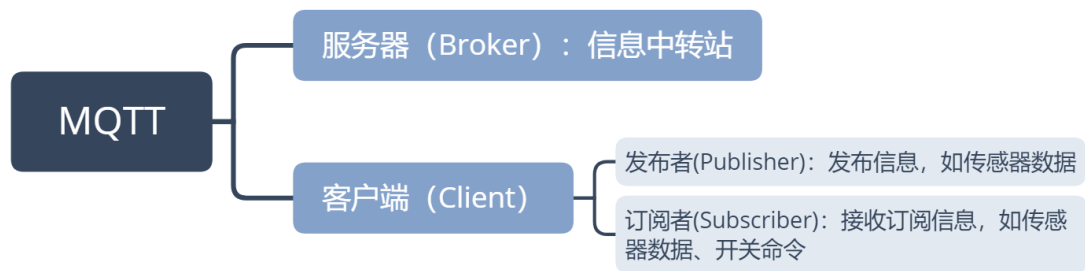


图 6-25 MQTT 角色说明

确定了角色后是如何传输数据呢？下表示 MQTT 最基本的数据帧格式，例如温度传感器发布主题“Temperature”编号,消息是“25”（表示温度）。那么所有订阅了这个主题编号的客户端（手机应用）就会收到相关信息，从而实现通信。如下表所示：

MQTT 数据帧格式	
Topic ID (主题编号)	Message (消息)
Temperature	25

图 6-26 MQTT 数据格式

由于特殊的发布/订阅机制，服务器不需要存储数据（当然也可以在服务器的设备上建立一个客户端来订阅保存信息），因此非常适合海量设备的传输。

人生苦短，而 MicroPython 已经封装好了 MQTT 客户端的库文件。让我们的应用变得简单美妙。K210 的 MQTT 模块文件为例程文件夹里面的 simple.py 文件。使用方法如下：

构造函数
<code>client=mqtt. MQTTClient (client_id, server, port)</code>
构建 MQTT 客户端对象。 client_id: 客户端 ID，具有唯一性； server: 服务器地址，可以是 IP 或者网址； port:服务器端口。（默认是 1883，服务器通常采用的端口，也可以自定义。）
使用方法
<code>client.connect()</code>

连接到服务器。
<code>client.publish(TOPIC,message)</code>
发布。TOPIC：主题编号；message：信息内容，例：'Hello 01Studio!'
<code>client.subscribe(TOPIC)</code>
订阅。TOPIC：主题编号。
<code>client.set_callback(callback)</code>
设置回调函数。callback：订阅后如果接收到信息，就执行相名称的回调函数。
<code>client.check_msg()</code>
检查订阅信息。如收到信息就执行设置过的回调函数 callback。

表 6-6 MQTT 客户端对象

由于客户端分为发布者和订阅者角色，因此为了方便大家更好理解，本实验分开两个案例来编程，分别为发布者和订阅者。再结合 MQTT 网络调试助手来测试。代表编写流程图如下：

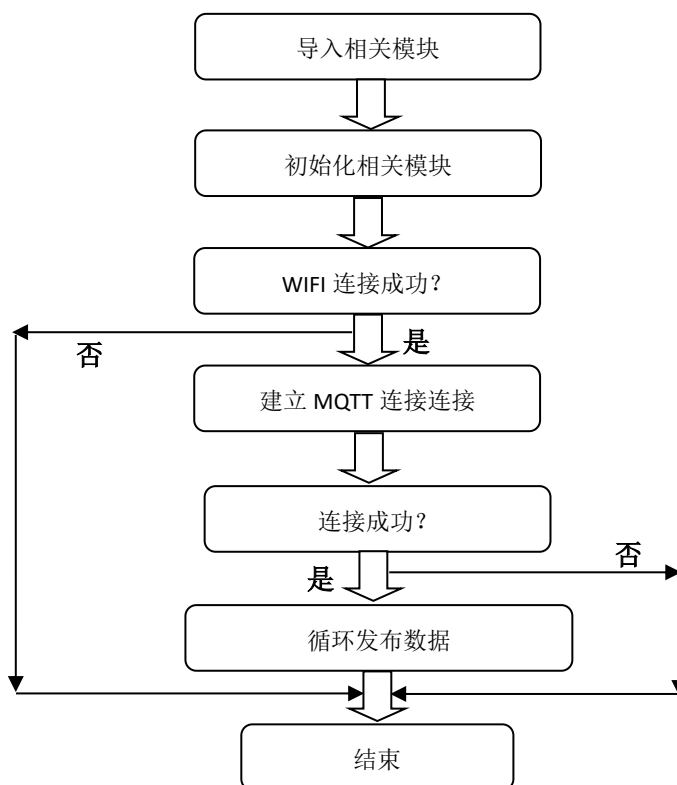


图 6-27 发布者代码编写流程

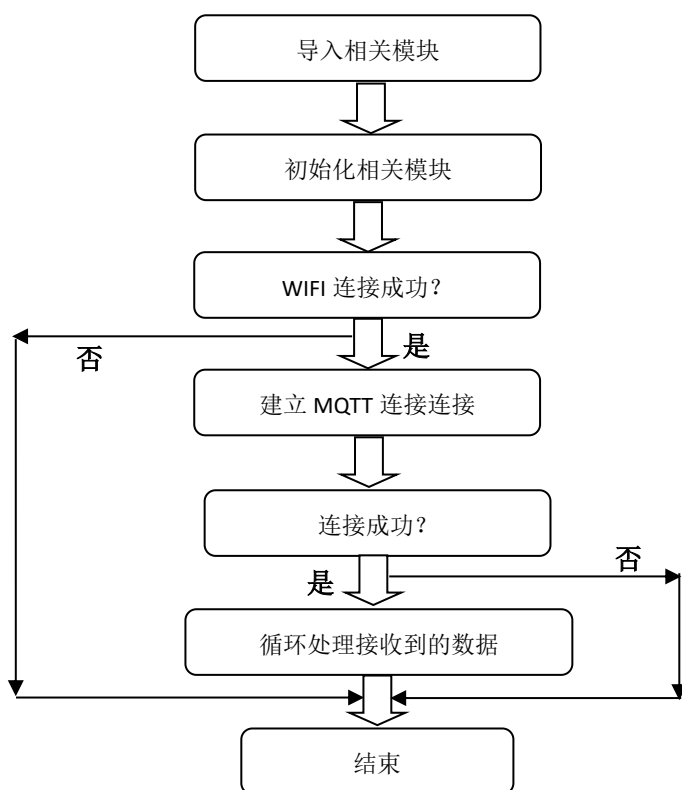


图 6-28 订阅者代码编写流程

发布者（publish）参考代码：

```

...

实验名称：MQTT 通信
版本：v1.0
日期：2019.8
作者：01Studio
说明：编程实现 MQTT 通信，实现发布数据。
...

import network,time
from machine import UART,Timer
from Maix import GPIO
from fpioa_manager import fm
from simple import MQTTClient

```

```

SSID='01Studio' # WiFi 账号
KEY='88888888' # WiFi 密码

##### WiFi 模块初始化 #####
#使能引脚初始化
fm.register(8, fm.fpioa.GPIOHS0, force=True)
wifi_en=GPIO(GPIO.GPIOHS0, GPIO.OUT)

#串口初始化
fm.register(7, fm.fpioa.UART2_TX, force=True)
fm.register(6, fm.fpioa.UART2_RX, force=True)
uart = UART(UART.UART2,115200,timeout=1000,read_buf_len=4096)

#WiFi 使能函数
def wifi_enable(en):
    global wifi_en
    wifi_en.value(en)

def wifi_init():
    global uart
    wifi_enable(0)
    time.sleep_ms(200)
    wifi_enable(1)
    time.sleep(2)
    uart = UART(UART.UART2,115200,timeout=1000, read_buf_len=4096)
    tmp = uart.read()
    uart.write("AT+UART_CUR=921600,8,1,0,0\r\n")
    print(uart.read())

```

```

    uart = UART(UART.UART2,921600,timeout=1000, read_buf_len=10240) #
important! baudrate too low or read_buf_len too small will loose data

    uart.write("AT\r\n")

    tmp = uart.read()

    print(tmp)

    if not tmp.endswith("OK\r\n"):

        print("reset fail")

        return None

    try:

        nic = network.ESP8285(uart)

    except Exception:

        return None

    return nic

#####

##### 主程序 #####

#####

#构建 WiFi 对象并使能

wlan = wifi_init()

#正在连接印提示

print("Trying to connect... (may take a while)...")

#连接网络

wlan.connect(SSID,KEY)

#打印 IP 相关信息

print(wlan.ifconfig())

```

```

#发布数据任务

def MQTT_Send(tim):

    client.publish(TOPIC, 'Hello 01Studio!')

SERVER = 'mqtt.p2hp.com'

PORT = 1883

CLIENT_ID = '01Studio-K210' # 客户端 ID

TOPIC = '/public/01Studio/1' # TOPIC 名称

client = MQTTClient(CLIENT_ID, SERVER, PORT)

client.connect()

#定时器 0 初始化, 周期 1 秒

tim = Timer(Timer.TIMER0, Timer.CHANNEL0, mode=Timer.MODE_PERIODIC,

            period=1000, callback=MQTT_Send)

while True:

    pass

```

订阅者（subscribe）参考代码：

```

'''

实验名称: MQTT 通信

版本: v1.0

日期: 2019.8

作者: 01Studio

说明: 编程实现 MQTT 通信, 实现订阅数据。

'''

import network,time

from machine import UART,Timer

from Maix import GPIO

```

```

from fpioa_manager import fm
from simple import MQTTClient

SSID='01Studio' # WiFi 账号
KEY='88888888' # WiFi 密码


##### WiFi 模块初始化 #####
#使能引脚初始化
fm.register(8, fm.fpioa.GPIOHS0, force=True)
wifi_en=GPIO(GPIO.GPIOHS0, GPIO.OUT)

#串口初始化
fm.register(7, fm.fpioa.UART2_TX, force=True)
fm.register(6, fm.fpioa.UART2_RX, force=True)
uart = UART(UART.UART2,115200,timeout=1000,read_buf_len=4096)

#WiFi 使能函数
def wifi_enable(en):
    global wifi_en
    wifi_en.value(en)

def wifi_init():
    global uart
    wifi_enable(0)
    time.sleep_ms(200)
    wifi_enable(1)
    time.sleep(2)
    uart = UART(UART.UART2,115200,timeout=1000, read_buf_len=4096)
    tmp = uart.read()

```

```

    uart.write("AT+UART_CUR=921600,8,1,0,0\r\n")
    print(uart.read())
# important! baudrate too low or read_buf_len too small will loose data
    uart = UART(UART.UART2,921600,timeout=1000, read_buf_len=10240)
    uart.write("AT\r\n")
    tmp = uart.read()
    print(tmp)
    if not tmp.endswith("OK\r\n"):
        print("reset fail")
        return None
    try:
        nic = network.ESP8285(uart)
    except Exception:
        return None
    return nic

#####

##### 主程序 #####

#####

#构建 WiFi 对象并使能
wlan = wifi_init()

#正在连接印提示
print("Trying to connect... (may take a while)...")

#连接网络
wlan.connect(SSID,KEY)

#打印 IP 相关信息

```



```

print(wlan.ifconfig())

#设置 MQTT 回调函数,有信息时候执行
def MQTT_callback(topic, msg):
    print('topic: {}'.format(topic))
    print('msg: {}'.format(msg))

#接收数据任务
def MQTT_Rev(tim):
    try:
        client.check_msg()
    except OSError:
        pass

SERVER = 'mqtt.p2hp.com'
PORT = 1883
CLIENT_ID = '01Studio-ESP32' # 客户端 ID
TOPIC = '/public/01Studio/1' # TOPIC 名称

client = MQTTClient(CLIENT_ID, SERVER, PORT) #建立客户端对象
client.set_callback(MQTT_callback) #配置回调函数
client.connect()
client.subscribe(TOPIC) #订阅主题

#定时器 0 初始化,周期 300ms,执行 MQTT 通信接收任务
tim = Timer(Timer.TIMER0, Timer.CHANNEL0, mode=Timer.MODE_PERIODIC,
            period=300, callback=MQTT_Rev)

while True:
    pass

```

从以上代码可以看到发布者和订阅者的编程方式相近，另外本实验需要一个 MQTT 服务器（Broker），这里使用的是跟我们将用到的 MQTT 在线网络助手同一个服务器和端口。

```
SERVER = 'mqtt.p2hp.com'

PORT = 1883
```

● 实验结果：

我们将 MQTT 示例程序中的 simple.py 文件拷贝到 K210 文件系统，然后在 MaixPy IDE 分别运行上述代码。

为了方便测试，我们可以使用 MQTT 网络助手进行调试。这里推荐一个在线 MQTT 网络调试助手：<http://mqtt.p2hp.com/websocket/>

打开上面网址，即可看到 MQTT 在线调试助手。可以配置基本信息，这里默认即可，点击连接。

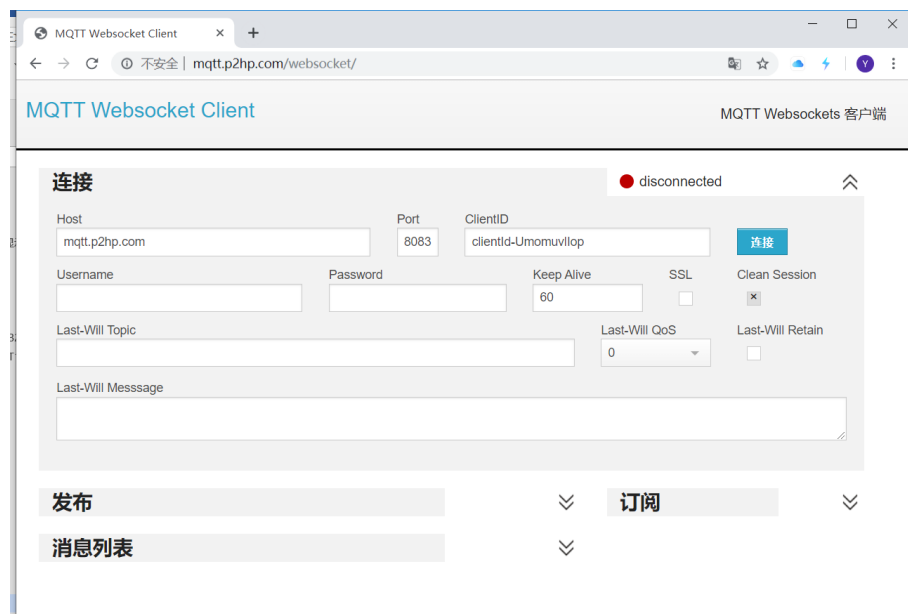


图 6-29 MQTT 助手

连接成功可以看到显示 Connected。

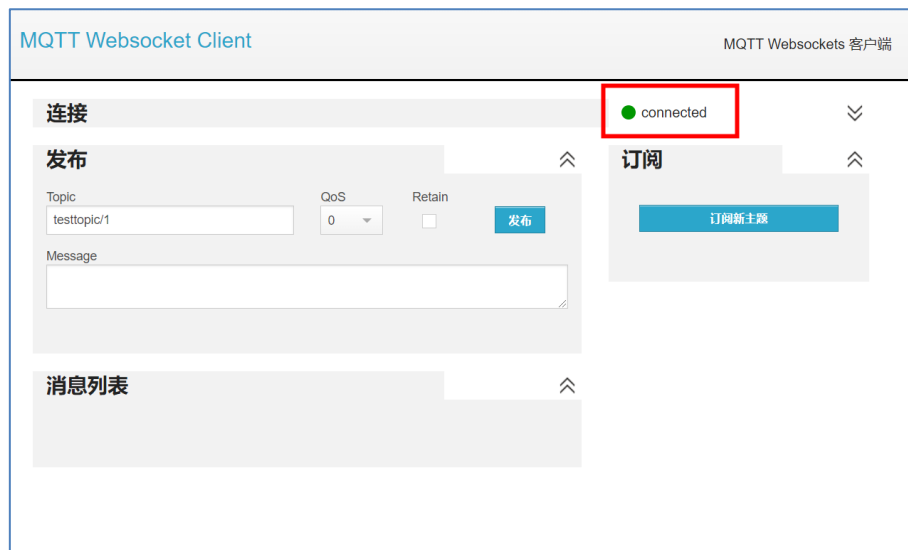


图 6-30 MQTT 助手连接成功

测试“发布者”代码：

我们先测试“发布者”代码。在 MQTT 助手中订阅主题修改为：'/public/01Studio/1'（跟代码发布的主题一致），QOS 选择 0 即可。然后点击订阅主题。

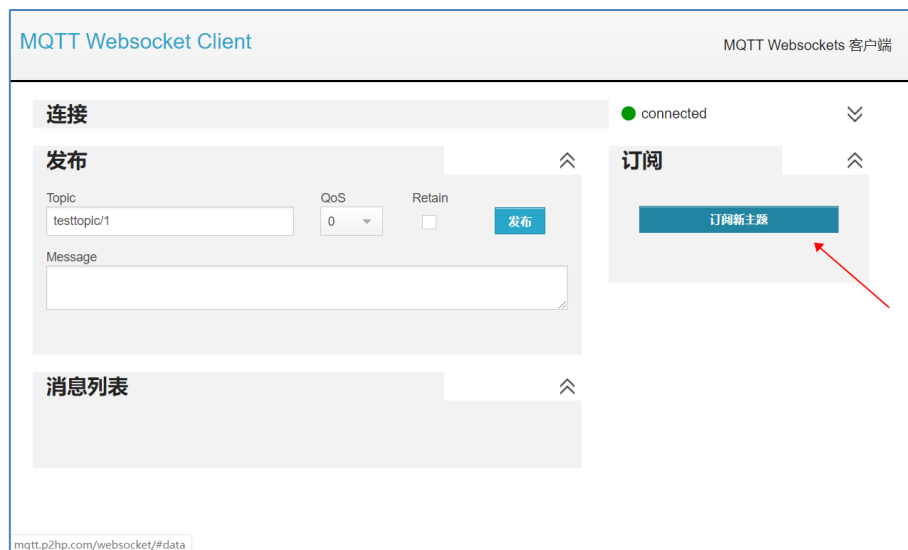


图 6-31 订阅主题

输入：/public/01Studio/1 ， Qos 选择 0。

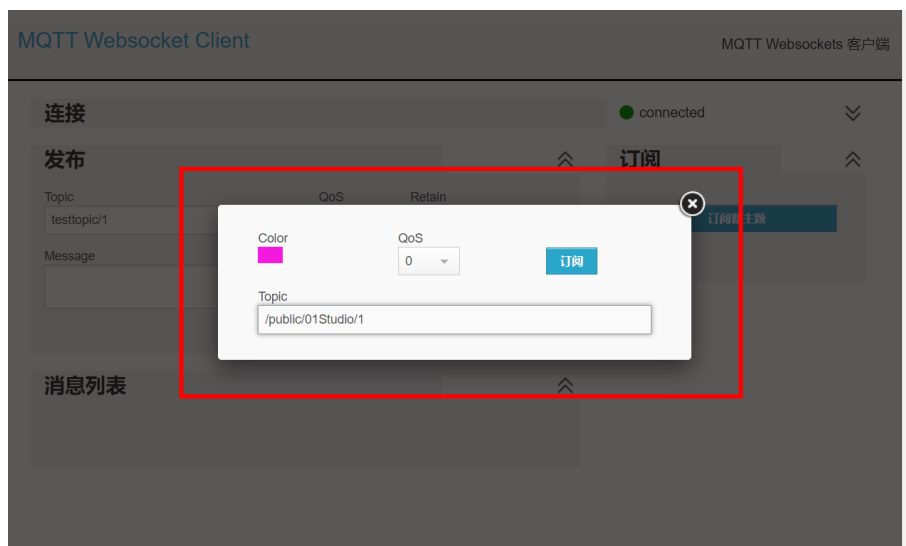


图 6-32 订阅主题

订阅后,运行程序,可以看到接收框收到来自开发板发布的信息。

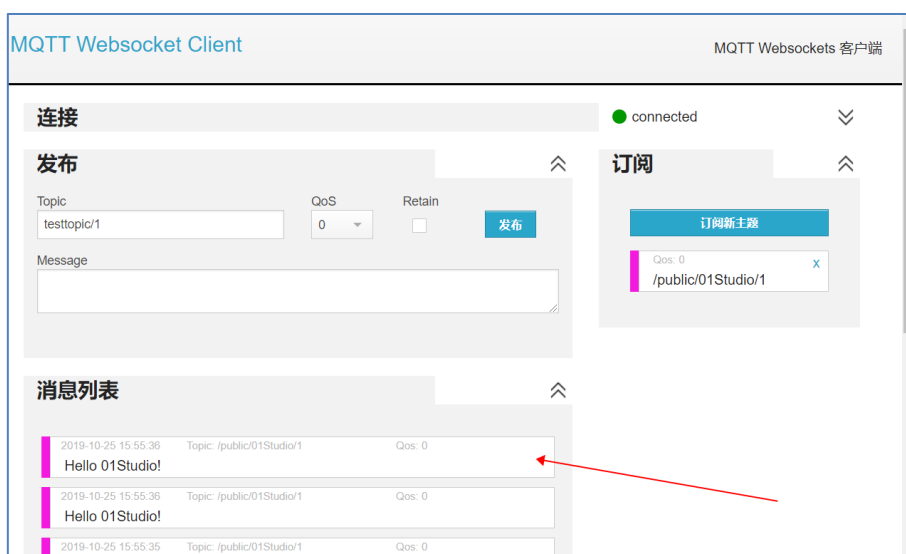


图 6-33 MQTT 助手收到开发板发布的信息

测试“订阅者”代码:

“订阅者”代码测试方法跟“发布者”相反。在电脑 MQTT 助手中发布主题修改为: '/public/01Studio/1' (跟代码发布的主题一致。) 在下方空白框输入“Hello 01Studio!”。

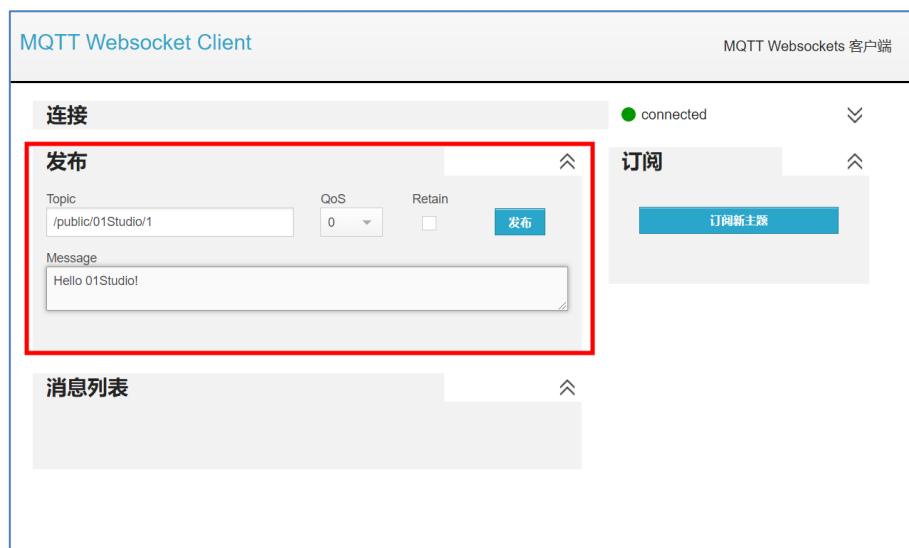


图 6-34 输入即将发布的主题和信息

开发板运行“订阅者”代码，成功连接后回到 MQTT 助手点击【发布】发布信息，可以在开发板的 REPL 看到接收到的订阅信息“Hello 01Studio!”被打印出来了（数据格式为字节数据）。

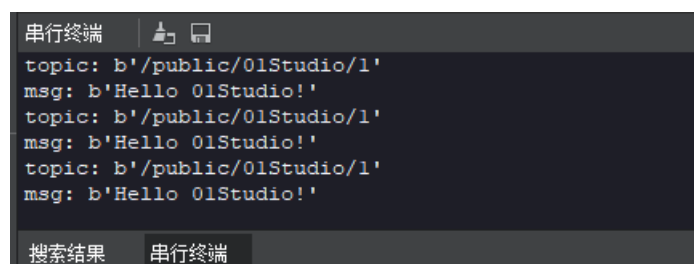


图 6-35 开发板接收到相关信息并打印

当使用 main.py 脱机运行时候，IDE 的串口终端和 putty 并不能打印信息。可以使用串口助手观察。波特率为 115200。

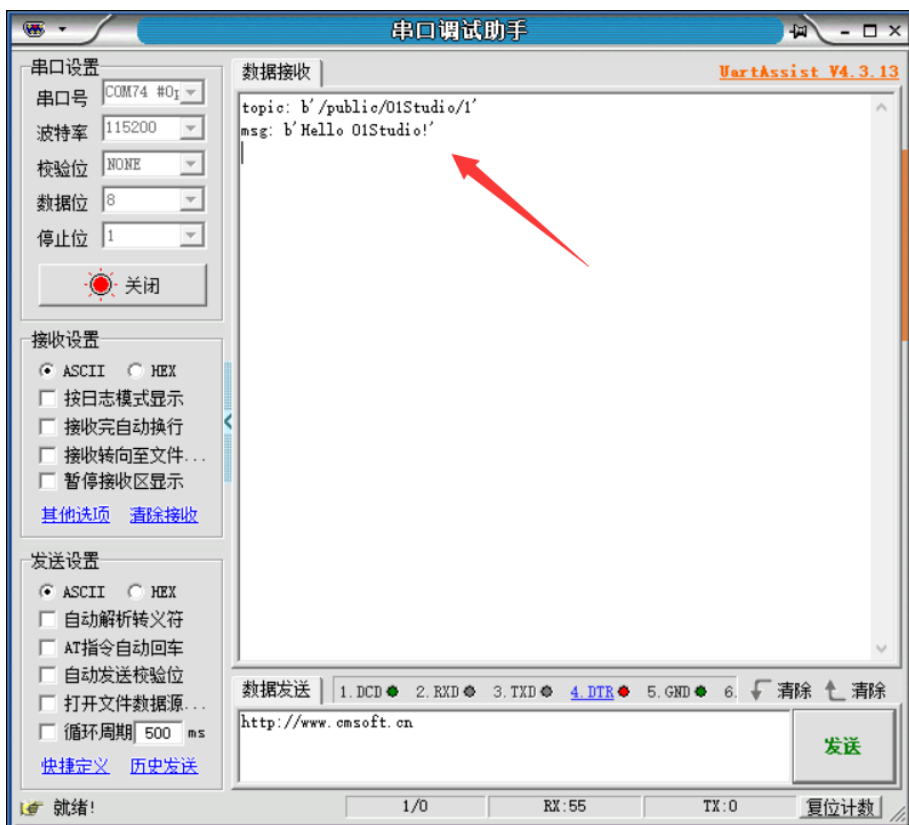


图 6-36 串口调试助手

当然你也可以在同一个 MQTT 在线助手下测试发布和订阅功能来做一些测试，只需要将主题设置一致即可，如下图所示：

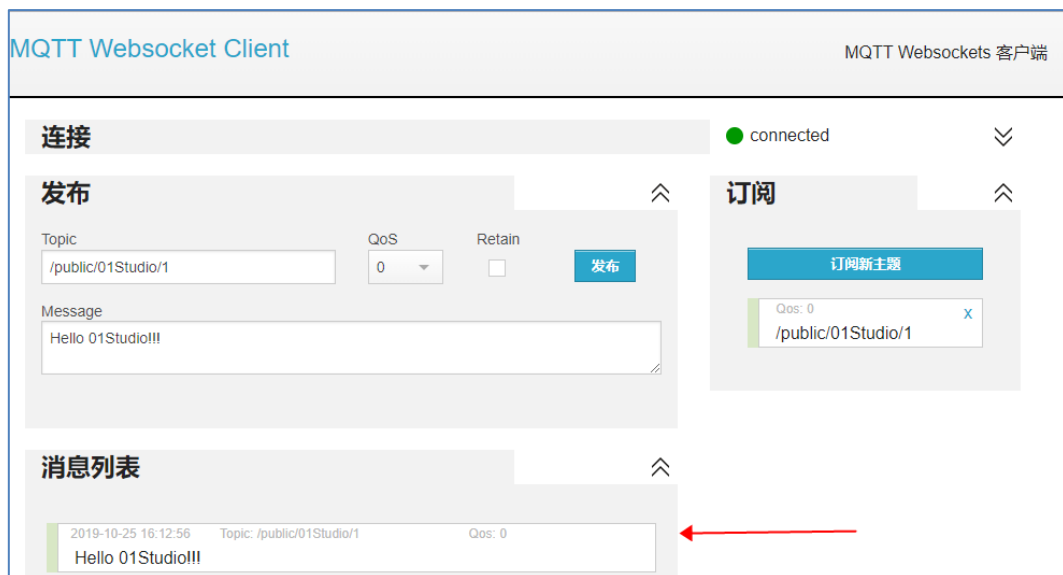


图 6-37 客户端同时发布和订阅信息

- **总结:**

通过本节我们了解了 MQTT 通信原理以及成功实现通信。目前市面上大部分物联网云平台支持 MQTT，原理大同小异。大家可以基于不同平台协议来开发，实现自己的物联网设备远程连接。

6.3 pyIOT（无线通信模块）

pyIOT 开源项目由 01Studio 发起，旨在为市面上成熟的串口物联网模块开发 micropython 库，如：WiFi、BLE、ZigBee、LORA、NB-IoT、4G、GPS(北斗)等，让用户可以快速实现各类物联网相关应用。

项目地址：<https://github.com/01studio-lab/pyIOT>

6.3.1 pyIOT-BLE TLS-01 蓝牙模块

- **前言：**

低功耗蓝牙透传模块日常生活使用非常多，例如手环，通过手机可以快速实现蓝牙无线连接。从而控制硬件设备。

- **实验平台：**

达芬奇 MicroPython 开发套件和 pyIOT-TSL01 蓝牙透传模块。

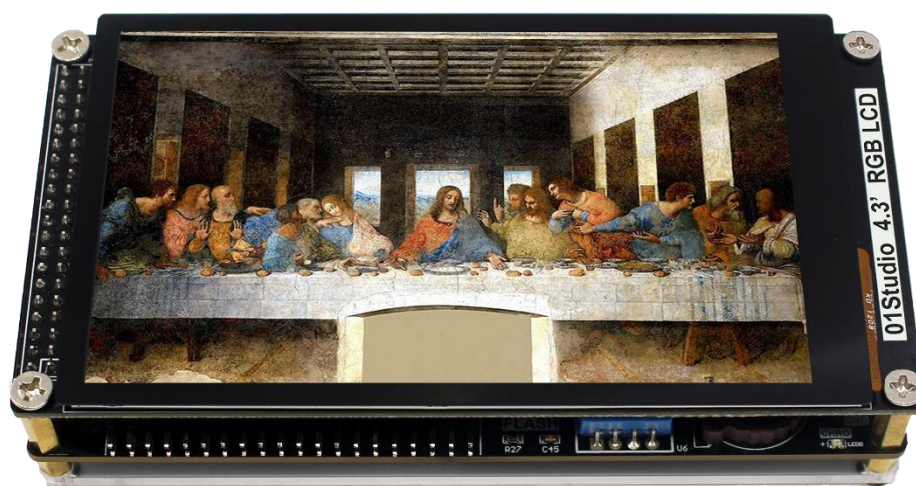


图 6-38

- **实验目的：**

通过手机软件蓝牙连接模块，跟开发板进行通讯，LCD 显示接收到的字符。并识别特定字符，点亮 LED。

- **实验讲解：**

pyIOT-TLS01 是基于 WeBee TLS-01 串口蓝牙透传模块，产品参数如下：

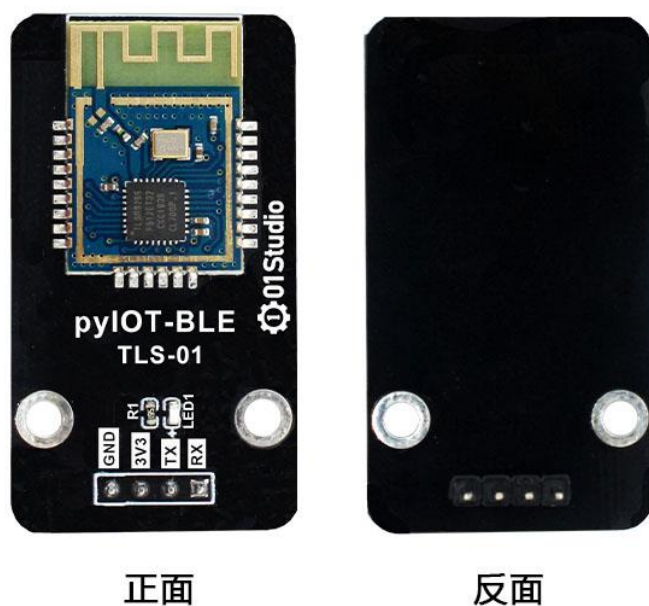


图 6-39

功能参数	
蓝牙主控	TLSR8266
控制方式	UART (串口)
蓝牙版本	BLE 4.0 (从机)
功耗	工作电流: <8.8mA, 广播电流: <1mA
引脚	GND, 3.3V, TX, RX
模块尺寸	4.5*2.5cm

表 6-7

01Studio 已经封装好了对应的 python 库，用户可以使用 python 编程直接使用基本的蓝牙功能。下面是 TLS01 库函数说明：

构造函数
<pre>tls01.TLS01(uart, baud_rate=9600)</pre> 创建 TLS01 蓝牙模块对象。 【uart】 串口编号 【baud_rate】 波特率，默认是 9600

使用方法
TLS01.Name_Check()
查看模块广播名称，返回字符。
TLS01.BI_Check()
查看蓝牙模块广播间隔，返回类型： <code>int</code>
TLS01.Baudrate_Check()
返回波特率
TLS01.Name_Set(name)
设置模块广播名称； 【name】 广播名字，字符类型，如： <code>01Studio</code>
TLS01.BI_Set(BI)
设置模块广播间隔； 【BI】 蓝牙广播间隔，间隔越短，手机搜索到设备越快。例： <code>100</code> 表示 <code>100ms</code>
TLS01.Baudrate_Set(Baudrate)
设置模块串口波特率； 【Baudrate】 波特率值，例： <code>9600</code> ， <code>115200</code>
TLS01.RST()
复位模块
TLS01.RESET()
重置模块，恢复出厂设置。

图 6-40 TLS01 对象

蓝牙串口模块的使用已经很简单了，让我们不用去考虑蓝牙协议栈的事情，现在在这个基础上再使用 `micropython` 封装相关库，就变得更易用了，编程流程如下：

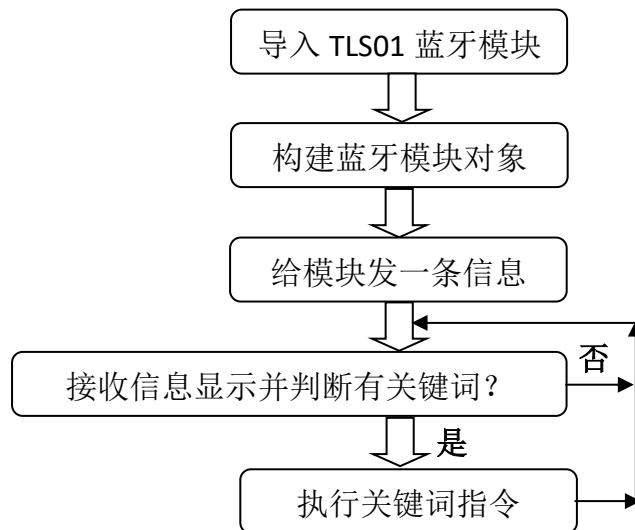


图 6-41 代码流程图

参考代码如下：

```

...

实验名称：串口蓝牙模块通信（pyIOT-BLE TLS01 蓝牙串口模块 by WeBee）
版本：v1.0
日期：2021.5
作者：01Studio
说明：通过编程实现蓝牙模块数据收发，并执行关键词指令，点亮 LED
实验平台：01Studio-达芬奇
...

#导入相关模块

from tls01 import TLS01

from machine import Pin

from tftlcd import LCD43R

import time

#定义常用颜色

RED = (255,0,0)
  
```

```

GREEN = (0,255,0)
BLUE = (0,0,255)
BLACK = (0,0,0)

#####

# 构建 4.3 寸 LCD 对象并初始化
#####

d = LCD43R(portrait=1) #默认方向

#画接收框
d.fill((255,255,255))
d.printStr('BLE Message Receive', 80, 20, BLACK, size=3)
d.printStr('Receive:', 10, 100, RED, size=3)
d.drawRect(10, 150, 450, 100, BLUE, border=5) #画矩形

#构建 LED4 对象,蓝灯
LED = Pin('C7', Pin.OUT)

#构建蓝牙模块对象（串口 2）
BLE=TLS01(2,9600) #设置串口号 3 和波特率,TX--A2,RX--YA3

#####信息透传#####
BLE.uart.write('Hello 01Studio!')#给手机发送一条数据

num = 0 #接收计数

#接收信息
while True:

    if BLE.uart.any(): #查询是否有信息

```

```

text = BLE.uart.read(128) #默认单次最多接收 128 字节'''
print(text)

#接收数据 LCD 显示
num =num +1
d.printStr('Num:'+str(num), 150, 100, BLACK, size=3)
d.printStr(str(text), 20, 180, BLACK, size=3)

#判断关键词指令，打开或关闭 LED4 蓝灯
if text == b'LEDON':
    LED.value(1)

if text == b'LEDOFF':
    LED.value(0)

```

● 实验结果:

本实验需要安装蓝牙模块配套的 APP，目前仅支持安卓系统。将 零一科技 (01Studio) 达芬奇 MicroPython 开发套件配套资料\02-示例程序\3.通讯实验\6.pyIOT 无线模块\1.TLS-01 蓝牙模块\02-手机 APP 下的安卓应用 BLE SPS.apk

将蓝牙模块通过 4P 排线跟达芬奇开发板连接：注意排针顺序跟接口对应。切勿接反电源导致短路。



图 6-42 GND 往最低部

插入后达芬奇上电，打开 APP，点击右上角 scan。

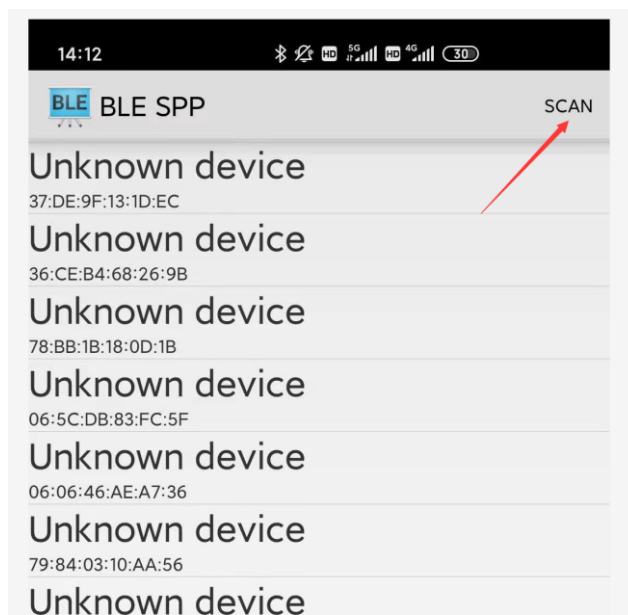


图 6-43

搜索出名字为 BLE SPS 时候，点击进入，自动连接蓝牙模块：

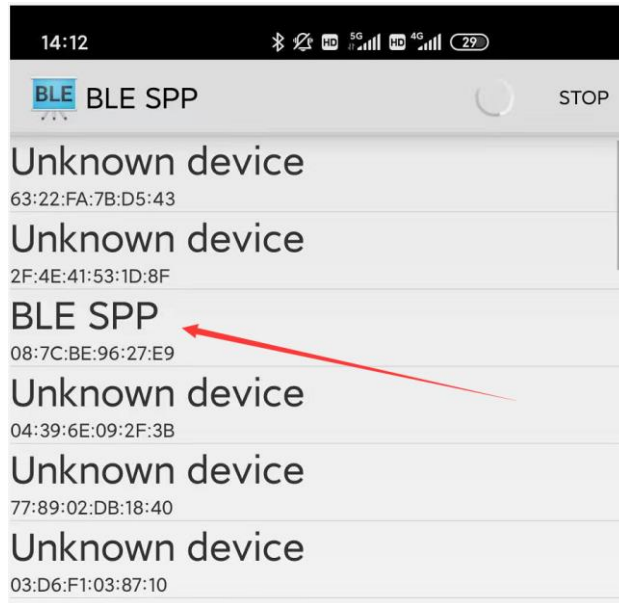


图 6-44

连接成功后模块的蓝色 LED 熄灭，表示连接成功。

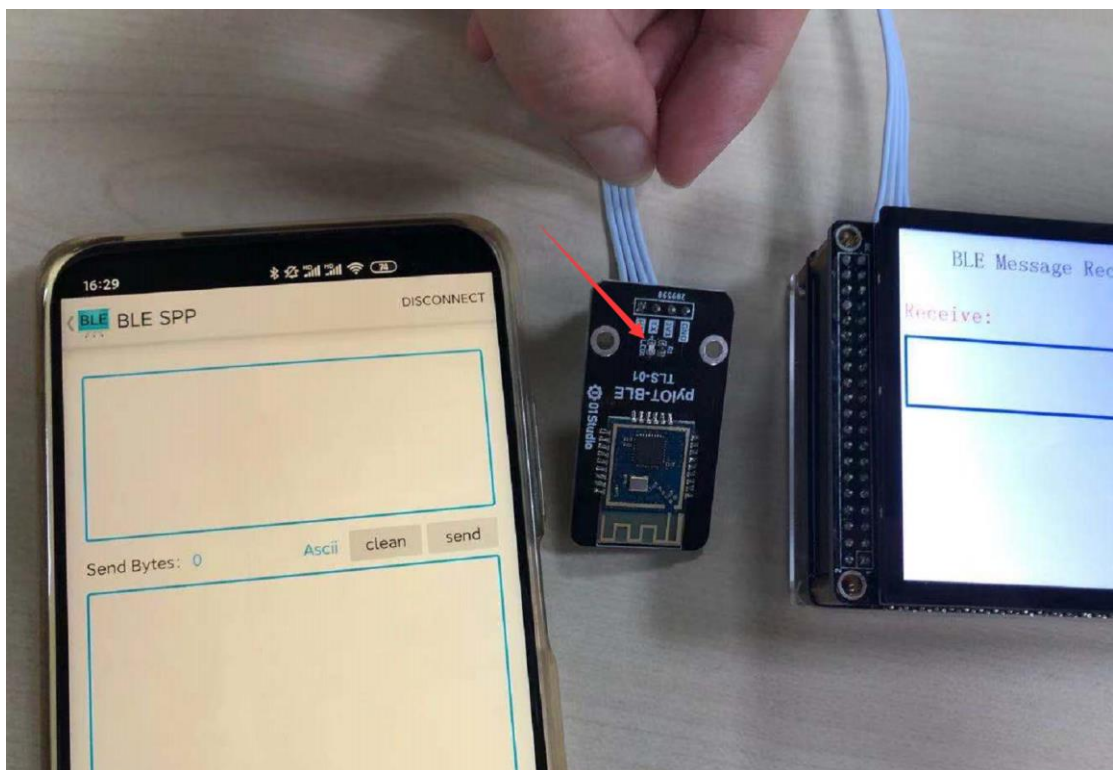


图 6-45

连接成功后可以相互收发数据：

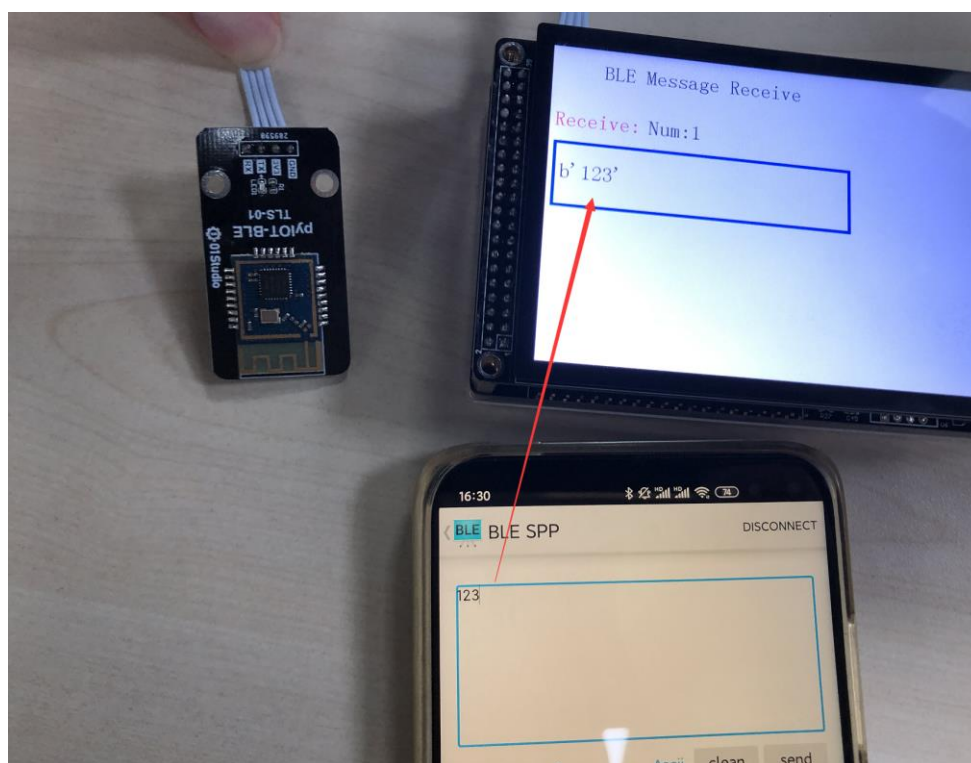


图 6-46

发送“LEDON”和“LEDOFF”关键词，分别点亮和关闭 LED4 蓝灯。

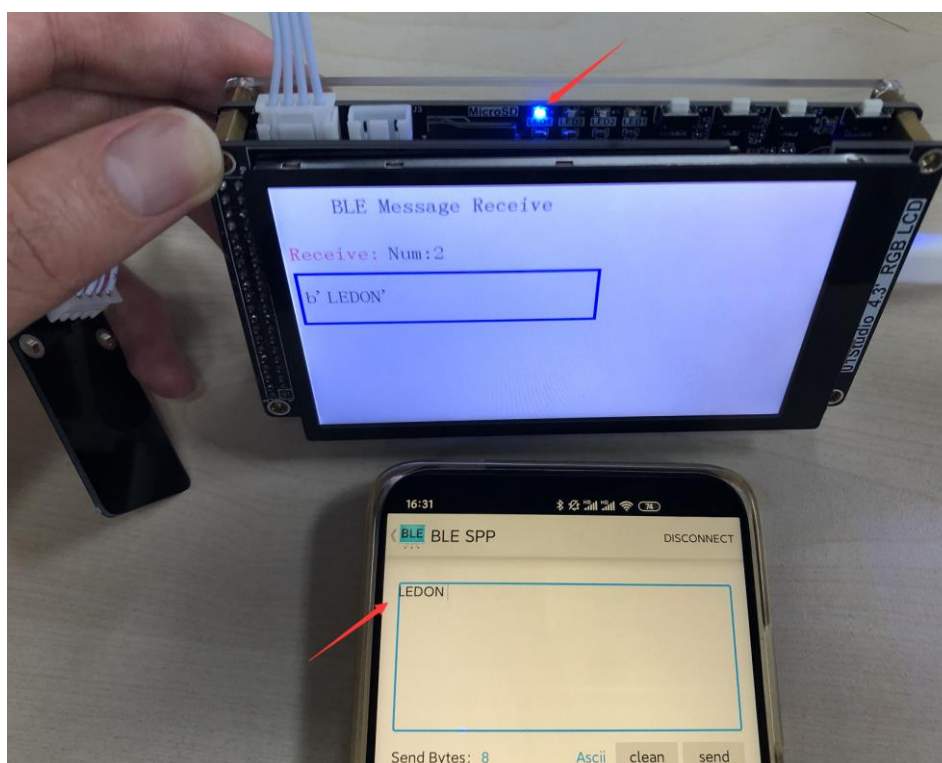


图 6-47 指令点亮 LED4

- **总结:**

蓝牙串口透传模块的使用已经非常简单，赋予 `micropython` 库后使用起来则更为方便，其本质是对串口指令的封装，有兴趣用户可以查看源代码，结合模块手册为模块增加更多的库函数功能。

6.3.2 pyIOT-GPS ATGM336H 卫星定位模块

- **前言：**

全球卫星定位系统的用途非常广泛，为船舶、汽车、飞机或其它移动资产进行定位。

全球有 4 大卫星定位系统，GPS 系统（美国）、BDS 系统（中国北斗）、GLONASS 系统（俄罗斯）、伽利略系统（欧盟）。目前国内比较常用的是 GPS 系统和北斗导航系统。

- **实验平台：**

达芬奇 MicroPython 开发套件和 pyIOT-GPS ATGM336H 卫星定位模块。

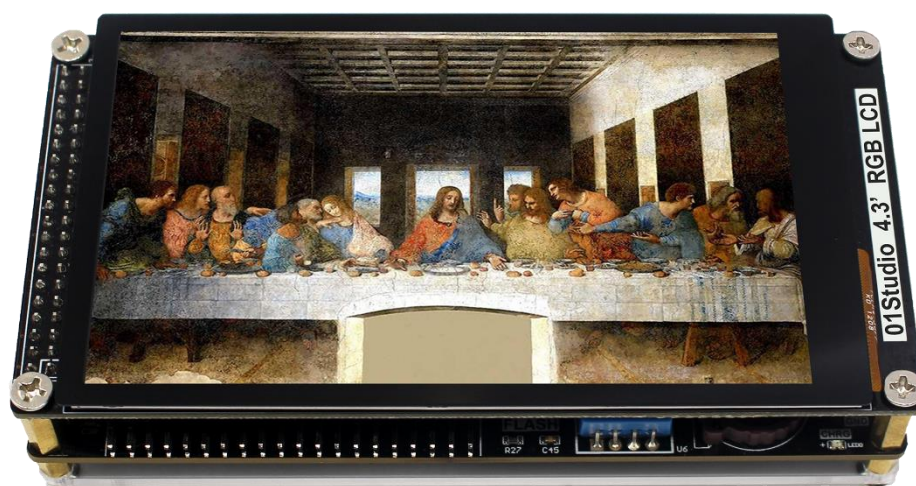


图 6-48

- **实验目的：**

编程实现获取 GPS 定位信息。

- **实验讲解：**

pyIOT-GPS ATGM336H 是标准的串口卫星定位模块，通过串口单向输出卫星信息，产品参数如下：

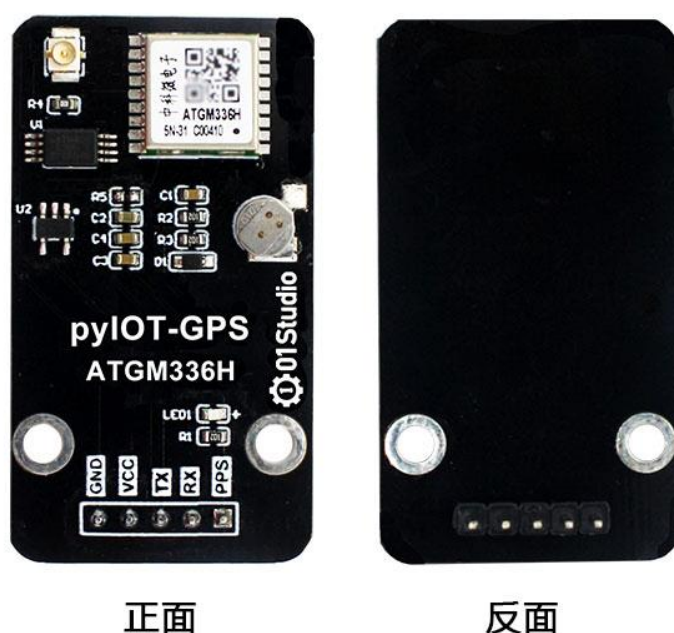


图 6-49

功能参数	
主控芯片	ATGM336H-5N
卫星信号	GPS/BDS/GLONASS
波特率	9600bps
工作电压	3.3V-5V
串口电平	3.3V 或 5V (自适应)
定位精度	2.5m (开阔地点)
功耗	工作：<25mA, 待机：<10uA (@3.3V)
模块尺寸	4.2*2.5cm

表 6-8

01Studio 已经封装好了对应的 python 库，用户可以使用 python 编程直接使获取模块 GPS 信息， 我们只需要对 GPS 信息进行解析并显示即可。关于 ATGM336H 信息的详细说明可以看手册：[零一科技（01Studio）达芬奇 MicroPython 开发套件配套资料 latest\02-示例程序\3.通讯实验\6.pyIOT 无线模块\2.ATGM336H GPS 模块\03-核心模块手册](#)。代码编写流程如下：

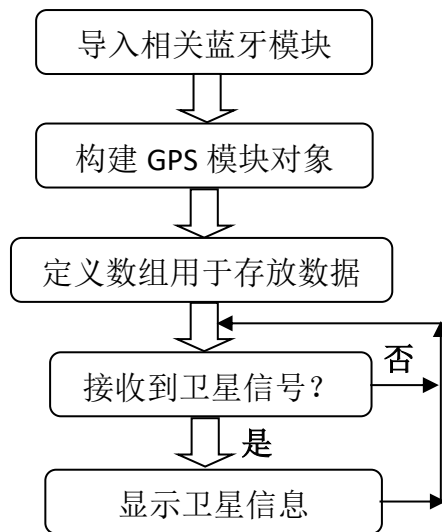


图 6-50 代码流程图

参考代码如下：

```

...

实验名称：pyIOT-GPS 模块（中科微 ATGM336H GPS 模块）
版本：v1.0
日期：2021.1
作者：01Studio
说明：编程实现串口获取 GPS 数据并显示
...

#导入串口模块

from machine import UART
from tftlcd import LCD43M
import time

#定义常用颜色

RED = (255,0,0)
GREEN = (0,255,0)
BLUE = (0,0,255)

```

```

BLACK = (0,0,0)

#####

# 构建 4.3 寸 LCD 对象并初始化

#####

d = LCD43M(portrait=1) #默认方向

#画接收框

d.fill((255,255,255))

d.printStr('GPS Location', 80, 20, BLUE, size=4)


#GPS 数据说明

#消息 ID: $GNGGA, 双模 GPS

#[0]定位点 UTC 时间

#[1]纬度[2]纬度方向[3]经度[4]经度方向

#[5]GPS 定位状态

#[6]卫星数量

#[7]水平精度衰减因子

#[8]海平面高度[9]高度单位

GPS_INFO=[' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' '] #14 个数据

k1='$GNGGA,' #关键词, 双模 GPS 数据


#构建蓝牙模块对象（串口）

GPS=UART(3,9600) #设置串口号 3 和波特率,TX--Y9,RX--Y10


#接收信息

while True:

    if GPS.any(): #查询是否有信息

        text0 = GPS.read(1024) #默认单次最多接收 128 字节'''

```

```

#print(text0) #原始数据

text=str(text0) #将数据转成字符

#找到双模定位
if text.find(k1) != -1 :
    begin_num=text.find(k1)+7 #起始字符
    for i in range(14):
        while text[begin_num]!=',' :
            GPS_INFO[i] = GPS_INFO[i]+str(text[begin_num])
            begin_num=begin_num+1
        begin_num=begin_num+1

    print(GPS_INFO) #双模 GPS 数据

#时间
time=GPS_INFO[0].split('.')
hh=int(int(time[0])/10000)+8 #北京时间东八区
mm=int(int(time[0])%10000/100)
ss=int(int(time[0])%100)
print('Time: '+str(hh)+':'+str(mm)+':'+str(ss))
d.printStr('Time: '+str(hh)+':'+str(mm)+':'+str(ss), 10, 100,
          BLACK, size=3)

#经纬度
print(GPS_INFO[1]+' '+GPS_INFO[2])
print(GPS_INFO[3]+' '+GPS_INFO[4])
d.printStr(GPS_INFO[1]+' '+GPS_INFO[2], 10, 150, BLACK,
          size=3)

```

```

d.printStr(GPS_INFO[3]+' '+GPS_INFO[4], 10, 200, BLACK,
           size=3)

#定位状态, 1 为定位成功
print('GPS State: '+GPS_INFO[5])
d.printStr('GPS State: '+GPS_INFO[5], 10, 250, BLACK, size=3)

#卫星数量
print('Satellites: '+GPS_INFO[6])
d.printStr('Satellites: '+GPS_INFO[6], 10, 300, BLACK,
           size=3)

#水平精度衰减因子
print('Horizontal precision attenuation factor:
      '+GPS_INFO[7])

#海拔高度
print('altitude: '+GPS_INFO[8]+GPS_INFO[9])
d.printStr('altitude: '+GPS_INFO[8]+GPS_INFO[9], 10, 350,
           BLACK, size=3)

#清空数据
for i in range(14):
    GPS_INFO[i]=' '

```

● 实验结果:

将 pyIOT-GPS ATGM336H 模块连接到达芬奇开发板 pyIOT 接口, 注意引脚顺序对上。同时模块接上 GPS 专用的陶瓷天线。



图 6-51 模块连接

GPS 模块需放在室外空旷位置搜索卫星信号（部分地方阳台也可以，取决于遮挡），当搜索到卫星信号时，模块的蓝灯会周期性闪烁。然后可以看到 LCD 显示出当前 GPS 模块发过来的信息：

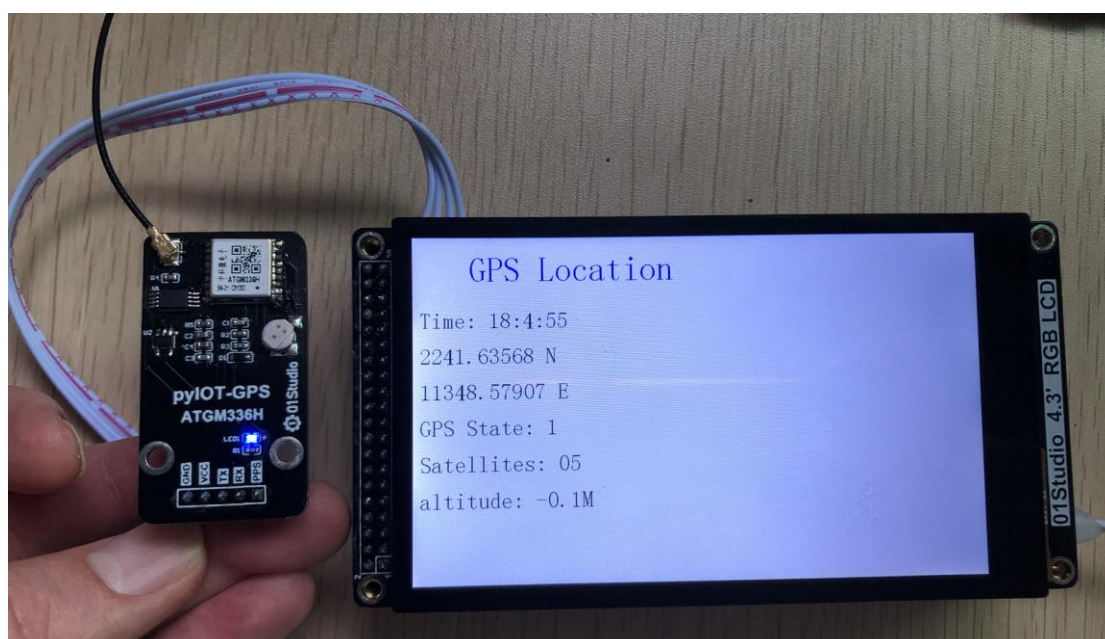


图 6-52 显示 GPS 信息

● 总结：

串口 GPS 模块的使用非常简单，但用途却非常广。有兴趣的用户可以用来打造自己的全球定位仪。

MicroBit 从0到1

用方块开始你的编程之旅

01Studio团队 编著



01Studio
-让编程变得简单有趣-

MicroPython 从0到1

用python做嵌入式编程

(基于pyBoard STM32F405平台)

01Studio团队 编著

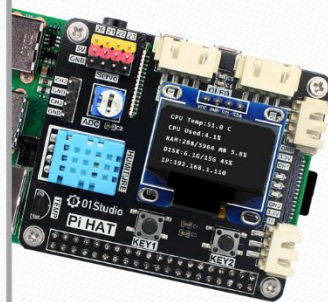


01Studio
-让编程变得简单有趣-

树莓派从0到1

(基于树莓派4B平台)

01Studio团队 编著



01Studio
-让编程变得简单有趣-



关注公众号，免费下载

